Introduction to Scientific Computing
(Beräkningsmatematik)

Lecture note sets

Andrew R. Winters
Division of Applied Mathematics
Department of Mathematics
Linköping University
andrew.ross.winters (at) liu.se

August 24, 2026

Contents

Page

Note Set 1: Concepts and finite precision

Sources of error 3

Finite precision 7

Note Set 2: Nonlinear equations and fixed point methods

Root finding problem 12

Fixed point iterative method 17

Note Set 3: Linear systems of equations

Basics and computational primitives 22

Solving linear systems 29

Note Set 4: Solving linear systems: LU and iterative methods

LU factorization 35

Stationary iterative methods 41

Note Set 5: Solving linear systems: Rectangular matrices

Overdetermined systems and **QR** factorization 48

Note Set 6: Polynomial interpolation

Global polynomial interpolation 56

Piecewise polynomial interpolation 63

Note Set 7: Approximating a function and its derivative(s)

Improving the global approximation 69

Discrete differentiation 75

Note Set 8: Numerical integration (quadrature)

Newton-Cotes quadrature 82

Gaussian quadrature 88

Note Set 9: Ordinary differential equations: Basics and Euler

Background and forward Euler 95

Consistency, convergence and stability of Euler methods 101

Note Set 10: Ordinary differential equations: Runge-Kutta

Runge-Kutta methods 107

Outlook to numerical partial differential equations 112

Appendix A: Note sets for removed topics

Solving linear systems: Descent methods 118

Note Set 1: Concepts and Finite Precision

The focus of this course is to provide an introduction to scientific computing. The field of numerical analysis is a powerful one and offers a multitude of tools to approximate the solution of a broad range of problems across the natural sciences. First, these physical problems are interpreted into the language of mathematics, then we use computer procedures (or *algorithms*) to determine a numerical answer. This course will teach how to examine and analyze these numerical approximations and interpret them in the context of the original problem.

Throughout the course, we will revisit many of the mathematical topics you have seen in the past from the simple, like addition or division, to the more complex, like computing the derivative of a function. However, on a computer, we must keep in mind that the mathematics performed herein is done in a *discrete sense*. Therefore, we no longer deal in so-called "infinite precision," and errors arise in the computations in a variety of ways. So, there is a fundamental principle we follow throughout this course:

> Computers are very powerful and can solve problems we otherwise could not. But, computers are **"dumb"** in that they lack intuition and just follow instructions. It is up to us to be cautious and interpret numerical results to ensure they make sense.

1.1 Sources of error

Once we move to approximating the solution of a given mathematical problem using a computer there are numerous sources of error. Some of the most important of these errors are:

Modeling errors that arise from fundamental assumptions made about the physical problem under consideration. For example, ignoring air resistance to examine projectile motion or neglecting friction for an object sliding on a surface. In this course we do not worry about these modeling errors. However, it is important to keep in mind, if there are such errors in the model of a physical situation, increasing the accuracy of a numerical solution **will not** improve the results.

Observational errors (also referred to as *errors in the input data*) are typically unavoidable. They can come from the limited accuracy of instrument measurements or the representation of a real number in a number system with a fixed number of digits.

Example 1.1 (Observational errors). (1) The radius of an electron is given by $(2.81777 + \varepsilon) \times 10^{-13}$ cm with $|\varepsilon| \leq 0.00011$ margin of error. (2) If we represent the number e in an eight digit number system then $e = 2.718281828459045\ldots \approx 2.7182818$ ⧫

Always remember that these errors cannot be removed, and they will affect the accuracy of any computations in which they are used.

Rounding/chopping errors arise from performing computations using a fixed number of digits.

Example 1.2 (Rounding/chopping error). Again, assume we work in a number system with only eight digits such that $e \doteq 2.7182818$ (the notation $\doteq$ means "is rounded to" in finite arithmetic). Now we compute $10 + e$:

$$10 + e = 12.7182818 \quad \text{is rounded to} \quad 12.718282$$
$$10 + e = 12.7182818 \quad \text{is chopped to} \quad 12.718281$$

Note, we first compute the addition with "more digits" than the number systems support and <u>then</u> we round or chop the result. This is also how a computer will operate (as discussed later). Therefore, the result of the computation remains limited to the eight digits of the number system. ⧫

Truncation errors (we will also refer to these errors as *approximation errors* or *discretization errors* in this course) are the main source of errors that we deal with in computational mathematics. These errors generally occur when we replace a computationally unsolvable problem, like an infinite series expansion, with a "nearby" problem that is computationally tractable.

EXAMPLE 1.3 (TRUNCATION ERROR). If we approximate the exponential function with the first ten terms of its Taylor series expansion

$$e^x = \sum_{k=0}^{\infty} \frac{x^k}{k!} \approx \sum_{k=0}^{9} \frac{x^k}{k!},$$

then the truncation error is given by

$$R_T := \left| e^x - \sum_{k=0}^{9} \frac{x^k}{k!} \right| = \left| \sum_{k=0}^{\infty} \frac{x^k}{k!} - \sum_{k=0}^{9} \frac{x^k}{k!} \right| = \left| \sum_{k=10}^{\infty} \frac{x^k}{k!} \right|$$

Note that in this case the truncation error is present even for infinite precision (that is with any number of digits of accuracy) because it is simply the remainder from Taylor's theorem. ♦

We now have some notion of where errors in approximations come from. Next, we need to answer the questions of: How do we measure the size of the errors? How can these errors possibly compound one another and influence the accuracy of a numerical solution?

EXAMPLE 1.4. As an illustration, take the exact value $c_E = \pi$, the approximate value $c_A \doteq 3.14159$, and say we want to quantify the error between the two. We can take the difference between the two values

$$c_E - c_A = \pi - 3.14159 = 0.00000265358\ldots \approx 3 \times 10^{-6} = 0.3 \times 10^{-5}$$

Alternatively, we can divide the difference by the true value to find

$$\frac{c_E - c_A}{c_E} = \frac{\pi - 3.14159}{\pi} = 0.000000844663\ldots \approx 8.5 \times 10^{-7} = 0.85 \times 10^{-6}$$

Note, we have rounded these values such that they can act as a *bound* on their respective errors. ♦

We specify a general notion of these two error types.

DEFINITION 1.1 (ABSOLUTE AND RELATIVE ERROR). Given the exact value of a quantity c_E and its approximate value c_A we define two common types of error:

Absolute error in c_A is $\Delta c = c_E - c_A$.

Relative error in c_A is $\dfrac{\Delta c}{c_E} = \dfrac{c_E - c_A}{c_E}$ provided $c_E \neq 0$.

◀

We typically only have a bound on the error of an approximation. However, this is still useful as it provides practical information such as how "close" the approximation gets as well as providing an upper and lower bound.

EXAMPLE 1.5. Continuing from the previous example we have the bounds

$$|\Delta c| \leq 3 \times 10^{-6}, \quad \text{and} \quad \left| \frac{\Delta c}{c_E} \right| \leq 8.5 \times 10^{-7}$$

on the absolute and relative errors, respectively. From this we have several *equivalent* statements that can be made about the errors. For the absolute error of the approximate value of $\pi \doteq 3.14159$:

- $c_A = 3.14159, \quad |\Delta c| \leq 3 \times 10^{-6}$
- $c_E = 3.14159 \pm 3 \times 10^{-6}$
- $3.141587 \leq c_E \leq 3.141593$

The relative error is often discussed in terms of percentages. In this case the relative error is, at most, $8.5 \times 10^{-5}\%$. ♦

There are two mechanisms, previous mentioned, that reduce the number of digits in finite arithmetic (i.e. numerical computation): *rounding* and *chopping*. We now clarify a systematic approach to these two processes.

Definition 1.2 (Rounding and chopping). Let η be the leading digit of a number that corresponds to positions *to the right* of the nth decimal. For example, in the number 4.9283η the value of η is to the right of the 4th decimal place.

Rounding: If $\eta < 0.5 \times 10^{-n}$, then the nth decimal place is left *unchanged*. If $\eta > 0.5 \times 10^{-n}$, then the nth decimal place is *increased by one*. In the case where $\eta = 0.5 \times 10^{-n}$, the nth decimal is increased by one *if it is odd* and left unchanged *if it is even*. This procedure is referred to rounding-to-even.

Chopping: Is much simpler and all decimals beyond the nth place *are ignored*.

◄

Example 1.6. For the following we will round the values to three decimal places:

$$\begin{aligned} 7.92492312 &\text{ is rounded to } 7.925 \\ 0.547234 &\text{ is rounded to } 0.547 \\ 0.322577264 &\text{ is rounded to } 0.322 \\ 0.927521 &\text{ is rounded to } 0.928 \end{aligned}$$

Chopping to three decimal places gives:

$$\begin{aligned} 7.92492312 &\text{ is chopped to } 7.924 \\ 0.547234 &\text{ is chopped to } 0.547 \\ 0.322577264 &\text{ is chopped to } 0.322 \\ 0.927521 &\text{ is chopped to } 0.927 \end{aligned}$$

♦

With some basic knowledge of how mathematical operations are performed in a finite number system we require a systematic way to interpret errors and examine "how good" an approximation might be.

Definition 1.3 (Correct decimals and significant digits). If the absolute error of a quantity $|c_E - c_A| \leq 0.5 \times 10^{-t}$, then the approximate value c_A has t *correct decimals*. Furthermore, when an approximate value has $t > 0$ correct decimals, the digits in the positions $\geq 10^{-t}$ are *significant digits*. Note, that leading zeros **do not** count as significant digits and only indicate the position of the decimal. ◄

Example 1.7. From this definition we examine the correct decimals and significant digits for several approximations. We also write them in different forms:

$$x_E = 1.25, \quad x_A = 1.24 \quad \text{and} \quad Z = 84.278 \pm 0.6 \times 10^{-3} \quad \text{and} \quad y = 0.09352 \pm 0.5 \times 10^{-5}.$$

For the first quantities we compute the absolute error and the accuracy to be

$$|x_E - x_A| = 0.01 = 0.1 \times 10^{-1} \qquad \text{Correct decimals: 1} \qquad \text{Significant digits: 2}$$

Next for the value Z we have

$$Z = 84.278 \pm 0.6 \times 10^{-3} \quad \Rightarrow \quad |\Delta Z| = 0.6 \times 10^{-3} \leq 0.5 \times 10^{-2} \quad \text{Correct decimals: 2} \qquad \text{Significant digits: 4}$$

Finally, we examine the accuracy of the final quantity of interest

$$y = 0.09352 \pm 0.5 \times 10^{-5} \quad \Rightarrow \quad |\Delta y| = 0.5 \times 10^{-5} \quad \text{Correct decimals: 5} \qquad \text{Significant digits: 4}$$

Notice here that the leading zero *does not* get counted as a significant digit. These examples serve to illustrate the subtleties of working with the definition of correct decimals and significant digits. Also highlighting the care that one must take in the computation and comparisons. ⧫

Another measure of accuracy, that involves less rounding, uses the relative error of a computation. It is a dimensionless value; that is, it has no assigned units and is a "pure" number. This is referred to as the number of *accurate digits*.

Definition 1.4 (Accurate digits). We say that an approximate value has

$$-\log_{10}\left|\frac{c_E - c_A}{c_E}\right|$$

accurate digits which is chopped to the nearest integer. We do so because discussing fractional digits of accuracy is not meaningful. ◀

Example 1.8. We return to the example where $\pi \doteq 3.14159$ which had the relative error $\frac{c_E - c_A}{c_E} = 8.5 \times 10^{-7}$. Then we can compute

$$-\log_{10}\left|\frac{c_E - c_A}{c_E}\right| \approx 6.073316$$

which says that the approximation $c_A = 3.14159$ has six digits of accuracy for π. ⧫

Remark 1.1. It is important to note that the number of significant digits and the number of accurate digits may or may not differ. In the case of our π approximate value above they happen to be the same. ⋈

In practice, if we use approximate values to compute other values the errors will, unsurprisingly, *propagate* and give rise to errors in the result. What impact can this have on accuracy?

Example 1.9. Suppose we are evaluating on a calculator that can represent up to six digits that uses rounding-to-even arithmetic. We wish to evaluate the function

$$f(x) = x(\sqrt{1+x} - \sqrt{x}).$$

Below we collect the this function evaluation for increasing values of x comparing the numbers produced by the 6-digit calculator and the true values of $f(x)$ chopped to six digits:

x	Approximate $f(x)$	True $f(x)$
1	.414210	.414214
10	1.54340	1.54347
100	4.99000	4.98756
1000	15.8000	15.8074
10000	50.0000	49.9987
100000	100.000	158.113

What happened?! For smaller values of x an error occurs but they are within reason of the previous examples. However, as x grows the errors become significantly worse to a point where we lose **all** correct decimals. ⧫

It turns out that we must be cautious when performing arithmetic operations (like subtraction) as well as functions evaluations (like the square root) as their combination of errors may compound. To fully understand the mechanism behind losing accuracy in finite arithmetic we need to investigate further how numbers are represented on a computer.

1.2 Finite precision

In everyday life we represent numbers in a base-10 system. However, a computer or calculator often uses a different base for its number system, typically base-2 (binary) or base-16 (hexadecimal). An example of a base-10 number representation is

$$5732.093 = 5 \times 10^3 + 7 \times 10^2 + 3 \times 10^1 + 2 \times 10^0 + 0 \times 10^{-1} + 9 \times 10^{-2} + 3 \times 10^{-3}.$$

For this number we required four digits before the decimal point and three after. Obviously, this will not always be the case for an arbitrary real number and we may require more/less digits before and/or after the decimal point. So, instead of fixing the decimal to represent a given amount of digits before and after the decimal (for example 4.3 above) we should allow the decimal point to ***float***. To do so, numbers are represented in an exponential format (like the scientific notation we have been using).

Modern computers use this *floating point representation* to approximate real numbers. It is built upon a particular base and range of resolvable exponential values.

Definition 1.5 (Floating point number system). A floating point number system $\mathcal{F} \subset \mathbb{R}$ is characterized by four integers:

- The base (or radix) β
- The precision t
- The exponent bounds $L \leq e \leq U$

A real number represented in $\mathcal{F}$ takes the form

$$y = m \times \beta^e$$

where

$$m = \pm d_0.d_1 d_2 \dots d_t, \quad 0 \leq d_i \leq \beta - 1 \text{ for } i = 1, 2, \dots, t, \quad 1 \leq |m| \leq \beta$$

where m is the *mantissa* and the exponent satisfies $L \leq e \leq U$. ◀

Every floating point number is a real number, but not vice versa! Essentially, the floating point system defines a mapping from the real numbers, i.e., for $x \in \mathbb{R}$ we have $\texttt{fl}(x) : \mathbb{R} \to \mathcal{F}$. Many such mappings are possible, but not all are robust or reliable. What can we say about the accuracy of a given floating point number system?

Example 1.10. Suppose we can represent a number *exactly* as $x = (d_0.d_1 d_2 \dots d_t d_{t+1} d_{t+2} \dots) \times \beta^e$ and we have its corresponding floating point value $\texttt{fl}(x) = (d_0.d_1 d_2 \dots d_t) \times \beta^e$. Then the relative error of the mapping to $\mathcal{F}$ is

$$\frac{|x - \texttt{fl}(x)|}{|x|} = \frac{(0.00 \dots 0 d_{t+1} d_{t+2} \dots) \times \beta^e}{(d_0.d_1 d_2 \dots d_t d_{t+1} d_{t+2} \dots) \times \beta^e} \leq \begin{cases} \mu = \beta^{-t} & \text{chopping} \\ \mu = \frac{1}{2}\beta^{-t} & \text{rounding-to-even} \end{cases}$$

where μ is referred to as the *unit roundoff*. Alternatively, we can write the relative error as

$$\texttt{fl}(x) = (1 + \delta)x, \quad |\delta| \leq \mu,$$

which reveals more clearly that the mapping from infinite precision into finite precision *perturbs* the data by an amount δ. ⧫

Remark 1.2. It is important to note that the relative error of the floating point representation is *independent* of the magnitude of x. If the result of a computation is a floating point number with $e > U$, then the value is outside the range of the mapping $\texttt{fl}$ that takes $\mathbb{R}$ into $\mathcal{F}$. In practice, when this occurs on a computer it will issue an error. This is known as *overflow*. Similarly, if a value has an exponent $e < L$ it will cause an *underflow* error to occur. ⋈

DEFINITION 1.6 (IEEE STANDARD FLOATS). The two most common floating point number systems with rounding-to-even are:

$$\underline{\text{Single precision}}\text{: } \beta = 2, \quad t = 23, \quad L = -126, \quad U = 127, \quad \mu = 2^{-24} \approx 5.96 \times 10^{-8}$$

$$\underline{\text{Double precision}}\text{: } \beta = 2, \quad t = 52, \quad L = -1022, \quad U = 1023, \quad \mu = 2^{-53} \approx 1.11 \times 10^{-16}$$

So, single precision has the possibility of, approximately, 7 digits of accuracy and double precision has 15 digits of accuracy. Note, in *most* programming languages, computations **default** to using double precision floating point numbers. ◀

We return to the question errors in finite arithmetic. In particular, how do they behave and how can they accumulate? We have already noted that mapping an exact value into a floating point number systems perturbs the value on the order of unit roundoff. Can we explain what happened in our previous example where we lost a great deal of accuracy?

To explore this error propagation we must be systematic. So, suppose we want to solve a problem specified through some data y to yield a solution s (note y and s could be a collection of real scalars). We represent this problem in a general form

$$s = g(y).$$

For example, this could be a function evaluation or finding the root of a polynomial. It is just a general problem structure. Now, the desired solution (from infinite precision arithmetic) would be $s = g(y)$. But, on the computer we now know that we store $\texttt{fl}(y) = (1+\delta)y$ such that there is small roundoff error for the input data. We can then state the exact solution of the perturbed problem to be

$$\tilde{s} = g(\texttt{fl}(y)).$$

How much does this perturbation (or small change) in the input data $\texttt{fl}(y)$ alter the solution when we evaluate the operation g? We can quantify this by computing the so-called **forward errors** as

$$|\Delta s| = |s - \tilde{s}| \qquad \text{and} \qquad \frac{|\Delta s|}{|s|} = \frac{|s - \tilde{s}|}{|s|}$$

which result from uncertainty in the input data y because of the floating point representation. However, this is a bit optimistic because it assumes we have knowledge of the exact solution. Can we develop a bound on how these errors behave? This leads to the important concept of *conditioning* and numerical stability.

DEFINITION 1.7 (CONDITION NUMBER). Assume the problem of interest $g(y)$ has a Taylor expansion around the point of interest y then

$$g(\texttt{fl}(y)) = g(y + \delta y) = g(y) + \delta y g'(y) + \mathcal{O}((\delta y)^2) \quad \Rightarrow \quad g(\texttt{fl}(y)) - g(y) \approx \delta y g'(y),$$

such that

$$|g(y) - g(\texttt{fl}(y))| \leq |y g'(y)||\delta|.$$

Then we have for the relative error

$$\frac{|g(y) - g(\texttt{fl}(y))|}{|g(y)|} \leq \frac{|y g'(y)|}{|g(y)|}|\delta| := \kappa|\delta|,$$

where κ is the condition number. ◀

The condition number κ bounds the relative error by an amplification factor on the magnitude of the perturbation done to the input data. It serves to give an idea of how much the uncertainty of the input data specifying the problem can change the solution. Problems with large condition numbers are *ill-conditioned* and can be prone to numerical instability or loss in accuracy. That is, large condition numbers tell us when errors *cannot* be expected to remain comparable to unit roundoff errors.

This concept of stability, illustrated in the Figure 1, is decomposed into two parts: *Forward stability* is related to the approximate (or computed) solution $\tilde{s} = \texttt{fl}(g(y))$ being "close" to the exact solution $s(y)$. Thus, it is concerned with what the computation produces as a result. We measure it with the forwards errors described above. *Backward stability* is related to the value $\tilde{s}$ and whether it is the exact solution of a perturbed problem. Thus, it is concerned with the problem itself and how prone it is to small changes in the problem data. Examining backward stability is typically more difficult than forward stability because we must find a value $\tilde{y}$ in the input data such that

$$g(\tilde{y}) = \tilde{s} = \texttt{fl}(g(y)).$$

Then we can measure the **backward error**

$$|\Delta y| = |y - \tilde{y}| \quad \text{and} \quad \frac{|\Delta y|}{|y|} = \frac{|y - \tilde{y}|}{|y|}.$$

In essense, forward stability asks the question "How close to the true answer is the numerical answer?"; backward stability asks the inverse of this question "How close to the true question is the question you answered numerically?"

Remark 1.3. In this course we are most concerned with forward errors. Backward errors are important, but because they inherenetly has this inverse nature to the problem they are left as a topic for a more advanced numerics course. ⋈

This formalism clarifies where the errors we discussed earlier arise in computations. We see that *observational errors (or errors in the input data)* come from representing values in a floating point number system. *Rounding errors* can appear in input data as well as solution data. Finally, *truncation errors* influence that solution data because they are fundamentally contained inside a discrete formula or algorithm.

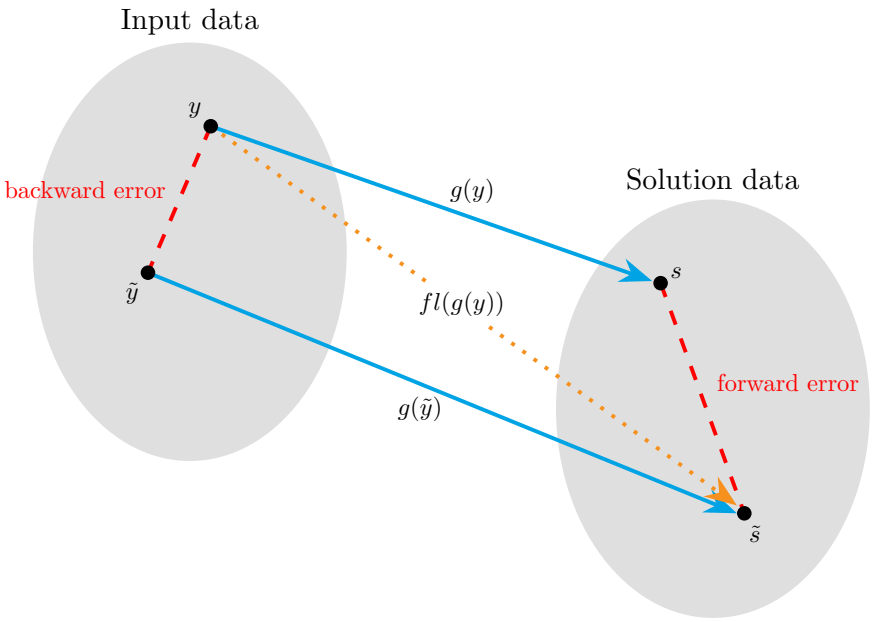


Figure 1: Illustration of the forward and backward errors. Solid lines represent the exact solution of a problem and the dotted line is the approximate solution produced by a finite formula evaluation or algorithm.

EXAMPLE 1.11. Compute the condition numbers for the function evaluation problem with $g_1(y) = \sqrt{y}$ and $g_2(y) = e^y$. It is straightforward to compute that

$$\kappa_1 = \frac{|yg'(y)|}{|g(y)|} = \frac{|y\frac{1}{2\sqrt{y}}|}{|\sqrt{y}|} = \frac{\sqrt{y}}{2\sqrt{y}} = \frac{1}{2},$$
$$\kappa_2 = \frac{|yg'(y)|}{|g(y)|} = \frac{|ye^y|}{|e^y|} = |y|.$$

So, the conditioning of the square root function is very good because it is independent of the size of y. However, the condition of the exponential function grows linearly in y. This makes sense because taking very large numbers in the exponent can easily lead to overflow. ⧫

Now we see that the condition of evaluating the square root is well-conditioned. So, again, we ask the question: What went wrong when we tried to compute $f(x) = x(\sqrt{1+x} - \sqrt{x})$?

EXAMPLE 1.12. Consider the sum of two numbers where

$$x = 151.72899, \quad y = -151.71422, \quad x + y = 0.01477$$

and

$$\tilde{x} = 151.73, \quad \tilde{y} = -151.71, \quad \tilde{x} + \tilde{y} = 0.02$$

The respective sums do not agree to any number of digits! Further, the relative error of the two sums is $|0.01477 - 0.02|/0.01477 = 0.35$ ⧫

The sum of two numbers when the magnitudes are close but the signs are opposite is ill-conditioned. This numerical issue is known as ***cancellation***. In general, cancellation error occurs whenever the result of addition or subtraction is much smaller in magnitude than the operands.

This highlights an important difference when we work in finite arithmetic. If $\odot$ denotes any of the arithmetic operations $+, -, \times, /$ then we know that $\odot\colon \mathbb{R} \to \mathbb{R}$. Also, we know that the floating point numbers are a subset of the real numbers. So, these arithmetic operations are applicable **but** $\odot\colon \mathcal{F} \to \mathbb{R}$.

Remark 1.4. Infinite arithmetic operations applied to floating point numbers are not closed! ⋈

We must define floating point arithmetic to be closed to ensure that applying operations to a floating point number will result in a floating point number. This also ensures that the error estimates discussed earlier remain valid for floating point arithmetic. This is done in the previously mentioned IEEE standards.

DEFINITION 1.8. Let $\odot$ denote any of the arithmetic operations $+, -, \times, /$ take $x, y \in \mathcal{F}$, then

$$\texttt{fl}(x \odot y) = (x \odot y)(1 + \delta), \quad |\delta| \leq \mu,$$

where μ is unit roundoff error. ◀

Remark 1.5. Even though we have error estimates and that floating point arithmetic is closed there are *consequences when we restriction to finite precision.* Many of the usual mathematical laws are no longer valid in finite precision. For instance

$$\texttt{fl}(\texttt{fl}(a+b)+c) \neq \texttt{fl}(a+\texttt{fl}(b+c)),$$

and we lose the associative law of addition. ⋈

Finally, we have diagnosed the problem and loss of accuracy we encountered in the previous example! Let us revisit it and determine how to improve the computation.

EXAMPLE 1.13. We want to evaluate the function $f(x) = x(\sqrt{1+x} - \sqrt{x})$ for large values like $x = 10000$. We see that the issue is cancellation because for large values $\sqrt{1+x} \approx \sqrt{x}$. In order to fix this we rewrite the expression $f(x)$ to remove the subtraction by multiplying by the conjugate:

$$f(x) = x(\sqrt{1+x} - \sqrt{x})\frac{\sqrt{1+x} + \sqrt{x}}{\sqrt{1+x} + \sqrt{x}} = \frac{x(1+x-x)}{\sqrt{1+x} + \sqrt{x}} = \frac{x}{\sqrt{1+x} + \sqrt{x}}$$

We know that the conditioning of the square root function is good and the addition of two numbers is also well conditioned when they have the same sign. Now we can compute in 6-digit arithmetic rounding-to-even and find the is value 49.9988, which is much closer to the true value of (chopped to six digits) 49.9987 ♦

To remove cancellation error we had to rewrite the problem and avoid subtracting numbers which are large and close in value. In general, removing cancellation error may be fixable with a clever algebraic manipulation of a problem or may show ill-conditioning and is not fixable. Below we provide two more examples of removing cancellation error from a finite computation.

EXAMPLE 1.14. Use a truncated Taylor series to evaluate $e^{-5.5}$. We know that we have the complete Taylor expansion

$$e^{-x} = \sum_{k=0}^{\infty} \frac{(-1)^k}{k!} x^k = 1 - x + \frac{x^2}{2} - \frac{x^3}{3!} + \frac{x^4}{4!} + \dots$$

This is an alternating series, so in finite precision there will cancellation errors as we compute this sum. Evaluating the series we have $e^{-5.5} = 1.0000 - 5.5000 + 15.125 - 27.729 + 38.128 + \dots$

After some index k the terms will stop affecting the result because the alternating large magnitude terms in the "middle" use most of the finite digit representation. The exact solution is $e^{-5.5} = 0.004086771438\dots$ If we use single precision and truncate the series at $k = 28$ we have

$$e^{-5.5} \approx \sum_{k=0}^{28} \frac{(-1)^k}{k!} x^k = 0.0040874378$$

We rewrite the original expansion to remove its alternating nature

$$e^{-x} = \frac{1}{e^x} = \frac{1}{1 + x + \frac{x^2}{2} + \frac{x^3}{3!} + \dots}$$

Now all terms have the same sign and there is no longer cancellation and if we compute (again in single precision) the truncated series with $k = 21$ we have

$$e^{-5.5} = \frac{1}{e^{5.5}} \approx \frac{1}{\sum_{k=0}^{21} \frac{x^k}{k!}} = 0.004086771$$

♦

EXAMPLE 1.15. We know the quadratic formula to compute the two roots of the polynomial $p(x) = ax^2 + bx + c$ where

$$x_{1,2} = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

There will be cancellation in one of the roots depending on the sign of b. This is particularly relevant if $b^2 \gg |4ac| \to |b| \approx \sqrt{b^2 - 4ac}$. Therefore, we can think of one root as well-behaved and the other as poorly behaved. We rewrite the quadratic formula to compute the roots by

$$x_1 = \frac{-(b + \text{sign}(b)\sqrt{b^2 - 4ac})}{2a}, \quad \text{sign}(b) = \begin{cases} 1 & b \geq 0 \\ -1 & b < 0 \end{cases}$$

Now that we have one root we use it to get the other in a more numerically stable fashion. From the original root formulas we find

$$x_1 x_2 = \frac{c}{a} \quad \Rightarrow \quad x_2 = \frac{c}{a x_1}.$$

♦

Note Set 2: Nonlinear equations and fixed point methods

We now know how a computer represents floating point numbers, how it manipulates them, and understand where errors can occur and/or accumulate. So let's start solving some problems! For this we move onto nonlinear algebraic problems. We investigate the solution of scalar functions with a single variable. The strategy we use here is one of the core tenets of this course as well as scientific computing as a whole: Replace a problem you **cannot** solve with an approximate problem that you *can*.

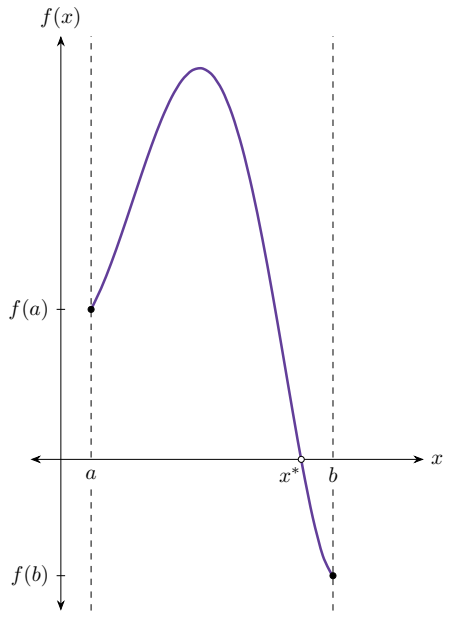
2.1 Root finding problem

A problem that frequently arises in scientific work is to determine the root(s) of an equation

$$f(x) = 0.$$

Such functions could be polynomials, e.g. $f(x) = x^5 + 3x^2 - 6x + 2$, or a transendental function, e.g. $f(x) = \cos(x) + e^x$. In rare cases, the roots of an equation are known explicitly (like the quadratic formula). However, in practice, the best we can hope to obtain are approximate solutions created through a sequence of steps (such as an iterative process) until the solution x^* is deemed "good enough" and $|f(x^*)|$ is small (in some measure).

For this problem we consider a real valued continuous function $f(x)$ on the interval $[a, b]$. Further, we take the sign of the function to switch inside the interval, i.e. $f(a) < 0$, $f(b) > 0$ or $f(a) > 0$, $f(b) < 0$. Because of this swap in sign and the continuity of the function an intermediate value in (a, b) is guaranteed to be a root, x^*. This result is sometimes referred to as *Bolzano's theorem*. A sketch of this situation is given below at the right.



So we know a root exists, but we want to approximate its value. For simplicity, we assume that there is only one root in the interval (a, b). Then a simple strategy to determine the root of the function $f(x)$ is to repeatedly halve the interval $[a, b]$ to recursively locate the root.

Definition 2.1 (Bisection method). Initially, take m to be the midpoint of $[a, b]$

$$m = a + \frac{b-a}{2} = \frac{a+b}{2}.$$

If $f(m) = 0$ we are done. Otherwise, we compare the signs of $f(a)$ and $f(m)$ to shrink the interval and then reapply this process. We check

- If $f(a)$ and $f(m)$ have the same sign, $x^* \in (m, b)$ so $a = m$ and $b = b$.
- If $f(a)$ and $f(m)$ have opposite signs, $x^* \in (a, m)$ so $a = a$ and $b = m$.

◀

However, we see that the check for the root $f(x^*) = 0$ is quite strick given what we know about floating point arithmetic, roundoff errors and error propagation. As such, we "soften" the requirement from a so-called hard zero to be within a small tolerance, ε, near zero:

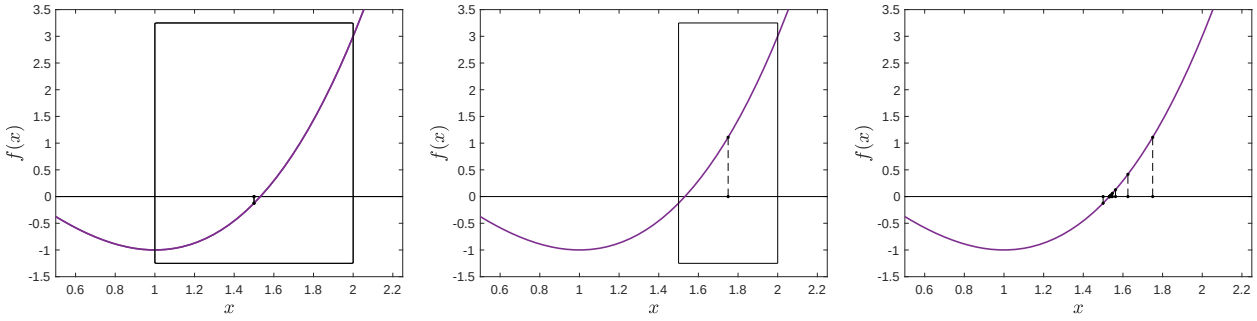
$$|f(x^*)| \leq \varepsilon.$$

Basically, the above definition is a set of instructions, or *algorithm*, to determine the root of a function. In scientific computing algorithms are typically presented in **pseudocode**, which is an informal interpretation of a set of instructions into simplified programming language:

Algorithm 2.1: *Bisection method*: Simple strategy to compute the root $f(x) = 0$

Procedure Bisection Method
Input: a, b, $f(x)$, `tol //e.g. tol` $= 10^{-6}$
$m \leftarrow \frac{a+b}{2}$
while $|f(m)| >$ `tol` **do**
  if $f(a) \cdot f(m) < 0$ **then**
    $b \leftarrow m$
  else
    $a \leftarrow m$
  $m \leftarrow \frac{a+b}{2}$
$x^* \leftarrow m$
Output: x^*
End Procedure Bisection Method

EXAMPLE 2.1. Use the bisection method to compute the root of $f(x) = x^3 - 3x + 1$ in $[1, 2]$ with $\varepsilon = 10^{-11}$. We illustrate the first few iterations and see that the root looks to be found in the so-called "eyeball" norm.



However, to achieve the tolerance of ε for the root takes 38 iterations to find $x^* = 1.532088886237034$ ♦

We see that the uncertainty associated with the root x^* in the bisection method has the size $b-a$ initially, then it is $\frac{b-a}{2}$ in the first subinterval, $\frac{b-a}{2^2}$ in the second subinterval, and so on. Therefore, the uncertainty is halved in each step.

DEFINITION 2.2 (BISECTION ERROR). The bisection method has the error

$$\epsilon_k = |x^* - x_k| \leq \frac{|b-a|}{2^k}, \quad k \geq 0,$$

in its kth iteration. ◄

The bisection method is a root finding technique that repeatedly halves the interval and *brackets* the value of the unknown root x^*. This combined with the fact that the error $|x^* - x_k| \to 0$ as $k \to \infty$ guarantees that the bisection method *will always work*. But how fast does x_k approach x^*? This introduces the concept of **convergence** for a numerical procedure. If we compare successive errors of the bisection method we see

$$\frac{\epsilon_{k+1}}{\epsilon_k} = \frac{|x^* - x_{k+1}|}{|x^* - x_k|} = \frac{\frac{|b-a|}{2^{k+1}}}{\frac{|b-a|}{2^k}} = \frac{1}{2}$$

DEFINITION 2.3 (BISECTION CONVERGENCE RATE). The bisection method converges at a *linear* rate. ◀

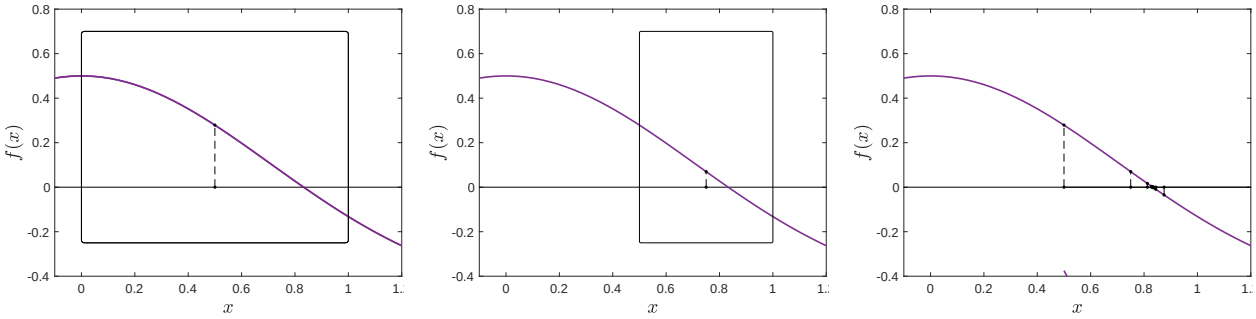
The error bound is also useful as we can use it to estimate the number of iterations needed by the bisection method to reach a certain tolerance level:

$$\frac{b-a}{2^k} \leq \varepsilon \quad \Rightarrow \quad 2^k \geq \frac{b-a}{\varepsilon}$$

where we used that $0 < \varepsilon < 1$.

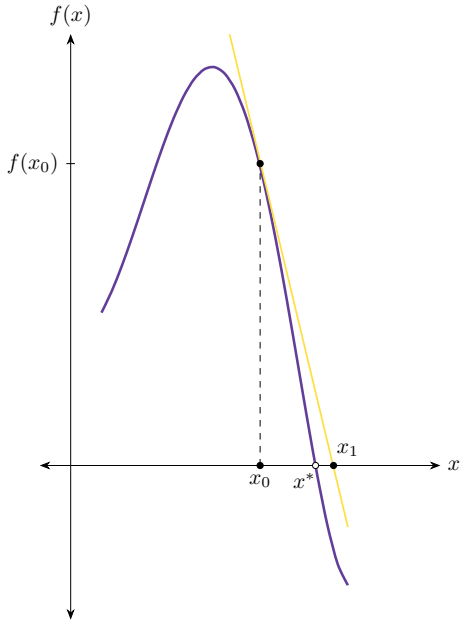
DEFINITION 2.4 (BISECTION ITERATION ESTIMATE). The number of iterations needed for the bisection method to reach a tolerance of ε is approximately $k = \log_2\left(\frac{b-a}{\varepsilon}\right)$. ◀

EXAMPLE 2.2. Use the bisection method to compute the root of $f(x) = e^{-x^2} - \frac{1}{2}$ in $[0, 1]$ with $\varepsilon = 10^{-11}$. We illustrate the first few iterations and see that the root looks to be found in the so-called "eyeball" norm.



We can now estimate that it will take $k \approx 36.54$ iterations for the error to drop below the set tolerance. For this example, we reach the tolerance in 36 iterations to find $x^* = 0.8325546111591$. Here the estimated number of iterations was quite accurate. However, recall from the previous example that slightly more iterations than predicted were necessary to achieve the same error tolerance. ♦

In general, the bisection method is guaranteed to converge, but it is ***slow***. Its simplicity is the main cause for such a sluggish convergence rate. The method does not take any information about the function $f(x)$ into account beyond its sign at the interval bounds. More sophisticated (and faster) root finding techniques can be found if we examine the function $f(x)$ as well as its properties and behavior.



A natural first idea comes from Calculus to approximate a function locally with its tangent line at a given point, say x_0:

$$y - f(x_0) = f'(x_0)(x - x_0).$$

We take the next approximation of the root, x_1, to be the intersection between the tangent line and the x-axis as shown at the left. This strategy gives an alternative root finding technique:

DEFINITION 2.5 (NEWTON-RAPHSON METHOD). Given a point x_k the next approximation to the root is

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}$$

provided $f'(x_k) \neq 0$. ◀

The Newton-Raphson method can also be derived by taking a Taylor expansion of $f(x)$ near the root x^* and truncating at the first two terms. Such a Taylor expansion is only valid in a region near the root x^*. So, in contrast to the bisection, the Newton-Raphson method requires a good *initial guess* x_0 to begin the iteration!

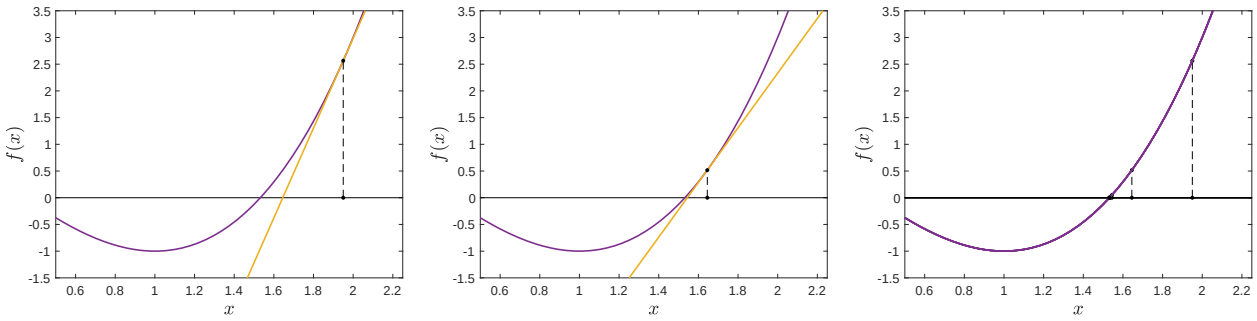
We provide pseudocode for the Newton-Raphson root finding method. In this pseudocode we introduce an alternative *stopping condition* (sometimes called a *kill condition*) for the iteration. This kill condition uses a relative comparison between the current and previous values of the approximate root. We also set a parameter to stop the algorithm after a maximum number of iterations. This is important to prevent an infinite loop if the algorithm fails to converge for a given initial guess x_0.

Algorithm 2.2: *Newton-Raphson method*: Strategy to compute the root $f(x) = 0$ using the function and its derivative

Procedure Newton-Raphson Method
Input: x_0, $f(x)$ $f'(x)$, `tol // e.g. tol` $= 10^{-6}$

$max_{its} \leftarrow 50$ `// maximum number of iterations allowed`
$x_{old} \leftarrow x_0$
for $k = 1$ **to** max_{its} **do**
  $x_{new} \leftarrow x_{old} - \frac{f(x_{old})}{f'(x_{old})}$
  if $\frac{|x_{new} - x_{old}|}{|x_{new}|} <$ `tol` **then**
    break
  $x_{old} \leftarrow x_{new}$
Output: x_{new}
End Procedure Newton-Raphson Method

EXAMPLE 2.3. Use the Newton-Raphson method to approximate the root of $f(x) = x^3 - 3x + 1$ in $[1, 2]$ with $\varepsilon = 10^{-11}$. We use the initial guess $x_0 = 1.9$ and illustrate the first few iterations.



To achieve the tolerance of ε for the root takes 6 iterations to find $x_{new} = 1.532088886237956$ ♦

We see that the method approaches the root much faster than the bisection method. To quantify this we recall the error for the kth approximate root is $\epsilon_k = x^* - x_k$. Then, from the form of the Newton-Raphson iteration, we develop an equation for the next error value

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)} \quad \Rightarrow \quad \epsilon_{k+1} = \epsilon_k + \frac{f(x^* - \epsilon_k)}{f'(x^* - \epsilon_k)}.$$

If we assume enough continuity then we perform Taylor expansions of the function $f(x)$ and its first derivative about the root x^* to find

$$\epsilon_{k+1} = \epsilon_k + \frac{f(x^*) - \epsilon_k f'(x^*) + \frac{\epsilon_k^2}{2} f''(x^*) + \mathcal{O}(\epsilon_k^3)}{f'(x^*) - \epsilon_k f''(x^*) + \mathcal{O}(\epsilon_k^2)} = \epsilon_k + \frac{0 - \epsilon_k f'(x^*) + \frac{\epsilon_k^2}{2} f''(x^*) + \mathcal{O}(\epsilon_k^3)}{f'(x^*) - \epsilon_k f''(x^*) + \mathcal{O}(\epsilon_k^2)},$$

where we used that $f(x^*) = 0$. Provided the first derivative is nonzero at x_k the error of the $k + 1$ iteration

becomes

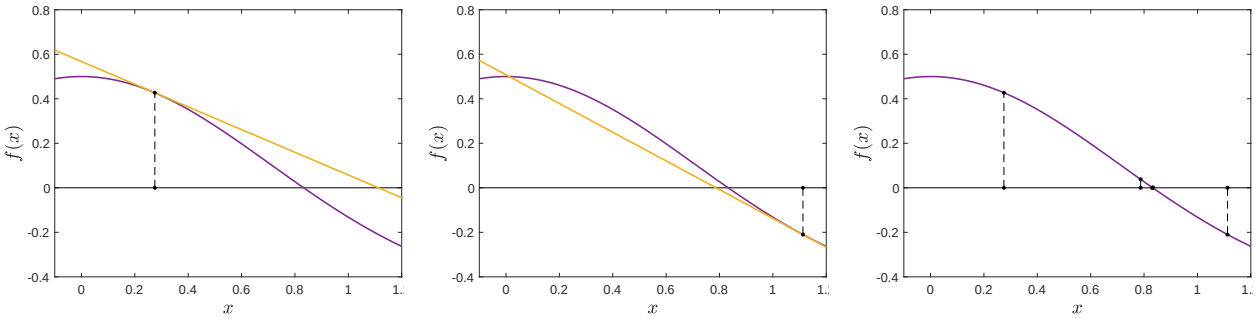
$$\epsilon_{k+1} = \epsilon_k - \epsilon_k \left(\frac{1 - \frac{\epsilon_k}{2} \frac{f''(x^*)}{f'(x^*)} + \mathcal{O}(\epsilon_k^2)}{1 - \epsilon_k \frac{f''(x^*)}{f'(x^*)} + \mathcal{O}(\epsilon_k^2)} \right) = \epsilon_k - \epsilon_k \left(1 + \frac{\epsilon_k}{2} \frac{f''(x^*)}{f'(x^*)} + \mathcal{O}(\epsilon_k^2) \right) = -\frac{\epsilon_k^2}{2} \frac{f''(x^*)}{f'(x^*)} + \mathcal{O}(\epsilon_k^3)$$

through polynomial long devision. Finally, we obtain a statement regarding the ratio of two subsequent errors truncated to its leading order term

$$\frac{\epsilon_{k+1}}{\epsilon_k^2} = -\frac{f''(x^*)}{2f'(x^*)}.$$

DEFINITION 2.6 (NEWTON-RAPHSON CONVERGENCE RATE). If an initial guess x_0 is near the root of the function $f(x)$ in an interval around the root $[x^* - r, x^* + r]$, $f'(x) \neq 0$, and $f''(x)$ is continuous in said interval, then the rate of convergence for the Newton-Raphson method is *quadratic*. ◀

EXAMPLE 2.4. Use the Newton-Raphson method to approximate the root of $f(x) = e^{-x^2} - \frac{1}{2}$ in $[0, 1]$ with $\varepsilon = 10^{-11}$. We use the initial guess $x_0 = 0.275$ and illustrate the first few iterations.



To achieve the tolerance of ε for the root takes 6 iterations to find $x_{new} = 0.832554611157698$. However, if we select the initial condition $x_0 = 0.1$ the method fails to converge and $x_k \to \infty$. ♦

We see that when we have a good initial guess x_0 and the assumptions on $f(x)$ and its derivative hold in the interval of interest, the Newton-Raphson method performs *very well*. In practice, it could be difficult (or even impossible) to know the derivative of the function $f(x)$. We can circumvent the need to know this derivative by replacing it with an approximation.

DEFINITION 2.7 (SECANT METHOD). Replacing the derivative term in the Newton-Raphson method with a difference approximation we find the *secant method*:

$$x_{k+1} = x_k - \frac{f(x_k)}{\frac{f(x_k) - f(x_{k-1})}{x_k - x_{k-1}}} = x_k - \frac{f(x_k)(x_k - x_{k-1})}{f(x_k) - f(x_{k-1})},$$

where we now require the current guess x_k as well as the previous guess x_{k-1} to advance the iteration. ◀

Note, the secant method *still* requires a good initial guess of the root to begin the iteration. Otherwise it is possible for the iteration to diverge, just like with the original Newton-Raphson method. The secant method contains more information about the function than the bisection method, but less information than the Newton-Raphson method. So, what about the rate of convergence for the secant method?

DEFINITION 2.8 (SECANT METHOD CONVERGENCE RATE). With a suitable initial guess the secant method exhibits *superlinear* convergence, where the actual convergence rate is ≈ 1.618. (Demonstrating this superlinear convergence rate is quite technical and can be found elsewhere.) ◀

These three methods (bisection, Newton-Raphson, and secant) to determine the root of an equation are far from exhaustive. Many techniques have been developed over the years, but, in the end, *we can discuss this problem more generally and design a framework to solve these root finding problems.*

2.2 Fixed point iterative method

We can compactly write the iteration to find the root of a function as

$$x_{k+1} = \varphi(x_k).$$

For example, the Newton-Raphson method takes $\varphi(x_k) = x_k - \frac{f(x_k)}{f'(x_k)}$. Because x^* is a root of the function $f(x)$ then we see that

$$x^* = \varphi(x^*).$$

So x^* is a *fixed point* of the auxiliary function $\varphi(x)$.

Definition 2.9 (Fixed point iterative method). For a given mapping $\varphi(x) : [a, b] \to \mathbb{R}$ such that $x^* = \varphi(x^*)$ when $f(x^*) = 0$ then

$$x_{k+1} = \varphi(x_k).$$

This is a *fixed point iteration*, which defines a nonlinear recurrence to determine the root of $f(x)$. Note, this still requires an initial guess of the value $x_0 \in [a, b]$ to begin the process. ◀

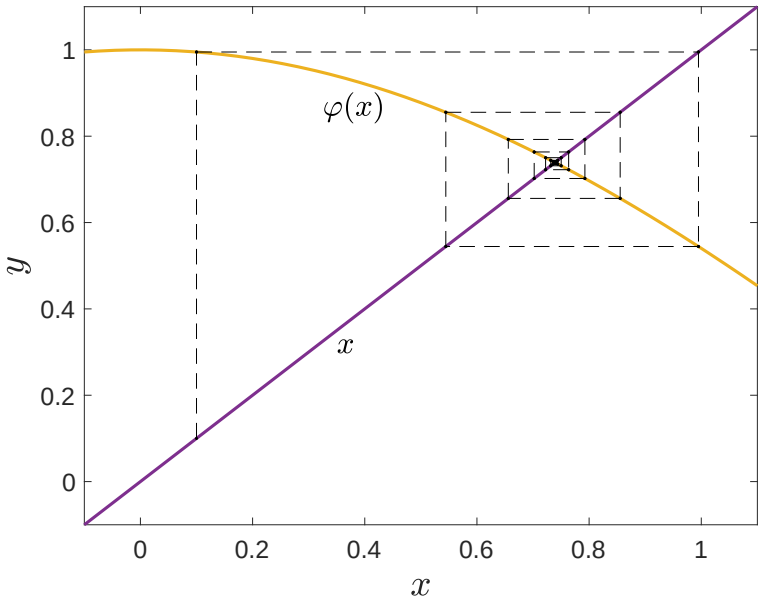
This formalism is incredibly powerful because we can, algebraically, **always** convert a root finding problem into a fixed point problem. However, we have a great deal of freedom in the construction of the mapping function $\varphi(x)$ because as long as it has the form

$$\varphi(x) = x + F(f(x)), \quad F(f(x^*)) = F(0) = 0,$$

for some continuous function F, then x^* is a fixed point.

Definition 2.10 (Picard iteration). The simplest choice for the mapping is $\varphi(x) = x + f(x)$. ◀

Example 2.5. Use a Picard iteration to determine a root of the function $f(x) = \cos(x) - x$ in $[0, 1]$ with $\varepsilon = 10^{-6}$. Use the value $x_0 = 0.1$ as an initial guess. Here the mapping function will be $\varphi(x) = \cos(x)$. We present a list of the iteration values as well as a visualizaition of the iterative process where we see it "falls" onto the fixed point in a spiral fashion. It takes 34 iterations to achieve the desried error tolerance.



k	x_k	$f(x_k)$
0	0.1	0.895004165
1	0.995004165	-0.450504769
2	0.544499395	0.310887310
⋮	⋮	⋮
18	0.738766351	0.000533479
19	0.739299830	-0.000359337
20	0.738940493	0.000242063
21	0.739182556	-0.000163052
22	0.739019504	0.000109836
23	0.739129340	-0.000073986
24	0.739055354	0.000049838
25	0.739105192	-0.000033571
26	0.739071620	0.000022614
27	0.739094235	-0.000015233
28	0.739079001	0.000010261
29	0.739089263	-0.000006912
30	0.739082351	0.000004656
31	0.739087007	-0.000003136
32	0.739083870	0.000002112
33	0.739085983	-0.000001423
34	0.739084560	0.000000958

♦

What about convergence of the fixed point method? Heuristically, from the previous example, we see that the iterative procedure remained in a "box" dictated by the interval of interest. We can devise a sufficient condition on the mapping $\varphi(x)$ to guarantee convergence to a fixed point in a general interval $[a, b]$. First we note the following:

DEFINITION 2.11 (CONTRACTION MAPPING). The continuous function $\varphi : [a, b] \to [a, b]$ is a *contraction mapping* on $[a, b]$ if there exists a constant $0 < L < 1$ such that

$$|\varphi(x) - \varphi(y)| \leq L|x - y|,$$

for all $x, y \in [a, b]$. ◀

This is a special case of the function $\varphi(x)$ satisfying a *Lipschitz condition* where L is called the *Lipschitz coefficient*. It is the fact that $0 < L < 1$ that makes the function $\varphi(x)$ a contraction mapping.

Consider the error of the fixed point iteration

$$x^* - x_{k+1} = \varphi(x^*) - \varphi(x_k).$$

If we apply the mean value theorem (assuming that $\varphi(x)$ is differentiable) then we have for some value ξ_k between x^* and x_k that

$$x^* - x_{k+1} = \varphi'(\xi_k)(x^* - x_k).$$

When it is true that $|\varphi'(\xi_k)| = L < 1$ for some constant L, then $\varphi(x)$ is a contraction mapping. Therefore, we have a bound on the error at the current iteration

$$|x^* - x_{k+1}| \leq L|x^* - x_k| \leq L^2|x^* - x_{k-1}| \leq \cdots \leq L^{k+1}|x^* - x_0|,$$

where $L^{k+1} \to 0$ as $k \to \infty$. Thus, the error is guaranteed to tend towards zero if the first derivative of the mapping function $\varphi(x)$ is bounded between zero and one. Further, we see by comparing successive errors of the fixed point iteration that

$$\frac{|x^* - x_{k+1}|}{|x^* - x_k|} \leq L.$$

So, the fixed point iteration converges with at least a *linear rate*, similar to the bisection method. This allows us to define a sufficient condition for the fixed point iteration to converge:

DEFINITION 2.12 (FIXED POINT CONVERGENCE). The fixed point iteration has, at least, a linear convergence rate in $[a, b]$ provided $\varphi(x)$ is a contraction and the initial guess x_0 is in the interval of contraction. ◀

Remark 2.1. We say the convergence is *at least* a linear rate. This is because, as already discussed, the Newton-Raphson method is a fixed point iteration but its error decays quadratically. We will investigate this rate of convergence a bit later. Overall, the fixed point method is nice because it is very easy to implement (see the pseudocode) and analyze; however, it might not be faster than bisection. ⋈

Algorithm 2.3: *Fixed point iteration*: General strategy to compute the fixed point $x^* = \varphi(x^*)$

Procedure Fixed Point Iteration
Input: x_0, $\varphi(x)$, `tol` `//e.g.` `tol` $= 10^{-6}$

$max_{its} \leftarrow 50$ `// maximum number of iterations allowed`
$x_{old} \leftarrow x_0$
for $k = 1$ **to** max_{its} **do**
  $x_{new} \leftarrow \varphi(x_{old})$
  if $\frac{|x_{new} - x_{old}|}{|x_{new}|} <$ `tol` **then**
    break
  $x_{old} \leftarrow x_{new}$
Output: x_{new}
End Procedure Fixed Point Iteration

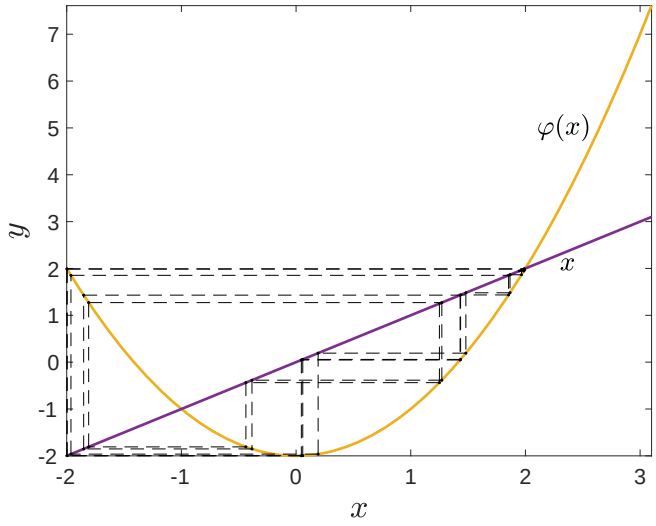
EXAMPLE 2.6. We can now explain why the Picard iteration converged to a root in the previous example. In that case $\varphi(x) = \cos(x)$, so $\varphi'(x) = \sin(x)$ on $[0, 1]$ and we see that the mapping is a contraction

$$|\cos(x) - \cos(y)| \leq L|x - y|,$$

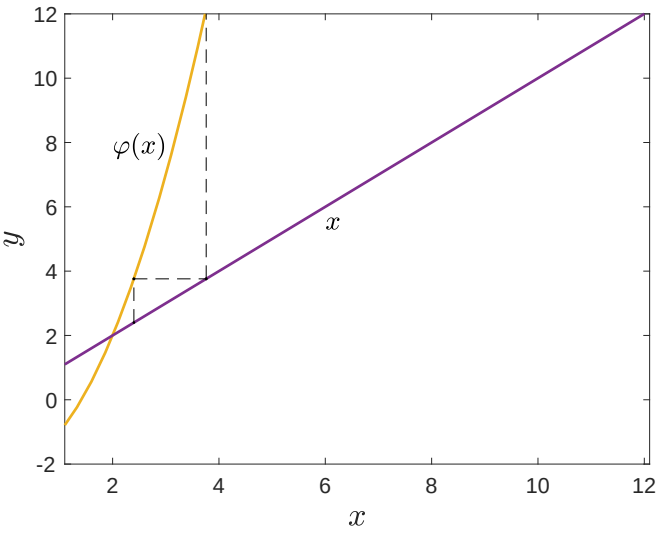
with coefficient $L = \sin(1)$. ♦

Conversely, we see that the if the mapping function is not a contraction then the error will grow with each step of the iteration because the root is actually **repelled** by the mapping!

EXAMPLE 2.7. Determine a root of the polynomial $f(x) = x^2 - x - 2$ in the interval $[1, 3]$ using a fixed point method. It is easy to calculate that the root we seek is $x^* = 2$. Say we take the mapping function $\varphi_1(x) = x^2 - 2$ such that $\varphi_1'(x) = 2x$. We see for $x > \frac{1}{2}$ that $|\varphi_1'(x)| > 1$. So, for all points in the interval $[1, 3]$ the iteration will fail because the mapping is not a contraction and the point $x^* = 2$ repels the iteration. We show this below for two initial guesses $x_0 = 1.25$, which "bounces" around never locating the root, and $x_0 = 2.5$, which diverges away toward infinity.

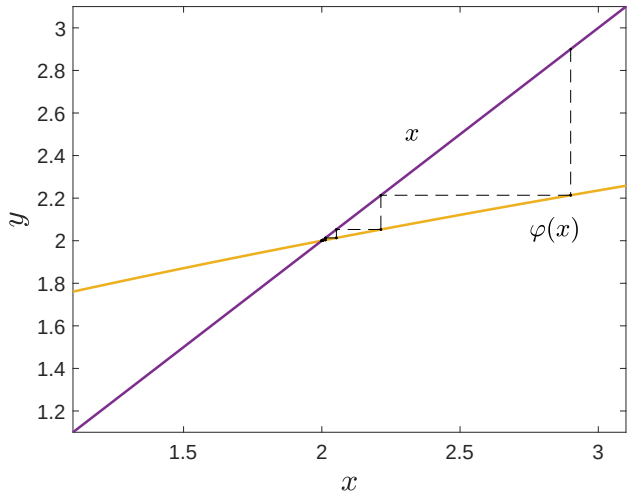


k	x_k	$f(x_k)$
0	1.25	-1.6875
1	-0.437500000	-1.371093750
2	-1.808593750	3.079605102
3	1.271011352	-1.655541494
4	-0.384530141	-1.467606428
5	-1.852136570	3.282546444
6	1.430409874	-1.384337465
7	0.046072408	-2.043949741
8	-1.997877333	3.989391171
9	1.991513838	-0.025386469
10	1.966127368	-0.100470539
11	1.865656828	-0.384981425
12	1.480675403	-1.288275753
13	0.192399649	-2.155382024
14	-1.962982374	3.816282179
15	1.853299804	-0.418579639
16	1.434720165	-1.376298212
17	0.058421952	-2.055008828
⋮	⋮	⋮



k	x_k	$f(x_k)$
0	2.4	1.36
1	3.76	8.3776
2	12.1376	133.18373
3	145.32133	20,970.96871

If, however, we take the mapping $\varphi_2(x) = \sqrt{x+2}$ then we have a contraction because $\varphi'(x) = \frac{1}{2\sqrt{x+2}} < 1$ for all points in the interval $[1, 3]$. Now the fixed point method zig-zags toward the root with an initial guess $x_0 = 2.9$. The iteration terminates after 11 iterations once a tolerance of $\varepsilon = 10^{-7}$ is reached.



k	x_k	$f(x_k)$
0	2.9	-0.686405637
1	2.213594362	-0.160890203
2	2.052704158	-0.039571237
3	2.013132921	-0.009852381
4	2.003280539	-0.002460572
5	2.000819966	-0.000614985
6	2.000204981	-0.000153736
7	2.000051244	-0.000038433
8	2.000012811	-0.000009608
9	2.000003202	-0.000002402
10	2.000000800	-0.000000600
11	2.000000200	-0.000000150

♦

We discussed that the fixed point method will converge with at least a linear rate if $\varphi(x)$ is a contraction mapping. In what situations can we expect more rapid convergence?

Definition 2.13 (Fixed point convergence rate). Assume the mapping $\varphi(x)$ is $m+1$ times differentiable in an interval (a, b) near the fixed point x^*. If $\varphi^{(i)}(x^*) = 0$ for $1 \leq i \leq m$ and $\varphi^{(m+1)}(x^*) \neq 0$ then the ratio of subsequent errors is given by

$$\frac{x^* - x_{k+1}}{(x^* - x_k)^{m+1}} = -\frac{\varphi^{(m+1)}(x^*)}{(m+1)!},$$

and the fixed point iteration converges for $x_0 \in (a, b)$ with order $m+1$. ◀

For example, the Newton-Raphson method takes the mapping function to be $\varphi(x) = x - \frac{f(x)}{f'(x)}$. It is straightforward to compute

$$\varphi'(x) = \frac{f(x)f''(x)}{(f'(x))^2}$$

such that $\varphi'(x^*) = 0$ and $\varphi''(x^*) = \frac{f''(x^*)}{f'(x^*)}$. So, Newton-Raphson is quadratically convergent, matching the Taylor expansion analysis we performed in the previous section. There exist fixed point iteration methods that are quadratically convergent, but do not require knowledge of a function's derivative:

Definition 2.14 (Steffensen's method). The mapping function

$$\varphi(x) = x - \frac{(f(x))^2}{f(x + f(x)) - f(x)}$$

defines a fixed point iteration that is quadratically convergent. ◀

Up to now we have only considered simple roots of the function $f(x)$, that is $f(x^*) = 0$ but $f'(x^*) \neq 0$. What if the root has a higher *multiplicity*? For roots with higher multiplicity the convergence rate of a fixed point iteration will **slow down**.

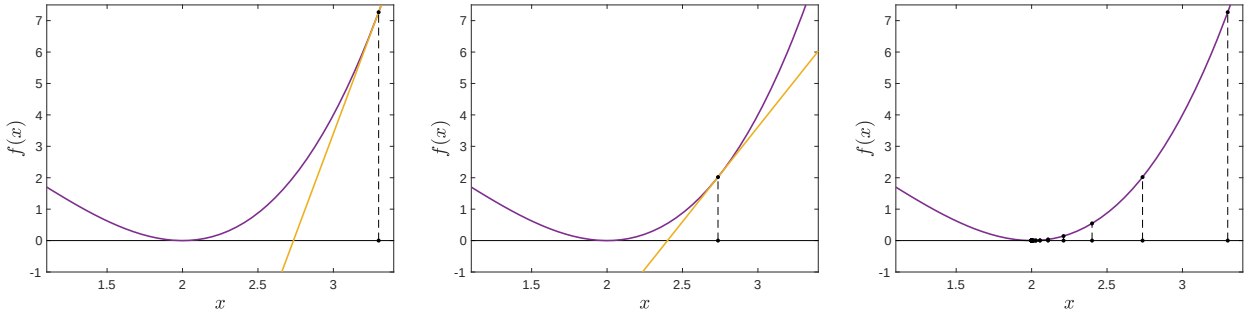
Definition 2.15 (Convergence order reduction). If the function $f(x)$ has a root of multiplicity m, i.e., $f(x^*) = f'(x^*) = \cdots = f^{(m-1)}(x^*) = 0$ and $f^{(m)}(x^*) \neq 0$, then quadratically convergent fixed point methods (like Newton-Raphson or Steffensen) will converge at a *linear rate*. ◀

Definition 2.16 (Modified Newton-Raphson). If we know the multiplicity, m, of the root x^* it is possible to modify the Newton-Raphson method

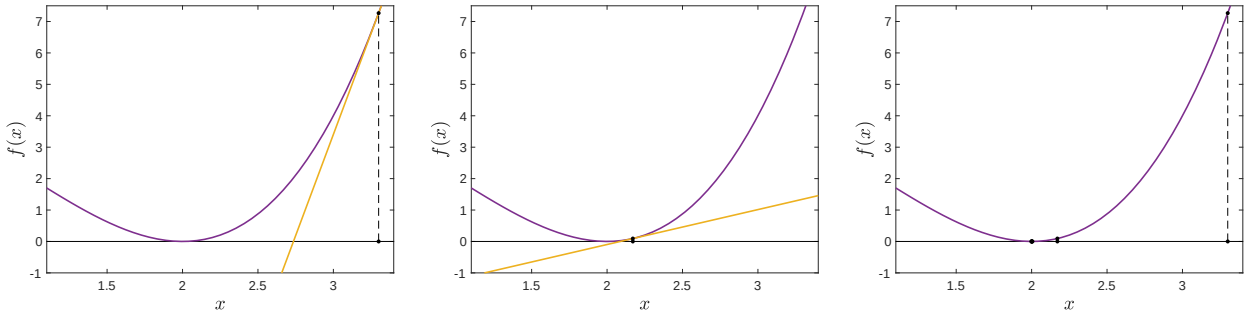
$$x_{k+1} = x_k - m\frac{f(x_k)}{f'(x_k)},$$

and recover quadratic convergence. ◀

Example 2.8. Use the traditional Newton-Raphson as well as the modified Newton-Raphson method to approximate the root of the polynomial $f(x) = x^3 - 3x^2 + 4$ in the interval $[1, 3]$ with initial guess $x_0 = 3.3$ and kill tolerance $\varepsilon = 10^{-7}$. We start with the traditional Newton-Raphson and see that the method gets in the neighborhood of the root $x = 2$ very quickly, but the convergence slows considerably as the slope of the tangent line approaches zero. It takes 29 iterations to reach the termination condition and $x^* \approx 2.000000003196132$.



We can factor the polynomial of interest to see $f(x) = (x-2)^2(x+1)$ to see that the root $x^* = 2$ has multiplicity $m = 2$. Therefore, we modify the Newton-Raphson method appropriately and see that the convergence order becomes rapid once again. We converge in *five* iterations to an approximate root $x^* \approx 2.000000000014988$.



♦

To halt the iterative process of finding the root we have defined two stopping conditions in the pseudocodes above. One examines the residual, that is how close $f(x_{new})$ is to zero. The other examine the relative change in the value of x_{new} compared to x_{old} and terminates once these changes are deemed "small." In practice, a *combination* of these two flavours of kill condition is typically needed in order to increase the reliability of the iterative process to accurately find a root.

We will revisit fixed point iteration methods as they are quite useful in certain circumstances to approximate the solution of a **system** of linear equations. But this will require some generalisations of the stopping conditions as well as re-interpreting what a "contraction" means in the multi-variable context.

As a final note, although outside the scope of this course, systems of nonlinear equations and locating their fixed points is an important aspect of numerical optimization. There multidimensional versions of the Newton-Raphson or secant (sometimes referred to as quasi-Newton) methods are crucial to approximate these values.

Note Set 3: Linear systems of equations

We expand our focus from solving equations with a single variable to consider solving linear systems of equations with multiple variables. This task is pervasive throughout computational science as many applications require the manipulation and solution of formulas that involve matrices. In particular, we will focus on the computation of a solution vector $\mathbf{x}$ for the general equation

$$\mathbf{A}\mathbf{x} = \mathbf{b},$$

where $\mathbf{A}$ is a matrix and $\mathbf{b}$ is some right-hand-side. However, just as with single variable computations, we must revisit our knowledge of linear algebra and understand how it translates onto computer. This includes examining errors introduced by the computation, how they affect the accuracy of the discrete solution, and how to estimate the "computational cost" of solving the systems with a particular algorithm.

Regarding notation for the discussion of linear algebra. In the *note sets and on the lecture slides* we denote scalars as lower case italic, e.g. c, vectors with lower case roman bold, e.g. $\mathbf{x}$, and matrices as upper case roman bold, e.g. $\mathbf{A}$. For *board work or hand-written notes* scalars are still lower case variables but vectors are denoted with lower case and a single underline, e.g. $\underline{x}$, and matrices are upper case with a double underline, e.g. $\underline{\underline{A}}$.

3.1 Basics and computational primitives

We first review some concepts from linear algebra in order to discuss and analyze vector and matrix operations on a computer. Therefore, these formulas and notions may be familiar, but we will *contextualize* them knowing that we plan to use this information in a practical and discrete setting.

A vector $\mathbf{x} \in \mathbb{R}^n$ is a one dimensional ordered list of n real scalars. Vectors can add component-wise or be scaled by a constant factor in each component

$$\mathbf{x} + \mathbf{y} = \begin{pmatrix} x_1 + y_1 \\ \vdots \\ x_n + y_n \end{pmatrix}, \qquad \alpha\mathbf{x} = \begin{pmatrix} \alpha x_1 \\ \vdots \\ \alpha x_n \end{pmatrix}.$$

A set of vectors $\mathbf{x}_1, \ldots, \mathbf{x}_m$ are *linearly independent* if

$$\alpha_1\mathbf{x}_1 + \cdots + \alpha_m\mathbf{x}_m = 0 \quad \Rightarrow \quad \alpha_i = 0 \ \ \forall i.$$

If not, then the vectors are *linearly dependent*, i.e. one of the vectors can be written as a linear combination of the others. This introduces the notion of a basis that can *span* a vector space. The *canonical basis* for $\mathbb{R}^n$ are the vectors $\mathbf{e}_1, \mathbf{e}_2, \ldots, \mathbf{e}_n$ where

$$\mathbf{e}_1 = \begin{pmatrix} 1 \\ 0 \\ \vdots \\ 0 \end{pmatrix}, \quad \mathbf{e}_2 = \begin{pmatrix} 0 \\ 1 \\ \vdots \\ 0 \end{pmatrix}, \quad \cdots, \quad \mathbf{e}_n = \begin{pmatrix} 0 \\ 0 \\ \vdots \\ 1 \end{pmatrix}.$$

This defines a vector of ones as $\mathbf{e} = \mathbf{e}_1 + \mathbf{e}_2 + \cdots \mathbf{e}_n = (1, 1, \ldots, 1)^T$, where T denotes the *transpose*. Two important ways to multiply vectors are the **inner product**

$$\mathbf{x}^T\mathbf{y} = x_1y_1 + x_2y_2 + \cdots + x_ny_n,$$

where the result is a scalar quantity and the **outer product**

$$\mathbf{x}\mathbf{y}^T = \begin{pmatrix} x_1y_1 & x_1y_2 & \cdots & x_1y_n \\ x_2y_1 & x_2y_2 & \cdots & x_2y_n \\ \vdots & \vdots & \ddots & \vdots \\ x_ny_1 & x_ny_2 & \cdots & x_ny_n \end{pmatrix},$$

where the result is a square matrix. Also, if two vectors are **orthogonal** then their inner product is zero

$$\mathbf{x}^T\mathbf{y} = 0.$$

A square matrix $\mathbf{A} \in \mathbb{R}^{n\times n}$ represents a linear transformation and is an ordered arrangement of n^2 real scalars

$$\mathbf{A} = \begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{pmatrix}.$$

The transpose of a matrix exchanges its rows and columns

$$\mathbf{A}^T = \begin{pmatrix} a_{11} & a_{21} & \cdots & a_{n1} \\ a_{12} & a_{22} & \cdots & a_{n2} \\ \vdots & \vdots & \ddots & \vdots \\ a_{1n} & a_{2n} & \cdots & a_{nn} \end{pmatrix}.$$

Matrices can be also be added component-wise $\mathbf{C} = \mathbf{A} + \mathbf{B}$ or scaled by a constant factor α. Also, we can multiply two matrices together to obtain about matrix

$$\mathbf{A}\mathbf{B} = \mathbf{C},$$

or multiply a matrix times a vector to obtain a vector

$$\mathbf{A}\mathbf{x} = \mathbf{y}.$$

Remark 3.1. It is important to remember the following:

- Matrix multiplication *does not* commute.
- Matrix multiplication is distributive; $\mathbf{A}(\mathbf{B} + \mathbf{C}) = \mathbf{A}\mathbf{B} + \mathbf{A}\mathbf{C}$.
- Vector and matrix addition as well as matrix multiplication are associative;

$$\mathbf{x} + \mathbf{y} = \mathbf{y} + \mathbf{x}, \quad \mathbf{A} + \mathbf{B} = \mathbf{B} + \mathbf{A}, \quad (\mathbf{A}\mathbf{B})\mathbf{C} = \mathbf{A}(\mathbf{B}\mathbf{C}).$$

Keep in mind that on a computer aspects of these mathematical formulas *will not* hold, like the associativity of addition (see Note Set 1). ⋈

If the columns of the matrix $\mathbf{A}$ are linearly independent, then the matrix has full *rank*. This means that the equation

$$\mathbf{A}\mathbf{x} = \mathbf{b},$$

admits a unique solution $\mathbf{x}$ provided $\mathbf{b}$ is in the *range* of $\mathbf{A}$. This implies that the matrix $\mathbf{A}$ is *invertible* (i.e. it is nonsingular) and the solution to the above problem is given by

$$\mathbf{x} = \mathbf{A}^{-1}\mathbf{b}.$$

The identity matrix is a square matrix with ones along its main diagonal and zeros elsewhere

$$\mathbf{I} = \begin{pmatrix} 1 & 0 & 0 & \cdots & 0 \\ 0 & 1 & 0 & \cdots & 0 \\ 0 & 0 & 1 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & 1 \end{pmatrix} = \text{diag}(1, \ldots, 1).$$

Also, if a matrix $\mathbf{A}$ is nonsingular then $\mathbf{I} = \mathbf{A}\mathbf{A}^{-1} = \mathbf{A}^{-1}\mathbf{A}$.

To discuss analytical properties of vectors and matrices we require tools to measure concepts such as size and distance. For this we use **norms**.

Definition 3.1 (Vector norm). Consider two vectors $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$ and a scalar $\alpha \in \mathbb{R}$. A *vector norm* $\|\cdot\|$ is a mapping from $\mathbb{R}^n \to \mathbb{R}$ that satisfies:

1. $\|\mathbf{x}\| \geq 0$ and $\|\mathbf{x}\| = 0 \Leftrightarrow \mathbf{x} = \mathbf{0}$ (definiteness).
2. $\|\alpha\mathbf{x}\| = |\alpha|\|\mathbf{x}\|$ (homogeneity).
3. $\|\mathbf{x} + \mathbf{y}\| \leq \|\mathbf{x}\| + \|\mathbf{y}\|$ (triangle inequality). ◀

Example 3.1. The most common norms that we use are

$$\|\mathbf{x}\|_1 = \sum_{i=1}^{n} |x_i| \qquad \text{(1-norm)}$$

$$\|\mathbf{x}\|_\infty = \max_{i=1,\ldots,n} |x_i| \quad \text{(∞-norm)}$$

$$\|\mathbf{x}\|_2 = \sqrt{\sum_{i=1}^{n} |x_i|^2} \quad \text{(2-norm or Euclidean norm)}$$

Consider the vector $\mathbf{x} = (2, -3, 1, 4)^T$, then we have

$$\|\mathbf{x}\|_1 = |2|+|-3|+|1|+|4| = 10, \quad \|\mathbf{x}\|_2 = \sqrt{(2)^2 + (-3)^2 + (1)^2 + (4^2)} = \sqrt{4 + 9 + 1 + 16} = \sqrt{30}, \quad \|\mathbf{x}\|_\infty = 4$$

♦

With this notation the distance between two vector quantities is $\|\mathbf{x} - \mathbf{y}\|$. Thus, it is possible to describe the error between a vector and its approximate value.

Definition 3.2 (Absolute and relative error). Given the exact value of a vector quantity $\mathbf{x}$ and its perturbed value written in a floating point number system $\mathbf{x}_A = \mathbf{x}(1 + \delta) = \mathbf{x} + \delta\mathbf{x}$ we have:

Absolute error in $\mathbf{x}_A$ is $\|\delta\mathbf{x}\| = \|\mathbf{x} - \mathbf{x}_A\|$.

Relative error in $\mathbf{x}_A$ is $\dfrac{\|\delta\mathbf{x}\|}{\|\mathbf{x}\|} = \dfrac{\|\mathbf{x} - \mathbf{x}_A\|}{\|\mathbf{x}\|}$ provided $\|\mathbf{x}\| \neq 0$.

Note that $\delta\mathbf{x}$ is a vector, so there is an error in each component of the subtraction of $\mathbf{x}$ and $\mathbf{x}_A$. ◀

With any vector norm we can induce a matrix norm.

Definition 3.3 (Induced matrix norm). Take $\|\cdot\|$ to be any vector norm. The *induced matrix norm* maps $\mathbb{R}^{n\times n} \to \mathbb{R}$ and is defined as

$$\|\mathbf{A}\| = \sup_{\mathbf{x}\neq 0} \frac{\|\mathbf{A}\mathbf{x}\|}{\|\mathbf{x}\|} = \sup_{\|\mathbf{x}\|=1} \|\mathbf{A}\mathbf{x}\|$$

where the latter formulation is often more convenient to compute. For matrices $\mathbf{A}, \mathbf{B} \in \mathbb{R}^{n\times n}$ and $\alpha \in \mathbb{R}$ the induced norm satisfies:

1. $\|\mathbf{A}\| \geq 0$ and $\|\mathbf{A}\| = 0 \Leftrightarrow \mathbf{A} = \mathbf{0}$ (definiteness).
2. $\|\alpha\mathbf{A}\| = |\alpha|\|\mathbf{A}\|$ (homogeneity).
3. $\|\mathbf{A} + \mathbf{B}\| \leq \|\mathbf{A}\| + \|\mathbf{B}\|$ (triangle inequality).
4. $\|\mathbf{A}\mathbf{x}\| \leq \|\mathbf{A}\|\|\mathbf{x}\|$ (subordinate).
5. $\|\mathbf{A}\mathbf{B}\| \leq \|\mathbf{A}\|\|\mathbf{B}\|$ (submultiplicative).

◀

EXAMPLE 3.2. Some common matrix norms are

$$\|\mathbf{A}\|_1 = \max_{j=1,\ldots,n} \left\{ \sum_{i=1}^{n} |a_{ij}| \right\} \quad \text{(1-norm): max of column sums}$$

$$\|\mathbf{A}\|_\infty = \max_{i=1,\ldots,n} \left\{ \sum_{j=1}^{n} |a_{ij}| \right\} \quad (\infty\text{-norm}): \text{ max of row sums}$$

$$\|\mathbf{A}\|_2 = \sqrt{\max_{i=1,\ldots,n} \lambda_i \left(\mathbf{A}^T\mathbf{A}\right)} \quad \text{(2-norm): spectral norm}$$

Consider the matrix

$$\mathbf{A} = \begin{pmatrix} 3 & -7 & 5 \\ 11 & 3 & -2 \\ -1 & 8 & 10 \end{pmatrix}$$

then we find that the 1-norm and ∞-norm are

$$\|\mathbf{A}\|_1 = \max\{|3| + |11| + |-1|, |-7| + |3| + |8|, |5| + |-2| + |10|\} = \max\{15, 18, 17\} = 18,$$
$$\|\mathbf{A}\|_\infty = \max\{|3| + |-7| + |5|, |11| + |3| + |-2|, |-1| + |8| + |10|\} = \max\{15, 16, 19\} = 19.$$

⧫

We see that the first two matrix norms are straightforward to compute. However, the 2-norm is a considerable amount of work to compute because it involves the solution of an eigenvalue problem. Fortunately, we have the following:

DEFINITION 3.4. In a finite dimensional vector space all norms are *equivalent*, e.g.

$$\|\mathbf{x}\|_\infty \leq \|\mathbf{x}\|_2 \leq \sqrt{n}\|\mathbf{x}\|_\infty, \qquad \frac{1}{\sqrt{n}}\|\mathbf{A}\|_\infty \leq \|\mathbf{A}\|_2 \leq \sqrt{n}\|\mathbf{A}\|_\infty.$$

In other words, equivalence gives us that no matter what norm we decide to use to "measure" a vector/matrix its value in an alternative norm only differs up to some multiplicative constant(s). ◀

This is incredibly useful because equivalence tells us that we can measure values of a vector, a matrix, or errors in whatever norm is easiest to compute and we know their value will be related to other (possibly more difficult to compute norms). A particularly useful inequality between matrix norms is

$$\|\mathbf{A}\|_2 \leq \sqrt{\|\mathbf{A}\|_1\|\mathbf{A}\|_\infty}.$$

We are now equipped to discuss the *conditioning* of solving a the square linear system of equations $\mathbf{A}\mathbf{x} = \mathbf{b}$. In Note Set 1 we defined the condition number to be the relative change in the solution divided by the relative change in the data. For the problem under consideration, the data are $\mathbf{A}$ and $\mathbf{b}$ and the solution is $\mathbf{x}$. To simplify the discussion we start by only allowing perturbations in $\mathbf{b}$ while $\mathbf{A}$ is unchanged (i.e. represented exactly). If the right hand side is perturbed, so is the solution, and the problem becomes

$$\mathbf{A}(\mathbf{x} + \delta\mathbf{x}) = \mathbf{b} + \delta\mathbf{b}.$$

Because $\mathbf{Ax} - \mathbf{b} = \mathbf{0}$ and $\mathbf{A}$ is nonsingular we find

$$\delta\mathbf{x} = \mathbf{A}^{-1}\delta\mathbf{b}.$$

Taking the norm of this quantity and using that induced matrix norms are subordinate we find an estimate of the absolute error

$$\|\delta\mathbf{x}\| = \|\mathbf{A}^{-1}\delta\mathbf{b}\| \leq \|\mathbf{A}^{-1}\|\,\|\delta\mathbf{b}\|.$$

Similarly, from the original equation we have

$$\|\mathbf{b}\| = \|\mathbf{Ax}\| \leq \|\mathbf{A}\|\,\|\mathbf{x}\| \quad \Rightarrow \quad \frac{1}{\|\mathbf{x}\|} \leq \|\mathbf{A}\|\,\frac{1}{\|\mathbf{b}\|}$$

Then we find that the relative error (or perturbation) of the solution is bounded from above by the relative error (or perturbation) of the right hand side data

$$\frac{\|\delta\mathbf{x}\|}{\|\mathbf{x}\|} \leq \|\mathbf{A}\|\,\|\mathbf{A}^{-1}\|\,\frac{\|\delta\mathbf{b}\|}{\|\mathbf{b}\|},$$

scaled by the factor $\|\mathbf{A}\|\,\|\mathbf{A}^{-1}\|$. This indicates how *sensitive* the computed solution is to perturbations in the right hand side.

Remark 3.2. This numerical sensitivity and how the conditioning of the matrix $\mathbf{A}$ influences the reliability of solving a linear system of equations is further explored in the first set of computer exercises. ⋈

Definition 3.5 (Condition number for $\mathbf{A}$). The condition number κ of a nonsingular matrix $\mathbf{A}$ is

$$\kappa(\mathbf{A}) = \|\mathbf{A}\|\,\|\mathbf{A}^{-1}\|.$$

Note, the condition number depends on which matrix norm is used to compute it. Therefore, subscripts are assigned accordingly, e.g. $\kappa_1(\mathbf{A}) = \|\mathbf{A}\|_1\,\|\mathbf{A}^{-1}\|_1$. ◀

Remark 3.3. Although we derived the conditioning only for perturbations in the right hand side data $\mathbf{b}$, a similar statement holds when $\mathbf{A}$ is perturbed and the right hand side data is unchanged

$$\frac{\|\delta\mathbf{x}\|}{\|\mathbf{x}\|} \leq \kappa(\mathbf{A})\,\frac{\|\delta\mathbf{A}\|}{\|\mathbf{A}\|}.$$

In any induced matrix norm it is true that $\|I\| = 1$. From this we see

$$1 = \|I\| = \|\mathbf{A}\mathbf{A}^{-1}\| \leq \|\mathbf{A}\|\,\|\mathbf{A}^{-1}\| = \kappa(\mathbf{A}),$$

where we used that the identity $I = \mathbf{A}\mathbf{A}^{-1}$. So, a condition number of one is the best we can hope for, i.e. the relative perturbation in the solution has the *same size* as that of the data.

Therefore we say that a matrix with a small condition number (that is close to one) is **well-conditioned** whereas a matrix with a large condition number is **ill-conditioned**. As stated above, the condition number is used to estimate how sensitive the accuracy of solving the problem $\mathbf{Ax} = \mathbf{b}$ with a computer is to perturbations in the data. Heuristically, a condition number on the order of 10^t indicates that in floating point arithmetic, approximately t digits are "lost" (that is become incorrect) in numerically computing the solution $\mathbf{x}$. ⋈

Remark 3.4. Recall that in floating point arithmetic we defined the unit round off to be $\mu \approx 5.96 \times 10^{-8}$ and $\mu \approx 1.11 \times 10^{-16}$ for single and double precision, respectively. If a nonsingular matrix $\mathbf{A}$ has a condition number $\kappa(\mathbf{A}) \gtrsim \mu^{-1}$, then for *computational purposes* the matrix is **singular**. ⋈

Remark 3.5. Keep in mind that the conditioning of $\mathbf{A}$ is *independent* from the algorithm we choose to solve the problem $\mathbf{Ax} = \mathbf{b}$. A numerically stable algorithm still produces an inaccurate solution if $\mathbf{A}$ is ill-conditioned. ⋈

EXAMPLE 3.3. Compute the condition number $\kappa_\infty(\mathbf{A})$ for the matrices

(a) $\mathbf{A} = \frac{1}{2}\begin{pmatrix} 1 & 1 \\ 4 & -2 \end{pmatrix}$

(b) $\mathbf{A} = \frac{1}{2}\begin{pmatrix} 1 & 1 \\ 1+10^{-8} & 1-10^{-8} \end{pmatrix}$

(a): We first find that the inverse of the matrix is $\mathbf{A}^{-1} = \frac{1}{3}\begin{pmatrix} 2 & 1 \\ 4 & -1 \end{pmatrix}$. Then we compute the two matrix ∞-norms to be $\|\mathbf{A}\|_\infty = 3$ and $\|\mathbf{A}^{-1}\|_\infty = \frac{5}{3}$. Therefore, the condition number is $\kappa_\infty(\mathbf{A}) = \|\mathbf{A}\|_\infty\|\mathbf{A}^{-1}\|_\infty = 5$.

(b): We find that the inverse is $\mathbf{A}^{-1} = 10^8\begin{pmatrix} 10^{-8}-1 & 1 \\ 10^{-8}+1 & -1 \end{pmatrix}$. The ∞-norms are then $\|\mathbf{A}\|_\infty = 1$ and $\|\mathbf{A}^{-1}\|_\infty = 10^8 \max\{|10^{-8}-1|+1, 10^{-8}+2\} \approx 10^8$. So, $\kappa_\infty(\mathbf{A}) = \|\mathbf{A}\|_\infty\|\mathbf{A}^{-1}\|_\infty \approx 10^8$ is the condition number.

We see that the first matrix is well-conditioned while the second matrix is ill-conditioned. In fact, the matrix from part (b) is singular for computations in single precision and we expect a significant loss of accuracy (on the order of eight digits) in double precision.

To demonstrate this loss of accuracy we consider the right hand side to be $\mathbf{b} = (1,1)^T$. Then it is easy to see that the exact solution for either matrix is $\mathbf{x}_E = (1,1)^T$. If we approximate the solution in double precision using the (a) matrix we find that the relative error is $\|\delta\mathbf{x}\|_2/\|\mathbf{x}_E\|_2 = 2.22 \times 10^{-16}$ which corresponds to 15 accurate digits. If, however, the approximate solution is computed using the (b) matrix the relative error is $\|\delta\mathbf{x}\|_2/\|\mathbf{x}_E\|_2 = 6.10 \times 10^{-9}$ which corresponds to only eight accurate digits. ♦

We close this section with a final discussion about vectors and matrices in numerical calculations and how we should think about them in the context of using computers to solves linear algebra problems. We have seen, for example, that a matrix is nonsingular in theory, but in practice it is essentially singular. So an appropriate question is: What should we take into consideration concerning a matrix in the context of computational mathematics? One aspect concerns exploiting the so-called *structure* of a matrix. Some examples are:

- The matrix is *symmetric*: $\mathbf{A} = \mathbf{A}^T$.
- The matrix is *triangular* (the matrix is entirely zero above or below the main diagonal):

$$\mathbf{A} = \begin{pmatrix} * & 0 & 0 & 0 & 0 \\ * & * & 0 & 0 & 0 \\ * & * & * & 0 & 0 \\ * & * & * & * & 0 \\ * & * & * & * & * \end{pmatrix} \quad \text{or} \quad \mathbf{A} = \begin{pmatrix} * & * & * & * & * \\ 0 & * & * & * & * \\ 0 & 0 & * & * & * \\ 0 & 0 & 0 & * & * \\ 0 & 0 & 0 & 0 & * \end{pmatrix}$$

- The matrix is *banded* (only a few diagonals are nonzero):

$$\mathbf{A} = \begin{pmatrix} * & * & 0 & 0 & 0 \\ * & * & * & 0 & 0 \\ 0 & * & * & * & 0 \\ 0 & 0 & * & * & * \\ 0 & 0 & 0 & * & * \end{pmatrix}$$

- The matrix is *sparse* (most of the matrix elements are zero):

$$\mathbf{A} = \begin{pmatrix} * & 0 & 0 & * & 0 \\ 0 & * & 0 & * & 0 \\ 0 & * & * & 0 & * \\ 0 & 0 & 0 & * & 0 \\ * & 0 & 0 & 0 & * \end{pmatrix}$$

As computational scientists we can use prior knowledge about the matrix and its structure to design algorithms that save memory or computation time, both of which are limited on a computer.

We need a systematic way to describe how much a computation and/or algorithm "costs" on a computer. To estimate the amount of work for a certain computation we count the number of floating point operations (or **flops** for short). In this course we take each scalar addition, subtraction, multiplication, division, and function evaluation (e.g. taking a square root) to count as one flop. We will also keep track on the amount of memory needed to store the data as well as the solution. We discuss these costs in terms of *asymptotic estimates*, i.e what happens as the dimension n become sufficiently large. To do so, we employ **big-$\mathcal{O}$** notation to denote the complexity of a problem and ignore lower order terms.

EXAMPLE 3.4. Say we have two vectors $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$ and two square matrices $\mathbf{A}, \mathbf{B} \in \mathbb{R}^{n \times n}$. We will estimate the computational work for computing:

(a) The inner product of two vectors.

(b) The outer product of two vectors.

(c) The product of a matrix and a vector.

(d) The product of two matrices.

(a): From the definition of the inner product we know that $\mathbf{x}^T\mathbf{y} = x_1y_1 + \cdots + x_ny_n$. This requires n multiplications and $n-1$ additions. Therefore, the computational complexity of the inner product is $2n-1 = \mathcal{O}(n)$. With respect to memory, the inner product uses two n vectors and results in a scalar, which gives a memory estimate of $2n+1 = \mathcal{O}(n)$.

(b): The outer product takes two n vectors and produces a square $n \times n$ matrix

$$\mathbf{x}\mathbf{y}^T = \begin{pmatrix} x_1y_1 & x_1y_2 & \cdots & x_1y_n \\ x_2y_1 & x_2y_2 & \cdots & x_2y_n \\ \vdots & \vdots & \ddots & \vdots \\ x_ny_1 & x_ny_2 & \cdots & x_ny_n \end{pmatrix}.$$

Therefore, the memory consumption is $n^2 + 2n = \mathcal{O}(n^2)$. To form each entry of the matrix requires a multiplication, which means the computational complexity of the outer product is $n^2 = \mathcal{O}(n^2)$.

(c): The product of a matrix and a vector results in a vector. So, the memory complexity is $n^2+2n = \mathcal{O}(n^2)$. We can interpret the matrix-vector product as taking the inner product of each row of the matrix $\mathbf{A}$ with the vector $\mathbf{x}$

$$\mathbf{A}\mathbf{x} = \begin{pmatrix} \mathbf{a}_1^T\mathbf{x} \\ \vdots \\ \mathbf{a}_n^T\mathbf{x} \end{pmatrix}$$

Therefore, we perform n inner products, that is we perform n^2 multiplications and $n(n-1)$ additions. This results in a computational complexity of $2n^2 - n = \mathcal{O}(n^2)$.

(d): The product of two matrices is a matrix such that the memory complexity is $3n^2 = \mathcal{O}(n^2)$. Multiplying two matrices together can be interpreted as taking the matrix vector product $\mathbf{A}\mathbf{b}_i$, $i = 1, \ldots, n$ for each column of $\mathbf{B}$. Therefore, the computational complexity is $n(2n^2 - n) = 2n^3 - n^2 = \mathcal{O}(n^3)$. ♦

It is helpful to divide similar computational primitives into categories. To do so, we use the convention of the Basic Linear Algebra Subprogram (BLAS) hierarchy:

BLAS1: Scalar-vector and vector-vector operations: $\mathcal{O}(n)$ data, $\mathcal{O}(n)$ flops.

BLAS2: Matrix-vector operations: $\mathcal{O}(n^2)$ data, $\mathcal{O}(n^2)$ flops.

BLAS3: Matrix-matrix operations: $\mathcal{O}(n^2)$ data, $\mathcal{O}(n^3)$ flops.

This structure helps us organize our thoughts and orient ourselves with some basic questions when examining an algorithm: What is the input data and how is it stored? What are the operations and how much do they cost? What is the output data (or solution) and how is it stored?

3.2 Solving linear systems

Now we have an understanding and mechanisms to discuss the solution of the problem $\mathbf{Ax} = \mathbf{b}$, where the errors may come from and what can affect the solution's accuracy. So, we turn to a more practical questions: What method(s) are available to compute the solution $\mathbf{x}$? Is the algorithm reliable?

A first instinct would be to find the inverse, $\mathbf{A}^{-1}$, explicitly and then use matrix-vector multiplication to compute $\mathbf{x} = \mathbf{A}^{-1}\mathbf{b}$. This is a ***disastrous*** algorithm choice in terms of complexity as well as reliability. One reason is because even if a matrix $\mathbf{A}$ has some structure (like sparsity with lots of zeros) its inverse *will not* which is costly in terms of memory. Another is that the computational complexity to construct the matrix inverse is approximately $2n^3$ flops. This estimate comes from the naive approach of inverting a matrix by computing its constituent.

Instead, we solve the linear system by computing the inverse's *"action"* without explicitly forming it.

EXAMPLE 3.5. Consider a matrix $\mathbf{A}$ that is diagonal. So, for $n = 4$

$$\begin{pmatrix} a_{11} & 0 & 0 & 0 \\ 0 & a_{22} & 0 & 0 \\ 0 & 0 & a_{33} & 0 \\ 0 & 0 & 0 & a_{44} \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{pmatrix} = \begin{pmatrix} b_1 \\ b_2 \\ b_3 \\ b_4 \end{pmatrix}.$$

Then is trivial to compute the action of the inverse and find the solution

$$\begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{pmatrix} = \begin{pmatrix} b_1/a_{11} \\ b_2/a_{22} \\ b_3/a_{33} \\ b_4/a_{44} \end{pmatrix},$$

provided none of the diagonal entries are zero. ♦

What about for a general matrix $\mathbf{A}$? We know that any linear combination of the given equation system remains true. Also, because of the assumed linear independence of the system, taking such linear combinations preserves the unique solution $\mathbf{x}$. We use these facts to perform *elimination* and remove variables from the equations to reveal the solution.

DEFINITION 3.6 (ELEMENTARY ROW OPERATIONS). There are three elementary row operations that can be performed on a general matrix $\mathbf{A}$:

1. Interchange of two rows.
2. Multiplication of a row by a nonzero scalar.
3. Addition of a nonzero multiple of one row with another row.

These operations do not change the solution of the system. ◄

There are many possible "strategies" to perform elimination. The most commonly known strategy is *Gauss-Jordan elimination*.

EXAMPLE 3.6. Consider the system of equations

$$5x_1 + x_2 + 3x_3 = 9 \quad \text{(i)}$$
$$10x_1 + 5x_2 + 12x_3 = 27 \quad \text{(ii)}$$
$$5x_1 + 10x_2 + 23x_3 = 38 \quad \text{(iii)}$$

If we perform the operations (ii) $-$ 2(i) $\to$ (ii) and (iii) $-$ (i) $\to$ (iii), then the system becomes

$$\begin{aligned} 5x_1 + x_2 + 3x_3 &= 9 && \text{(i)} \\ 0x_1 + 3x_2 + 6x_3 &= 9 && \text{(ii)} \\ 0x_1 + 9x_2 + 20x_3 &= 29 && \text{(iii)} \end{aligned}$$

Then we combine and replace (i) $-\frac{1}{3}$(ii) $\to$ (i) and (iii) $-$ 3(ii) $\to$ (iii) to have

$$\begin{aligned} 5x_1 + 0x_2 + x_3 &= 6 && \text{(i)} \\ 0x_1 + 3x_2 + 6x_3 &= 9 && \text{(ii)} \\ 0x_1 + 0x_2 + 2x_3 &= 2 && \text{(iii)} \end{aligned}$$

Third, we calculate (i) $-\frac{1}{2}$(iii) $\to$ (i) and (ii) $-$ 3(iii) $\to$ (ii) to obtain the system now in diagonal form

$$\begin{aligned} 5x_1 + 0x_2 + 0x_3 &= 5 && \text{(i)} \\ 0x_1 + 3x_2 + 0x_3 &= 3 && \text{(ii)} \\ 0x_1 + 0x_2 + 2x_3 &= 2 && \text{(iii)} \end{aligned}$$

Finally, we divide by the appropriate constants $\frac{1}{5}$(i) $\to$ (i), $\frac{1}{3}$(ii) $\to$ (ii), and $\frac{1}{2}$(iii) $\to$ (iii) to get

$$\begin{aligned} x_1 + 0x_2 + 0x_3 &= 1 && \text{(i)} \\ 0x_1 + x_2 + 0x_3 &= 1 && \text{(ii)} \\ 0x_1 + 0x_2 + x_3 &= 1 && \text{(iii)} \end{aligned}$$

and determine that the solution is $\mathbf{x} = (1, 1, 1)^T$.

Next, we reinterpret the previous operations, but in the language of matrices. On the left hand side we have

$$\underbrace{\begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}}_{\mathbf{I}} = \underbrace{\begin{pmatrix} \frac{1}{5} & 0 & 0 \\ 0 & \frac{1}{3} & 0 \\ 0 & 0 & \frac{1}{2} \end{pmatrix} \begin{pmatrix} 1 & 0 & -\frac{1}{2} \\ 0 & 1 & -3 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & -\frac{1}{3} & 0 \\ 0 & 1 & 0 \\ 0 & -3 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 & 0 \\ -2 & 1 & 0 \\ -1 & 0 & 1 \end{pmatrix}}_{\mathbf{A}^{-1}} \underbrace{\begin{pmatrix} 5 & 1 & 3 \\ 10 & 5 & 12 \\ 5 & 10 & 23 \end{pmatrix}}_{\mathbf{A}}$$

Similarly, on the right hand side we have in matrix form

$$\underbrace{\begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix}}_{\mathbf{x}} = \underbrace{\begin{pmatrix} \frac{1}{5} & 0 & 0 \\ 0 & \frac{1}{3} & 0 \\ 0 & 0 & \frac{1}{2} \end{pmatrix} \begin{pmatrix} 1 & 0 & -\frac{1}{2} \\ 0 & 1 & -3 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & -\frac{1}{3} & 0 \\ 0 & 1 & 0 \\ 0 & -3 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 & 0 \\ -2 & 1 & 0 \\ -1 & 0 & 1 \end{pmatrix}}_{\mathbf{A}^{-1}} \underbrace{\begin{pmatrix} 9 \\ 27 \\ 38 \end{pmatrix}}_{\mathbf{b}}$$

To illustrate that we have computed the action of the inverse $\mathbf{A}^{-1}$ to solve the system but did not explicitly form its structure. ⧫

Remark 3.6. This reinforces the computational complexity discussion of explicitly forming the inverse and how expensive it can be. For this example, it relies on matrix-matrix multiplication. ⋈

Great! We can solve the system with Gauss-Jordan elimination (provided the matrix $\mathbf{A}$ is invertible), but can we do better? It turns out we do not need to transform the matrix into a diagonal form. It is *sufficient* to drive the system to become upper triangular. This will require fewer eliminations/operations.

Before we revisit the previous example through this new lens, we observe some important structure behind the algebra of Gauss-Jordan eliminations. To do so, we recast the elementary row operations into the

language of the canonical basis vectors $\mathbf{e}_i$, $i = 1, \ldots, n$. Note first that the identity matrix can be written in terms of the canonical basis vectors as

$$\mathbf{I} = \begin{pmatrix} \mathbf{e}_1^T \\ \mathbf{e}_2^T \\ \vdots \\ \mathbf{e}_n^T \end{pmatrix}$$

Another property of the canonical basis is that we can use one to extract a given row of the matrix $\mathbf{A}$, e.g. to extract the jth row of $\mathbf{A}$ we take $\mathbf{e}_j^T\mathbf{A}$. Now we will demonstrate how to compactly rewrite the elementary row operation of adding a nonzero multiple of one row with another row.

EXAMPLE 3.7. For the purpose of this example say $n = 4$. Then if we want to add four times the first row of $\mathbf{A}$ to its second row and put the result in the second row (while leaving all other rows untouched) we have

$$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 4 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \mathbf{A} = \begin{pmatrix} \mathbf{e}_1^T \\ \mathbf{e}_2^T + 4\mathbf{e}_1^T \\ \mathbf{e}_3^T \\ \mathbf{e}_4^T \end{pmatrix} \mathbf{A} = \left[\mathbf{I} + \begin{pmatrix} 0\mathbf{e}_1^T \\ 4\mathbf{e}_1^T \\ 0\mathbf{e}_1^T \\ 0\mathbf{e}_1^T \end{pmatrix} \right] \mathbf{A} = \left[\mathbf{I} + 4 \begin{pmatrix} 0 \\ 1 \\ 0 \\ 0 \end{pmatrix} \mathbf{e}_1^T \right] \mathbf{A} = \left[\mathbf{I} + 4\mathbf{e}_2\mathbf{e}_1^T \right] \mathbf{A}.$$

♦

This leads to a general form of this elementary row opeation:

DEFINITION 3.7 (GAUSS-JORDAN ELIMINATION MATRIX). If we want to add a nonzero scalar $\alpha \in \mathbb{R}$, $\alpha \neq 0$ times row j of a matrix $\mathbf{A}$ to row i and place the result in row i we use the expression

$$\left[\mathbf{I} + \alpha \mathbf{e}_i \mathbf{e}_j^T \right] \mathbf{A}.$$

Where the term in brackets is a type of *elementary matrix*. ◄

What does this new way of writing the elementary row operation give us? It turns out quite a lot! These elementary matrices satisfy two very useful properties. First, the inverse of these elementary matrices is

$$(\mathbf{I} + \alpha \mathbf{e}_i \mathbf{e}_j^T)^{-1} = \mathbf{I} - \alpha \mathbf{e}_i \mathbf{e}_j^T.$$

This comes directly from the orthogonality of the canonical basis. So, if $i \neq j$ then for a scalar α we have

$$(\mathbf{I} + \alpha \mathbf{e}_i \mathbf{e}_j^T)(\mathbf{I} - \alpha \mathbf{e}_i \mathbf{e}_j^T) = \mathbf{I} + \alpha \mathbf{e}_i \mathbf{e}_j^T - \alpha \mathbf{e}_i \mathbf{e}_j^T - \alpha^2 \mathbf{e}_i \mathbf{e}_j^T \mathbf{e}_i \mathbf{e}_j^T = \mathbf{I} - \alpha^2 \mathbf{e}_i (\mathbf{e}_j^T \mathbf{e}_i) \mathbf{e}_j^T = \mathbf{I}.$$

Second, if we perform two row operations back-to-back to different rows of $\mathbf{A}$ they do not influence one another. For example, if we add α times the first row to the second row and β times the first row to third row this gives

$$(\mathbf{I} + \alpha \mathbf{e}_2 \mathbf{e}_1^T)(\mathbf{I} + \beta \mathbf{e}_3 \mathbf{e}_1^T) = \mathbf{I} + \alpha \mathbf{e}_2 \mathbf{e}_1^T + \beta \mathbf{e}_3 \mathbf{e}_1^T + \alpha\beta \mathbf{e}_2 (\mathbf{e}_1^T \mathbf{e}_3) \mathbf{e}_1^T = \mathbf{I} + \alpha \mathbf{e}_2 \mathbf{e}_1^T + \beta \mathbf{e}_3 \mathbf{e}_1^T,$$

again due to the orthogonality of the basis vectors. So, the product of these elementary matrices just combines the nonzero terms.

EXAMPLE 3.8. Now, we revisit the Gauss-Jordan elimination example for the system

$$\begin{aligned} 5x_1 + x_2 + 3x_3 &= 9 \\ 10x_1 + 5x_2 + 12x_3 &= 27 \\ 5x_1 + 10x_2 + 23x_3 &= 38 \end{aligned} \quad \Rightarrow \quad \mathbf{A} = \begin{pmatrix} 5 & 1 & 3 \\ 10 & 5 & 12 \\ 5 & 10 & 23 \end{pmatrix}, \quad \mathbf{b} = \begin{pmatrix} 9 \\ 27 \\ 38 \end{pmatrix}.$$

As stated above, we will drive the matrix $\mathbf{A}$ to an upper triangular form as well as keep track of how the row operations affect the right hand side $\mathbf{b}$. The three row operations that transform $\mathbf{A}$ to an upper triangular form $\mathbf{U}$ are written in matrix form by

$$\mathbf{U} = \begin{pmatrix} 5 & 1 & 3 \\ 0 & 3 & 6 \\ 0 & 0 & 2 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & -3 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 & 0 \\ -2 & 1 & 0 \\ -1 & 0 & 1 \end{pmatrix} \begin{pmatrix} 5 & 1 & 3 \\ 10 & 5 & 12 \\ 5 & 10 & 23 \end{pmatrix} = \mathbf{M}_2\mathbf{M}_1\mathbf{A}$$

$$\mathbf{c} = \begin{pmatrix} 9 \\ 9 \\ 2 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & -3 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 & 0 \\ -2 & 1 & 0 \\ -1 & 0 & 1 \end{pmatrix} \begin{pmatrix} 9 \\ 27 \\ 38 \end{pmatrix} = \mathbf{M}_2\mathbf{M}_1\mathbf{b}.$$

We can use the elementary matrix notation to keep track of the row operations:

$$\mathbf{M}_1 = \begin{pmatrix} 1 & 0 & 0 \\ -2 & 1 & 0 \\ -1 & 0 & 1 \end{pmatrix} = \mathbf{I} - 2\mathbf{e}_2\mathbf{e}_1^T - \mathbf{e}_3\mathbf{e}_1^T = \left(\mathbf{I} - 2\mathbf{e}_2\mathbf{e}_1^T\right)\left(\mathbf{I} - \mathbf{e}_3\mathbf{e}_1^T\right), \quad \mathbf{M}_2 = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & -3 & 1 \end{pmatrix} = \mathbf{I} - 3\mathbf{e}_3\mathbf{e}_2^T$$

where we have used the fact that the multiplication of these elementary matrices just combine the nonzero terms below the diagonal.

This manipulation gives us an upper triangular system to solve

$$\mathbf{U} = \begin{pmatrix} 5 & 1 & 3 \\ 0 & 3 & 6 \\ 0 & 0 & 2 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} 9 \\ 9 \\ 2 \end{pmatrix} = \mathbf{c}.$$

This can be very efficiently solved with a **backward substitution** process. This begins by solving for the final component

$$x_3 = \frac{c_3}{u_{33}} = \frac{2}{2} = 1.$$

Then we work backwards through the vector of unknowns

$$x_2 = \frac{c_2 - u_{23}x_3}{u_{22}} = \frac{9 - (6)(1)}{3} = \frac{3}{3} = 1,$$

$$x_1 = \frac{c_1 - u_{12}x_2 - u_{13}x_3}{u_{11}} = \frac{9 - (1)(1) - (3)(1)}{5} = \frac{5}{5} = 1,$$

to complete the solution of the system. ♦

Generally, this gives a two step process to solve a linear system:

1. Drive the matrix to an upper triangular form with Gauss-Jordan elimination.
2. Solve the resulting upper triangular system with back substitution.

To reiterate, this produces the action of the matrix inverse without explicitly forming it. We generalize this back substitution process in the following pseudocode.

Algorithm 3.1: *Backward substitution*: Solve an upper triangular linear system

Procedure Backward Substitution
Input: **U**, **c**, n // `upper triangular square matrix, right hand side vector, and size`
$x_n \leftarrow c_n/u_{nn}$
for $i = n-1$ **to** 1 **step** -1 **do**
  for $j = i+1$ **to** n **do**
    $c_i \leftarrow c_i - u_{ij} * x_j$
  $x_i \leftarrow c_i/u_{ii}$
Output: `x` // `solution vector`
End Procedure Backward Substitution

What about the computational cost(s) of this new process? To drive the system into an upper triangular form requires $\frac{2}{3}n^3 + \mathcal{O}(n^2)$ operations. This estimate comes from the fact that the ith step of the Gauss-Jordan elimination requires (in the most expensive case) $(n-i)^2$ additions/subtractions, $(n-i)^2$ multiplications, and $(n-1)$ divisions. Note that the computational complexity still scales with the cube of the system size, but the coefficient is smaller than that for explicitly forming the inverse matrix. Solving the upper triangular system with a backward substitution (examining the pseudocode) requires:

$\underline{i = n}$**:** 1 multiplication/division, 0 addition/subtraction.

$\underline{i = n-1}$**:** 2 multiplications/divisions, 1 addition/subtraction.

$\underline{i = n-2}$**:** 3 multiplications/divisions, 2 additions/subtractions.

$\vdots$

$\underline{i = 2}$**:** $n-1$ multiplications/divisions, $n-2$ additions/subtractions.

$\underline{i = 1}$**:** n multiplications/divisions, $n-1$ additions/subtractions.

Thus, solving an $n \times n$ triangular linear system by back substitution requires $n^2 + \mathcal{O}(n)$ flops. So, for large system sizes the dominate amount of work is the computation to drive the system to upper triangular.

While we are discussing computational complexity, we return to the elementary matrix form and its structure because, from its construction, it is possible to save on computational and memory costs. A naive application of an elementary matrix to the right hand side vector **b** is a matrix-vector multiplication that we know has an $\mathcal{O}(n^2)$ cost in both computation and memory, i.e. it is a BLAS2 operation. However, we can *exploit the structure* to find

$$(\mathbf{I} + \alpha \mathbf{e}_i \mathbf{e}_j^T)\mathbf{b} = \mathbf{b} + \alpha \mathbf{e}_i (\mathbf{e}_j^T \mathbf{b}) = \mathbf{b} + \tilde{\alpha}\mathbf{e}_i = \tilde{\mathbf{b}}.$$

Here, we used a property of the identity matrix as well as grouped together the inner product of two vectors, which results in a scalar. The result was then combined into a new scalar value $\tilde{\alpha} = \alpha(\mathbf{e}_j^T \mathbf{b})$. We know that the inner product is an $\mathcal{O}(n)$ computation and the final step, $\mathbf{b} + \tilde{\alpha}\mathbf{e}_i$, is another n multiplications and n additions for the two vectors.

Therefore, the cost of applying an elementary matrix to a vector is reduced to an $\mathcal{O}(n)$ operation in terms of computation and memory. That is, exploiting the form on the elementary matrix transforms a BLAS2 operation **into** a BLAS1 operation! So, we see that knowledge of matrix structure and "clever" grouping of terms can save computation time and memory in practice.

Finally, we notice one more important detail and exploit the structure of the Gauss-Jordan elimination matrices (built from these elementary matrices):

EXAMPLE 3.9. We return to the previous example where we applied Gauss-Jordan eliminations to create an upper triangular system for $n = 3$:

$$\mathbf{U} = \begin{pmatrix} 5 & 1 & 3 \\ 0 & 3 & 6 \\ 0 & 0 & 2 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & -3 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 & 0 \\ -2 & 1 & 0 \\ -1 & 0 & 1 \end{pmatrix} \begin{pmatrix} 5 & 1 & 3 \\ 10 & 5 & 12 \\ 5 & 10 & 23 \end{pmatrix} = \mathbf{M}_2\mathbf{M}_1\mathbf{A}.$$

These $\mathbf{M}_i$, $i = 1, 2$ matrices are built from and have structure similar to the elementary matrices. That is, it is straightforward to compute their inverse because we just ***swap the signs*** of any values below the main diagonal:

$$\mathbf{M}_1^{-1} = \begin{pmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ 1 & 0 & 1 \end{pmatrix}, \quad \mathbf{M}_2^{-1} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 3 & 1 \end{pmatrix}.$$

Further, if we multiply these two matrices together it remains a lower triangular matrix (with ones along the main diagonal)

$$\mathbf{L} = \mathbf{M}_1^{-1}\mathbf{M}_2^{-1} = \begin{pmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ 1 & 3 & 1 \end{pmatrix}.$$

Therefore, we have an equivalent form for the original matrix $\mathbf{A}$ as the product of a lower triangular matrix and an upper triangular matrix, i.e.

$$\mathbf{LU} = \begin{pmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ 1 & 3 & 1 \end{pmatrix} \begin{pmatrix} 5 & 1 & 3 \\ 0 & 3 & 6 \\ 0 & 0 & 2 \end{pmatrix} = \begin{pmatrix} 5 & 1 & 3 \\ 10 & 5 & 12 \\ 5 & 10 & 23 \end{pmatrix} = \mathbf{A}.$$

This is a very useful ***factorization*** of the system. ♦

This general strategy of decomposing the matrix $\mathbf{A}$ into its upper and lower triangular parts will be expanded upon in the next note set. In particular, we will address how to exploit this upper/lower structure to create the action of the matrix inverse as well as what to do if the Gauss-Jordan elimination produces a zero value along the main diagonal (something that did not occur in our examples here).

Note Set 4: Solving linear systems: LU and iterative methods

We expand upon the work from the previous note set and go into detail for numerical methods to solve a square linear system of equations $\mathbf{Ax} = \mathbf{b}$. In the first half of this note set, we focus on a factorization technique that comes from our knowledge of Gauss-Jordan elimination. This involves working with triangular systems and then solving for the vectors of unknowns. This is a *direct* method to numerically solve the system. Next, we work with alternative *iterative* techniques to approximate the solution vector. To do so, we revisit the concept of fixed point iterations from Note Set 2. However, there are differences which we clarify such as what a contraction mapping means in the context of a linear system of equations.

4.1 LU factorization

From last time, we discussed a solution approach where we applied subsequent Gauss-Jordan eliminations and drove the system to an upper triangular form. The upper triangular system was then efficiently solved with a backward substitution strategy. At the end of the last note set we saw that the combined action of all the Gauss-Jordan eliminations was a *unit lower triangular matrix*. In other words, the values above the main diagonal are zero, the values along the main diagonal are all one, and the values below the main diagonal can be nonzero. In turn, this allowed us to write the matrix $\mathbf{A}$ as the multiplication of this unit lower triangular matrix and an upper triangular matrix

$$\mathbf{A} = \begin{pmatrix} 1 & 0 & \cdots & 0 \\ l_{21} & 1 & \ddots & \vdots \\ \vdots & & \ddots & 0 \\ l_{n1} & \cdots & l_{n,n-1} & 1 \end{pmatrix} \begin{pmatrix} u_{11} & u_{12} & \cdots & u_{1n} \\ 0 & u_{22} & \cdots & u_{2n} \\ \vdots & \ddots & \ddots & \vdots \\ 0 & \cdots & 0 & u_{nn} \end{pmatrix} = \mathbf{LU}.$$

This is a type of *factorization* strategy where we rewrite the matrix $\mathbf{A}$ into a form that is easier to solve. In particular, this is known as the **LU factorization** (also sometimes called the LU decomposition).

The LU factorization is a powerful technique to directly solve square linear systems of equations. However, we need to dig into the details to ensure the factorization and subsequent solution of the systems is reliable.

Recall that we interpreted Gauss-Jordan eliminations into a useful *elementary matrix* form. These matrices were particularly *easy to invert* and we saw that new transformations did not affect the previous ones:

Example 4.1. Let $n = 4$ and say that a transformation $\mathbf{M}_1$ has already zeroed out the first column of $\mathbf{A}$ below the main diagonal. If we apply a second transformation $\mathbf{M}_2$ we have the structure

$$\mathbf{M}_2\mathbf{M}_1\mathbf{A} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & -l_{32} & 1 & 0 \\ 0 & -l_{42} & 0 & 1 \end{pmatrix} \begin{pmatrix} * & * & * & * \\ 0 & * & * & * \\ 0 & * & * & * \\ 0 & * & * & * \end{pmatrix} = \begin{pmatrix} * & * & * & * \\ 0 & * & * & * \\ 0 & 0 & *' & *' \\ 0 & 0 & *' & *' \end{pmatrix},$$

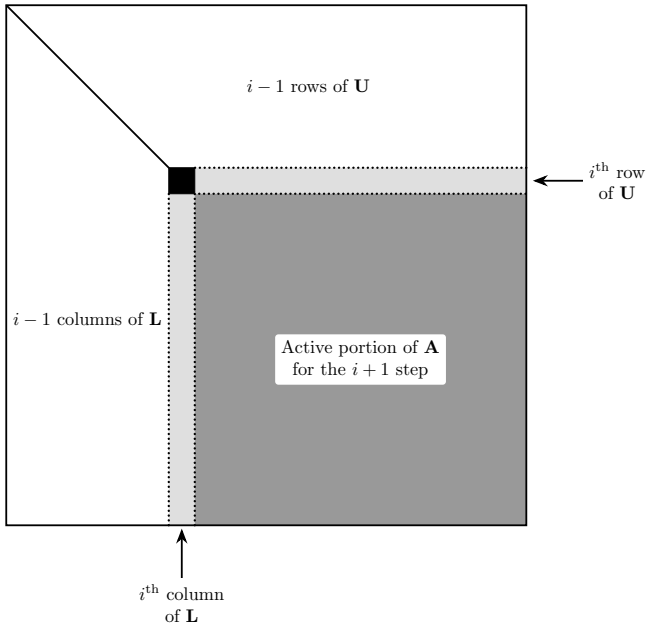
where the $*$ are placeholders for arbitrary values. We see that the application of the transformation $\mathbf{M}_2$ to $\mathbf{A}$ **does not** destroy the 0 elements a previous transformation $\mathbf{M}_1$ introduced in the first column. Also, the inverse of the transformation $\mathbf{M}_2$ is

$$\mathbf{M}_2^{-1} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & l_{32} & 1 & 0 \\ 0 & l_{42} & 0 & 1 \end{pmatrix},$$

where we simply flip the signs of the values below the main diagonal. ♦

This example reveals something else important: There is a "static" part of the matrix that remains unchanged by new transformations and an "active" part of the matrix where values are updated/changed. This is excellent news because we can exploit this structure to save on memory and create the **LU** factorization *in place*. Therefore, the storage complexity remains $\mathcal{O}(n^2)$, but we will not store any unnecessary zero values. Before we walk through an example of this in place strategy to factorize **A**, we give a visual representation (at the right) of what the factorization strategy produces during the ith step:

- Column i of the unit lower triangular matrix **L**.
- Row i of the upper triangular matrix **U**.
- Updates to the remaining "active" portion of **A** that has dimension $(n-i) \times (n-i)$.



Keep in mind that the ith step *does not* change the already created $i-1$ columns of **L** and $i-1$ rows of **U**. Also, implicit to this storage strategy is that the matrix **L** has ones along its main diagonal. So, we don't actually store them, but we must keep in mind when we go to solve the system.

This introduces an important distinction when working with matrices and other linear algebra objects in computational methods. A matrix is a *mathematical object* whereas storing the factorization of a matrix, such as the in place **LU**, is done in an array which is a *data structure*. We will distinguish between the two as we work through an example.

Remark 4.1. For examples throughout this note set, we will highlight within the array structure the new rows of **U** in pink, the new columns of **L** in blue, and the active part of **A** in orange. ⋈

EXAMPLE 4.2. Form the in place **LU** factorization of the matrix **A** where $n = 4$. We initialize the array with the same values as those in the matrix **A**:

$$\mathbf{A} = \begin{pmatrix} 5 & 1 & 3 & 1 \\ 10 & 5 & 12 & 3 \\ 5 & 10 & 23 & 5 \\ 15 & 6 & 19 & 7 \end{pmatrix}, \qquad \texttt{array} = \begin{bmatrix} 5 & 1 & 3 & 1 \\ 10 & 5 & 12 & 3 \\ 5 & 10 & 23 & 5 \\ 15 & 6 & 19 & 7 \end{bmatrix}.$$

Now, we apply the first set of Gauss-Jordan eliminations to zero out the first column

$$\begin{pmatrix} 5 & 1 & 3 & 1 \\ 0 & 3 & 6 & 1 \\ 0 & 9 & 20 & 4 \\ 0 & 3 & 10 & 4 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ -2 & 1 & 0 & 0 \\ -1 & 0 & 1 & 0 \\ -3 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 5 & 1 & 3 & 1 \\ 10 & 5 & 12 & 3 \\ 5 & 10 & 23 & 5 \\ 15 & 6 & 19 & 7 \end{pmatrix},$$

$$\texttt{array} = \begin{bmatrix} 5 & 1 & 3 & 1 \\ 10 & 5 & 12 & 3 \\ 5 & 10 & 23 & 5 \\ 15 & 6 & 19 & 7 \end{bmatrix} \quad \Rightarrow \quad \texttt{array} = \begin{bmatrix} 5 & 1 & 3 & 1 \\ 2 & 3 & 6 & 1 \\ 1 & 9 & 20 & 4 \\ 3 & 3 & 10 & 4 \end{bmatrix}.$$

Note that we flip the signs when placing the values into the array structure, which is effectively taking the "inverse" of the Gauss-Jordan elimination matrices. For the second step we zero out the second column and

update the remaining portion of $\mathbf{A}$

$$\begin{pmatrix} 5 & 1 & 3 & 1 \\ 0 & 3 & 6 & 1 \\ 0 & 0 & 2 & 1 \\ 0 & 0 & 4 & 3 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & -3 & 1 & 0 \\ 0 & -1 & 0 & 1 \end{pmatrix} \begin{pmatrix} 5 & 1 & 3 & 1 \\ 0 & 3 & 6 & 1 \\ 0 & 9 & 20 & 4 \\ 0 & 3 & 10 & 4 \end{pmatrix},$$

$$\texttt{array} = \begin{bmatrix} 5 & 1 & 3 & 1 \\ 2 & 3 & 6 & 1 \\ 1 & 9 & 20 & 4 \\ 3 & 3 & 10 & 4 \end{bmatrix} \quad \Rightarrow \quad \texttt{array} = \begin{bmatrix} 5 & 1 & 3 & 1 \\ 2 & 3 & 6 & 1 \\ 1 & 3 & 2 & 1 \\ 3 & 1 & 4 & 3 \end{bmatrix}.$$

We see that the first row of $\mathbf{U}$ and the first column of $\mathbf{L}$ are unaffected by this new update. Finally, we perform one more Gauss-Jordan elimination to complete the $\mathbf{LU}$ factorization

$$\begin{pmatrix} 5 & 1 & 3 & 1 \\ 0 & 3 & 6 & 1 \\ 0 & 0 & 2 & 1 \\ 0 & 0 & 0 & 1 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & -2 & 1 \end{pmatrix} \begin{pmatrix} 5 & 1 & 3 & 1 \\ 0 & 3 & 6 & 1 \\ 0 & 0 & 2 & 1 \\ 0 & 0 & 4 & 3 \end{pmatrix},$$

$$\texttt{array} = \begin{bmatrix} 5 & 1 & 3 & 1 \\ 2 & 3 & 6 & 1 \\ 1 & 3 & 2 & 1 \\ 3 & 1 & 4 & 3 \end{bmatrix} \quad \Rightarrow \quad \texttt{array} = \begin{bmatrix} 5 & 1 & 3 & 1 \\ 2 & 3 & 6 & 1 \\ 1 & 3 & 2 & 1 \\ 3 & 1 & 2 & 1 \end{bmatrix}.$$

Now the array above compactly stores the $\mathbf{LU}$ decomposition

$$\mathbf{A} = \begin{pmatrix} 5 & 1 & 3 & 1 \\ 10 & 5 & 12 & 3 \\ 5 & 10 & 23 & 5 \\ 15 & 6 & 19 & 7 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 2 & 1 & 0 & 0 \\ 1 & 3 & 1 & 0 \\ 3 & 1 & 2 & 1 \end{pmatrix} \begin{pmatrix} 5 & 1 & 3 & 1 \\ 0 & 3 & 6 & 1 \\ 0 & 0 & 2 & 1 \\ 0 & 0 & 0 & 1 \end{pmatrix} = \mathbf{LU}$$

of the matrix $\mathbf{A}$. ♦

Now we have a strategy to obtain and compactly store the $\mathbf{LU}$ factorization of the matrix $\mathbf{A}$. How do we actually solve the system? Recall we used back substitution to solve the upper triangular system $\mathbf{Ux} = \mathbf{c}$. But where does the vector $\mathbf{c}$ come from in this context? Through some algebraic manipulation we find that this is where the matrix $\mathbf{L}$ comes into play:

$$\mathbf{Ax} = \mathbf{b} \quad \Rightarrow \quad \mathbf{LUx} = \mathbf{b} \quad \Rightarrow \quad \begin{matrix} \mathbf{Lc} = \mathbf{b} \\ \mathbf{Ux} = \mathbf{c} \end{matrix}$$

So, the computation of the solution vector $\mathbf{x}$ comes from a three-step process:

1. Create the $\mathbf{LU}$ factorization of $\mathbf{A}$.
2. Compute the intermediate vector $\mathbf{c}$ by solving a lower triangular system using *forward substitution*.
3. The vector $\mathbf{c}$ becomes the right hand side for the previously discussed *backward substitution* strategy.

The forward substitution technique is very similar to backward substitution apart from the direction in which direction we iterate through the information. Provided the main diagonal entries of the matrix $\mathbf{L}$ are nonzero (as is the case for the $\mathbf{LU}$ factorization) we can compute the solution for $\mathbf{Lc} = \mathbf{b}$ as

$$c_1 = \frac{b_1}{l_{11}}, \qquad c_i = \frac{1}{l_{ii}} \left(b_i - \sum_{j=1}^{i-1} l_{ij} c_j \right), \; i = 2, \ldots, n.$$

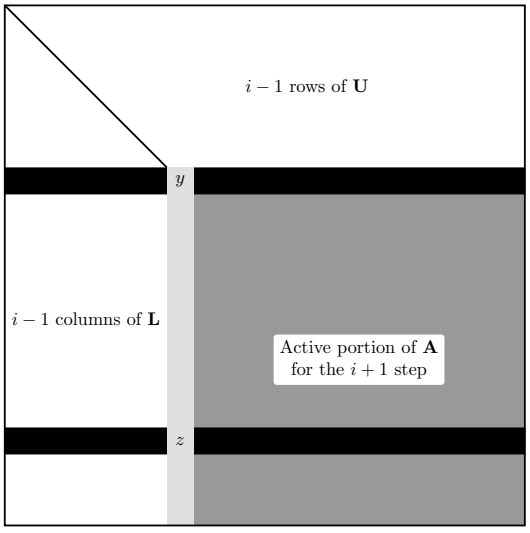
The computational complexity for the forward substitution is $\mathcal{O}(n^2)$ (similar to the backward substitution). Therefore, the overall complexity of the algorithm is dominated by the $\frac{2}{3}n^3 + \mathcal{O}(n^2)$ work necessary to create the **LU** factorization. Pseudocode for the forward substitution process is found below.

Algorithm 4.1: *Forward substitution*: Solve a lower triangular linear system

Procedure Forward Substitution
Input: **L**, **b**, n `// lower triangular square matrix, right hand side vector, and size`
$c_1 \leftarrow b_1/l_{11}$
for $i = 2$ **to** n **do**
  for $j = 1$ **to** $i - 1$ **do**
    $b_i \leftarrow b_i - l_{ij} * c_j$
  $c_i \leftarrow b_i/l_{ii}$
Output: **c** `// solution vector`
End Procedure Forward Substitution

Remark 4.2. This pseudocode is written for a general lower triangular system. For the **LU** factorization we know that the matrix **L** is unit lower triangular. Therefore, we do not need to perform the division of the l_{ii} elements, which decreases computational complexity. ⋈

Example 4.3. Continuing the previous example we solve the system with the right hand side $\mathbf{b} = (30, 82, 101, 123)^T$ such that the forward substitution yields the intermediate vector $\mathbf{c} = (30, 22, 5, 1)^T$ and the backward substitution gives the solution vector $\mathbf{x} = (4, 3, 2, 1)^T$. ⧫



Next, we address the stability of the **LU** factorization. In the examples considered so far the algorithm never "broke," but what happens if the entry along the main diagonal becomes zero (or close to it)? Then dividing by a small number in the Gauss-Jordan elimination becomes a very large number and we have seen that this can lead to accuracy issues when working in finite precision. We will denote the current element that will divide in the Gauss-Jordan elimination as the **pivot element**. To avoid these issues of dividing by a small pivot element we think back to the possible elementary row operations and recall that we are allowed to exchange rows of the matrix *without* changing the solution.

Definition 4.1 (Pivoting). The interchanging of rows in order to determine a suitably large nonzero pivot element is called **pivoting**. In general, pivoting is necessary to guarantee existence and numerical stability of the **LU** factorization. ◀

There are many strategies to select the pivot element. We choose a strategy referred to as ***row pivoting*** where we search the first column of the active sub-matrix and select the element of *maximum magnitude*. We illustrate this process on the left where we exchange the current row that has leading element y with the row that has leading element z with the largest magnitude throughout the whole column.

Remark 4.3. Note that partial row pivoting will also exchange rows of the lower triangular matrix **L**. ⋈

Remark 4.4. One consequence of partial row pivoting is that all the off-diagonal elements in the unit lower triangular matrix **L** will have a magnitude less than one. ⋈

EXAMPLE 4.4. To demonstrate the necessity of pivoting consider a 2×2 linear system where

$$\mathbf{A} = \begin{pmatrix} -10^{-9} & 1 \\ 1 & -1 \end{pmatrix}, \quad \mathbf{b} = \begin{pmatrix} 1 - 10^{-9} \\ 0 \end{pmatrix}.$$

Without pivoting we use elimination to find

$$\begin{pmatrix} -10^{-9} & 1 \\ 0 & -1 + 10^{9} \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} 1 - 10^{-9} \\ -1 + 10^{9} \end{pmatrix} \quad \Rightarrow \quad \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} \frac{(1-10^{-9})-1}{-10^{-9}} \\ 1 \end{pmatrix}.$$

The calculation of x_1 is ill-conditioned because it involves the subtraction of nearby numbers and accuracy will be lost in finite precision. However, if we pivot the two rows such that the largest value in the first column is on the main diagonal *and then* preform elimination we see

$$\begin{pmatrix} 1 & -1 \\ -10^{-9} & 1 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} 0 \\ 1 - 10^{-9} \end{pmatrix} \quad \Rightarrow \quad \begin{pmatrix} 1 & -1 \\ 0 & 1 - 10^{-9} \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} 0 \\ 1 - 10^{-9} \end{pmatrix}$$

Now the computation of the solution is well-conditioned and we correctly find that $\mathbf{x} = (1, 1)^T$. ♦

DEFINITION 4.2 (PERMUTATION MATRIX). We represent the exchange of two rows of a matrix $\mathbf{A}$ using a **permutation matrix** denoted as $\mathbf{P}$. It is essentially a scrambled version of the identity matrix. For example, take $n = 3$ then the permutation of $\mathbf{A}$

$$\mathbf{PA} = \begin{pmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \end{pmatrix} \begin{pmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{pmatrix} = \begin{pmatrix} a_{31} & a_{32} & a_{33} \\ a_{21} & a_{22} & a_{23} \\ a_{11} & a_{12} & a_{13} \end{pmatrix},$$

exchanges the first and third rows. ◀

Remark 4.5. Permutation matrices are a type of *orthogonal* matrix. One consequence of this is that they are trivial to invert because $\mathbf{P}^{-1} = \mathbf{P}^T$. ⋈

DEFINITION 4.3 (LU WITH PARTIAL ROW PIVOTING). If the matrix $\mathbf{A}$ is nonsingular then there exists a product of row permutations $\mathbf{P} = \mathbf{P}_{n-1} \cdots \mathbf{P}_1$, a unit lower triangular matrix $\mathbf{L}$ and an upper triangular matrix $\mathbf{U}$ such that

$$\mathbf{PA} = \mathbf{LU}.$$

That is, partial row pivoting is equivalent to computing the $\mathbf{LU}$ factorization of a permuted version of the original system $\mathbf{A}$. ◀

Remark 4.6. This only alters the solution strategy slightly. We simply must keep track of the permutations that occur because we must apply them to the right hand side as well to ensure the correct solution is computed:

$$\mathbf{Ax} = \mathbf{b} \quad \Rightarrow \quad \mathbf{PAx} = \mathbf{Pb} \quad \Rightarrow \quad \mathbf{LUx} = \widetilde{\mathbf{b}} \quad \Rightarrow \quad \begin{matrix} \mathbf{Lc} = \widetilde{\mathbf{b}} \\ \mathbf{Ux} = \mathbf{c} \end{matrix}$$

where we, again, apply a forward/backward substitution strategy to compute $\mathbf{x}$. ⋈

EXAMPLE 4.5. Use partial row pivoting and find the $\mathbf{LU}$ factorization of the matrix $\mathbf{A}$ and store the decomposition in place within an array. For the first step we pivot the first and third rows such that the pivot element is maximal and then perform Gauss-Jordan elimination

$$\mathbf{A} = \begin{pmatrix} 3 & 17 & 10 \\ 2 & 4 & -2 \\ 6 & 18 & -12 \end{pmatrix}, \quad \mathbf{P}_1 = \begin{pmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \end{pmatrix} \quad \Rightarrow \quad \begin{pmatrix} 6 & 18 & -12 \\ 0 & -2 & 2 \\ 0 & 8 & 16 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 \\ -\frac{1}{3} & 1 & 0 \\ -\frac{1}{2} & 0 & 1 \end{pmatrix} \begin{pmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \end{pmatrix} \begin{pmatrix} 3 & 17 & 10 \\ 2 & 4 & -2 \\ 6 & 18 & -12 \end{pmatrix}.$$

For the array storing the factorization in place we, as before, start by saving the original values and order of $\mathbf{A}$ in the array. We then permute the rows, compute the first column of the matrix $\mathbf{L}$ and apply the elimination to the "active" 2×2 portion of the matrix:

$$\texttt{array} = \begin{bmatrix} 3 & 17 & 10 \\ 2 & 4 & -2 \\ 6 & 18 & -12 \end{bmatrix} \overset{\text{permute}}{\Rightarrow} \begin{bmatrix} 6 & 18 & -12 \\ 2 & 4 & -2 \\ 3 & 17 & 10 \end{bmatrix} \overset{\text{get } \mathbf{L}_1}{\Rightarrow} \begin{bmatrix} 6 & 18 & -12 \\ \frac{1}{3} & 4 & -2 \\ \frac{1}{2} & 17 & 10 \end{bmatrix} \overset{\text{eliminate}}{\Rightarrow} \begin{bmatrix} 6 & 18 & -12 \\ \frac{1}{3} & -2 & 2 \\ \frac{1}{2} & 8 & 16 \end{bmatrix}.$$

The second step also requires a pivot to swap the second and third rows and we find

$$\begin{pmatrix} 6 & 18 & -12 \\ 0 & -2 & 2 \\ 0 & 8 & 16 \end{pmatrix}, \quad \mathbf{P}_2 = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \end{pmatrix} \quad \Rightarrow \quad \begin{pmatrix} 6 & 18 & -12 \\ 0 & 8 & 16 \\ 0 & 0 & 6 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & \frac{1}{4} & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \end{pmatrix} \begin{pmatrix} 6 & 18 & -12 \\ 0 & -2 & 2 \\ 0 & 8 & 16 \end{pmatrix}.$$

To update the in place storage of the factorization we have the steps

$$\texttt{array} = \begin{bmatrix} 6 & 18 & -12 \\ \frac{1}{3} & -2 & 2 \\ \frac{1}{2} & 8 & 16 \end{bmatrix} \overset{\text{permute}}{\Rightarrow} \begin{bmatrix} 6 & 18 & -12 \\ \frac{1}{2} & 8 & 16 \\ \frac{1}{3} & -2 & 2 \end{bmatrix} \overset{\text{get } \mathbf{L}_2}{\Rightarrow} \begin{bmatrix} 6 & 18 & -12 \\ \frac{1}{2} & 8 & 16 \\ \frac{1}{3} & -\frac{1}{4} & 2 \end{bmatrix} \overset{\text{eliminate}}{\Rightarrow} \begin{bmatrix} 6 & 18 & -12 \\ \frac{1}{2} & 8 & 16 \\ \frac{1}{3} & -\frac{1}{4} & 6 \end{bmatrix}.$$

For verification, we have the total permutation matrix

$$\mathbf{P} = \mathbf{P}_2\mathbf{P}_1 = \begin{pmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{pmatrix},$$

and we see that the factorization gives

$$\mathbf{PA} = \begin{pmatrix} 6 & 18 & -12 \\ 3 & 17 & 10 \\ 2 & 4 & -2 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 \\ \frac{1}{2} & 1 & 0 \\ \frac{1}{3} & -\frac{1}{4} & 1 \end{pmatrix} \begin{pmatrix} 6 & 18 & -12 \\ 0 & 8 & 16 \\ 0 & 0 & 6 \end{pmatrix} = \mathbf{LU},$$

as expected. ♦

We close our discussion on the **LU** factorization with a special case of the decomposition for a *symmetric positive definite (s.p.d)* matrix. A matrix is s.p.d. provided $\mathbf{x}^T\mathbf{A}\mathbf{x} > 0$ for any vector $\mathbf{x} \neq 0$. Symmetry ($\mathbf{A} = \mathbf{A}^T$) is easy to check, but positive definiteness is more subtle. A sufficient condition is that all the diagonal elements of $\mathbf{A}$ are positive and the largest element *is on* the diagonal

$$a_{kk} > 0, \quad k = 1, \ldots, n \qquad \text{and} \qquad \max_{i,j} |a_{ij}| = \max_k a_{kk}.$$

Remark 4.7. A *necessary and sufficient* condition for a matrix $\mathbf{A}$ to be s.p.d. is that all of the eigenvalues of the matrix are greater than zero. ⋈

Definition 4.4 (Cholesky factorization). Every symmetric positive definite matrix $\mathbf{A}$ can be factored symmetrically and *without* pivoting as $\mathbf{A} = \mathbf{C}\mathbf{C}^T$ where $\mathbf{C}$ is a lower triangular matrix. ◀

Remark 4.8. Because of the symmetry of $\mathbf{A}$ the computational complexity to compute the Cholesky factorization is halved, meaning it costs $\frac{1}{3}n^3 + \mathcal{O}(n^2)$.

Example 4.6. The matrix

$$\mathbf{A} = \begin{pmatrix} 25 & 15 & -5 \\ 15 & 18 & 0 \\ -5 & 0 & 11 \end{pmatrix}.$$

is clearly symmetric and satisfies the sufficient condition for positive definiteness. Therefore, applying the **LU** factorization technique to this matrix yields

$$\begin{pmatrix} 25 & 15 & -5 \\ 15 & 18 & 0 \\ -5 & 0 & 11 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 \\ \frac{3}{5} & 1 & 0 \\ -\frac{1}{5} & \frac{1}{3} & 1 \end{pmatrix} \begin{pmatrix} 25 & 15 & -5 \\ 0 & 9 & 3 \\ 0 & 0 & 9 \end{pmatrix} \quad \Rightarrow \quad \begin{pmatrix} 25 & 15 & -5 \\ 15 & 18 & 0 \\ -5 & 0 & 11 \end{pmatrix} = \begin{pmatrix} 5 & 0 & 0 \\ 3 & 3 & 0 \\ -1 & 1 & 3 \end{pmatrix} \begin{pmatrix} 5 & 3 & -1 \\ 0 & 3 & 1 \\ 0 & 0 & 3 \end{pmatrix},$$

which is the Cholesky decomposition $\mathbf{A} = \mathbf{C}\mathbf{C}^T$ because $\mathbf{A}$ is a s.p.d. matrix. ♦

4.2 Stationary iterative methods

The **LU** factorization is a powerful, general tool to solve linear systems. However, sometimes the matrix is too large to be stored in the computer memory making such a direct method difficult to use. Even more importantly, the computational cost of $\frac{2}{3}n^3$ for Gauss-Jordan elimination to create the **L** and **U** matrices is too expensive for such large systems. Instead, we *return* to iterative solution techniques that can reduce the computational cost to $\mathcal{O}(n^2)$. This is done by creating a solution algorithm that only requires matrix-vector multiplication (or something equivalently expensive) as well as vector addition/subtraction. Therefore, iterative methods is part of the BLAS2 hierarchy whereas factorization methods, like **LU**, are in BLAS3.

Recall that iterative methods to solve a problem repeat a task until the solution is deemed "good enough" in some error tolerance. So, we must again discuss the questions:

1. How do we know when an iterative technique will *converge*?
2. What is a good *initial guess* to kick start the iteration?
3. What is an appropriate condition to *stop* the iteration?

A strategy we explored previously for nonlinear equations of a single variable to create an iterative solution process was the fixed point iteration. An analogue of this idea to linear systems is the creation of a *stationary iterative method*. Again, we want to solve the problem $\mathbf{A}\mathbf{x} = \mathbf{b}$ and assume that the matrix $\mathbf{A}$ is nonsingular.

Definition 4.5 (Stationary Iterative Method). Let $\mathbf{T} \in \mathbb{R}^{n\times n}, \mathbf{c} \in \mathbb{R}^n$, and an initial guess $\mathbf{x}^{(0)} \in \mathbb{R}^n$ all be given. Then the operator $\mathbf{T}$ and vector $\mathbf{c}$ define a linear stationary method

$$\mathbf{x}^{(k+1)} = \mathbf{T}\mathbf{x}^{(k)} + \mathbf{c},$$

where $\mathbf{x}^{(0)}$ is the initial condition that defines a particular sequence of solutions. ◀

To define the pieces of the stationary iterative technique above we split the nonsingular matrix $\mathbf{A}$ into two matrices

$$\mathbf{A} = \mathbf{M} - \mathbf{N},$$

where $\mathbf{M}$ is also nonsingular (i.e. invertible) and $\mathbf{N}$ is an $n \times n$ matrix (the "leftovers"). Then our problem of interest is rewritten to become

$$\mathbf{A}\mathbf{x} = \mathbf{b} \quad \Rightarrow \quad \mathbf{M}\mathbf{x} = \mathbf{N}\mathbf{x} + \mathbf{b} \quad \Rightarrow \quad \mathbf{x} = \mathbf{M}^{-1}\mathbf{N}\mathbf{x} + \mathbf{M}^{-1}\mathbf{b}.$$

Thus, the *iteration matrix* for this splitting approach is $\mathbf{T} = \mathbf{M}^{-1}\mathbf{N}$ and $\mathbf{c} = \mathbf{M}^{-1}\mathbf{b}$.

Example 4.7. Show that the solution $\mathbf{x} = \mathbf{A}^{-1}\mathbf{b}$ is a fixed point of the stationary iterative method defined by the matrix splitting $\mathbf{A} = \mathbf{M} - \mathbf{N}$. We examine the right hand side of the equation to find

$$\begin{aligned} \mathbf{T}\mathbf{x} + \mathbf{c} = \mathbf{M}^{-1}\mathbf{N}\mathbf{x} + \mathbf{M}^{-1}\mathbf{b} &= \mathbf{M}^{-1}\mathbf{N}\mathbf{A}^{-1}\mathbf{b} + \mathbf{M}^{-1}\mathbf{b} \\ &= \mathbf{M}^{-1}(\mathbf{M} - \mathbf{A})\mathbf{A}^{-1}\mathbf{b} + \mathbf{M}^{-1}\mathbf{b} \\ &= \mathbf{A}^{-1}\mathbf{b} - \mathbf{M}^{-1}\mathbf{b} + \mathbf{M}^{-1}\mathbf{b} \\ &= \mathbf{A}^{-1}\mathbf{b} \\ &= \mathbf{x}. \end{aligned}$$

♦

Remark 4.9. An equivalent way to write the stationary iterative method is in terms of the *residual* in the kth iteration, $\mathbf{r}^{(k)} = \mathbf{b} - \mathbf{A}\mathbf{x}^{(k)}$, where

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \mathbf{M}^{-1}\mathbf{r}^{(k)}.$$

This is often referred to as the *preconditioner* form. Heuristically, this is because we select the matrix $\mathbf{M}$ such that $\mathbf{M}^{-1} \approx \mathbf{A}^{-1}$ albeit significantly cheaper to invert. ⋈

It is important to note that the components of the iterative technique, $\mathbf{T}$ and $\mathbf{c}$, ***do not*** depend on the iteration! This is a hallmark of stationary iterative techniques. Broadly, these methods (either written in the form from Definition 4.5 or in residual form) belong to the family of *Richardson* stationary methods.

Now that we have defined a fixed point iteration for the solution of a linear system, when will it work? To find out, we develop a condition to determine if the method will converge. As such, we require an equation for the error in a given iteration k, where

$$\mathbf{x}^{(k)} = \mathbf{T}\mathbf{x}^{(k-1)} + \mathbf{c}.$$

If we subtract the exact solution $\mathbf{x}$, which also satisfies the fixed point equation, we have

$$\boldsymbol{\epsilon}^{(k)} = \mathbf{x}^{(k)} - \mathbf{x} = \mathbf{T}(\mathbf{x}^{(k-1)} - \mathbf{x}) = \mathbf{T}\boldsymbol{\epsilon}^{(k-1)}$$

where the vector $\mathbf{c}$ cancels due to its independence from the iteration. Next, we use a suitable vector norm to measure the magnitude of each side of the equation. Further, we apply the fact that the error is recursive to see

$$\|\boldsymbol{\epsilon}^{(k)}\| = \|\mathbf{T}\boldsymbol{\epsilon}^{(k-1)}\| = \|\mathbf{T}^2\boldsymbol{\epsilon}^{(k-2)}\| = \cdots = \|\mathbf{T}^k\boldsymbol{\epsilon}^{(0)}\|$$

such that the error of the kth iteration depends on a power of the iteration matrix $\mathbf{T}$ and the initial error $\boldsymbol{\epsilon}^{(0)} = \mathbf{x}^{(0)} - \mathbf{x}$. Now, we apply the subordinate and submultiplicative properties of the induced matrix norm to find

$$\|\boldsymbol{\epsilon}^{(k)}\| = \|\mathbf{T}^k\boldsymbol{\epsilon}^{(0)}\| \leq \|\mathbf{T}^k\|\|\boldsymbol{\epsilon}^{(0)}\| \leq \|\mathbf{T}\|^k\|\boldsymbol{\epsilon}^{(0)}\|,$$

where the error in the stationary iteration goes to zero provided $\|\mathbf{T}\|^k \to 0$ as $k \to \infty$.

Definition 4.6 (Sufficient condition for convergence). If the norm of the iteration matrix $\mathbf{T}$ is less than unity, $\|\mathbf{T}\| < 1$, for a given nonsingular matrix $\mathbf{A}$, then the stationary iterative method converges. ◀

Remark 4.10. Note that this condition on the "magnitude" of the iteration matrix $\mathbf{T}$ is *reminiscent* of the definition for a contraction mapping in a fixed point iterative method from Note Set 2. ⋈

Remark 4.11. Keep in mind that the sufficient condition $\|\mathbf{T}\| < 1$ is rather vague because it does not specify the matrix norm we should use. However, as long as we satisfy this inequality in *some* matrix norm we know that the stationary iteration will converge, although this convergence can be quite slow. ⋈

Remark 4.12. In this course, we will only use this **sufficient** criterion to discuss convergence of a stationary iterative method. However, more rigorous conditions on the iteration matrix exist that are *necessary and sufficient* that rely on examining the eigenvalues of the matrix $\mathbf{T}$. ⋈

Next, we examine the computational cost of the stationary iterative method. For this, the matrix splitting construct is particularly useful where we keep in mind that the matrix $\mathbf{M}$ is nonsingular. Note, that the matrix form is a useful theoretical tool to estimate computational cost or discuss convergence but, as we will see in later examples, this is not how one implements this type of method in practice. The general algorithm for a single iteration of the stationary method built from a matrix splitting approach can be divided into four steps:

1. Solve $\mathbf{M}\mathbf{c} = \mathbf{b}$.
2. Compute the vector $\mathbf{v} = \mathbf{N}\mathbf{x}^{(k)}$.

3. Solve $\mathbf{M}\mathbf{z}^{(k)} = \mathbf{v}$.

4. Update the solution $\mathbf{x}^{(k+1)} = \mathbf{z}^{(k)} + \mathbf{c}$.

We see that the first step of the algorithm actually only has to be done *once* because it does not depend on the iteration. In contrast, the matrix solution in step three must be done in *every* iteration, which reinforces that the inversion of the matrix $\mathbf{M}$ should be *cheap*.

With this theoretical background we can now discuss two classical stationary iterative methods for the solution of linear systems. For this we consider a general, nonsingular matrix

$$\mathbf{A} = \begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{pmatrix}.$$

Further, we divide $\mathbf{A}$ into a matrix that contains only diagonal elements $\mathbf{D} = \operatorname{diag}(a_{11}, \ldots, a_{nn})$, a strictly lower triangular matrix $\mathbf{R}_L$, and a strictly upper triangular matrix $\mathbf{R}_U$

$$\mathbf{R}_L = \begin{pmatrix} 0 & 0 & 0 & \cdots & 0 \\ -a_{21} & 0 & 0 & \cdots & 0 \\ -a_{31} & -a_{32} & 0 & \cdots & 0 \\ \vdots & \vdots & \ddots & \ddots & \vdots \\ -a_{n1} & -a_{n2} & \cdots & -a_{n,n-1} & 0 \end{pmatrix}, \qquad \mathbf{R}_U = \begin{pmatrix} 0 & -a_{12} & -a_{13} & \cdots & -a_{1n} \\ 0 & 0 & -a_{23} & \cdots & -a_{2n} \\ \vdots & \vdots & \ddots & \ddots & \vdots \\ 0 & 0 & \cdots & 0 & -a_{n-1,n} \\ 0 & 0 & \cdots & 0 & 0 \end{pmatrix},$$

such that

$$\mathbf{A} = \mathbf{D} - \mathbf{R}_L - \mathbf{R}_U.$$

The signs here are useful to relate this division of the matrix $\mathbf{A}$ to the splitting form of the stationary iterative method.

Definition 4.7 (Jacobi iterative method). If we take the splitting matrices to be

$$\mathbf{M} = \mathbf{D} \quad \text{and} \quad \mathbf{N} = \mathbf{R}_L + \mathbf{R}_U,$$

we obtain iterative strategy of Jacobi

$$\mathbf{x}^{(k+1)} = \mathbf{T}\mathbf{x}^{(k)} + \mathbf{c} = \mathbf{D}^{-1}(\mathbf{R}_L + \mathbf{R}_U)\mathbf{x}^{(k)} + \mathbf{D}^{-1}\mathbf{b}.$$

◀

Example 4.8. Consider the three matrices

$$\mathbf{A}_1 = \begin{pmatrix} 9 & 1 & 1 \\ 2 & 10 & 3 \\ 3 & 4 & 11 \end{pmatrix}, \quad \mathbf{A}_2 = \begin{pmatrix} 2 & -1 & 0 & 0 & 0 & 0 & 0 \\ -1 & 2 & -1 & 0 & 0 & 0 & 0 \\ 0 & -1 & 2 & -1 & 0 & 0 & 0 \\ 0 & 0 & -1 & 2 & -1 & 0 & 0 \\ 0 & 0 & 0 & -1 & 2 & -1 & 0 \\ 0 & 0 & 0 & 0 & -1 & 2 & -1 \\ 0 & 0 & 0 & 0 & 0 & -1 & 2 \end{pmatrix}, \quad \mathbf{A}_3 = \begin{pmatrix} 4 & -1 & 0 & 0 & 0 & 0 & 0 \\ -1 & 4 & -1 & 0 & 0 & 0 & 0 \\ 0 & -1 & 4 & -1 & 0 & 0 & 0 \\ 0 & 0 & -1 & 4 & -1 & 0 & 0 \\ 0 & 0 & 0 & -1 & 4 & -1 & 0 \\ 0 & 0 & 0 & 0 & -1 & 4 & -1 \\ 0 & 0 & 0 & 0 & 0 & -1 & 4 \end{pmatrix},$$

and determine if the Jacobi iteration will converge. The associated iteration matrices are

$$\mathbf{T}_1 = \begin{pmatrix} 0 & -\frac{1}{9} & -\frac{1}{9} \\ -\frac{2}{10} & 0 & -\frac{3}{10} \\ -\frac{3}{11} & -\frac{4}{11} & 0 \end{pmatrix}, \quad \mathbf{T}_2 = \frac{1}{2}\begin{pmatrix} 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 \end{pmatrix}, \quad \mathbf{T}_3 = \frac{1}{4}\begin{pmatrix} 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 \end{pmatrix}.$$

We can compute that $\|\mathbf{T}_1\|_\infty = \frac{7}{11}$, $\|\mathbf{T}_2\|_1 = \|\mathbf{T}_2\|_\infty = 1$, and $\|\mathbf{T}_3\|_1 = \|\mathbf{T}_3\|_\infty = \frac{1}{2}$. So, we immediately know that Jacobi's iteration will converge for $\mathbf{A}_1$ and $\mathbf{A}_3$, but it is not clear if it will work for $\mathbf{A}_2$. However, if we compute the 2-norm of this matrix we find $\|\mathbf{T}_2\|_2 = 0.923879532511287$ and, therefore, Jacobi will converge. This reinforces that the choice of norm for our sufficient convergence condition is important. ♦

From the perspective of computational cost, we see that the preconditioning matrix $\mathbf{M}$ for Jacobi's iteration is diagonal and, therefore, trivial to invert requiring only n divisions. In the general algorithmic structure above we see that the second step, which is a matrix-vector product, is the most expensive part for a given iteration. Therefore, each step of the Jacobi iteration has an estimated cost of $2n^2 + \mathcal{O}(n)$.

Such an estimate of the work is nice, but in practice this matrix form *is not* an efficient way to implement the Jacobi iteration. For one thing, storing a diagonal matrix $\mathbf{D}$ is a waste of memory because of the large number of zero entries. Instead, it is useful to look at the index form of the Jacobi iterative method

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j=1, i \neq j}^{n} a_{ij} x_j^{(k)} \right), \quad i = 1, \ldots, n.$$

We see that the new approximation to the solution $\mathbf{x}^{(k+1)}$ is constructed from a linear combination of the right hand side information $\mathbf{b}$ and the *old* approximation $\mathbf{x}^{(k)}$. We provide pseudocode for the Jacobi iterative technique (again we set a maximum number of iterations to ensure that we do not create an infinite loop):

Algorithm 4.2: *Jacobi iteration*: Solve a nonsingular linear system with Jacobi's method

Procedure Jacobi Iteration
Input: $\mathbf{A}$, $\mathbf{b}$, $\mathbf{x}^{(0)}$, n, `tol`
$max_{its} \leftarrow 250$ `// maximum number of iterations allowed`
for $k = 1$ **to** max_{its} **do**
  for $i = 1$ **to** n **do**
    sum $\leftarrow 0$
    for $j = 1$ **to** n **do**
      if $j \neq i$ **then**
        sum $\leftarrow$ sum $+ a_{ij} x_j^{(0)}$
    $x_i^{(1)} \leftarrow (b_i - \text{sum})/a_{ii}$
  $\mathbf{r}^{(1)} \leftarrow \mathbf{b} - \mathbf{A}\mathbf{x}^{(1)}$
  if $\|\mathbf{r}^{(1)}\|_2 < \texttt{tol} \cdot \|\mathbf{b}\|_2$ **then**
    break
  $\mathbf{x}^{(0)} \leftarrow \mathbf{x}^{(1)}$
Output: $\mathbf{x}^{(1)}$ `//solution vector`
End Procedure Jacobi Iteration

Just as we saw with fixed point iterations we require some kind of *kill condition* to stop the iteration once the solution is deemed "good enough".

Definition 4.8 (Kill condition for iterative method). There are many ways to define a condition that terminates the iteration. Two such choices, where we select the vector 2-norm, are

$$\|\mathbf{x}^{(k+1)} - \mathbf{x}^{(k)}\|_2 < \texttt{tol} \qquad \text{or} \qquad \frac{\|\mathbf{r}^{(k+1)}\|_2}{\|\mathbf{b}\|_2} < \texttt{tol}.$$

The second stopping condition is the one given in the pseudocode above. ◄

The first stopping criterion compares the difference between components of the last two iterates k and $k+1$. Thus, it is possible that this kill condition could halt the iteration before an accurate solution is reached because it *does not* check the residual. In contrast, the first stopping condition is cheaper to evaluate because it does not require a matrix-vector product. However, the second kill condition is more robust in providing an accurate iterative solution to the problem.

Recall from Remark 4.8 there is an equivalent residual form of the stationary iterative method. This is convenient because we can ***reuse*** the residual computed for the stopping criterion to update the solution in the next iteration! We provide psudocode for an alternative, residual form of the Jacobi iteration where the preconditioner matrix is $\mathbf{M} = \mathbf{D}$.

Algorithm 4.3: *Jacobi iteration (residual form)*: Equivalent form of Jacobi's method

Procedure Jacobi Iteration (Residual Form)
Input: $\mathbf{A}$, $\mathbf{b}$, $\mathbf{x}^{(0)}$, n, `tol`
$max_{its} \leftarrow 250$ `// maximum number of iterations allowed`
$\mathbf{r}^{(0)} \leftarrow \mathbf{b} - \mathbf{A}\mathbf{x}^{(0)}$
for $k = 1$ **to** max_{its} **do**
  for $i = 1$ **to** n **do**
    $x_i^{(1)} \leftarrow x_i^{(0)} + r_i^{(0)}/a_{ii}$
  $\mathbf{r}^{(1)} \leftarrow \mathbf{b} - \mathbf{A}\mathbf{x}^{(1)}$
  if $\|\mathbf{r}^{(1)}\|_2 < \texttt{tol} \cdot \|\mathbf{b}\|_2$ **then**
    break
  $\mathbf{x}^{(0)} \leftarrow \mathbf{x}^{(1)}$
  $\mathbf{r}^{(0)} \leftarrow \mathbf{r}^{(1)}$
Output: $\mathbf{x}^{(1)}$ `//solution vector`
End Procedure Jacobi Iteration (Residual Form)

We see that in the Jacobi iteration we do not use the most recently available information when computing $x_i^{(k+1)}$. For example, $x_1^{(k)}$ is used in the computation of $x_2^{(k+1)}$ despite the fact that the component $x_1^{(k+1)}$ is already known. If we assume that iterative method is convergent, it would make sense to use the most up-to-date information to compute the solution within an iteration. This is the basic idea of the *Gauss-Seidel iterative method*.

Definition 4.9 (Gauss-Seidel method). If we take the splitting matrices to be

$$\mathbf{M} = \mathbf{D} - \mathbf{R}_L \quad \text{and} \quad \mathbf{N} = \mathbf{R}_U,$$

we obtain the Gauss-Seidel iterative strategy

$$\mathbf{x}^{(k+1)} = \mathbf{T}\mathbf{x}^{(k)} + \mathbf{c} = (\mathbf{D} - \mathbf{R}_L)^{-1}\mathbf{R}_U\mathbf{x}^{(k)} + (\mathbf{D} - \mathbf{R}_L)^{-1}\mathbf{b}.$$

◀

For the Gauss-Seidel method we see that the iteration matrix is $\mathbf{T} = (\mathbf{D} - \mathbf{R}_L)^{-1}\mathbf{R}_U$, so constructing $\mathbf{T}$ and determining the matrix norm to check the sufficient condition for convergence is unwieldy. Instead we define a stricter, but easier to check, condition:

Definition 4.10 (Diagonal dominance). Examining the rows of a matrix $\mathbf{A}$ where $1 \leq i \leq n$

$$|a_{ii}| \geq \sum_{j=1, j\neq i}^{n} |a_{ij}|,$$

then the matrix is said to be *weakly diagonally dominant*. If the equality is removed then the matrix is *strictly diagonally dominant*. ◀

Remark 4.13. If the matrix $\mathbf{A}$ is strictly diagonally dominant, then $\mathbf{A}$ is nonsingular and the associated Jacobi and Gauss-Seidel iterations converge for *any* initial guess $\mathbf{x}^{(0)}$. ⋈

Remark 4.14. For strictly diagonally dominant matrices finding the inverse of the preconditioner matrix $\mathbf{M} = \mathbf{D}$ for Jacobi or $\mathbf{M} = \mathbf{D} - \mathbf{R}_L$ for Gauss-Seidel is a fairly good, cheaper to invert approximation of the true matrix inverse $\mathbf{A}^{-1}$. ⋈

EXAMPLE 4.9. From this knowledge we immediately know that Gauss-Seidel will converge for the matrices $\mathbf{A}_1$ and $\mathbf{A}_3$ from the Example 4.8 because they are strictly diagonally dominant. The matrix $\mathbf{A}_2$ is only weakly diagonally dominant, but it is symmetric positive definite. Thus, Gauss-Seidel is still guaranteed to converge for any initial guess. ⧫

What about the computational cost of the Gauss-Seidel method? From the general algorithm above we see it still involves a matrix-vector multiplication in the second step. But the third step now requires the solution of a matrix problem. However, this can be done efficiently through a forward solve which is known to cost $n^2 + \mathcal{O}(n)$. Therefore, the total estimated cost is $3n^2 + \mathcal{O}(n)$. So, on paper, we see that the Gauss-Seidel iterative method is more expensive than Jacobi's method. However, *if both methods converge*, the Gauss-Seidel method will converge ***faster*** because it uses the most up-to-date information in each iteration.

Just as before, the matrix form is useful to examine the Gauss-Seidel method theoretically but has limited use in practice. We exploit the forward substitution nature of inverting the matrix $\mathbf{M}$ and write the index form of the method

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j=1}^{i-1} a_{ij} x_j^{(k+1)} - \sum_{j=i+1}^{n} a_{ij} x_j^{(k)} \right), \quad i = 1, \ldots, n.$$

This reformulation and exploitation of the forward solution algorithm puts the Gauss-Seidel method into a computational form that has a very similar cost (and structure) to the Jacobi iteration, but keep in mind Gauss-Seidel will be faster. We provide pseudocode for the Gauss-Seidel iterative technique below where, to save memory consumption, the updated solution continually overwrites the previous information:

Algorithm 4.4: *Gauss-Seidel iteration*: Solve a nonsingular linear system with Gauss-Seidel's method

```
Procedure Gauss-Seidel Iteration
Input: A, b, x, n, tol // x contains the initial guess
max_its ← 250 // maximum number of iterations allowed
for k = 1 to max_its do
    for i = 1 to n do
        sum ← 0
        for j = 1 to n do
            if j ≠ i then
                sum ← sum + a_ij x_j
        x_i ← (b_i − sum)/a_ii // update the information in the solution vector
    r ← b − Ax
    if ‖r‖_2 < tol · ‖b‖_2 then
        break
Output: x // solution vector
End Procedure Gauss-Seidel Iteration
```

Remark 4.15. If the matrix $\mathbf{A}$ is strongly diagonally dominant, then the Gauss-Seidel iterative method is guaranteed to converge for any initial guess $\mathbf{x}^{(0)}$. ⋈

EXAMPLE 4.10. We apply the Jacobi and Gauss-Seidel method to the three matrices given in Example 4.8. For the stopping criterion we use the residual check given in the pseudocode for Algorithms 4.2 and 4.4 with a tolerance $\texttt{tol} = 10^{-6}$. We ***manufacture*** the true solution to be $\mathbf{x}_{exact}$ to be a vector of all ones, where the right hand side $\mathbf{b}$ is set accordingly. We take the initial guess for the solution to be a vector of zeros, e.g. $\mathbf{x}^{(0)} = \mathbf{0}$. In the following table we give the number of iterations required for the Jacobi and Gauss-Seidel iterative methods to reach the set tolerance for each matrix:

Matrix	Dimension	Iterations k (Jacobi)	Iterations k (Gauss-Seidel)
$\mathbf{A}_1$	3×3	18	7
$\mathbf{A}_2$	7×7	163	81
$\mathbf{A}_3$	7×7	18	11

For these small "toy" examples we see that the number of iterations k is larger than the dimension of the system n. We can expect this because the motivation of iterative techniques was for very large linear system (i.e. n is large). In this case the iterative methods that cost $\mathcal{O}(n^2)$ scale better than direct methods that cost $\mathcal{O}(n^3)$. ⧫

Can these methods be improved and convergence toward the solution accelerated? Yes, there are many strategies to do this. In the realm of stationary iterative methods there are so-called *relaxation methods*. However, there are also more sophisticated *nonstationary* iterative methods. The main difference between the method types is that the iteration involves information that changes with each iteration.

To develop such nonstationary techniques ones views the solution of a linear system through a different lens and motivate a different iterative approach to solve the problem. This is the basis of so called *gradient methods*. These methods are outside the scope of this course but are very popular in the numerical linear algebra community. For those interested, there are supplementary notes on the topic of gradient methods available in this compendium.

Note Set 5: Solving linear systems: Rectangular matrices

We close our discussion on solving linear systems. In doing so we will generalize our tools to solve linear systems that are *rectangular*. In particular, the technique we will learn allows us to solve systems of equations where we have more equations than unknown variables. Such a system is called an *overdetermined* linear system. Basically, the idea is we know that we cannot solve all the equations simultaneously because we lack sufficient free variables. So, instead, we will compute a solution that is "ideal" and minimizes the error. That last statement was nice and vague: Which error? Minimized against what? We shall answer these questions and develop another flavour of matrix factorization tool in this note set.

5.1 Overdetermined systems and **QR** factorization

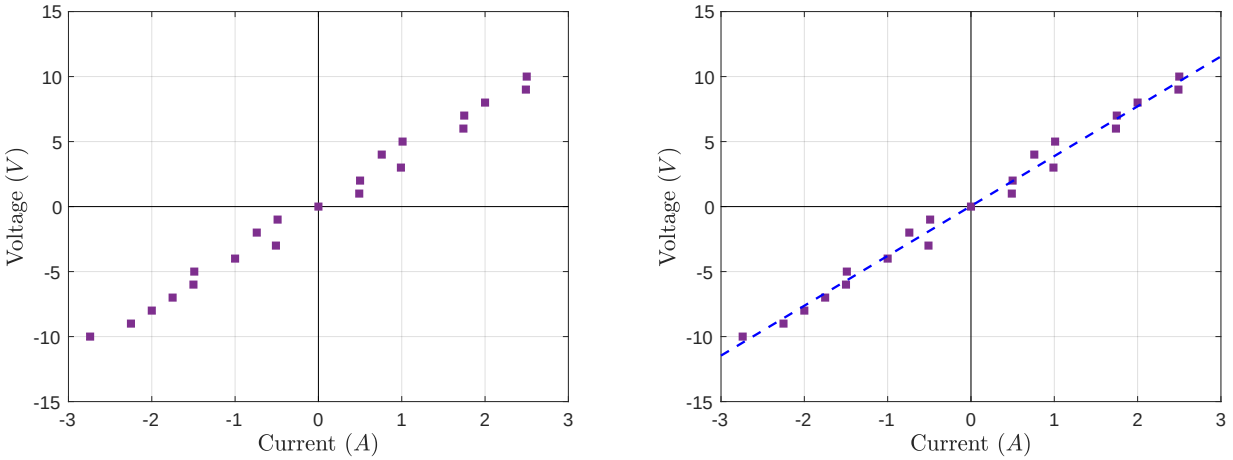
So far we have considered the solution of $\mathbf{Ax} = \mathbf{b}$ only when $\mathbf{A}$ is a square matrix. In this lecture we expand to solve a problem for a matrix $\mathbf{A}$ that is rectangular with size $m \times n$ where $m > n$ (often *much larger* in practice). This is referred to as an ***overdetermined*** linear system because the system has more equations to satisfy than the available number of variables.

We begin with an example to illustrate a situation where an overdetermined linear system is relevant.

Example 5.1. Suppose we have run an experiment to measure the current in a circuit at different voltages. The data from such an experiment is collected in the following table.

Voltage (V)	-10.0	-9.0	-8.0	-7.0	-6.0	-5.0	-4.0	-3.0	-2.0	-1.0	0.0	1.0	2.0	3.0	4.0	5.0	6.0	7.0	8.0	9.0	10.0
Current (A)	-2.74	-2.25	-2.0	-1.75	-1.5	-1.49	-1.0	-0.51	-0.74	-0.49	0.0	0.49	0.5	0.99	0.76	1.01	1.74	1.75	2.0	2.49	2.5

We know from Ohm's law that the voltage and current satisfy a linear relationship where the voltage is proportional to the current scaled by the resistance. If we make a scatter plot of the data, as shown on the left in the figure below, we see the general trend of this expected linear relationship. On the right we *fit* a linear function to the given data where said line is as close as possible to all the given data points.



It is the calculation of this linear fit that necessitates an overdetermined system of equations. This is because such a linear function $g(x) = t_1 + t_2 x$ has two *degrees of freedom* to be found from the larger set of data points. So, in this case we have 21 data points and only two unknowns. ♦

From this illustration, we see that using all the available information produces a more accurate approximation to the known linear relationship between voltage and current compared to using just two of the data points. In a perfect world, if we tested Ohm's law, all the experimental data would already lie on a straight line. But due to *measurement errors* (among others) we know that this will not happen. This is one motivation for supplying more equations than unknown variables because it serves to reduce the effect of such measurement errors.

We will discuss a systematic approach to solve such a rectangular, overdetermined linear system. For this presentation, we assume that the matrix $\mathbf{A} \in \mathbb{R}^{m\times n}$ has full column rank. Or, basically, we are assuming that the rectangular matrix $\mathbf{A}$ has linearly independent columns. This ensures that if $m = n$, then the square system has a solution because the matrix $\mathbf{A}$ would be invertible, i.e. $\mathbf{x} = \mathbf{A}^{-1}\mathbf{b}$.

Well hold on, from the dimensionality of the overdetermined problem and the full column rank of the matrix $\mathbf{A}$ could we not just "square" the matrix by multiplying both sides of the linear equation system with $\mathbf{A}^T$. That is, we simply rewrite the system into the equivalent form

$$\mathbf{A}\mathbf{x} = \mathbf{b} \qquad \Leftrightarrow \qquad \mathbf{A}^T\mathbf{A}\mathbf{x} = \mathbf{A}^T\mathbf{b}.$$

Here, the matrix $\mathbf{A}^T\mathbf{A}$ is now an $n \times n$ square invertible matrix. Further, the matrix $\mathbf{A}^T\mathbf{A}$ is symmetric positive definite by construction, so a method like Cholesky or Gauss-Seidel could solve the system in a straightforward way.

Remark 5.1. The formulation on the right hand side is referred to as the ***normal equations***. This manipulation is theoretically valid, but it is less clear whether or not such it will be practical. We are guaranteed that the matrix $\mathbf{A}^T\mathbf{A}$ is symmetric positive definite, but matrix multiplication is computationally expensive and could destroy any structure the matrix might have (like sparsity). Also, the conditioning of the normal equations is

$$\kappa_2(\mathbf{A}^T\mathbf{A}) = \kappa_2(\mathbf{A})^2.$$

Thus, if the original system is ill-conditioned solving the normal equations is even more so. ⋈

Remark 5.2. So, "squaring the system" may or may not be computationally practical and does not seem particularly useful. However, we will see the normal equations are a nice theoretical avenue on which to travel and create more stable and tractable strategies to solve the overdetermined linear system. ⋈

For the case of an overdetermined matrix $\mathbf{A} \in \mathbb{R}^{m\times n}$ when $m > n$, we must redefine what we mean by the phrase *solving the system*. In general, for the overdetermined system we say that

$$\mathbf{A}\mathbf{x} \simeq \mathbf{b}.$$

Here we use the symbol "$\simeq$" to indicate that we typically cannot satisfy all the equations simultaneously, but we seek a solution $\mathbf{x} \in \mathbb{R}^n$ such that $\mathbf{A}\mathbf{x} \in \mathbb{R}^m$ is as close as possible to $\mathbf{b} \in \mathbb{R}^m$. We need to clarify what is meant by the rather vague statement *as close as possible*, in order to determine a solution $\mathbf{x}$. To do this, we revisit the residual of the system, $\mathbf{r} = \mathbf{b} - \mathbf{A}\mathbf{x}$, which lies in the vector space $\mathbb{R}^m$.

Depending on how we construct the solution vector $\mathbf{x}$, the residual changes. We choose to measure the magnitude of the residual using the vector 2-norm which is

$$\|\mathbf{r}\|_2^2 = r_1^2 + \cdots + r_m^2 = \mathbf{r}^T\mathbf{r}.$$

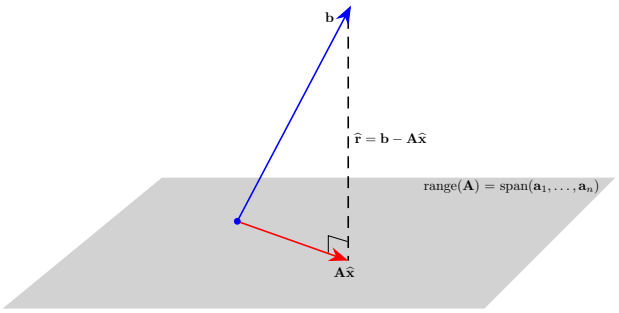
Then, we denote the solution we want as $\widehat{\mathbf{x}}$. It *minimizes* the size of the residual measured in the 2-norm.

Definition 5.1 (Least squares solution). Suppose we have an overdetermined system with $\mathbf{b} \in \mathbb{R}^m$, $\mathbf{A} \in \mathbb{R}^{m\times n}$, $m > n$ and full column rank. Then the *least squares solution* is a vector $\widehat{\mathbf{x}}$ that minimizes

$$\min_{\mathbf{x}\in\mathbb{R}^n} \|\mathbf{b} - \mathbf{A}\mathbf{x}\|_2.$$

In other words, the Euclidean norm of the residual is as small as possible for the solution $\widehat{\mathbf{x}}$. ◀

The geometry of the least square problem is illustrated on the right. We know that the n columns of the rectangular matrix $\mathbf{A}$ span a subspace of $\mathbb{R}^m$ because of the assumption that $m > n$. In general, the vector $\mathbf{b}$ will contain information that cannot be *reached* by a solution $\mathbf{x}$. Therefore, the best we can do is to find the solution $\widehat{\mathbf{x}}$ that captures as much information from $\mathbf{b}$ as possible. In turn, this minimizes the distance between the true value of $\mathbf{b}$ and its projection onto the subspace $\mathbb{R}^n$. As a consequence, the residual in the least squares problem should be orthogonal to the span of the columns of the matrix $\mathbf{A}$. From this fact we find that



$$\mathbf{A}^T\widehat{\mathbf{r}} = \mathbf{A}^T(\mathbf{b} - \mathbf{A}\widehat{\mathbf{x}}) = \mathbf{0} \qquad \Rightarrow \qquad \mathbf{A}^T\mathbf{A}\widehat{\mathbf{x}} = \mathbf{A}^T\mathbf{b}.$$

Hey, there are those normal equations again! This provides us an equivalent formulation (and different theoretical viewpoint) for the least squares solution.

Definition 5.2 (Least squares solution from the normal equations). A solution $\widehat{\mathbf{x}}$ of the least squares problem

$$\min_{\mathbf{x}\in\mathbb{R}^n} \|\mathbf{b} - \mathbf{A}\mathbf{x}\|_2$$

satisfies the normal equations

$$\mathbf{A}^T\mathbf{A}\widehat{\mathbf{x}} = \mathbf{A}^T\mathbf{b},$$

provided the columns of the matrix $\mathbf{A}$ are linearly independent. This ensures that the matrix $\mathbf{A}^T\mathbf{A}$ is symmetric positive definite. Thus, the least squares solution $\widehat{\mathbf{x}}$ is unique. ◀

It is straightforward to show that the solution $\widehat{\mathbf{x}}$ of the normal equations minimizes the 2-norm. Consider the vector $\mathbf{x} = \widehat{\mathbf{x}} + \mathbf{y}$, then the residual is $\mathbf{r} = \mathbf{b} - \mathbf{A}(\widehat{\mathbf{x}} + \mathbf{y}) = \widehat{\mathbf{r}} - \mathbf{A}\mathbf{y}$. Plus, we know that $\mathbf{A}^T\widehat{\mathbf{r}} = \mathbf{0}$ so

$$\begin{aligned}\|\mathbf{r}\|_2^2 = (\widehat{\mathbf{r}} - \mathbf{A}\mathbf{y})^T(\widehat{\mathbf{r}} - \mathbf{A}\mathbf{y}) &= \widehat{\mathbf{r}}^T\widehat{\mathbf{r}} - \widehat{\mathbf{r}}^T\mathbf{A}\mathbf{y} - \mathbf{y}^T\mathbf{A}^T\widehat{\mathbf{r}} + \mathbf{y}^T\mathbf{A}^T\mathbf{A}\mathbf{y} \\ &= \widehat{\mathbf{r}}^T\widehat{\mathbf{r}} - \mathbf{0} - \mathbf{0} + \mathbf{y}^T\mathbf{A}^T\mathbf{A}\mathbf{y} \\ &> \widehat{\mathbf{r}}^T\widehat{\mathbf{r}} = \|\widehat{\mathbf{r}}\|_2^2,\end{aligned}$$

as $\mathbf{y}$ is assumed to not be the zero vector, i.e., $\mathbf{y} \neq \mathbf{0}$.

The normal equations are convenient to work with in a theoretical setting because it restates the overdetermined $m \times n$ system ***into*** an $n \times n$ square linear system. This allows us to solve the overdetermined system in a straightforward fashion:

Definition 5.3 (Pseudoinverse). The linear least squares solution of the overdetermined system $\mathbf{A}\mathbf{x} \simeq \mathbf{b}$ can be computed from the normal equations through the use of the *pseudoinverse* $\mathbf{A}^\dagger$ defined by

$$\widehat{\mathbf{x}} = \mathbf{A}^\dagger\mathbf{b} \qquad \text{where} \qquad \mathbf{A}^\dagger = (\mathbf{A}^T\mathbf{A})^{-1}\mathbf{A}^T.$$

The matrix $\mathbf{A}^\dagger$ is also referred to as the Moore-Penrose inverse. ◀

Example 5.2. Let us revisit the problem concerning Ohm's law to determine a linear fit to the experimental data. We expect that the voltage is a linear function of the current. Therefore, we set up an overdetermined system

$$x_1 + x_2 I_j = V_j, \qquad j = 1, \ldots, 21,$$

where the voltage, V, and current, I, values are taken from the Table in Example 5.1. We compute the normal equations for this problem to obtain

$$\mathbf{A}^T\mathbf{A} = \begin{pmatrix} 21 & -0.24 \\ -0.24 & 51.7586 \end{pmatrix}, \qquad \mathbf{A}^T\mathbf{b} = \begin{pmatrix} 0 \\ 198.5 \end{pmatrix}.$$

This 2×2 system is well-conditioned to solve with the pseudoinverse and we determine $x_1 \approx 0.0438$ and $x_2 \approx 3.8353$, which yields the linear fit from before. ♦

Remark 5.3. Because the normal equations yield a symmetric positive definite matrix, there are many tools available to approximate the solution of the normal equations and produce the least squares solution for an overdetermined system. These include techniques of varying computational complexity, like the Cholesky factorization or Gauss-Seidel stationary iteration. ⋈

Though convenient theoretically, the normal equations suffer from several issues, as discussed previously. These include storage expense or the computational complexity of forming the matrix-matrix product for the left hand side. Most importantly, the normal equations can be *very* poorly conditioned numerically. This is particularly true if the columns of the rectangular matrix $\mathbf{A}$ are close to linearly dependent.

Example 5.3. Consider the rectangular matrix

$$\mathbf{A} = \begin{pmatrix} 1 & 1 \\ 10^{-6} & 0 \\ 0 & 10^{-6} \end{pmatrix}.$$

Then when we form the normal equations we find that

$$\mathbf{A}^T\mathbf{A} = \begin{pmatrix} 1 + 10^{-12} & 1 \\ 1 & 1 + 10^{-12} \end{pmatrix}.$$

Unfortunately, this matrix is nearly singular because the condition number $\kappa_\infty(\mathbf{A}^T\mathbf{A}) \approx 10^{12}$. So, forming the pseudoinverse $\mathbf{A}^\dagger$ can lead to excessive round-off errors to compute the least squares solution. ♦

How do we compute the least squares solution in a numerically stable way? To do this we will perform another type of factorization on the rectangular matrix $\mathbf{A}$. First, however, we need to define a useful type of matrix.

Definition 5.4 (Orthogonal matrix). An *orthogonal matrix* $\mathbf{Q}$ is a particular square matrix where the column vectors are mutually orthogonal. Further, each of the columns has a Euclidean length of one. Therefore, the columns of the matrix $\mathbf{Q}$ are orthonormal and it satisfies the condition

$$\mathbf{Q}\mathbf{Q}^T = \mathbf{Q}^T\mathbf{Q} = \mathbf{I}.$$

That is, an orthogonal matrix is trivial to invert because we only have to take its transpose. ◀

Orthogonal transformations are particularly useful for the least squares problem because they *preserve* the Euclidean length:

$$\|\mathbf{Q}\mathbf{x}\|_2^2 = (\mathbf{Q}\mathbf{x})^T\mathbf{Q}\mathbf{x} = \mathbf{x}^T\mathbf{Q}^T\mathbf{Q}\mathbf{x} = \mathbf{x}^T\mathbf{x} = \|\mathbf{x}\|_2^2.$$

This means that applying an orthogonal transformation to the least squares problem ***does not*** change the minimal solution $\widehat{\mathbf{x}}$.

This leads to another important factorization technique.

Definition 5.5 (**QR** factorization). Every overdetermined matrix $\mathbf{A} \in \mathbb{R}^{m\times n}$ where $m \geq n$ and with full column rank can be decomposed into an orthogonal matrix $\mathbf{Q} \in \mathbb{R}^{m\times m}$ and an upper triangular matrix $\mathbf{R} \in \mathbb{R}^{n\times n}$ where the factorization takes the form

$$\mathbf{A} = \mathbf{Q}\begin{pmatrix} \mathbf{R} \\ \mathbf{0} \end{pmatrix},$$

where the rectangular portion of zeros has a size $\mathbf{0} \in \mathbb{R}^{(m-n)\times n}$. ◀

Remark 5.4. The **QR** factorization plays a role in the least squares problem analogous to the role of the **LU** factorization in square linear systems. ⋈

Remark 5.5. When $m > n$, as is the case for overdetermined linear systems, there is a compressed form of the **QR** factorization. Expanding the factorization we see

$$\mathbf{A} = \mathbf{Q}\begin{pmatrix}\mathbf{R}\\ \mathbf{0}\end{pmatrix} = \begin{pmatrix}\mathbf{q}_1 & \mathbf{q}_2 & \cdots & \mathbf{q}_m\end{pmatrix}\begin{pmatrix} r_{11} & r_{12} & \cdots & r_{1n}\\ 0 & r_{22} & \cdots & r_{2n}\\ \vdots & & \ddots & \vdots\\ 0 & 0 & \cdots & r_{nn}\\ 0 & 0 & \cdots & 0\\ \vdots & \vdots & & \vdots\\ 0 & 0 & \cdots & 0\end{pmatrix},$$

and note that the vectors $\mathbf{q}_{n+1}, \ldots, \mathbf{q}_m$ are always multiplied by zero. In some sense, we can interpret this to mean that no information about $\mathbf{A}$ is "lost" if we ignore them and reduce the factorization to be

$$\mathbf{A} = \begin{pmatrix}\mathbf{q}_1 & \mathbf{q}_2 & \cdots & \mathbf{q}_n\end{pmatrix}\begin{pmatrix} r_{11} & r_{12} & \cdots & r_{1n}\\ 0 & r_{22} & \cdots & r_{2n}\\ \vdots & & \ddots & \vdots\\ 0 & 0 & \cdots & r_{nn}\end{pmatrix} = \widehat{\mathbf{Q}}\mathbf{R}.$$

This form is called a ***thin*** **QR** factorization because the number of columns in $\widehat{\mathbf{Q}}$ is n rather than m. ⋈

Remark 5.6. Under the assumption that the columns of $\mathbf{A}$ are linearly independent the upper triangular matrix $\mathbf{R}$ is invertible. ⋈

If we substitute the thin **QR** factorization into the least squares solution we find

$$\begin{aligned}\widehat{\mathbf{x}} = \mathbf{A}^{\dagger}\mathbf{b} = (\mathbf{A}^T\mathbf{A})^{-1}\mathbf{A}^T\mathbf{b} &= ((\widehat{\mathbf{Q}}\mathbf{R})^T(\widehat{\mathbf{Q}}\mathbf{R}))^{-1}(\widehat{\mathbf{Q}}\mathbf{R})^T\mathbf{b}\\ &= (\mathbf{R}^T\widehat{\mathbf{Q}}^T\widehat{\mathbf{Q}}\mathbf{R})^{-1}\mathbf{R}^T\widehat{\mathbf{Q}}^T\mathbf{b}\\ &= (\mathbf{R}^T\mathbf{R})^{-1}\mathbf{R}^T\widehat{\mathbf{Q}}^T\mathbf{b}\\ &= \mathbf{R}^{-1}\mathbf{R}^{-T}\mathbf{R}^T\widehat{\mathbf{Q}}^T\mathbf{b}\\ &= \mathbf{R}^{-1}\widehat{\mathbf{Q}}^T\mathbf{b}.\end{aligned}$$

Remark 5.7. It is important to point out that the columns of the "thin" matrix $\widehat{\mathbf{Q}}$ are orthonormal so $\widehat{\mathbf{Q}}^T\widehat{\mathbf{Q}} = \mathbf{I} \in \mathbb{R}^{n\times n}$. However, when $m > n$ the matrix $\widehat{\mathbf{Q}}\widehat{\mathbf{Q}}^T \neq \mathbf{I} \in \mathbb{R}^{m\times m}$, so the rectangular matrix $\widehat{\mathbf{Q}}$ is ***not*** an orthogonal matrix. A simple explanation for this is because we clipped out the $\mathbf{q}_{n+1}, \ldots, \mathbf{q}_m$ columns from the (square) orthogonal matrix $\mathbf{Q}$ to create its thin version $\widehat{\mathbf{Q}}$.

By applying the thin **QR** factorization we see that it is possible to determine the least squares solution *without* forming the normal equations. Further, the use of the **QR** factorization to solve the least squares problem ***avoids*** numerical stability issues seen when computing the normal equations. Generally, one version of the **QR** factorization to solve the least squares problem can be separated into the following steps:

1. Compute the thin $\widehat{\mathbf{Q}}$ and $\mathbf{R}$ factors of the matrix $\mathbf{A}$. This has an overall computational cost of $2mn^2 + \mathcal{O}(mn)$ flops when $\mathbf{A}$ is $m \times n$.

2. Form the right hand side vector $\mathbf{c} = \widehat{\mathbf{Q}}^T\mathbf{b}$. This is a rectangular matrix-vector product and costs $2mn + \mathcal{O}(m)$ flops.

3. Determine the least squares solution from the system $\mathbf{R}\widehat{\mathbf{x}} = \mathbf{c}$. This is efficiently done with the backsubstitution algorithm and costs $n^2 + \mathcal{O}(n)$ flops.

Steps 2 and 3 are straightforward for us because we know how to compute them with knowledge from note set 3. So, we only need to address how to complete step 1. How do we determine the orthogonal matrix in practice? One way to do this is to use the Gram-Schmidt orthogonalization process to find the columns of $\widehat{\mathbf{Q}}$. Given the matrix

$$\mathbf{A} = (\mathbf{a}_1 \quad \mathbf{a}_2 \quad \cdots \quad \mathbf{a}_n)$$

written in terms of its columns, then

$$\mathbf{q}_1 = \frac{\mathbf{a}_1}{\|\mathbf{a}_1\|_2}, \qquad \mathbf{u}_2 = \mathbf{a}_2 - (\mathbf{a}_2^T\mathbf{q}_1)\mathbf{q}_1 \quad \text{where} \quad \mathbf{q}_2 = \frac{\mathbf{u}_2}{\|\mathbf{u}_2\|_2},$$

$$\mathbf{u}_{k+1} = \mathbf{a}_{k+1} - (\mathbf{a}_{k+1}^T\mathbf{q}_1)\mathbf{q}_1 - \cdots - (\mathbf{a}_{k+1}^T\mathbf{q}_k)\mathbf{q}_k \quad \text{where} \quad \mathbf{q}_{k+1} = \frac{\mathbf{u}_{k+1}}{\|\mathbf{u}_{k+1}\|_2}.$$

The resulting thin **QR** factorization created by the Gram-Schmidt process is given by

$$\mathbf{A} = (\mathbf{a}_1 \quad \mathbf{a}_2 \quad \cdots \quad \mathbf{a}_n) = (\mathbf{q}_1 \quad \mathbf{q}_2 \quad \cdots \quad \mathbf{q}_n) \begin{pmatrix} \mathbf{a}_1^T\mathbf{q}_1 & \mathbf{a}_2^T\mathbf{q}_1 & \cdots & \mathbf{a}_n^T\mathbf{q}_1 \\ 0 & \mathbf{a}_2^T\mathbf{q}_2 & \cdots & \mathbf{a}_n^T\mathbf{q}_2 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \mathbf{a}_n^T\mathbf{q}_n \end{pmatrix} = \widehat{\mathbf{Q}}\mathbf{R}.$$

EXAMPLE 5.4. Use the thin **QR** factorization to compute the least squares solution of the overdetermined problem $\mathbf{Ax} = \mathbf{b}$ where

$$\mathbf{A} = \begin{pmatrix} 3 & -6 \\ 4 & -8 \\ 0 & 1 \end{pmatrix}, \qquad \mathbf{b} = \begin{pmatrix} -1 \\ 7 \\ 2 \end{pmatrix}.$$

Applying the Gram-Schmidt process we compute the factorization to be

$$\widehat{\mathbf{Q}} = \begin{pmatrix} \frac{3}{5} & 0 \\ \frac{4}{5} & 0 \\ 0 & 1 \end{pmatrix}, \qquad \mathbf{R} = \begin{pmatrix} \mathbf{a}_1^T\mathbf{q}_1 & \mathbf{a}_2^T\mathbf{q}_1 \\ 0 & \mathbf{a}_2^T\mathbf{q}_2 \end{pmatrix} = \begin{pmatrix} 5 & -10 \\ 0 & 1 \end{pmatrix}.$$

The right hand side of the overdetermined system becomes

$$\mathbf{c} = \widehat{\mathbf{Q}}^T\mathbf{b} = \begin{pmatrix} \frac{3}{5} & \frac{4}{5} & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} -1 \\ 7 \\ 2 \end{pmatrix} = \begin{pmatrix} 5 \\ 2 \end{pmatrix}.$$

We then solve the resulting square system $\mathbf{R}\widehat{\mathbf{x}} = \mathbf{c}$ with backsubstitution to find $\widehat{x}_2 = 2$ and $\widehat{x}_1 = 5$. ♦

Remark 5.8. We see that there is a lot of subtraction during the Gram-Schmidt process. Thus, it is also prone to numerical round-off errors. Therefore, this process to solve the least squares problem can still suffer from numerical stability issues, albeit different than those from the normal equations! ⋈

In the pseudocode provided below we use a modified form of the Gram-Schmidt orthogonalization process that is less sensitive to the accumulation of round-off errors. The classical and modified Gram-Schmidt are identical in exact arithmetic (i.e. if the calculations were done using pen and paper); however, in finite arithmetic, like that on a computer, there are differences due to floating point errors. The difference is subtle. In the classical Gram-Schmidt process shown above, we take each vector, one-by-one, and make it orthogonal to all previous vectors. For the modified Gram-Schmidt, we take each vector, and modify all forthcoming vectors to be orthogonal to it.

The algorithm takes the (possibly rectangular) matrix $\mathbf{A}$ as an input and then produces the matrices $\widehat{\mathbf{Q}}$ and $\mathbf{R}$ from the thin version of the factorization as the output. Note that the algorithm *does not* overwrite

the values stored in the matrix $\mathbf{A}$. It is straightforward to use $\widehat{\mathbf{Q}}$ to formulate the right hand side vector $\mathbf{c} = \widehat{\mathbf{Q}}^T\mathbf{b}$, then one can feed the the matrix $\mathbf{R}$ and vector $\mathbf{c}$ into the backward substitution algorithm from note set 3. This would complete the procedure and determine the least squares solution $\widehat{\mathbf{x}} = \mathbf{R}^{-1}\widehat{\mathbf{Q}}^T\mathbf{b}$.

Algorithm 5.1: *Thin* **QR**: Compute the orthogonal matrix $\widehat{\mathbf{Q}}$ and upper triangular matrix $\mathbf{R}$

Procedure Thin QR
Input: $\mathbf{A}$, n // $m \times n$ `matrix with` $m \geq n$`, number of columns` n
`//save the columns of` $\mathbf{A}$ `in vectors for the modified Gram-Schmidt process`
for $j = 1$ **to** n **do**
  $\mathbf{v}_j \leftarrow \mathbf{a}_j$
for $j = 1$ **to** n **do**
  $R_{jj} \leftarrow \|\mathbf{v}_j\|_2$
  $\widehat{\mathbf{q}}_j \leftarrow \mathbf{v}_j / R_{jj}$
  for $k = j + 1$ **to** n **do**
    $R_{jk} = \widehat{\mathbf{q}}_j^T \mathbf{v}_k$
    $\mathbf{v}_k = \mathbf{v}_k - R_{jk}\widehat{\mathbf{q}}_j$
Output: $\widehat{\mathbf{Q}}$, $\mathbf{R}$ // ``thin'' `orthogonal matrix and upper triangular matrix`
End Procedure Thin QR

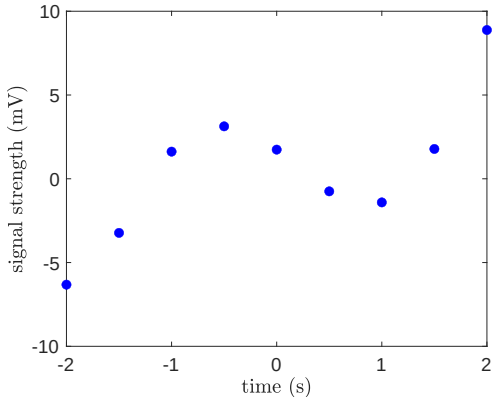
Remark 5.9. There is a more sophisticated technique to find the complete orthogonal matrix $\mathbf{Q} \in \mathbb{R}^{m\times m}$ using *Householder reflectors*. This tactic is more stable than the modified Gram-Schmidt process and is related closely to the geometry of the least squares problem. It is covered in the numerical linear algebra course TANA15. ⋈

The **QR** factorization allows one, in a numerically stable fashion, to solve an overdetermined system of equations. For this reason it has many uses across data science, statistics, and optimization. In particular, given a set of data, the **QR** factorization can be used to determine the least-squares solution to create a curve that fits to the general trend of the data. We explore this in a final example.

Example 5.5. Suppose we have taken measurements of the signal output by a small electronic device and obtain the following data:

time (s)	-2.00	-1.50	-1.00	-0.50	0.00	0.50	1.00	1.50	2.00
Signal strength (mV)	-6.32	-3.23	1.62	3.13	1.74	-0.75	-1.41	1.78	8.88

Looking at the raw data it is difficult to discern any trend and/or properties. Therefore, we plot the data in the figure below to examine its general behaviour.



We notice two things about the shape of this data:

1. It crosses the t-axis three times, so it decent approximation could be with a cubic polynomial.
2. It looks somewhat like a sine curve, although the data is not periodic.

From these observations we decide that one ansatz for the curve fitting can be a combination of these two (odd) functions

$$f(t) = c_1 + c_2 t + c_3 t^3 + c_4 \sin(t).$$

The coefficients $\mathbf{c} = (c_1, c_2, c_3, c_4)^T \in \mathbb{R}^{4\times 1}$ are the unknowns in our curve ansatz.

From the given data point in the table above (t_j, y_j), $j = 1, \ldots, 9$ we can create a rectangular system of equations $\mathbf{Ac} = \mathbf{b}$ to determine the four unknown coefficients. The corresponding matrix $\mathbf{A}$ and right hand side vector $\mathbf{b}$ are

$$\mathbf{A} = \begin{pmatrix} 1 & t_1 & t_1^3 & \sin(t_1) \\ \vdots & \vdots & \vdots & \vdots \\ 1 & t_9 & t_9^3 & \sin(t_9) \end{pmatrix} \in \mathbb{R}^{9\times 4}, \qquad \mathbf{b} = \begin{pmatrix} y_1 \\ \vdots \\ y_9 \end{pmatrix} \in \mathbb{R}^{9\times 1}$$

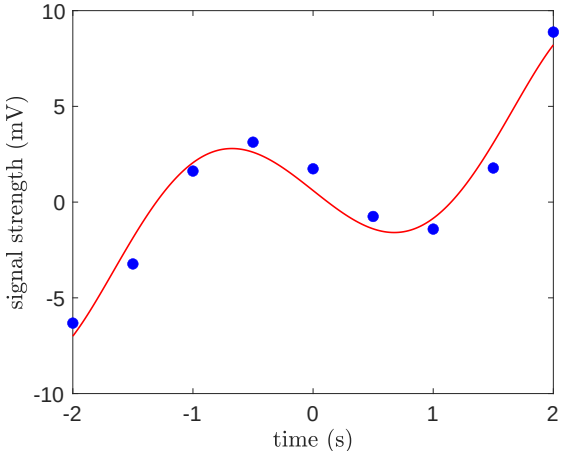
We apply the thin $\mathbf{QR}$ factorization from Alg. 5.1 to decompose the matrix $\mathbf{A}$ into the orthogonal matrix $\widehat{\mathbf{Q}} \in \mathbb{R}^{9\times 4}$ and the upper triangular matrix $\mathbf{R} \in \mathbb{R}^{4\times 4}$. It is then straightforward to find the least-squares solution for the system as follows:

$$\mathbf{Ac} = \mathbf{b} \quad \Rightarrow \quad \widehat{\mathbf{Q}}\mathbf{Rc} = \mathbf{b} \quad \Rightarrow \quad \mathbf{c} = \mathbf{R}^{-1}\widehat{\mathbf{Q}}^T\mathbf{b}$$

where the "action" of the inverse matrix $\mathbf{R}$ can be computed efficiently via a backsubstitution algorithm. The coefficients computed from this process rounded to four decimals are

$$\mathbf{c} = \begin{pmatrix} c_1 \\ c_2 \\ c_3 \\ c_4 \end{pmatrix} = \begin{pmatrix} 0.6044 \\ 54.8136 \\ -5.9573 \\ -59.7869 \end{pmatrix}$$

Additionally, we plot the resulting curve fitting $f(t)$ below. We see that it does an adequate job capturing the trend of the given data.



Note, the curve fitting could be improved if we consider a different ansatz curve for $f(t)$. For instance, we can incorporate higher polynomial powers x^5 which would add another coefficient and the matrix system would need to be solved again. ♦

Note Set 6: Polynomial interpolation

Suppose we only have a discrete set of data points that describe an underlying mathematical function. We have seen that we can "fit" a function (or solution) to the data, but what if we want an approximation to this unknown function that recovers these known values exactly? This introduces the topic of ***interpolation***. In particular, we focus on using polynomial functions to capture the known values while also providing an approximation between the data points. A main motivation of using polynomials for this task is that they are easy to understand, easy to manipulate (e.g. differentiate and integrate) and numerically cheap to evaluate on a computer.

6.1 Global polynomial interpolation

To begin, suppose we know the values of some function $f(x)$ on a set of $n+1$ *different* points. That is, we have a set of points $x_0, \ldots, x_n$ (or $\{x_i\}_{i=0}^n$) and we know function data

$$f(x_i) = f_i \quad \text{where} \quad i = 0, \ldots, n.$$

The goal of a *global polynomial interpolation* procedure is to determine a polynomial function of degree n, denoted as $p_n(x)$, such that

$$p_n(x_i) = f_i \quad \text{where} \quad i = 0, \ldots, n.$$

That is, the interpolating polynomial and the original function ***must*** match at the known data points. We can then use $p_n(x)$ to approximate the function $f(x)$ for any value of x within the interval formed by $x_0, \ldots, x_n$.

Remark 6.1. If we use the polynomial $p_n(x)$ to approximate the function $f(x)$ outside this interval it is known as *extrapolation*. It is important to remember that extrapolation is inherently unreliable. ⋈

So we want to find this polynomial function given some function data. Effectively, this means that we ensure that the known data points are *roots* of the polynomial interpolant. This provides $n+1$ unknown parameters for $n+1$ constraints to determine an nth order interpolating polynomial function.

Definition 6.1 (Uniqueness of polynomial interpolant). Given $n+1$ points $x_0, \ldots, x_n$ with corresponding values $f_0, \ldots, f_n$ there is at most one polynomial function $p_n(x)$ of degree less than or equal to n such that

$$p_n(x_i) = f_i$$

where $i = 0, \ldots, n$. ◀

This statement follows directly from the Fundamental Theorem of Algebra. Suppose we have two polynomials, $p_n(x)$ and $r_n(x)$, that interpolate the same set of data points such that $p_n(x_i) = r_n(x_i) = f_i$ for $i = 0, \ldots, n$. Then their difference is also a polynomial $q_n(x) = p_n(x) - r_n(x)$ that satisfies $q_n(x_i) = 0$ for $i = 0, \ldots, n$. However, since the number of roots of a nonzero polynomial is equal to its degree, it follows that $q_n(x) \equiv 0$. Thus, $p_n(x) = r_n(x)$.

Though the interpolating polynomial is unique, we know that there are many equivalent representations of a polynomial function (like factoring its roots). We have seen previously that in numerical computations it is very important ***how*** we represent and manipulate information in order to maintain accuracy. This is also true in forming a polynomial interpolant.

As a first investigation, we represent the interpolating polynomial in a "standard" form in terms of *monomials*

$$p_n(x) = \alpha_0 + \alpha_1 x + \alpha_2 x^2 + \cdots + \alpha_n x^n,$$

which is a linear combination of increasing powers of the variable x. Now we apply the constraints that the interpolating polynomial must satisfy the given function data and the given points $x_0, \ldots, x_n$ to see

$$\begin{aligned}
\alpha_0 + \alpha_1 x_0 + \alpha_2 x_0^2 + \cdots + \alpha_n x_0^n &= f_0, \\
\alpha_0 + \alpha_1 x_1 + \alpha_2 x_1^2 + \cdots + \alpha_n x_1^n &= f_1, \\
\alpha_0 + \alpha_1 x_2 + \alpha_2 x_2^2 + \cdots + \alpha_n x_2^n &= f_2, \\
&\vdots \\
\alpha_0 + \alpha_1 x_n + \alpha_2 x_n^2 + \cdots + \alpha_n x_n^n &= f_n.
\end{aligned}$$

Equivalently, this can be written in matrix-vector form as

$$\begin{pmatrix} 1 & x_0 & x_0^2 & \cdots & x_0^n \\ 1 & x_1 & x_1^2 & \cdots & x_1^n \\ 1 & x_2 & x_2^2 & \cdots & x_2^n \\ \vdots & \vdots & \vdots & & \vdots \\ 1 & x_n & x_n^2 & \cdots & x_n^n \end{pmatrix} \begin{pmatrix} \alpha_0 \\ \alpha_1 \\ \alpha_2 \\ \vdots \\ \alpha_n \end{pmatrix} = \begin{pmatrix} f_0 \\ f_1 \\ f_2 \\ \vdots \\ f_n \end{pmatrix} \quad \text{or} \quad \mathbf{V}\boldsymbol{\alpha} = \mathbf{f}.$$

EXAMPLE 6.1. Say we are given the data points for a function

x	-2	-1	0	1	2
f	-9	-15	-5	-3	39

The corresponding linear system for the polynomial interpolant written in terms of monomials is then

$$\begin{pmatrix} 1 & -2 & 4 & -8 & 16 \\ 1 & -1 & 1 & -1 & 1 \\ 1 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 2 & 4 & 8 & 16 \end{pmatrix} \begin{pmatrix} \alpha_0 \\ \alpha_1 \\ \alpha_2 \\ \alpha_3 \\ \alpha_4 \end{pmatrix} = \begin{pmatrix} -9 \\ -15 \\ -5 \\ -3 \\ 39 \end{pmatrix}.$$

This a dense, nonsingular matrix that must be solved. We can use an **LU** factorization with row pivoting to solve this linear system and determine the coefficients of the interpolating polynomial to be

$$(\alpha_0\,,\,\alpha_1\,,\,\alpha_2\,,\,\alpha_3\,,\,\alpha_4)^T = (-5\,,\,4\,,\,-7\,,\,2\,,\,3)^T.$$

This computation yields the fourth degree interpolating polynomial $p_4(x) = -5 + 4x - 7x^2 + 2x^3 + 3x^4$ for the data. ⧫

We know that, computationally, solving such a dense linear system requires $\mathcal{O}(n^3)$ operations to create the matrix factorization. More importantly, is this solution reliable?

Remark 6.2. The matrix $\mathbf{V}$ is known as a *Vandermonde matrix*. In general, this type of matrix is dense and suffers from ill-conditioning. ⋈

So it turns out no! Not only is forming and solving this linear system computationally expensive, we can't even count on it to give a good interpolating polynomial.

As mentioned above there are multiple ways to represent a polynomial. More concretely, we can decide on a particular set of *basis functions* in which to write the interpolating polynomial. The set of monomials (also called the monic basis) is only one, particularly simple, set of basis polynomial functions.

Remark 6.3. Recall from linear algebra that a set of n basis vectors can represent all members of the vector space $\mathbb{R}^n$. Analogously, the space of nth degree polynomial functions $\mathbb{P}^n$ can be represented in terms of a set of polynomial basis functions. ⋈

Thus, if we rewrite the interpolating polynomial in terms of a different set of basis functions we can improve the reliability of the resulting interpolating polynomial.

We explore two popular alternative forms of the interpolating polynomial. The first is due to Newton, where the idea is to write the interpolating polynomial in a factored form. Note, because we already know that the location of the roots should be $x_0, \ldots, x_n$ this is quite straightforward. The Newton basis polynomials are given by

$$N_0(x) = 1, \quad N_1(x) = (x - x_0), \quad N_2(x) = (x - x_0)(x - x_1), \quad \cdots, \quad N_n(x) = \prod_{j=0}^{n-1}(x - x_j).$$

Remark 6.4. Note that the Newton basis and the monic basis both satisfy a *hierarchy* where the first basis function is a constant, the next is linear, then a quadratic, then a cubic, and so on. ⋈

The interpolating polynomial is then written as a combination of the Newton basis polynomials

$$p_n(x) = \sum_{i=0}^{n} \beta_i N_i(x),$$

where the coefficients β_j must be determined. Now when we apply the interpolation constraints of the given function data we obtain a linear system of the form:

$$\begin{pmatrix} 1 & 0 & \cdots & 0 & 0 \\ 1 & x_1 - x_0 & 0 & \cdots & 0 \\ \vdots & \vdots & \ddots & \ddots & \vdots \\ 1 & x_{n-1} - x_0 & \cdots & \prod_{j=0}^{n-2}(x_{n-1} - x_j) & 0 \\ 1 & x_n - x_0 & \cdots & \prod_{j=0}^{n-2}(x_n - x_j) & \prod_{j=0}^{n-1}(x_n - x_j) \end{pmatrix} \begin{pmatrix} \beta_0 \\ \beta_1 \\ \vdots \\ \beta_{n-1} \\ \beta_n \end{pmatrix} = \begin{pmatrix} f_0 \\ f_1 \\ \vdots \\ f_{n-1} \\ f_n \end{pmatrix}.$$

By changing the basis and how we represented the polynomial now the linear system is lower triangular! Provided all the of the interpolation points $x_0, \ldots, x_n$ are distinct, this matrix is invertible.

For the general case, we use forward substitution to determine the coefficients:

$$\beta_0 = f_0, \quad \beta_1 = \frac{f_1 - f_0}{x_1 - x_0}, \quad \beta_2 = \frac{f_2 - f_0 - \frac{x_2 - x_0}{x_1 - x_0}(f_1 - f_0)}{(x_2 - x_0)(x_2 - x_1)}, \quad \text{etc.}$$

The analytical expressions for the coefficients become increasingly complicated and unwieldy. Notice that the coefficients all involve a ratio of difference(s) between the known function data and the distance(s) between the interpolation points. We can exploit this fact and recursively take differences between function data and interpolation points to build the necessary coefficients, e.g.,

$$f[x_0] = f(x_0) = f_0, \quad f[x_0, x_1] = \frac{f_1 - f_0}{x_1 - x_0}, \quad f[x_0, x_1, x_2] = \frac{f[x_1, x_2] - f[x_0, x_1]}{x_2 - x_0}.$$

Definition 6.2 (Newton divided difference). The kth *divided difference* of f is given by

$$f[x_0, \ldots, x_k] = \frac{f[x_1, \ldots, x_k] - f[x_0, \ldots, x_{k-1}]}{x_k - x_0},$$

with respect to the points $x_0, \ldots, x_k$ ◀

The divided differences gives a straightforward algorithm to determine the coefficients of the interpolating polynomial for the Newton basis. Conveniently, we can arrange the data needed for the interpolating polynomial into a *Newton divided difference table*. This is a convenient organizational technique for the data needed to build this particular form of the interpolating polynomial.

For instance, if we have four data points the structure of the Newton divided difference table is shown at the right. One can actually take any connected path through the table to define the coefficients of the interpolating polynomial. Each path will change the particular value in front of the terms in the Newton basis. But the actual interpolant does not change because the interpolating polynomial is unique! In the figure at the right, we indicate the *typical* path one takes through the divided difference table to construct the Newton interpolating polynomial.

$$
\begin{array}{llll}
x_0 & f_0 & & \\
 & & f[x_0,x_1] & \\
x_1 & f_1 & & f[x_0,x_1,x_2] \\
 & & f[x_1,x_2] & & f[x_0,x_1,x_2,x_3] \\
x_2 & f_2 & & f[x_1,x_2,x_3] \\
 & & f[x_2,x_3] & \\
x_3 & f_3 & &
\end{array}
$$

DEFINITION 6.3 (NEWTON INTERPOLATING POLYNOMIAL). The typical form of the interpolating polynomial created with Newton's strategy is

$$
\begin{aligned}
p_n(x) &= f_0 + f[x_0,x_1](x-x_0) + f[x_0,x_1,x_2](x-x_0)(x-x_1) + \cdots + f[x_0,\ldots,x_n](x-x_0)(x-x_1)\cdots(x-x_{n-1}) \\
&= f_0 + f[x_0,x_1]N_1(x) + f[x_0,x_1,x_2]N_2(x) + \cdots + f[x_0,\ldots,x_n]N_n(x),
\end{aligned}
$$

where $N_i(x)$ is the Newton basis described above with a given data set $(x_i, f(x_i))$ and $i = 0,\ldots,n$. ◀

EXAMPLE 6.2. Use the data set from EXAMPLE 6.1 to create a Newton interpolating polynomial. First, we compute the values for the Newton divided difference table:

x	$f(x)$	$f[\cdot,\cdot]$	$f[\cdot,\cdot,\cdot]$	$f[\cdot,\cdot,\cdot,\cdot]$	$f[\cdot,\cdot,\cdot,\cdot,\cdot]$
-2	-9				
		-6			
-1	-15		8		
		10		-4	
0	-5		-4		3
		2		8	
1	-3		20		
		42			
2	39				

The values we use for the interpolant are highlighted and we find the Newton polynomial to be

$$p_4(x) = -9 - 6(x+2) + 8(x+1)(x+2) - 4x(x+1)(x+2) + 3x(x-1)(x+1)(x+2).$$

As expected, if we expand the Newton interpolating polynomial into monomial form it is identical to the previous computation in the monic basis with the Vandermonde matrix. ♦

The second common tactic to rewrite the interpolating polynomial we cover is due to Lagrange. Here, we use an alternative set of polynomial functions as our basis:

DEFINITION 6.4 (LAGRANGE POLYNOMIALS). Given a set of distinct interpolation points $x_0,\ldots,x_n$ the *Lagrange polynomials* are n^{th} degree polynomials defined as

$$\ell_i(x) = \prod_{\substack{j=0 \\ j\neq i}}^{n} \frac{x - x_j}{x_i - x_j}, \quad \text{where} \quad i = 0,\ldots,n.$$

By design the Lagrange polynomials satisfy the identity

$$\ell_i(x_k) = \begin{cases} 1, & \text{if } i = k \\ 0, & \text{if } i \neq k \end{cases}$$

also known as the cardinality property. ◀

Remark 6.5. Notice that the Lagrange basis polynomials ***are not*** hierarchical! Each one of them is a polynomial of degree n. ⋈

What is gained when we move to this more sophisticated set of basis functions? It turns out it is trivial to write the interpolating polynomial in Lagrange form

$$p_n(x) = \sum_{i=0}^{n} \gamma_i \ell_i(x).$$

because of the cardinality property of the basis functions $\ell_i(x)$. When we apply the interpolation constraints to the interpolating polynomial written in Lagrange form we obtain the system

$$\begin{pmatrix} 1 & 0 & \cdots & 0 & 0 \\ 0 & 1 & \cdots & 0 & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & \cdots & 1 & 0 \\ 0 & 0 & \cdots & 0 & 1 \end{pmatrix} \begin{pmatrix} \gamma_0 \\ \gamma_1 \\ \vdots \\ \gamma_{n-1} \\ \gamma_n \end{pmatrix} = \begin{pmatrix} f_0 \\ f_1 \\ \vdots \\ f_{n-1} \\ f_n \end{pmatrix}.$$

Definition 6.5 (Lagrange interpolating polynomial). Given a set of distinct interpolation points $x_0, \ldots, x_n$ with corresponding data $f_0, \ldots, f_n$ then

$$p_n(x) = \sum_{i=0}^{n} f_i \ell_i(x),$$

is the *Lagrange interpolating polynomial*. ◀

Example 6.3. Use the data set from Example 6.1 to create a Lagrange interpolating polynomial. We already have the function data, so once we construct the appropriate Lagrange polynomials we are done:

$$\begin{aligned}
\ell_0 &= \frac{(x+1)(x-0)(x-1)(x-2)}{(-2+1)(-2)(-2-1)(-2-2)} &&= \frac{(x+1)(x-0)(x-1)(x-2)}{24} \\
\ell_1 &= \frac{(x+2)(x-0)(x-1)(x-2)}{(-1+2)(-1)(-1-1)(-1-2)} &&= \frac{(x+2)(x-0)(x-1)(x-2)}{-6} \\
\ell_2 &= \frac{(x+2)(x+1)(x-1)(x-2)}{(0+2)(0+1)(0-1)(0-2)} &&= \frac{(x+2)(x+1)(x-1)(x-2)}{4} \\
\ell_3 &= \frac{(x+2)(x+1)(x-0)(x-2)}{(1+2)(1+1)(1)(1-2)} &&= \frac{(x+2)(x+1)(x-0)(x-2)}{-6} \\
\ell_4 &= \frac{(x+2)(x+1)(x-0)(x-1)}{(2+2)(2+1)(2)(2-1)} &&= \frac{(x+2)(x+1)(x-0)(x-1)}{24}
\end{aligned}$$

Therefore, the Lagrange interpolating polynomial $p_4(x) = \sum_{i=0}^{4} f_i \ell_i(x)$ is

$$\begin{aligned}
p_4(x) = &-9\left[\frac{(x+1)(x-0)(x-1)(x-2)}{24}\right] - 15\left[\frac{(x+2)(x-0)(x-1)(x-2)}{-6}\right] - 5\left[\frac{(x+2)(x+1)(x-1)(x-2)}{4}\right] \\
&- 3\left[\frac{(x+2)(x+1)(x-0)(x-2)}{-6}\right] + 39\left[\frac{(x+2)(x+1)(x-0)(x-1)}{24}\right]
\end{aligned}$$

Again, if we expand the Lagrange representation into monic form we recover the same polynomial. ♦

Now we have several strategies to create the polynomial interpolant. Next, we address an efficient way to evaluate the resulting polynomial function. Consider the monic form of a polynomial $p_n(x) = \alpha_0 + \alpha_1 x + \alpha_2 x^2 + \cdots + \alpha_n x^n$. Naively, for a given value of x, one could evaluate each of the polynomial powers, scale with the appropriate constant α_i, and add them together to obtain the result. **But**, this amounts to $\mathcal{O}(n^2)$ complexity and is prone to cancellation errors because the power of x could be very large. To reduce the computational cost and sensitivity of the polynomial evaluation, we use a *factoring trick*:

Definition 6.6 (Horner's rule). To reduce the computation cost, we evaluate the monic polynomial in the form

$$p_n(x) = \alpha_0 + x\,(\alpha_1 + x\,(\alpha_2 + x\,(\alpha_3 + \cdots + x\,(\alpha_{n-1} + x\alpha_n)\cdots)))$$

which requires only n multiplications and n additions, i.e. $\mathcal{O}(n)$. It also reduces cancellation errors. ◀

Algorithm 6.1: *Horner's Rule*: Efficiently evaluate a polynomial at a given value

Procedure Horner's Rule
Input: $\boldsymbol{\alpha}$, x, n `// vector of coefficients, value, and polynomial degree`
$p \leftarrow \alpha_n$
for $k = n-1$ **to** 0 **step** -1 **do**
  $p \leftarrow p * x + \alpha_k$
Output: p `//` `evaluation of` $p_n(x)$
End Procedure Horner's Rule

Remark 6.6. Newton's form of the interpolating polynomial can also be rewritten with Horner's rule as

$$p_n(x) = f_0 + (x - x_0)(f[x_0, x_1] + (x - x_1)(f[x_0, x_1, x_2] + \cdots + (x - x_{n-2})(f[x_0, \cdots, x_{n-1}] + f[x_0, \cdots, x_n](x - x_{n-1})\cdots))$$

and improve performance provided one gives the procedure the interpolation nodes $x_0, \ldots, x_n$ as well. ⋈

Remark 6.7. Here we summarize the computational costs to construct the different forms of the polynomial interpolant as well as its evaluation:

	Computing interpolation coefficients	Evaluation of $p_n(x)$
Monomial form	$\mathcal{O}(n^3)$	$\mathcal{O}(n)$
Newton form	$\mathcal{O}(n^2)$	$\mathcal{O}(n)$
Lagrange form	0	$\mathcal{O}(n^2)$

Notice, that the work can be reduced by considering different basis representations but it has to be done at some point. This highlights an overarching principle in numerical mathematics. There is computational work that must be performed, but we can concentrate it in different areas depending on the formulation. But remember, *there is no free lunch*! ⋈

Remark 6.8. Unfortunately, the current form of the Lagrange interpolating polynomial **cannot** be factored according to Horner's rule. However, it is possible to recast the Lagrange basis functions into the so-called *barycentric formulation* that can exploit Horner's rule, but this topic is outside the scope of this course. ⋈

Finally, we discuss the error of the global interpolating polynomial in approximating a particular function $f(x)$. Note, we **will not** discuss convergence because it is very mathematically technical. Instead, we will only examine the pointwise error between a function and an nth degree polynomial interpolant.

Definition 6.7 (Pointwise error). Let $[a, b]$ be an interval on which $f(x)$ is $(n+1)$-times differentiable. Further, we assume that the interpolation nodes $x_0, \ldots, x_n$ as well as a value x are all contained within $[a, b]$. Then the *pointwise error* between the function and a global polynomial interpolant for values $f_0, \ldots, f_n$ is

$$E_n(x) = f(x) - p_n(x) = \frac{(x - x_0)(x - x_1)\cdots(x - x_n)}{(n+1)!} f^{(n+1)}(\xi) = \frac{\Psi_n(x)}{(n+1)!} f^{(n+1)}(\xi),$$

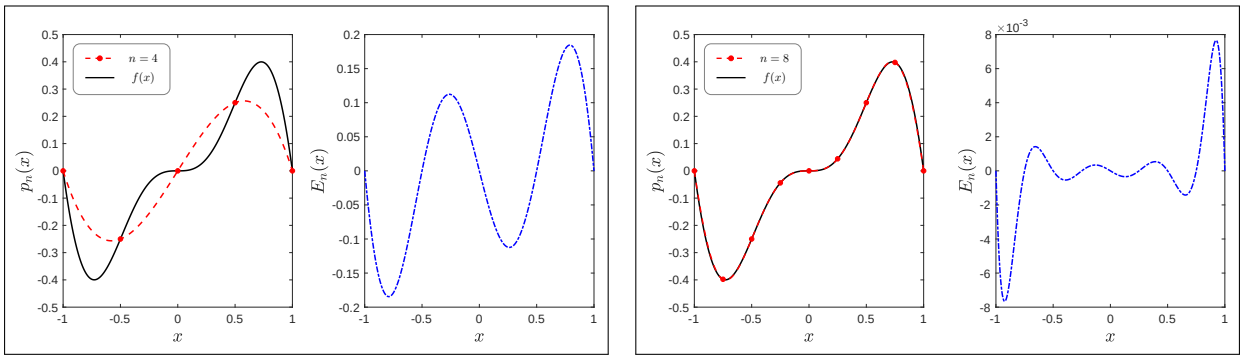
for some value of $\xi \in [a, b]$ and we introduce a compact notation $\Psi_n(x) = (x - x_0)(x - x_1)\cdots(x - x_n)$. ◀

From the construction of the interpolating polynomial we know that the error at the interpolation nodes is zero, i.e. $E_n(x_i) = 0$ for $i = 0, \ldots, n$. Note that this pointwise error tends to be very oscillatory. Also, if the derivative of the function $f^{(n+1)}(x)$ is bounded on $[a, b]$ then the error can be estimated. However, we can encounter issues if $f^{(n+1)}(x)$ grows faster than the factorial $(n+1)!$ or if $\Psi_n(x)$ is large.

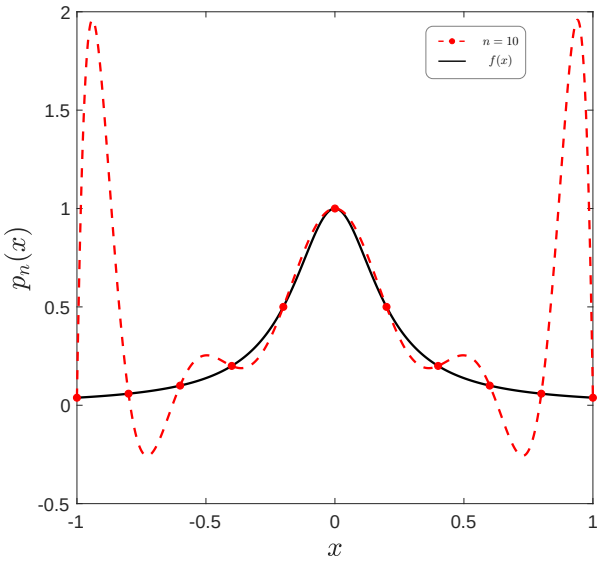
EXAMPLE 6.4. Examine the pointwise error of a fourth degree and eighth degree interpolating polynomial of the function $f(x) = x^2 \sin(\pi x)$ on the interval $[-1, 1]$ with uniformly spaced points. For the two interpolating polynomials the equally spaced nodes will be

$$\{x_i\}_{i=0}^{4} = \left\{-1, -\frac{1}{2}, 0, \frac{1}{2}, 1\right\} \quad \text{and} \quad \{x_i\}_{i=0}^{8} = \left\{-1, -\frac{3}{4}, -\frac{1}{2}, -\frac{1}{4}, 0, \frac{1}{4}, \frac{1}{2}, \frac{3}{4}, 1\right\}.$$

The higher order derivatives of the function $f(x)$ are nicely bounded and the pointwise error is dominated by the factorial term. Therefore, we expect the pointwise error to behave well in that its overall magnitude should shrink if we construct an interpolant with more points.



We examine this trend in the plots of the interpolating polynomial, the function $f(x)$, and the pointwise error above on the left for $n = 4$ and the right for $n = 8$. We observe that the pointwise error indeed becomes smaller with more interpolation nodes. Although, notice that the error remains the largest near the endpoints of the interval. ♦



Is this always the case that more *uniformly* spaced interpolation points yields a more accurate global interpolating polynomial to a function? No! This is not the case. To demonstrate, we consider the function

$$f(x) = \frac{1}{1 + 25x^2}$$

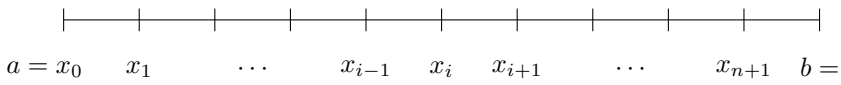
We see in the figure on the left that near the endpoints the interpolating polynomial is a very poor approximation of the function $f(x)$ although in the interior it looks quite good. This observation is confirmed with the pointwise error where the interior is significantly more accurate than close to the boundary. The ringing near the boundaries are known as ***Runge phenomena***. One cause is the natural, oscillatory behaviors of a high order polynomial function. Next, we examine how to alleviate this spurious ringing and obtain better accuracy with regards to interpolation near the boundary.

6.2 Piecewise polynomial interpolation

Previously, we constructed a global polynomial function, $p_n(x)$, to interpolate a function at a given set of data points. As we increase the number of interpolation points, the global polynomial gets a higher degree. In order to capture the given data this global polynomial oscillates, sometimes a great deal, in between the interpolation points even when the function $f(x)$ we want to approximate is *nonoscillatory*. For instance, recall the interpolation of the function $f(x) = (1 + 25x^2)^{-1}$.

To remove this oscillatory behavior of a high degree polynomial interpolant we alter the interpolation strategy and move from a global perspective to a **local** one. The motivation for this change comes from the observation that we get a poor global approximation with a low degree polynomial but no wild oscillations, whereas a high degree polynomial is accurate in parts of the interval but can have wild oscillations in another part of the interval.

Suppose we still have an interval $[a, b]$ and a set of data points $\{x_i\}_{i=0}^n$. For simplicity we assume that these data points are uniformly spaced. Now, we will *subdivide* the interval $a = x_0 < x_1 < \cdots < x_n = b$



This produces n sub-intervals $[x_{i-1}, x_i]$ where $i = 1, \ldots, n$. Within each of these sub-intervals we then use a low degree polynomial to approximate the function $f(x)$. The global interpolant is then the union of these local, low degree polynomial approximations on the nonoverlapping sub-intervals.

But how do we construct these local approximations? We constrain this procedure with the requirement that the global approximation is *smooth* by enforcing continuity as well as continuity of some number of its derivatives. This is the principle behind constructing ***spline interpolation***.

Definition 6.8 (Spline interpolation). A function $s_{2m+1}(x)$ is a spline of degree $2m + 1$ when $s_m(x)$ is a union of piecewise polynomials of degree $2m + 1$ such that $s_{2m+1}(x)$ **and** its first $2m$ derivatives are continuous. When the spline is constructed such that $s_{2m+1}(x_i) = f(x_i)$, $i = 0, 1, \ldots, n$ it is a *spline interpolant*. ◀

Remark 6.9. Two popular forms are the *linear spline* when $m = 0$ and the *cubic spline* when $m = 1$. ⋈

First, we discuss the linear spline interpolant. As such, suppose we have the set of data points $\{(x_i, f_i)\}_{i=0}^n$ that we want to interpolate. We produce sub-intervals $[x_{i-1}, x_i]$ and on each sub-interval we approximate the function with a straight line. Thus, the linear spline $s_1(x)$ is composed of n line segments with the form

$$s_1(x) = a_i + b_i(x - x_{i-1}), \quad \text{where} \quad x_{i-1} \le x \le x_i.$$

We must determine the $2n$ coefficients of the different line segments. We enforce the continuity of the linear spline between the sub-intervals, which gives the conditions

$$a_i + b_i(x - x_{i-1}) = a_{i+1}, \quad \text{where} \quad i = 1, \ldots, n - 1.$$

This leaves $2n - (n - 1) = n + 1$ degrees of freedom. To close the system and explicitly determine the spline coefficients we remove these degrees of freedom with the requirement that the linear spline take the given function values at the sub-interval boundaries

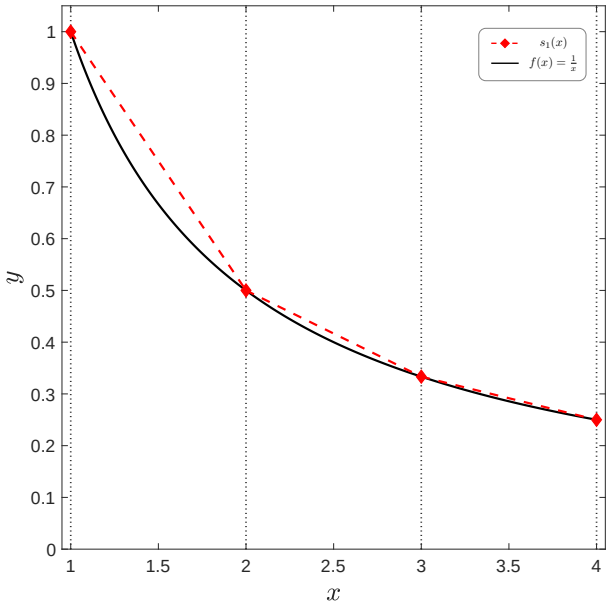
$$s_1(x_i) = f_i, \quad \text{where} \quad i = 0, \ldots, n.$$

This yields a general form for the linear spline in each sub-interval

Definition 6.9 (Linear spline interpolant). Given a set of data points $\{(x_i, f_i)\}_{i=0}^n$ in the interval $[a, b]$ where the values $\{x_i\}_{i=0}^n$ are distinct, the linear interpolating spline can be written as

$$s_1(x) = f_{i-1} + \frac{f_i - f_{i-1}}{x_i - x_{i-1}}(x - x_{i-1}), \quad \text{where} \quad x_{i-1} \le x \le x_i$$

and $i = 1, \ldots, n$. ◀



Let us examine how the linear spline performs.

EXAMPLE 6.5. Construct a linear spline for the data points

$$\{(x_i, f_i)\}_{i=0}^{3} = \left\{(1,1)\ ,\ \left(2,\frac{1}{2}\right)\ ,\ \left(3,\frac{1}{3}\right)\ ,\ \left(4,\frac{1}{4}\right)\right\}$$

and compare it to the function $f(x) = x^{-1}$. We divide the interval $[1,4]$ into three sub-intervals and compute the linear spline to be

$$s_1(x) = \begin{cases} 1 + \frac{\frac{1}{2}-1}{2-1}(x-1) = 1 - \frac{1}{2}(x-1), & \text{if } 1 \le x \le 2 \\ \frac{1}{2} + \frac{\frac{1}{3}-\frac{1}{2}}{3-2}(x-2) = \frac{1}{2} - \frac{1}{6}(x-2), & \text{if } 2 \le x \le 3 \\ \frac{1}{3} + \frac{\frac{1}{4}-\frac{1}{3}}{4-3}(x-3) = \frac{1}{3} - \frac{1}{12}(x-3), & \text{if } 3 \le x \le 4 \end{cases}$$

We plot the linear spline and visually compare the approximation to the known function $f(x) = \frac{1}{x}$.

Additionally, we use vertical dotted lines to distinguish between the different sub-intervals. We see that the linear spline is, essentially, a mathematical and systematic strategy to **connect-the-dots** between the different known function points. Note, we used very few sub-intervals to demonstrate the idea behind the linear spline. Thus, we see that the approximation is quite crude. ♦

Locally, within each of the sub-intervals, we use a linear polynomial to approximate the function. Therefore, we have the same pointwise error estimate from before inside each sub-interval. We use this to state an error estimate for the linear spline **globally**. To do so, we denote the size of each sub-interval to be

$$h_i = x_i - x_{i-1}$$

where $i = 1, \ldots, n$.

DEFINITION 6.10 (GLOBAL ERROR OF LINEAR SPLINE). For a function $f(x)$ that has two continuous derivatives on the interval $[a,b]$ the *global error* of the linear spline is bounded by

$$E_n^{s_1} = \max_{x\in[a,b]} |f(x) - s_1(x)| \le \frac{h^2}{8} \max_{x\in[a,b]} |f''(x)| \sim \mathcal{O}(h^2),$$

where $h = \max_{i=1,\ldots,n} \{h_i\}$ is the size of the largest sub-interval. ◄

We see that the global error shrinks quadratically as we add more sub-intervals for the linear spline. This means that if we double the number of sub-intervals the error reduces by a factor of four. So, the linear spline is a second order method to approximate a suitably smooth function $f(x)$.

EXAMPLE 6.6. Verify that the linear spline converges at second order to the function $f(x) = \frac{1}{x}$ on the interval $[1,4]$. We note that the function is twice differentiable in the given interval, so the global error estimate is valid. We compute the error for increasing values of n on a uniform set of points $\{x_i\}_{i=0}^{n}$. In turn, this increases the number of sub-intervals and decreases the value of h.

Let us examine the ratio of two errors where the number of points differ by a factor of two. From above we have the global error estimates for the linear spline

$$E_n^{s_1} \approx h^2, \quad \text{and} \quad E_{2n}^{s_1} \approx \left(\frac{h}{2}\right)^2 = \frac{h^2}{2^2} \quad \Rightarrow \quad \frac{E_n^{s_1}}{E_{2n}^{s_1}} \approx 2^2$$

n	$E_n^{s_1}$	EOC
2	0.25000	—
4	0.08547	1.54
8	0.02666	1.68
16	0.00755	1.82
32	0.00202	1.90
64	0.00052	1.96

If we take the logarithm and manipulate we see that this error ratio can reveal the experimental order of convergence (EOC) for the approximation

$$EOC \equiv \frac{\log\left(\frac{E_n^{s1}}{E_{2n}^{s1}}\right)}{\log(2)} \approx 2$$

We collect values of the error for different numbers of sub-intervals as well as the EOC in the table above. Notice that the EOC manipulation above was always done approximately. That is why for very coarse resolutions of the linear spline the EOC is worse than for fine resolutions where it becomes closer to the expected value. ♦

We see that this *connect-the-dots* style of linear spline works, but straight lines are a poor approximation for curves. Therefore, we can improve the accuracy of the spline interpolant by using a higher degree polynomial within each sub-interval.

Next, we examine *cubic splines* where the function is approximated on the interval $[a, b]$ within each sub-interval with a cubic polynomial with the general form

$$s_3(x) = a_i + b_i\left(\frac{x - x_{i-1}}{h_i}\right) + c_i\left(\frac{x - x_{i-1}}{h_i}\right)^2 + d_i\left(\frac{x - x_{i-1}}{h_i}\right)^3, \quad \text{where} \quad x_{i-1} \le x \le x_i$$

and $i = 1, \ldots, n$. So we must determine $4n$ coefficients to define the cubic spline. We enforce continuity of the spline as well as its first and second derivatives

$$\lim_{x \to x_i^+} s_3(x) = \lim_{x \to x_i^-} s_3(x), \quad \lim_{x \to x_i^+} s_3'(x) = \lim_{x \to x_i^-} s_3'(x), \quad \text{and} \quad \lim_{x \to x_i^+} s_3''(x) = \lim_{x \to x_i^-} s_3''(x)$$

for $i = 1, \ldots, n-1$ in the interval $[a, b]$. The continuity of the spline provides $3(n-1)$ conditions, so we have a remaining $4n - 3(n-1) = n + 3$ degrees of freedom. The interpolation requirements $s_3(x_i) = f_i$, $i = 0, \ldots, n$ are an additional $n + 1$ conditions. To remove the remaining two degrees of freedom we apply two *boundary conditions*. The simplest of which are the following:

Definition 6.11 (Natural boundary condition). We take interpolating cubic spline to satisfy

$$s_3''(x_0) = 0 \quad \text{and} \quad s_3''(x_n) = 0,$$

at the boundaries of the interval $[a, b]$. These boundary conditions are *natural* in the sense that the spline is assumed to be a straight line outside the interval and its second derivative vanishes. ◀

Remark 6.10. There are other types of boundary conditions available to "close" the spline. However, they often require additional knowledge about the derivative(s) of the function $f(x)$. ⋈

Remark 6.11. There are many strategies to determine the coefficients of the cubic spline interpolant. In the following, we present a construction strategy *that is different than the book*. ⋈

In order to build a cubic spline we first assume we have a set of n data points $\{(x_i, f_i)\}_{i=0}^n$ with the corresponding sub-intervals $[x_{i-1}, x_i]$, $i = 1, \ldots, n$. Next, we consider the second derivative of the spline $s_3''(x)$ that we know to be a linear polynomial within each of the sub-intervals. Further, we say that the second derivative of the cubic spline evaluates to $n + 1$ *yet to be determined* values

$$s_3''(x_i) = M_i, \quad \text{where} \quad i = 0, \ldots, n.$$

For this discussion we consider the arbitrary interval $[x_{i-1}, x_i]$. We impose that $s_3''(x)$ is continuous at the internal nodes to find

$$s_3''(x) = M_{i-1}\frac{(x_i - x)}{h_i} + M_i\frac{(x - x_{i-1})}{h_i}, \quad \text{in} \quad x_{i-1} \le x \le x_i$$

where $i = 1, \ldots, n$. We integrate twice to find the expression for the first derivative and the cubic spline

$$s_3'(x) = -M_{i-1}\frac{(x - x_i)^2}{2h_i} + M_i\frac{(x - x_{i-1})^2}{2h_i} + \gamma_{i-1}, \quad \text{in} \quad x_{i-1} \le x \le x_i$$

$$s_3(x) = M_{i-1}\frac{(x_i - x)^3}{6h_i} + M_i\frac{(x - x_{i-1})^3}{6h_i} + \gamma_{i-1}(x - x_{i-1}) + \tilde{\gamma}_{i-1}, \quad \text{in} \quad x_{i-1} \le x \le x_i$$

To eliminate the $2n$ constants of integration, γ_{i-1} and $\tilde{\gamma}_{i-1}$ with $i = 1, \ldots, n$, we apply the interpolation conditions as well as the continuity of the spline $s_3(x)$

$$s_3(x_0) = f_0, \quad s_3(x_n) = f_n, \quad \text{and} \quad \lim_{x \to x_i^+} s_3(x) = \lim_{x \to x_i^-} s_3(x) = f_i$$

for $i = 1, \ldots, n-1$. From this we determine that

$$\gamma_{i-1} = \frac{f_i - f_{i-1}}{h_i} - \frac{h_i}{6}(M_i - M_{i-1}), \quad \tilde{\gamma}_{i-1} = f_{i-1} - M_{i-1}\frac{h_i^2}{6}, \quad \text{where} \quad i = 1, \ldots, n.$$

Substituting these values and enforcing the continuity of the first derivative of the cubic spline

$$\lim_{x \to x_i^+} s_3'(x) = \lim_{x \to x_i^-} s_3'(x),$$

gives us (after some algebraic manipulation)

$$\frac{h_i}{6}M_{i-1} + \frac{h_i}{3}M_i + \frac{f_i - f_{i-1}}{h_i} = -\frac{h_{i+1}}{3}M_i - \frac{h_{i+1}}{6}M_{i+1} + \frac{f_{i+1} - f_i}{h_{i+1}}, \quad \text{where} \quad i = 1, \ldots, n-1.$$

We separate the *still unknown* values of M_i from the known values and gather terms to find

$$\frac{h_i}{6}M_{i-1} + \frac{h_{i+1} + h_i}{6}M_i + \frac{h_{i+1}}{6}M_{i+1} = \frac{(f_{i+1} - f_i)}{h_{i+1}} - \frac{(f_i - f_{i-1})}{h_i}.$$

Then, we multiply through by the value $6/(h_{i+1} + h_i)$ and introduce the constants (for convenience)

$$\mu_i = \frac{h_i}{h_{i+1} + h_i}, \quad \sigma_i = \frac{h_{i+1}}{h_{i+1} + h_i}, \quad \text{and} \quad \eta_i = \frac{6}{h_{i+1} + h_i}\left(\frac{(f_{i+1} - f_i)}{h_{i+1}} - \frac{(f_i - f_{i-1})}{h_i}\right),$$

where $i = 1, \ldots, n-1$. This reveals that the problem to find the cubic spline coefficients is a *tridiagonal linear system*

$$\begin{pmatrix} \mu_1 & 2 & \sigma_1 & 0 & \cdots & \cdots & 0 \\ 0 & \mu_2 & 2 & \sigma_2 & 0 & \cdots & \vdots \\ \vdots & \ddots & \ddots & \ddots & \ddots & \ddots & \vdots \\ \vdots & \ddots & 0 & \mu_{n-2} & 2 & \sigma_{n-2} & 0 \\ 0 & \cdots & \cdots & 0 & \mu_{n-1} & 2 & \sigma_{n-1} \end{pmatrix} \begin{pmatrix} M_0 \\ M_1 \\ \vdots \\ M_{n-1} \\ M_n \end{pmatrix} = \begin{pmatrix} \eta_1 \\ \eta_2 \\ \vdots \\ \eta_{n-2} \\ \eta_{n-1} \end{pmatrix}.$$

This is a system of $n+1$ equations for $n-1$ unknown values. *Finally*, we impose the general boundary conditions $s_3''(x_0) = M_0 = f''(x_0)$ and $s_3''(x_n) = M_n = f''(x_n)$ that account for the curvature of the function. This yields a square system of equations

$$\begin{pmatrix} 2 & \sigma_1 & 0 & \cdots & 0 \\ \mu_2 & 2 & \sigma_2 & & \vdots \\ 0 & \ddots & \ddots & \ddots & 0 \\ \vdots & & \mu_{n-2} & 2 & \sigma_{n-2} \\ 0 & \cdots & 0 & \mu_{n-1} & 2 \end{pmatrix} \begin{pmatrix} M_1 \\ M_2 \\ \vdots \\ M_{n-2} \\ M_{n-1} \end{pmatrix} = \begin{pmatrix} \eta_1 - M_0\mu_1 \\ \eta_2 \\ \vdots \\ \eta_{n-2} \\ \eta_{n-1} - M_n\sigma_{n-1} \end{pmatrix}.$$

For natural boundary conditions $M_0 = M_n = 0$, otherwise we require some information about the second derivative of the function we want to interpolate.

Remark 6.12. The system is nonsingular because the constants $\mu_i, \sigma_i < 1$, so the tridiagonal matrix is strongly diagonally dominant. In fact this linear system is symmetric positive definite. Therefore, we can apply Cholesky or Gauss-Seidel to solve it. ⋈

Once we solve this square, nonsingular linear system we have all the necessary information for our cubic spline, even though it might not seem like it. We collect the pieces of the interpolating cubic spline:

DEFINITION 6.12 (NATURAL CUBIC SPLINE INTERPOLANT). Given the coefficients $\{M_i\}_{i=0}^n$ found by solving the tridiagonal system with natural boundary conditions then the *cubic spline interpolant* is given by

$$s_3(x) = M_{i-1}\frac{(x_i - x)^3}{6h_i} + M_i\frac{(x - x_{i-1})^3}{6h_i} + \gamma_{i-1}(x - x_{i-1}) + \tilde{\gamma}_{i-1}, \quad \text{in} \quad x_{i-1} \le x \le x_i$$

with the coefficients

$$\gamma_{i-1} = \frac{f_i - f_{i-1}}{h_i} - \frac{h_i}{6}(M_i - M_{i-1}), \quad \tilde{\gamma}_{i-1} = f_{i-1} - M_{i-1}\frac{h_i^2}{6},$$

where $i = 1, \ldots, n$. ◀

Oy, that was a lot work! But now we have a more sophisticated spline interpolant for a function $f(x)$.

EXAMPLE 6.7. Construct a natural cubic spline interpolant for the data points

$$\{(x_i, f_i)\}_{i=0}^3 = \left\{(1,1)\ ,\ \left(2, \frac{1}{2}\right)\ ,\ \left(3, \frac{1}{3}\right)\ ,\ \left(4, \frac{1}{4}\right)\right\},$$

and compare it to the function $f(x) = x^{-1}$. From the natural boundary conditions we know that $M_0 = M_3 = 0$ and because the nodes are uniform we have that $\mu_i = \sigma_i = \frac{1}{2}$ and the right hand side values are

$$\eta_1 = \frac{6}{2}\left(\frac{1}{3} - \frac{1}{2} - \left(\frac{1}{2} - 1\right)\right) = 3\left(-\frac{1}{6} + \frac{1}{2}\right) = 1, \quad \eta_2 = \frac{6}{2}\left(\frac{1}{4} - \frac{1}{3} - \left(\frac{1}{3} - \frac{1}{2}\right)\right) = 3\left(-\frac{1}{12} + \frac{1}{6}\right) = \frac{1}{4}.$$

This simplifies the tridiagonal matrix into a the 2×2 linear system

$$\begin{pmatrix} 2 & \frac{1}{2} \\ \frac{1}{2} & 2 \end{pmatrix}\begin{pmatrix} M_1 \\ M_2 \end{pmatrix} = \begin{pmatrix} 1 \\ \frac{1}{4} \end{pmatrix} \quad \Rightarrow \quad \begin{pmatrix} M_1 \\ M_2 \end{pmatrix} = \begin{pmatrix} \frac{1}{2} \\ 0 \end{pmatrix}.$$

We also compute the other necessary constants needed for the cubic spline

$$i = 1: \quad \gamma_0 = f_1 - f_0 - \frac{1}{6}(M_1 - M_0) = \frac{1}{2} - 1 - \frac{1}{6}\left(\frac{1}{2} - 0\right) = -\frac{7}{12}, \quad \tilde{\gamma}_0 = f_0 - M_0\frac{h_i^2}{6} = 1 - 0 = 1$$

$$i = 2: \quad \gamma_1 = -\frac{1}{12}, \quad \tilde{\gamma}_1 = \frac{5}{12}$$

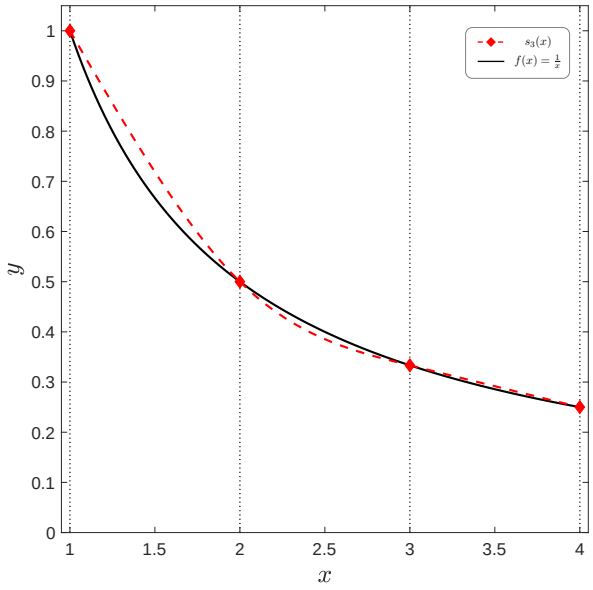
$$i = 3: \quad \gamma_2 = -\frac{1}{12}, \quad \tilde{\gamma}_2 = \frac{1}{3}$$

We then assemble the complete cubic spline interpolant

$$s_3(x) = \begin{cases} \frac{1}{12}(x-1)^3 - \frac{7}{12}(x-1) + 1, & \text{if } 1 \le x \le 2 \\ -\frac{1}{12}(x-3)^3 - \frac{1}{12}(x-2) + \frac{5}{12}, & \text{if } 2 \le x \le 3 \\ -\frac{1}{12}(x-3) + \frac{1}{3}, & \text{if } 3 \le x \le 4 \end{cases}$$

We plot the cubic interpolating spline on the left at the top of the next page. ⧫

Remark 6.13. Note that in the example we used uniform nodes, but the splines (either linear or cubic) can use nonuniform nodes to define the sub-intervals. In the case of the cubic spline interpolant it simply makes the linear system to determine the coefficients slightly more complicated, but still solvable. ⋈



We see from the previous example that the coarse cubic spline interpolant resembles the linear spline interpolant for the function $f(x) = x^{-1}$. So what advantage does the more sophisticated cubic spline interpolant give us? In general, the local error within each sub-interval will be smaller because the local cubic function can take the curvature of the function into account. Therefore, we can create a bound for the global error of the cubic spline interpolant.

DEFINITION 6.13 (GLOBAL ERROR OF CUBIC SPLINE). If $s_3(x)$ is an interpolatory cubic spline constructed with natural boundary conditions then its *global error* is bounded by

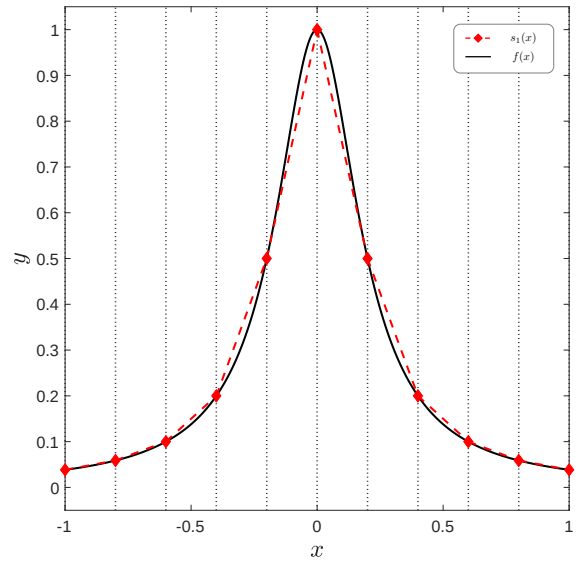
$$E_n^{s_3} = \max_{x\in[a,b]} |f(x) - s_3(x)| \leq \frac{5h^4}{384} \max_{x\in[a,b]} |f^{(4)}(x)| \sim \mathcal{O}(h^4)$$

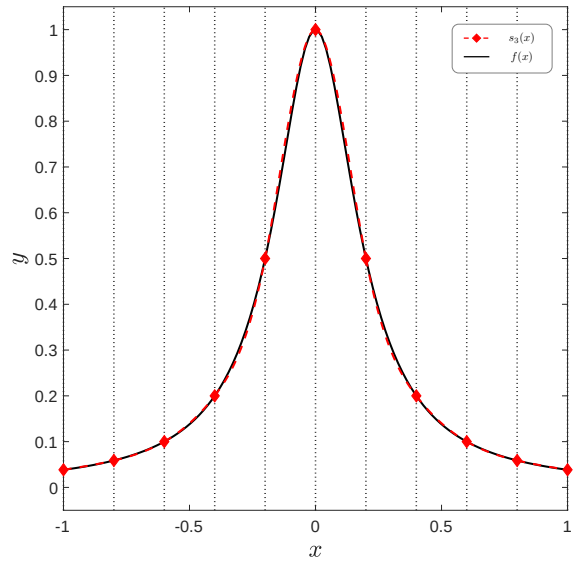
provided the function $f(x)$ has four continuous derivatives. ◀

Remark 6.14. The global error of the cubic spline interpolant also depends on the boundary conditions used to create it. In fact, the accuracy and convergence rate of the cubic spline can be improved if boundary conditions that account for derivative information of the underlying function $f(x)$ are used. ⋈

We see that creating a spline interpolant with higher degree polynomials increases the global accuracy as well as increases how quickly the spline interpolant converges to the function $f(x)$. In fact, we see that if we double the number of sub-intervals the global error of the cubic spline interpolant reduces by a factor of sixteen. So, the cubic spline is a fourth order method to approximate a suitably smooth function $f(x)$.

But what about our original goal of *removing* the Runge phenomena which can occur for the global interpolation strategy? We revisit creating an interpolant of the function $f(x) = (1 + 25x^2)^{-1}$ but this time use a linear spline (shown below on the left) and a cubic spline (shown below on the right) each using $n = 10$ sub-intervals with a *uniform* size. Again, we indicate the sub-intervals using dotted lines.





We see that the piecewise interpolation methods **both** avoid the generation of those spurious oscillations near the boundaries of the interval. Visually, we see that for the function $f(x) = (1 + 25x^2)^{-1}$, which possesses a larger amount of curvature, is better approximated by the cubic spline for the same number of sub-intervals.

Note Set 7: Approximating a function and its derivative(s)

We have seen that we can use interpolation to approximate a function $f(x)$ using a discrete set of known function values at a unique set of points. However, we saw that a global approximation could lead to large, spurious oscillations in the approximation near the endpoints of the interval, so-called Runge phenomena. We could remedy this issue if we changed ***how*** the polynomial interpolant was created moving from a global approximation to a local one. This was the idea behind splines.

In this note set we will remedy this issue for a global interpolation by changing ***where*** the interpolation occurs. In particular, we will use very specific sets of interpolation nodes that are nonuniform but concentrate the nodes close to the interval boundary (where the pointwise error was the largest). After this we will discuss strategies to approximate the derivative of a function in a global sense.

7.1 Improving the global approximation

We return to the approximation of a function $f(x)$ with a global interpolating polynomial that, recall, was denoted $p_n(x)$. This interpolating polynomial is *unique* for a given set of data points

$$\{(x_i, f(x_i))\}_{i=0}^n = \{(x_i, f_i)\}_{i=0}^n$$

although it can be written in a variety of different forms, like with the Lagrange basis polynomials. Further, we had the pointwise error between the function and its polynomial interpolant of the form

$$E_n(x) = f(x) - p_n(x) = \frac{(x-x_0)(x-x_1)\cdots(x-x_n)}{(n+1)!} f^{(n+1)}(\xi) = \frac{\Psi_n(x)}{(n+1)!} f^{(n+1)}(\xi),$$

for some value of $\xi \in [a, b]$. Ideally, the pointwise error should become smaller as the number of nodes increases in the polynomial interpolation. However, we have seen that this may not be the case particularly for uniformly spaced interpolation points. Such nodes could result in the Runge phenomena, as we saw in the previous note set. So, should we even expect this error to shrink? Can we approximate a function arbitrarily well with a global interpolating polynomial? It turns out ***yes*** we can, due to a Theorem by Weierstrass. But Weierstrass' work is a mathematical result that has to do with the existence of this so-called "best" approximation, but it is difficult to apply the result in practice.

We at least know that what we aim to do is possible. That is, use a global polynomial to represent a function where the pointwise error vanishes as the degree of the polynomial interpolation increases

$$\lim_{n\to\infty} |E_n(x)| = \lim_{n\to\infty} |f(x) - p_n(x)| = 0.$$

If we examine this pointwise error piece-by-piece we see that it involves three terms:

1. The polynomial $\Psi_n(x)$.
2. The higher order derivative $f^{(n+1)}(\xi)$ with $\xi \in [a, b]$.
3. The factorial $(n+1)!$

Therefore, the magnitude of the pointwise error is a balancing act between the product (or division) of these three components. Over which of these terms do we have some type of *control*? The factorial term is directly related to the degree of the polynomial interpolant we construct. The higher order derivative term makes

an implicit assumption that $f^{(n+1)}(\xi)$ does not grow too rapidly with n. But we have no real "control" over the given function or how its derivatives behave. What we ***can*** do is force the product

$$\Psi_n(x) = (x - x_0)(x - x_1)\cdots(x - x_n) = \prod_{i=0}^{n}(x - x_i),$$

to be as small as possible on the given interval $[a, b]$.

For the simple choice of equally spaced interpolation points, the magnitude of the polynomial term $\Psi_n(x)$ increases as one moves from the interior of the interval toward the endpoints. Further, this magnitude becomes larger and more pronounced as the number of interpolation nodes n increases. This is why the global interpolating polynomial becomes less reliable as one approaches the leftmost or rightmost endpoint of the interval $[a, b]$, causing Runge phenomena.

Fortunately, we are free to change the location of the nodes on which the interpolation is built. This is true for the global interpolation strategy as well as the piecewise strategy with splines. We have this freedom, but how do we get a "better" set of interpolation points?

Though seemingly unrelated, to answer this question we have to take a brief detour to discuss orthogonal polynomials. Conceptually, we can think of functions in a similar fashion as we think about vectors in linear algebra. As such, we say that polynomials of degree n form a function space $\mathbb{P}^n$ (analogous to a vector space $\mathbb{R}^n$). We have already seen that members of this space of polynomials $\mathbb{P}^n$ can be represented with many different basis functions. For example, the monic polynomials

$$\{1, x, x^2, x^3, \ldots, x^n\},$$

the Newton polynomial basis $\{N_i(x)\}_{i=0}^n$ or the Lagrange basis $\{\ell_i(x)\}_{i=0}^n$. But, as we discussed before, some of the basis representations for the unique interpolating polynomial are easier to work with than others. Just like working with vectors, an orthogonal basis will make the representation and approximation much "cleaner".

So, we must clarify what it means for two polynomial functions to be orthogonal. Continuing the analogue to linear algebra, we need a mechanism to compute the inner product between two polynomials. That is, we need a mathematical tool to multiply two polynomials and obtain a scalar value. To do this we define the following:

Definition 7.1 (Inner product of two polynomials). On a given interval $[a, b]$ the *inner product* between two polynomials $p(x)$ and $q(x)$ is defined by

$$(p, q) = \int_a^b p(x)\, q(x)\, w(x)\, \mathrm{d}x,$$

where $w(x)$ is a positive weight function. ◀

Remark 7.1. Now it is well-defined what we mean if two polynomials are *orthogonal* because then

$$(p, q) = 0,$$

or their inner product is zero. ⋈

The subject of orthogonal polynomials is vast and we will only scratch the surface here. Two particularly useful families of orthogonal polynomials that appear in many aspects of mathematics (not just in scientific computing) are the *Legendre polynomials* and the *Chebyshev polynomials*.

Definition 7.2 (Legendre polynomials). On the interval $[-1, 1]$ with the weight function $w(x) = 1$ the set of Legendre polynomials of order zero up to order n are defined by the three-term recurrence relation

$$L_0(x) = 1, \qquad L_1(x) = x, \qquad L_{k+1} = \frac{2k+1}{k+1} x L_k(x) - \frac{k}{k+1} L_{k-1}(x),$$

for $k = 1, \ldots, n$. ◀

Remark 7.2. Because the weight function for the Legendre family of polynomials is just the constant function $w(x) = 1$, they are very useful to develop approximations to an integral. We will revisit the Legendre family of polynomials in the next note set when we talk about Gauss-type integral approximations. ⋈

EXAMPLE 7.1. Write out the next three Legendre polynomials $L_2(x)$, $L_3(x)$, $L_4(x)$. We start with the $k = 1$ case to generate

$$L_2(x) = \frac{2+1}{1+1}xL_1(x) - \frac{1}{1+1}L_0(x) = \frac{3}{2}x(x) - \frac{1}{2}(1) = \frac{1}{2}(3x^2 - 1).$$

Next, for $k = 2$ we find

$$L_3(x) = \frac{2(2)+1}{2+1}xL_2(x) - \frac{2}{2+1}L_1(x) = \frac{5}{3}x\left(\frac{1}{2}(3x^2-1)\right) - \frac{2}{3}(x) = \frac{1}{2}(5x^3 - 3x).$$

Finally, for $k = 3$ the recursion produces

$$L_4(x) = \frac{2(3)+1}{3+1}xL_3(x) - \frac{3}{3+1}L_2(x) = \frac{7}{4}x\left(\frac{1}{2}(5x^3-3x)\right) - \frac{3}{4}\left(\frac{1}{2}(3x^2-1)\right) = \frac{1}{8}(35x^4 - 30x^2 + 3).$$

♦

Of particular interest to the topic of interpolation are another set of orthogonal polynomials.

DEFINITION 7.3 (CHEBYSHEV POLYNOMIALS). On the interval $[-1, 1]$ with weight function

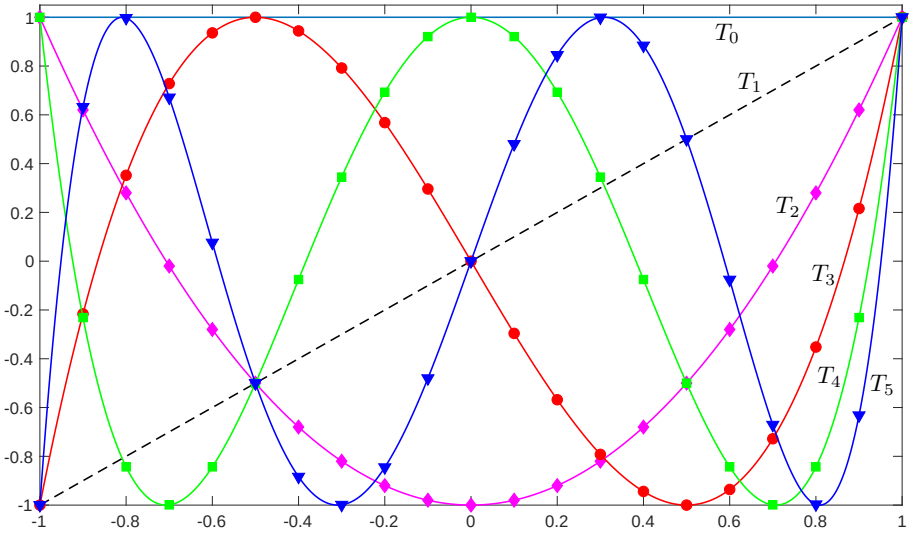
$$w(x) = \frac{1}{\sqrt{1-x^2}},$$

the set of Chebyshev polynomials of order zero up to order n are defined by the three-term recurrence relation

$$T_0(x) = 1, \qquad T_1(x) = x, \qquad T_{k+1} = 2xT_k(x) - T_{k-1}(x),$$

for $k = 1, \ldots, n$. ◀

For either the Legendre or the Chebyshev polynomials an important observation is that a polynomial of degree k contains *only even* degree terms when k is even and *only odd* degree terms when k is odd. Perhaps more important is an equioscillation property that can be observed visually. We plot the first six Chebyshev polynomials T_k, $k = 0, \ldots, 5$ below.



We see that the successive extremums of the polynomial functions $T_k(x)$ are equal in magnitude and alternate sign. This is of interest for us because such a property tends to uniformly distruibute the pointwise error within the interval when we use interpolation to approximate an arbitrary continuous function $f(x)$.

This equi-oscillation has particularly useful implications for polynomial interpolation because the maximum value of the pointwise error over the interval is minimized when the interpolation points are chosen to be the extremums of the Chebyshev polynomials of appropriate degree. So, the Chebyshev polynomials give us a set of interpolations nodes such that the monic polynomial term $\Psi_n(x)$ is as small as possible, which was our goal.

Great! We have identified the new set of interpolations points. Now we need to determine an explicit form for the locations of these extremums. In order to do so, we note the following usual property of the Chebyshev polynomial family.

Remark 7.3. An equivalent form of the Chebyshev polynomials is

$$T_k(x) = \cos(k \cos^{-1}(x)), \quad x \in [-1, 1],$$

where $k = 0, 1, \ldots, n$. ⋈

Though it is not obvious, this representation is still a polynomial function. This can be verified through the application of several trigonometric identities. From this trigonometric form of the Chebyshev polynomials it is straightforward to determine the value of the extremums.

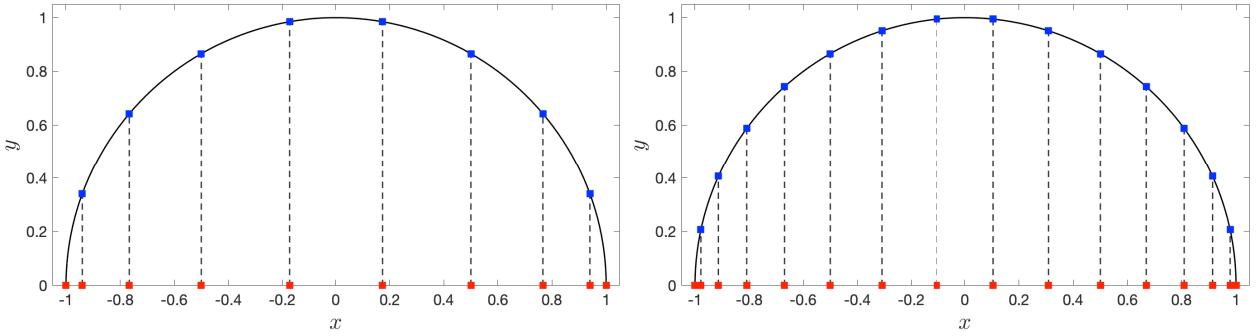
Definition 7.4 (Chebyshev extremums). A Chebyshev polynomial of degree k takes on $k+1$ extremums at the nodes

$$x_j = -\cos\left(\frac{j\pi}{k}\right),$$

where $j = 0, \ldots, k$. All the extremums occur in the interval $[-1, 1]$ and the endpoints are *included*. ◀

From here on we refer to this set of extreme values as the *Chebyshev nodes*. Note, they are not uniformly spaced in the interval $[-1, 1]$. But they are symmetrically distributed around the point $x = 0$ and concentrated near the ends of the interval. So, heuristically, the pointwise error in the global interpolation is reduced because we put more nodes near the endpoints of the interval (where the pointwise error was the worst) and fewer points in the interior of the interval (where the pointwise error was better).

Remark 7.4. Before we examine their performance in creating a global polynomial interpolant we note a final interesting feature about the Chebyshev nodes. We know that they are not uniformly distributed in the interval $[-1, 1]$ but they ***are*** spaced uniformly on the unit circle. We present this in the figures below for $n = 9$ on the left and $n = 15$ on the right.



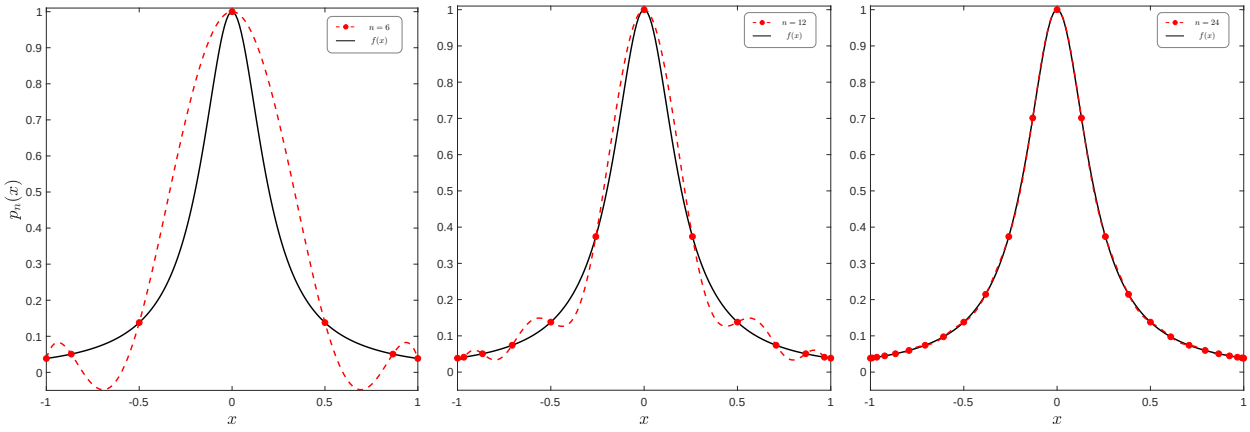
The uniform distribution of point on the unit circle are then projected onto the interval $[-1, 1]$ and results in their clustering near the endpoints because we project values from a curve onto a straight line. Also, we see that as the number of Chebyshev nodes increases, the interior has nearly uniform nodes but there is still clustering near the endpoints of the interval. ⋈

Next, we use the Chebyshev nodes to create a global interpolating polynomial for a function $f(x)$.

EXAMPLE 7.2. We revisit the function

$$f(x) = \frac{1}{1+25x^2}$$

that gave us trouble with uniformly spaced interpolation nodes and produced Runge phenomena. For this we construct the Lagrange interpolating polynomial basis and select the Chebyshev nodes as the interpolation points. We plot three instances of the global interpolating polynomial below for increasing values of the degree of the polynomial interpolant $n = 6$, $n = 12$, and $n = 24$.



We see that increasing the polynomial degree of the interpolation no longer produces Runge phenomena. The Chebyshev nodes has removed this deleterious effect. Moreover, we see that the use of the Chebyshev nodes more evenly distributes the pointwise error throughout the interval. ⧫

Now that we use the Chebyshev nodes the Runge phenomena are gone. Furthermore, we now observe the desired behavior that the global interpolating polynomial becomes more accurate as the degree increases. In fact, it appears to do so quite rapidly! For example, look above at the approximation when we double the number of Chebyshev nodes from $n = 12$ to $n = 24$. The convergence of the global interpolating polynomial now depends solely on the behavior on the derivatives of the function $f(x)$.

DEFINITION 7.5 (CONVERGENCE OF GLOBAL INTERPOLATION). Suppose we create a global polynomial interpolant for the function $f(x)$ on $n+1$ Chebyshev nodes in the interval $[-1, 1]$. If the function $f(x)$ is continuous and bounded on $[-1, 1]$ ***and*** it is also continuous and bounded on $[-1, 1]$ up to its mth derivative then the pointwise error decays according to

$$|E_n(x)| = |f(x) - p_n(x)| \sim \mathcal{O}(n^{-m}).$$

This is a negative power on the degree of the interpolating polynomial. In other words, the error in the global polynomial interpolant decays according to the "smoothness" of the function $f(x)$. ◀

Remark 7.5. This accounts for the rapid rate of convergence for the function $f(x) = (1 + 25x^2)^{-1}$. It is infinitely differentiable and bounded in the interval $[-1, 1]$. Therefore, the convergence actually occurs faster than $\mathcal{O}(n^{-m})$ no matter how large we take m. This is commonly called *exponential accuracy.* ⋈

EXAMPLE 7.3. We examine the behavior of this type of convergence that depends on how many times a function is differentiable. To do so, we consider the interval $[-1, 1]$ and take the very smooth function $f(x) = (1 + 25x^2)^{-1}$ (where $m = \infty$) and the function $f(x) = |x|$ that is continuous but has only one derivative (so $m = 1$). We plot the pointwise error as a function of the polynomial degree on the top right of the next page. Note this figure is in a semilog scale. This is to demonstrate that as we increase the polynomial order n the pointwise error (in the scale of the common logarithm $\log_{10}$) decays along a straight line for the infinitely differentiable function whereas the convergence stagnates for $f(x) = |x|$ because it lacks sufficient smoothness. Also, notice that eventually, if n is large enough, the pointwise error for the global interpolating polynomial and $f(x) = (1+25x^2)^{-1}$ reaches the magnitude of double precision roundoff. ⧫

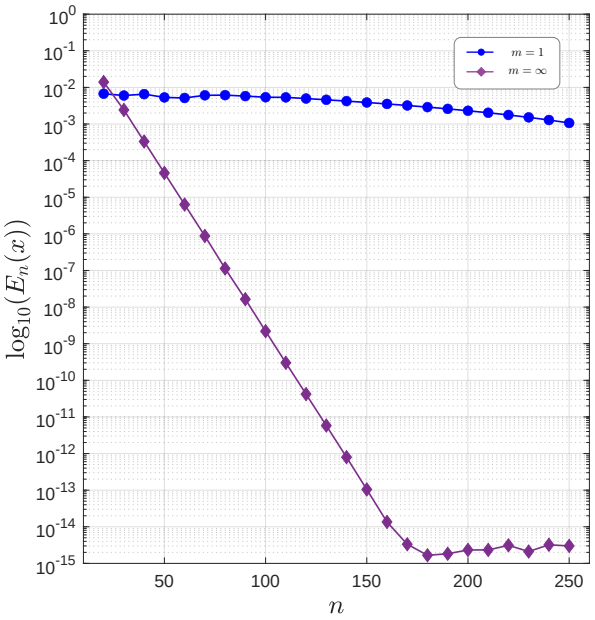
Of course, one may have no choice in placing the interpolation nodes, either because of existing measured data or because a particular distribution (like uniformly space) is required for other reasons. But when one does have the freedom to choose the locations of the interpolation points, which is often the case for problems in scientific computing, the Chebyshev nodes are an excellent choice.

We can also use the Chebyshev nodes to approximate a function that is not in the interval $[-1, 1]$. To do this we use a simple variable transformation:

DEFINITION 7.6 (AFFINE MAP). For a general interval of interest $[a, b]$ it is possible to rewrite any value $x \in [a, b]$ with the *affine map*

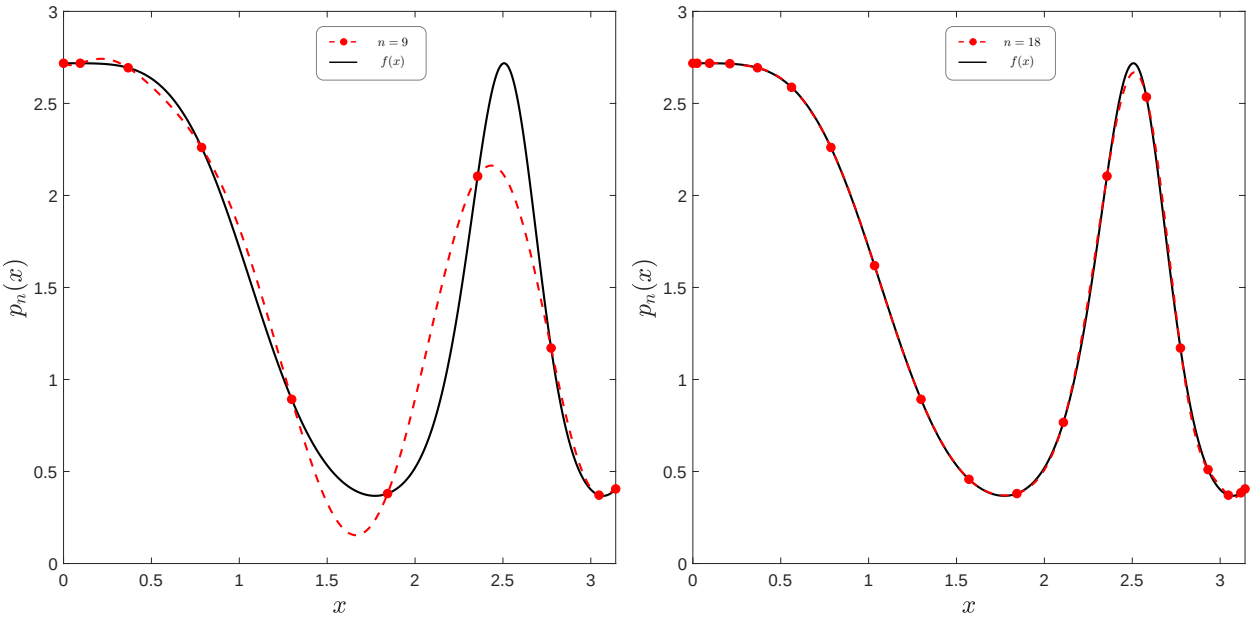
$$x = \frac{b-a}{2}\xi + \frac{b+a}{2},$$

such that x is in terms of a variable $\xi \in [-1, 1]$. ◀



We close this discussion on interpolation with a final example using a high degree global interpolating polynomial defined on the Chebyshev nodes on an interval that is not $[-1, 1]$.

EXAMPLE 7.4. We create a global interpolating polynomial in Lagrange form for the function $f(x) = e^{\cos(x^2)}$ on the interval $[0, \pi]$. The function is infinitely differentiable and its derivatives are bounded on the interval $[0, \pi]$. Thus, we expect the global interpolating polynomial to perform well when the interpolation points are chosen to be the Chebyshev nodes. We plot the interpolant against the function $f(x)$ for a visual comparison with global polynomial degrees of $n = 9$ (at the left) and $n = 18$ (at the right). We see that doubling the points improves the approximation considerably in the eyeball norm.



♦

7.2 Discrete differentiation

We now have the ability to approximate a function $f(x)$ with a polynomial as long as we are given some discrete set of data points $\{x_i, f(x_i)\}_{i=0}^n$. But often problems we are interested in involve not only function data, but derivatives as well. This is especially true to numerically approximate the solution of ordinary or partial differential equations. This is because we wish to approximate any derivative(s) to reduce a differential equation into a form that we can more easily solve. Such as the tools we learned from the linear algebra portion of the course.

The numerical approximation of derivatives will look very familiar because we will motivate them from Taylor's theorem and other background knowledge we have from single variable Calculus. For example, recall the limit definition of the derivative

$$f'(x) = \lim_{h\to 0} \frac{f(x+h) - f(x)}{h}.$$

To simplify the discussion of discrete derivatives we make the assumption that the data points $\{x_i\}_{i=0}^n$ are *uniformly spaced* such that the interval $[a, b]$ is divided into equal pieces each with length h:

$$h = \frac{b-a}{n}, \qquad x_i = a + ih, \qquad i = 0, \ldots, n.$$

This motivates that we can approximate the derivative of a function with a *finite difference*

$$f'(x_i) \approx \frac{f(x_i + h) - f(x_i)}{h} = \frac{f(x_{i+1}) - f(x_i)}{h} = \frac{f_{i+1} - f_i}{h}, \qquad i = 0, \ldots, n.$$

This is referred to as a *forward* difference because of how the data is accessed by the approximation. What kind of error did we just make? If we examine the Taylor expansion of the function about the point x_i we have

$$f(x_i + h) = f(x_i) + hf'(x_i) + \frac{h^2}{2} f''(x_i) + \frac{h^3}{6} f'''(\xi),$$

for some value of $\xi \in (x_i, x_i + h)$. Rearranging this truncated Taylor expansion we find the error between the true derivative and the forward difference approximation to be

$$f'(x_i) - \left(\frac{f(x_i + h) - f(x_i)}{h} \right) = -\frac{h}{2} f''(x_i) - \frac{h^2}{6} f'''(\xi).$$

Remark 7.6. Note that we have implicitly assumed that the function $f(x)$ is smooth such that higher order derivatives of the function exist and are bounded, such that applying Taylor's theorem is valid. ⋈

Definition 7.7 (Forward finite difference). A first order approximation to the derivative of a function $f(x)$ on the interval $[a, b]$ is given by the *forward difference*

$$f'(x_i) \approx \frac{f_{i+1} - f_i}{h}, \qquad i = 0, \ldots, n.$$

where $h = \frac{b-a}{n}$. The error in the approximation is $\mathcal{O}(h)$ such that doubling the number of approximation points decreases the error by a factor of two. ◀

Remark 7.7. Analogously, we can approximate the derivative with a *backward finite difference*

$$f'(x_i) \approx \frac{f_i - f_{i-1}}{h}, \qquad i = 0, \ldots, n.$$

where $h = \frac{b-a}{n}$. This is also a first order approximation of the derivative. ⋈

The choice of forward or backward finite differences can be motivated by the physics of the problem you want to solve (See Note Set 10 for details). But a more pressing issue is that we need to address what to do at the boundaries of the interval $[a, b]$.

EXAMPLE 7.5. We want to apply the forward finite difference to approximate the derivative of $f(x)$ on the interval $[a, b]$. This is okay until we reach the right most point in the domain, then the formula becomes

$$f'(x_n) \approx \frac{f_{n+1} - f_n}{h}.$$

This requires the knowledge of $f_{n+1} = f(x_{n+1})$, which is information we do not have! Therefore, at the right most endpoint we switch to a backward difference to select information we ***do*** have

$$f'(x_n) \approx \frac{f_n - f_{n-1}}{h}.$$

Further, we can rewrite the approximation of the derivative into a matrix-vector structure with

$$\begin{pmatrix} f'_0 \\ f'_1 \\ \vdots \\ f'_{n-1} \\ f'_n \end{pmatrix} = \frac{1}{h} \begin{pmatrix} -1 & 1 & & & \\ & -1 & 1 & & \\ & & \ddots & \ddots & \\ & & & -1 & 1 \\ & & & -1 & 1 \end{pmatrix} \begin{pmatrix} f_0 \\ f_1 \\ \vdots \\ f_{n-1} \\ f_n \end{pmatrix} \qquad \text{or} \qquad \mathbf{f}' = \mathbf{D}_x \mathbf{f}.$$

♦

Here the elements of $\mathbf{D}_x$ that are not shown are zeroes, a convention we use throughout this note set. We call the matrix $\mathbf{D}_x$ a *differentiation matrix*. Each row of $\mathbf{D}_x$ gives the weights of the finite difference formula being used at one of the nodes. Thus, multiplication of the differentiation matrix from the left with the vector of function values yields the derivative approximation at all the nodes.

Remark 7.8. We are free to choose whatever finite difference formula we want in each row of the differentiation matrix $\mathbf{D}_x$. However, it makes sense to make the rows as similar as possible because it makes it easier to examine the overall error of the derivative approximation (among other things). ⋈

What about other finite difference approximations? Can we construct an approximation that is more accurate? We can, and it basically all boils down to playing a game with algebra and different Taylor expansion coefficients.

Consider the forward and backward Taylor expansions for a function around the point x_i:

$$f(x_i + h) = f(x_i) + hf'(x_i) + \frac{h^2}{2} f''(x_i) + \frac{h^3}{6} f'''(x_i) + \frac{h^4}{24} f^{(4)}(\xi),$$
$$f(x_i - h) = f(x_i) - hf'(x_i) + \frac{h^2}{2} f''(x_i) - \frac{h^3}{6} f'''(x_i) + \frac{h^4}{24} f^{(4)}(\xi).$$

If we subtract these two expressions and manipulate we obtain a higher order approximation to the first derivative of the function $f(x)$:

$$f'(x_i) - \left(\frac{f(x_i + h) - f(x_i - h)}{2h} \right) = -\frac{h^2}{6} f'''(x_i) + \mathcal{O}(h^4).$$

DEFINITION 7.8 (CENTRAL DIFFERENCE APPROXIMATION). A second order approximation to the first derivative of the function $f(x)$ on $[a, b]$ is given by

$$f'(x_i) \approx \frac{f(x_i + h) - f(x_i - h)}{2h}, \qquad i = 0, \ldots, n$$

where $h = \frac{b-a}{n}$. The error is on the order of $\mathcal{O}(h^2)$ so doubling the number of points decreases the error by a factor of four. ◀

Remark 7.9. Notice, to increase the approximation accuracy we require *more* neighbour information to the function at point x_i. In this instance, the central difference approximation needs information to the left and to the right of the point of interest. ⋈

Including more function information to approximate the derivative at the point x_i is referred to as *widening the stencil* of the finite difference. Below we give two example stencils:



This is okay in the interior of the interval $[a, b]$ where this extra function information is available. However, the issue of approximating the derivative at the boundaries of the interval becomes relevant again. How do we give the finite difference approximation the necessary information to approximate the derivative when the stencil requires information we do not have? There are three options typically employed:

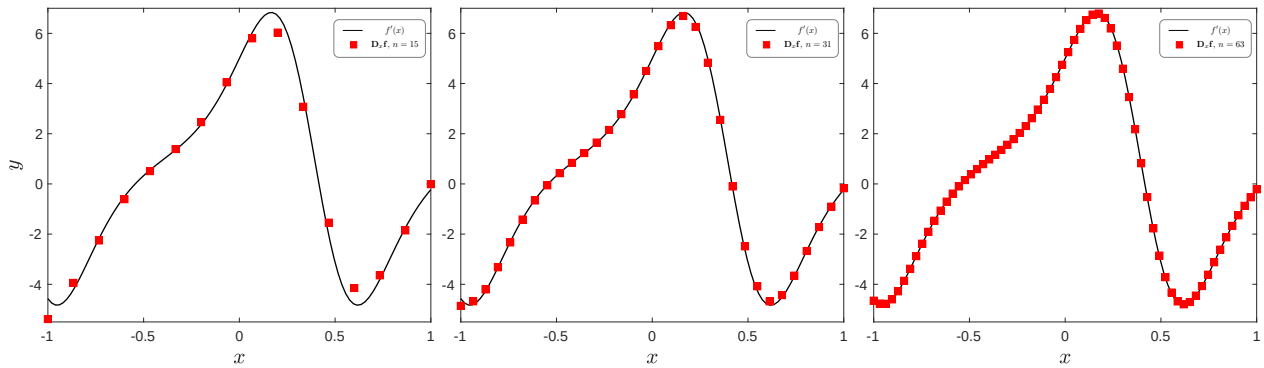
1. *Switch* to a lower order derivative approximation near the boundary that requires less function information.
2. *Bias* the stencil of the finite difference to keep the same order on the error but only take information available from within the interval.
3. Introduce *ghost points* outside the interval that get assigned the unknown information by some means.

Remark 7.10. We mention ghost points for completeness but will not address how this is done in this course. Just know that manufacturing this unknown information outside the interval $[a, b]$ can be done in many ways, e.g., with extrapolation using an interpolating polynomial of the function $f(x)$ (or its data). ⋈

EXAMPLE 7.6. Approximate the first derivative of the function $f(x) = x + e^{\sin(4x)}$ on the interval $[-1, 1]$ using a central difference in the interior and forward/backward differences on the boundary. First we assemble the derivative matrix we use for the approximation that takes the form

$$\begin{pmatrix} f_0' \\ f_1' \\ f_2' \\ \vdots \\ f_{n-1}' \\ f_n' \end{pmatrix} = \frac{1}{h} \begin{pmatrix} -1 & 1 & & & & \\ -\frac{1}{2} & 0 & \frac{1}{2} & & & \\ & -\frac{1}{2} & 0 & \frac{1}{2} & & \\ & & \ddots & \ddots & \ddots & \\ & & & -\frac{1}{2} & 0 & \frac{1}{2} \\ & & & & -1 & 1 \end{pmatrix} \begin{pmatrix} f_0 \\ f_1 \\ f_2 \\ \vdots \\ f_{n-1} \\ f_n \end{pmatrix}.$$

Notice, we have the total derivative approximation with the forward difference on the left boundary, the backward difference on the right boundary, and then a repeating stencil in the interior with the central difference. We plot the values of the derivative approximation below against the known derivative of the function $f'(x) = 1 + 4\cos(4x)e^{\sin(4x)}$ for an increasing number of nodes with $n = 15$, $n = 31$, and $n = 63$.



These plots highlight that the finite difference approximation only "knows" an approximation to the derivative at these discrete data points. The finite difference gives no information about the derivative of the function between the points. ⧫

Okay, so we can handle boundaries. However, switching to a lower order finite difference approximation at the boundaries will affect the overall error (or quality) of the approximate derivative. Instead, we seek to bias the finite difference approximation near the endpoints of the interval to maintain the same order of accuracy. However, we know that to do this we will need more function information. A (slightly restrictive) rule of thumb is that if we want an mth order finite difference approximation to the first derivative we need at least $m+1$ nodes of information. For example, a first order approximation required two nodes and we expect that a second order approximation needs three nodes.

To create a biased finite difference for the first derivative we will use the *method of undetermined coefficients* which is best demonstrated through example.

EXAMPLE 7.7. Create a second order forward biased finite difference for the first derivative of $f(x)$. We expect this finite difference approximation to require three points of information to obtain the desired error of second order. Therefore, this biased finite difference approximation will take the general form

$$f'(x_i) \approx \frac{\gamma f_i + \eta f_{i+1} + \rho f_{i+2}}{h},$$

for the unknown real number coefficients γ, η, and ρ. We use Taylor expansions of the second two terms to find

$$f_{i+1} = f(x_i + h) = f(x_i) + hf'(x_i) + \frac{h^2}{2}f''(x_i) + \frac{h^3}{6}f'''(x_i) + \frac{h^4}{24}f^{(4)}(\xi_1),$$
$$f_{i+2} = f(x_i + 2h) = f(x_i) + 2hf'(x_i) + \frac{(2h)^2}{2}f''(x_i) + \frac{(2h)^3}{6}f'''(x_i) + \frac{(2h)^4}{24}f^{(4)}(\xi_2),$$

for some values of $\xi_1 \in (x_i, x_i + h)$ and $\xi_2 \in (x_i, x_i + 2h)$. Now we gather like terms to find

$$\frac{\gamma f_i + \eta f_{i+1} + \rho f_{i+2}}{h} = \frac{\gamma + \eta + \rho}{h}f_i + (\eta + 2\rho)f'(x_i) + \frac{h}{2}(\eta + 4\rho)f''(x_i) + \mathcal{O}(h^2).$$

For consistency to the first derivative we require the first coefficient on the right hand side to be zero and the second coefficient to be one. To maintain second order accuracy we require the third coefficient on the right hand side to vanish. This produces three equations for the three unknown coefficients:

$$\gamma + \eta + \rho = 0, \qquad \eta + 2\rho = 1, \qquad \eta + 4\rho = 0.$$

It is straightforward to determine the coefficients to be

$$\gamma = -\frac{3}{2}, \qquad \eta = 2, \qquad \rho = -\frac{1}{2},$$

which gives the second order forward biased difference approximation

$$f'(x_i) = \frac{-3f_i + 4f_{i+1} - f_{i+2}}{2h} + \mathcal{O}(h^2).$$

⧫

Remark 7.11. A biased backward difference formula that is second order accurate is

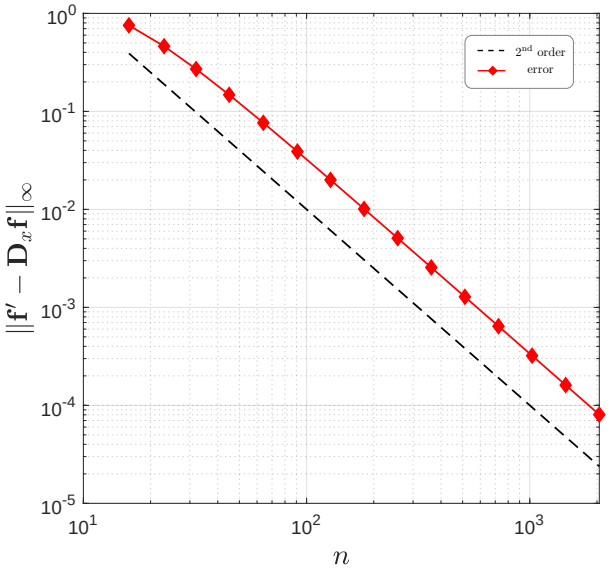
$$f'(x_i) = \frac{f_{i-2} - 4f_{i-1} + 3f_i}{2h} + \mathcal{O}(h^2),$$

where the coefficients can be found in a similar way as above. ⋈

We next examine the error behavior of the finite difference approximation.

EXAMPLE 7.8. We revisit the approximation of the derivative $f(x) = x + e^{\sin(4x)}$ this time using the second order biased finite difference at the boundary and central difference in the interior. The derivative matrix now has the structure

$$\begin{pmatrix} f_0' \\ f_1' \\ f_2' \\ \vdots \\ f_{n-1}' \\ f_n' \end{pmatrix} = \frac{1}{h} \begin{pmatrix} -\frac{3}{2} & 2 & -\frac{1}{2} & & & \\ -\frac{1}{2} & 0 & \frac{1}{2} & & & \\ & -\frac{1}{2} & 0 & \frac{1}{2} & & \\ & & \ddots & \ddots & \ddots & \\ & & & -\frac{1}{2} & 0 & \frac{1}{2} \\ & & & \frac{1}{2} & -2 & \frac{3}{2} \end{pmatrix} \begin{pmatrix} f_0 \\ f_1 \\ f_2 \\ \vdots \\ f_{n-1} \\ f_n \end{pmatrix}$$



We plot the ∞-norm of the errors at the nodal values of the derivative approximation for increasing values of n. Further, the error plot for the finite difference approximation is presented in a log-log scale at the right. The slope of the line in the error plot is two compared to the dashed reference line, as we expect for a second order approximation. We reiterate that examining errors in the logarithmic scale is often more meaningful to reinforce the mathematical theory of an approximation. ♦

What if the equation we want to solve contains second derivatives? We need a finite difference approximation for those as well. Just as before we examine Taylor expansions and create the finite difference approximation to a given derivative. Recall the expansions

$$f(x_i + h) = f(x_i) + hf'(x_i) + \frac{h^2}{2}f''(x_i) + \frac{h^3}{6}f'''(x_i) + \frac{h^4}{24}f^{(4)}(x_i) + \mathcal{O}(h^5),$$
$$f(x_i - h) = f(x_i) - hf'(x_i) + \frac{h^2}{2}f''(x_i) - \frac{h^3}{6}f'''(x_i) + \frac{h^4}{24}f^{(4)}(x_i) + \mathcal{O}(h^5).$$

If we add these two terms we see that the odd derivatives in the expansion cancel and the even derivatives remain to have

$$f(x_i + h) + f(x_i - h) = 2f(x_i) + h^2 f''(x_i) + \frac{h^4}{12}f^{(4)}(x_i) + \mathcal{O}(h^6).$$

Manipulating this expression we find

$$f''(x_i) = \frac{f(x_i + h) - 2f(x_i) + f(x_i - h)}{h^2} - \frac{h^2}{12}f^{(4)}(x_i) + \mathcal{O}(h^4).$$

DEFINITION 7.9 (CENTRAL DIFFERENCE FOR SECOND DERIVATIVE). A second order approximation to the second derivative of the function $f(x)$ on the interval $[a, b]$ is given by a *central difference*

$$f''(x_i) \approx \frac{f_{i+1} - 2f_i + f_{i-1}}{h^2},$$

where $h = \frac{b-a}{n}$. The approximation is second order accurate so the error is on the order of $\mathcal{O}(h^2)$. ◀

Remark 7.12. We see that the stencil widens as well for the finite difference to approximate a derivative of higher degree. ⋈

Remark 7.13. It is also possible to bias the second order finite difference approximation as either forward or backward to avoid reaching unavailable data. For example, the forward biased finite difference of the second derivative that remains second order accurate in the error, i.e. $\mathcal{O}(h^2)$, is

$$f''(x_i) = \frac{2f_i - 5f_{i+1} + 4f_{i+2} - f_{i+3}}{h^2} + \mathcal{O}(h^2)$$

Note that the stencil widens further in the biased approximation in order to have enough degrees of freedom to bias the approximation of the second degree derivative and maintain second order accuracy in the error. ⋈

In general, a second order accurate finite difference approximation to the second derivative is given by

$$\begin{pmatrix} f_0'' \\ f_1'' \\ f_2'' \\ \vdots \\ f_{n-1}'' \\ f_n'' \end{pmatrix} = \frac{1}{h^2} \begin{pmatrix} 2 & -5 & 4 & -1 & & \\ 1 & -2 & 1 & & & \\ & 1 & -2 & 1 & & \\ & & \ddots & \ddots & \ddots & \\ & & & 1 & -2 & 1 \\ & & -1 & 4 & -5 & 2 \end{pmatrix} \begin{pmatrix} f_0 \\ f_1 \\ f_2 \\ \vdots \\ f_{n-1} \\ f_n \end{pmatrix} \qquad \text{or} \qquad \mathbf{f}'' = \mathbf{D}_{xx}\mathbf{f}.$$

Again, the finite difference derivative matrix for the second derivative $\mathbf{D}_{xx}$ has the biased approximation on the left and right endpoints and a repeating central stencil in the interior of the interval.

Remark 7.14. Note, that we could construct a second derivative approximation matrix by multiplying two first order derivative matrices such that

$$\mathbf{D}_{xx} = \mathbf{D}_x\mathbf{D}_x.$$

This is completely valid, but would further *widen* the stencils in the matrix $\mathbf{D}_{xx}$. In practice, we want to keep the stencils as narrow as possible while maintaining accuracy. ⋈

To close this discussion we examine how to *assemble* such a derivative matrix in practice. Here, there will be a slight change such that we do not have to bias the derivative approximation because we *know* a certain amount of boundary information.

Example 7.9. Suppose we want to numerically approximate the solution to the two-point boundary value problem

$$u''(x) + 4u(x) = 0, \qquad u(0) = -2, \quad u\left(\frac{\pi}{4}\right) = 1.$$

We will convert this linear second order ordinary differential equation (ODE) into a linear system. That is, something we know how to solve numerically. To make the indexing a bit easier (and match the indexing from the linear algebra portion of the course) say we want to solve the problem on $n+1$ subintervals which means that the uniformly spaced points are given by

$$h = \frac{b-a}{n+1}, \qquad x_i = a + ih, \qquad i = 0, \ldots, n+1,$$

where the interval $[a, b] = \left[0, \frac{\pi}{4}\right]$. The second order derivative can be approximated on the interior points x_i where $i = 1, \ldots, n$ in a matrix-form which gives a discrete version of the second order ODE:

$$\frac{1}{h^2} \begin{pmatrix} 1 & -2 & 1 & & & & \\ & 1 & -2 & 1 & & & \\ & & 1 & -2 & 1 & & \\ & & & \ddots & \ddots & \ddots & \\ & & & & 1 & -2 & 1 \\ & & & & & 1 & -2 & 1 \end{pmatrix} \begin{pmatrix} u_0 \\ u_1 \\ u_2 \\ \vdots \\ u_n \\ u_{n+1} \end{pmatrix} + 4 \begin{pmatrix} u_1 \\ u_2 \\ \vdots \\ u_{n-1} \\ u_n \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ \vdots \\ 0 \\ 0 \end{pmatrix}$$

where $\mathbf{u} = (u_0, u_1, \ldots, u_n, u_{n+1})^T$ are the unknown quantities which we want to determine. This matrix form has n rows and $n+2$ columns. However, we already know the two of the values

$$u_0 = -2, \qquad \text{and} \qquad u_{n+1} = 1,$$

because of the boundary conditions given by the problem formulation. Applying the boundary conditions shrinks the system by two unknown values to become just in terms of the unknown interior values. We manipulate the equations and multiply through by $-h^2$ to find

$$\begin{pmatrix} 2 & -1 & & & \\ -1 & 2 & -1 & & \\ & \ddots & \ddots & \ddots & \\ & & -1 & 2 & -1 \\ & & & -1 & 2 \end{pmatrix} \begin{pmatrix} u_1 \\ u_2 \\ \vdots \\ u_{n-1} \\ u_n \end{pmatrix} - 4h^2 \begin{pmatrix} u_1 \\ u_2 \\ \vdots \\ u_{n-1} \\ u_n \end{pmatrix} = \begin{pmatrix} 2-4h^2 & -1 & & & \\ -1 & 2-4h^2 & -1 & & \\ & \ddots & \ddots & \ddots & \\ & & -1 & 2-4h^2 & -1 \\ & & & -1 & 2-4h^2 \end{pmatrix} \begin{pmatrix} u_1 \\ u_2 \\ \vdots \\ u_{n-1} \\ u_n \end{pmatrix} = \begin{pmatrix} u_0 \\ 0 \\ \vdots \\ 0 \\ u_{n+1} \end{pmatrix}.$$

This is now a linear system of the form $\mathbf{Ax} = \mathbf{b}$ where

$$\mathbf{A} = \begin{pmatrix} 2-4h^2 & -1 & & & \\ -1 & 2-4h^2 & -1 & & \\ & \ddots & \ddots & \ddots & \\ & & -1 & 2-4h^2 & -1 \\ & & & -1 & 2-4h^2 \end{pmatrix}, \quad \mathbf{x} = \begin{pmatrix} u_1 \\ u_2 \\ \vdots \\ u_{n-1} \\ u_n \end{pmatrix} \quad \text{and} \quad \mathbf{b} = \begin{pmatrix} u_0 \\ 0 \\ \vdots \\ 0 \\ u_{n+1} \end{pmatrix} = \begin{pmatrix} -2 \\ 0 \\ \vdots \\ 0 \\ 1 \end{pmatrix}.$$

Further, we have seen a tridiagonal matrix like this before and know that it is symmetric positive definite (provided h is small enough). So we have a few options to solve this system with either the Cholesky factorization, Jacobi or Gauss-Seidel. ♦

Note Set 8: Numerical integration (quadrature)

We continue the discussion of discrete calculus by next examining integration. The overall goal is that we want to compute the value of a definite integral over a given interval

$$I(f) = \int_a^b f(x)\,\mathrm{d}x$$

which is a type of *functional*, i.e., a mathematical expression that takes in a function as an argument and returns a number. Finding such definite integrals have many important uses such as computing the mass of a physical quantity.

But what if we only know this function $f(x)$ on a discrete set of points $\{(x_i, f_i)\}_{i=0}^n$? Can we find an approximation to the integral $I(f)$? We have seen that discretely differentiating a function is fairly straightforward with finite difference approximations built from Taylor series. Given some function data we can find an approximate derivative. However, given that same function data, how would we approximate its definite integral over an interval $[a, b]$? Numerically, this is slightly more cumbersome and we will use tools from polynomial interpolation to make this possible.

In general, this discrete version of the Calculus matches the continuous case. Derivatives are typically straightforward to find for an arbitrary function using chain rule, product rule, etc. Finite differences give an analogously straightforward way to approximate derivatives from function data points. However, it is often harder to compute the exact value of a definite integral for a general function. This carries over to the discrete case and we have to be clever to develop rules that tell the computer how to effectively approximate an integral from a given data set or integrand function.

8.1 Newton-Cotes quadrature

The numerical approximation of definite integrals is known as *numerical quadrature*. This name stems from old methods of approximating areas of irregular objects with small rectangles, a process formalized by Riemann into the standard form of integration we learn in Calculus courses. As such, similar to starting from the limit definition of the derivative for finite differences, we take a cue from the Riemann sums that define the integral. That is, the integral will be approximated by a weighted sum of the integrand $f(x)$ evaluated at a set of sample points within the interval $[a, b]$. This offers a general form for the *quadrature rule* defined at $n + 1$ points:

Definition 8.1 (General quadrature rule). Given a set of points where $a \leq x_0 < x_1 < \cdots < x_n \leq b$ then the integral can be approximated by the *quadrature rule*

$$I(f) \approx I_n(f) = \sum_{i=0}^{n} \omega_i f(x_i)$$

where the integrand $f(x)$ is evaluated at the set $\{x_i\}_{i=0}^n$ *quadrature nodes* and the set of multipliers $\{\omega_i\}_{i=0}^n$ are the *quadrature weights*. ◀

Remark 8.1. Note that the quadrature nodes in this definition do not necessarily include the end points of the interval $[a, b]$. ⋈

Great! We are slowly turning this into a problem which we can solve numerically. It just remains that we need to determine this set of quadrature nodes and, more importantly, the quadrature weights. Just like

with interpolation, we start with a simple choice and say that the quadrature nodes are *uniformly spaced* on the interval. We actually have a choice here whether to include the boundary nodes or not. Depending on this choice, the locations of the quadrature nodes will change.

DEFINITION 8.2 (UNIFORM QUADRATURE NODES). On the interval $[a, b]$ the set of uniform nodes to be used in a quadrature rule take the general form

$$x_i = x_0 + ih \quad \text{where} \quad i = 0, \ldots, n.$$

However, the values of x_0, x_n, and h change slightly if we want to include the endpoints (CLOSED) or exclude the endpoints (OPEN)

$$\begin{aligned} \text{CLOSED quadrature nodes:} \quad & x_0 = a, \quad x_n = b, \quad \text{where} \quad h_c = \frac{b-a}{n} \quad n \geq 1 \\ \text{OPEN quadrature nodes:} \quad & x_0 = a + h_o, \quad x_n = b - h_o, \quad \text{where} \quad h_o = \frac{b-a}{n+2} \quad n \geq 0 \end{aligned}$$

Be aware that h is defined differently for open and closed quadrature nodes! ◀

We have the quadrature nodes, but where does these quadrature weights $\{\omega_i\}_{i=0}^n$ come from? Once we know them we have a complete quadrature rule to approximate integrals.

If we return to the original problem of computing $I(f)$ for the moment we realize something. We are operating under the assumption that we have been given a discrete set of points $\{(x_i, f_i)\}_{i=0}^n$ where we have information about the integrand $f(x)$. This is remarkably similar to the problem of interpolation. We may not have any clue how to integrate the function $f(x)$ analytically (in fact it may not even possess an antiderivative). But, we know how to integrate polynomials and it is actually very simple. So, we are going to construct an interpolating polynomial of the integrand to reduce the complexity. Then we will reuse the interpolation points as the quadrature nodes (that is the two node sets are *collocated*) such that the determining the quadrature weights reduces to integrating simple polynomial functions.

Remark 8.2. The use of polynomial interpolation on *uniformly spaced* quadrature points to derive quadrature formulas is a general strategy known as ***Newton-Cotes quadrature***. ⋈

Remark 8.3. We choose to approximate the integrand with a polynomial written in the Lagrange basis. We do this to determine a quadrature rule that ***does not*** depend on the integrand function $f(x)$. ⋈

We will derive a few classic examples of the quadrature weights before giving a general form. Suppose we approximate the integrand at a single value, the midpoint of the interval. This is equivalent to using a constant polynomial function ($n = 0$) to approximate $f(x)$

$$p_0(x) = f(x_0) = f_0, \quad \text{where} \quad x_0 = \frac{b+a}{2}$$

Then the integral is approximated by the formula

$$I_0(f) = \int_a^b f_0 \, \mathrm{d}x = (b-a) f_0$$

This is the so-called *midpoint quadrature rule* where we have one quadrature point at x_0 and the corresponding quadrature weight is the width of the interval

$$\omega_0 = b - a$$

What is the approximation error of this quadrature rule? To determine this quantity we return to Taylor's theorem and examine the difference between the true integral and the quadrature rule. To do this we expand the integrand about the single integration point $x_0 = (b+a)/2$ and truncate the expansion to have

$$f(x) \approx f(x_0) + (x - x_0) f'(x_0) + \frac{(x - x_0)^2}{2} f''(\xi), \quad \text{where} \quad \xi \in [a, b].$$

Comparing the true integral with the midpoint quadrature rule we find that

$$I(f) - I_0(f) = \int_a^b \left(f(x_0) + (x - x_0)f'(x_0) + \frac{(x - x_0)^2}{2} f''(\xi) \right) \mathrm{d}x - (b - a)f_0.$$

The first term cancels because it is precisely the midpoint rule approximation and we integrate the next two terms to find

$$I(f) - I_0(f) = f'(x_0) \left[\frac{(x - x_0)^2}{2} \right]_{x=a}^{b} + f''(\xi) \left[\frac{(x - x_0)^3}{6} \right]_{x=a}^{b} = 0 + \frac{h_o^3}{3} f''(\xi)$$

where the first term eliminates when we substitute $x_0 = (b + a)/2$ and we apply the definition of the node spacing for the value $h_o = (b - a)/2$ Pay close attention to the node spacing! This is an open quadrature formula. Then we have the following:

Definition 8.3 (Midpoint quadrature rule). The definite integral on the interval $[a, b]$ of function $f(x)$ with at least two continuous derivatives can be approximated at the midpoint of the interval $x_0 = \frac{b+a}{2}$ with the formula

$$I_0(f) = (b - a)f_0 = 2h_o f_0,$$

where $h_o = \frac{b-a}{2}$ is the node spacing for the open quadrature formula with $n = 0$ and

$$E_0 = I(f) - I_0(f) = \frac{h_o^3}{3} f''(\xi),$$

is the associated error of the quadrature formula. ◀

This is nice and simple, but this was just assuming we knew a single value of the function in the interval $[a, b]$. If we use a higher degree polynomial to approximate the integrand $f(x)$ then the quadrature formula will be better. What about if we approximate the integrand with a linear polynomial ($n = 1$) using data from the two endpoints of the interval

$$f(x) \approx f_0 \frac{x - x_1}{x_0 - x_1} + f_1 \frac{x - x_0}{x_1 - x_0} = f_0 \frac{x - b}{a - b} + f_1 \frac{x - a}{b - a} = \frac{1}{b - a} \left(f(b)(x - a) - f(a)(x - b) \right).$$

Note that we will collocate the interpolation and the quadrature nodes which are $x_0 = a$ and $x_1 = b$. So, we are creating a closed quadrature rule because the endpoints are included. We substitute this into the definite integral to determine the *trapezoid rule*

$$I_1(f) = \frac{1}{b - a} \int_a^b f(b)(x - a) - f(a)(x - b) \, \mathrm{d}x = \cdots = \frac{b - a}{2} (f(a) + f(b)).$$

Definition 8.4 (Trapezoid quadrature rule). The definite integral on the interval $[a, b]$ of function $f(x)$ with at least two continuous derivatives can be approximated at the endpoints of the interval $x_0 = a$ and $x_1 = b$ with the formula

$$I_1(f) = \omega_0 f(x_0) + \omega_1 f(x_1) = \frac{b - a}{2} (f(a) + f(b)),$$

such that the quadrature weights are

$$\omega_0 = \omega_1 = \frac{b - a}{2}.$$

Further, we have that

$$E_1 = I(f) - I_1(f) = -\frac{h_c^3}{12} f''(\xi), \quad \text{where} \quad h_c = b - a,$$

is the associated error of the quadrature formula. ◀

For one more example, say we use a quadratic interpolating polynomial ($n = 2$) to approximate the integrand. We use the endpoint of the interval as well as the midpoint as the interpolation points

$$x_0 = a, \quad x_1 = \frac{b+a}{2}, \quad x_2 = b,$$

to have

$$f(x) \approx f_0 \frac{(x-x_1)(x-x_2)}{(x_0-x_1)(x_0-x_2)} + f_1 \frac{(x-x_0)(x-x_2)}{(x_1-x_0)(x_1-x_2)} + f_2 \frac{(x-x_0)(x-x_1)}{(x_2-x_0)(x_2-x_1)}.$$

We substitute this into the definite integral and obtain a quadrature rule known as *Simpson's rule*.

Definition 8.5 (Simpson's quadrature rule). The definite integral on the interval $[a, b]$ of function $f(x)$ with at least four continuous derivatives can be approximated using the quadrature nodes

$$x_0 = a, \quad x_1 = \frac{b+a}{2}, \quad x_2 = b,$$

with the formula

$$I_2(f) = \omega_0 f(x_0) + \omega_1 f(x_1) + \omega_2 f(x_2) = \frac{h_c}{3}(f_0 + 4f_1 + f_2), \quad \text{where} \quad h_c = \frac{b-a}{2},$$

such that the quadrature weights are

$$\omega_0 = \omega_2 = \frac{h_c}{3} \quad \text{and} \quad \omega_1 = \frac{4h_c}{3},$$

Further, we have that

$$E_2 = I(f) - I_2(f) = -\frac{h_c^5}{90} f^{(4)}(\xi), \quad \text{where} \quad h_c = \frac{b-a}{2},$$

is the associated error of the quadrature formula. ◀

Remark 8.4. Keep track on the value of the quadrature node spacing h because its value changes for each of the quadrature methods! Further, the change depends on whether the quadrature rule is open or closed *as well as* the number of quadrature nodes used in the integral approximation. ⋈

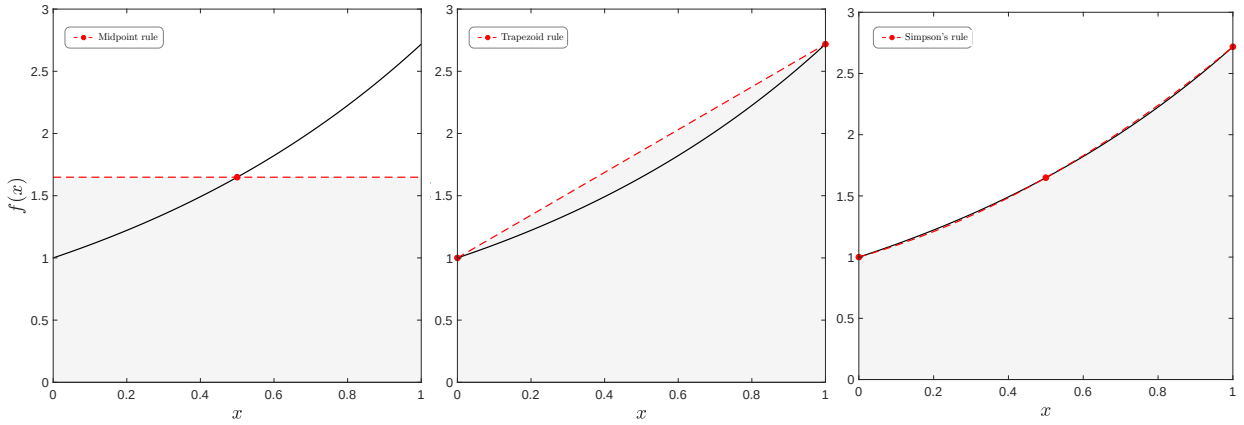
Example 8.1. Use the midpoint, trapezoid and Simpson's quadrature rules to approximate the value of the definite integral

$$I(e^x) = \int_0^1 e^x \, \mathrm{d}x = e - 1 = 1.7182818284\ldots$$

The three approximations are given by

$$\begin{aligned}
I_0(e^x) &= (1-0) f\left(\frac{1-0}{2}\right) && = e^{\frac{1}{2}} && = 1.648721270\ldots \\
I_1(e^x) &= \frac{1-0}{2}(f(0) + f(1)) && = \frac{1}{2}(1+e) && = 1.859140914\ldots \\
I_2(e^x) &= \frac{\frac{1-0}{2}}{3}\left(f(0) + 4f\left(\frac{1+0}{2}\right) + f(1)\right) && = \frac{1}{6}(1 + 4e^{\frac{1}{2}} + e) && = 1.718861151\ldots
\end{aligned}$$

Where we see that Simpson's quadrature rule is the most accurate, as we expect, because it uses the most information about the function to create the approximation. We illustrate how the three methods approximate the area below the curve $f(x) = e^x$ in the figures.



Upon this visual inspection we see that the Simpson's rule is very similar to the exact function. ♦

Remark 8.5. In general, for a given set of quadrature nodes (either open or closed) the corresponding quadrature weights can be found by integrating the Lagrange basis polynomials defined on the associated quadrature nodes

$$\omega_i = \int_a^b \ell_i(x)\,\mathrm{d}x, \quad \text{where} \quad i = 0, \ldots, n$$

which is laborious, but doable because the basis functions are all polynomials. ⋈

Just as we have with discrete differentiation sometimes the quadrature rule can integrate a function exactly. This *degree of exactness* is defined to be the maximal polynomial order for which the quadrature is exact.

Definition 8.6 (Degree of exactness). The *degree of exactness* (sometimes called the degree of precision) of a quadrature formula $I_n(f)$, is the maximum integer m such that

$$I(x^k) - I_n(x^k) = 0, \;\; 0 \le k \le m, \quad \text{and} \quad I(x^{m+1}) - I_n(x^{m+1}) \neq 0.$$

◀

Remark 8.6. Keep in mind that this is different than the accuracy order in the error formula for the quadrature formula $E_n = I(f) - I_n(f) = \mathcal{O}(h^r)$ which is referred to as the "order of the infinitesimal". ⋈

Sometimes the degree of exactness is higher than you might expect. We will see in the next example that the midpoint rule, that uses just a constant value to approximate the integrand $f(x)$, is quite accurate.

Example 8.2. Determine the degree of exactness for the midpoint rule and the trapezoid rule. For this we first compute the exact integrals of polynomials of order zero, one and two:

$$I(1) = \int_a^b 1\,\mathrm{d}x = b - a, \qquad I(x) = \int_a^b x\,\mathrm{d}x = \frac{b^2 - a^2}{2}, \qquad I(x^2) = \int_a^b x^2\,\mathrm{d}x = \frac{b^3 - a^3}{3}.$$

Applying the midpoint rule we have

$$I_0(1) = (b-a) f\left(\frac{b+a}{2}\right) = b - a = I(1), \qquad I_0(x) = (b-a) f\left(\frac{b+a}{2}\right) = (b-a)\left(\frac{b+a}{2}\right) = \frac{b^2 - a^2}{2} = I(x),$$

$$I_0(x^2) = (b-a) f\left(\frac{b+a}{2}\right) = (b-a)\left(\frac{b+a}{2}\right)^2 = \frac{b^3 + 2ab^2 - 2a^2b - a^3}{4} \neq I(x^2)$$

So, the midpoint rule has degree of exactness $m = 1$. Next, applying the trapezoid rule we have

$$I_1(1) = \frac{b-a}{2}(f(a) + f(b)) = \frac{b-a}{2}(1+1) = b - a = I(1),$$
$$I_1(x) = \frac{b-a}{2}(f(a) + f(b)) = \frac{b-a}{2}(a+b) = \frac{b^2-a^2}{2} = I(x,)$$
$$I_1(x^2) = \frac{b-a}{2}(f(a) + f(b)) = \frac{b-a}{2}(a^2+b^2) = \frac{b^3+a^2b-ab^2-a^3}{2} \neq I(x^2).$$

So, the trapezoid rule also has degree of exactness $m = 1$. ⧫

Remark 8.7. That the midpoint quadrature, which is an open Newton-Cotes method, has a higher degree of exactness than expected is a general result. Open Newton-Cotes methods that use an odd number of quadrature nodes obtain an extra degree of exactness. We saw this in the example above that the midpoint quadrature rule, which uses one node, has an extra degree of exactness. That is, we expected it to only have exactness for constant functions but it is exact for linear functions as well! ⋈

These Newton-Cotes quadrature methods are simple in principle, but to create very high accuracy quadrature rule on the entire interval $[a, b]$ is unwieldy. However we can use a defining feature of the definite integral that we can break-up the total integral as the sum of two integrals, e.g.

$$I(f) = \int_a^b f(x)\,\mathrm{d}x = \int_a^c f(x)\,\mathrm{d}x + \int_c^b f(x)\,\mathrm{d}x$$

where $c = \frac{b+a}{2}$. We can then apply a simple Newton-Cotes quadrature formula to these two definite integrals now each with a smaller interval. For example, if we apply the trapezoid rule we have

$$I_1(f) = \frac{h_I}{2}(f(a) + f(c)) + \frac{h_I}{2}(f(c) + f(b)) = \frac{h_I}{2}(f(a) + 2f(c) + f(b)), \qquad h_I = \frac{b-a}{2}$$

where h_I is the width of each of the sub-intervals.

Remark 8.8. This idea of breaking the larger interval into sub-intervals is a ***composite quadrature rule***. It has a similar spirit to piecewise polynomial interpolation where we use several low order approximations on sub-intervals and add the results together. ⋈

Below we collect composite versions of the three quadrature rules discussed previously:

Definition 8.7 (Composite quadrature formulas). Composite forms of the midpoint, trapezoid, and Simpson's quadrature rules on the interval $[a, b]$ for an appropriately smooth integrand $f(x)$ are given below. Here, we divide the interval into n sub-intervals of equal size with the nodes

$$x_i = a + ih, \quad h = \frac{b-a}{n}, \quad \text{where} \quad i = 0, \ldots, n.$$

For the formulas below we assume that the value of n is an ***even*** integer. Then the composite quadrature rules are

$$I_0^{\text{comp}}(f) = h\left(f\left(\frac{x_0+x_1}{2}\right) + f\left(\frac{x_1+x_2}{2}\right) + \cdots + f\left(\frac{x_{n-1}+x_n}{2}\right)\right),$$
$$I_1^{\text{comp}}(f) = \frac{h}{2}\left(f(x_0) + 2f(x_1) + \cdots + 2f(x_{n-1}) + f(x_n)\right),$$
$$I_2^{\text{comp}}(f) = \frac{h}{3}\left(f(x_0) + 4f(x_1) + 2f_(x_2) + 4f(x_3) + \cdots + 2f(x_{n-2}) + 4f(x_{n-1}) + f(x_n)\right),$$

with corresponding errors

$$E_0^{\text{comp}} = \frac{(b-a)h^2}{24}f''(\xi), \qquad E_1^{\text{comp}} = -\frac{(b-a)h^2}{12}f''(\xi), \qquad E_2^{\text{comp}} = -\frac{(b-a)h^4}{180}f^{(4)}(\xi).$$

The error estimate changes because it is the sum of the error estimates over the n sub-intervals. ◀

We apply the composite versions of the midpoint and Simpson's quadrature formulas to examine the error. The accompanying figure illustrates how the composite quadrature rule divides the total discrete integral into pieces.

EXAMPLE 8.3. We return to the approximation of the integral

$$I(e^x) = \int_0^1 e^x \, \mathrm{d}x = e - 1 = 1.7182818284\ldots$$

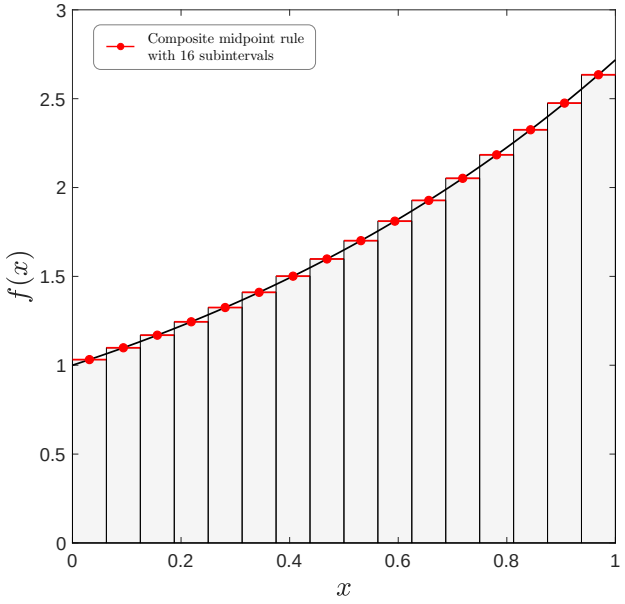
However, we now apply a composite form of the midpoint and Simpson's quadrature rules on 16 subintervals to find

$$I_0^{\text{comp}}(e^x) = 1.718002192\ldots$$
$$I_2^{\text{comp}}(e^x) = 1.718281974\ldots$$

where we compute the corresponding errors

$$|I(e^x) - I_0^{\text{comp}}(e^x)| \approx 2.79636406 \times 10^{-4}$$
$$|I(e^x) - I_2^{\text{comp}}(e^x)| \approx 1.45592846 \times 10^{-7}$$

where we see that the composite Simpson's rule performs very well. ♦



8.2 Gaussian quadrature

Can we make the quadrature rules more exact than what we obtain with Newton-Cotes type formulas? Yes, and to do so we will again consider forming an approximation over the entire interval $[a, b]$ similar to what was done for the global approximation of a function. Recall that, in general, the quadrature rule has the form

$$I(f) \approx I_n(f) = \sum_{i=0}^{n} \omega_i f(x_i).$$

Before, we made the simplifying assumption that the quadrature nodes were going to be *uniformly* spaced in the interval $[a, b]$. However, we now remove that assumption. This means in the sum above we have $2n + 2$ parameters to determine the quadrature nodes and the quadrature weights. Intuition tells us that the maximal degree of exactness possible would be $m = 2n + 1$. So, this is now our goal. We want to find the quadrature nodes and quadrature weights that maximize the degree of exactness.

This is the basic idea behind the family of *Gauss quadrature* rules. Note that just like when we switched to the Chebyshev nodes for the global interpolation, these Gauss quadratures assume that we are free to **select** an arbitrary set of quadrature nodes and that the corresponding function data will be available. However, from this assumption we can gain a lot of ground in the approximation of definite integrals.

The selection of these quadrature nodes and weights is related to the families of orthogonal polynomials introduced in a previous note set. Thus, we will restrict ourselves to the interval $[-1, 1]$ for the time being such that we can determine the quadrature rule with maximum degree of exactness. To illustrate this more accurate quadrature procedure let us say we want to use two quadrature points to approximate the integral

$$\int_{-1}^{1} f(x) \, \mathrm{d}x \approx \omega_0 f(x_0) + \omega_1 f(x_1).$$

From above we know that for two quadrature points the maximal degree of exactness is $m = 2(1) + 1 = 3$. Therefore, we must find the quadrature nodes and weights such the following four integrals are all exact:

$$\omega_0 + \omega_1 = \int_{-1}^{1} 1 \, \mathrm{d}x \stackrel{!}{=} 2, \qquad \omega_0 x_0 + \omega_1 x_1 = \int_{-1}^{1} x \, \mathrm{d}x \stackrel{!}{=} 0,$$

$$\omega_0 x_0^2 + \omega_1 x_1^2 = \int_{-1}^{1} x^2 \, \mathrm{d}x \stackrel{!}{=} \frac{2}{3}, \quad \text{and} \quad \omega_0 x_0^3 + \omega_1 x_1^3 = \int_{-1}^{1} x^3 \, \mathrm{d}x \stackrel{!}{=} 0.$$

Forgoing the algebra for the sake of brevity, we solve these four equations for the four unknowns to find

$$x_0 = -\frac{1}{\sqrt{3}}, \quad x_1 = \frac{1}{\sqrt{3}}, \quad \omega_0 = \omega_1 = 1.$$

This yields a particular instance of a Gauss-type quadrature

Remark 8.9. Notice that these quadrature nodes form an *open* quadrature rule, because the endpoints are not included. This is not an accident. Just as we saw for Newton-Cotes, open quadrature rule can possess a higher degree of accuracy than their closed counterparts. ⋈

Definition 8.8 (Two-point Legendre-Gauss quadrature). On the interval $[-1, 1]$ the two-point *Legendre-Gauss* quadrature rule for an integrand with four continuous derivatives uses the quadrature nodes and weights

$$x_0 = -\frac{1}{\sqrt{3}}, \quad x_1 = \frac{1}{\sqrt{3}}, \quad \omega_0 = \omega_1 = 1,$$

and is defined as

$$I_1^{\mathrm{LG}} = f\left(-\frac{1}{\sqrt{3}}\right) + f\left(\frac{1}{\sqrt{3}}\right)$$

with the corresponding error bound

$$E_1^{\mathrm{LG}} = \frac{2^5(2!)^4}{5(4!)^3} f^{(4)}(\xi) = \frac{1}{135} f^{(4)}(\xi), \quad \text{where} \quad \xi \in [-1, 1]$$

where we see the necessity that the integrand function $f(x)$ is smooth with bounded derivatives. ◀

Example 8.4. We return to the example of approximating the integral of e^x now on the interval $[-1, 1]$

$$I(e^x) = \int_{-1}^{1} e^x \, \mathrm{d}x = e - e^{-1} = 2.350402387\ldots$$

now using the two-point Legendre-Gauss quadrature rule. This yields the approximation

$$I_1^{\mathrm{LG}}(e^x) = f\left(-\frac{1}{\sqrt{3}}\right) + f\left(\frac{1}{\sqrt{3}}\right) = e^{-\frac{1}{\sqrt{3}}} + e^{\frac{1}{\sqrt{3}}} = 2.342696087\ldots$$

For comparison Simpson's quadrature rule on the same problem gives a less accurate approximation with

$$I_2(e^x) = \frac{\frac{1-(-1)}{2}}{3}\left(f(-1) + 4f\left(\frac{1+(-1)}{2}\right) + f(1)\right) = \frac{1}{3}(e^{-1} + 4 + e) = 2.3620537565\ldots$$

This is easier to see when comparing the errors

$$|I(e^x) - I_1^{\mathrm{LG}}(e^x)| \approx 7.7062 \times 10^{-3}, \qquad |I(e^x) - I_2(e^x)| \approx 1.16513 \times 10^{-2}.$$

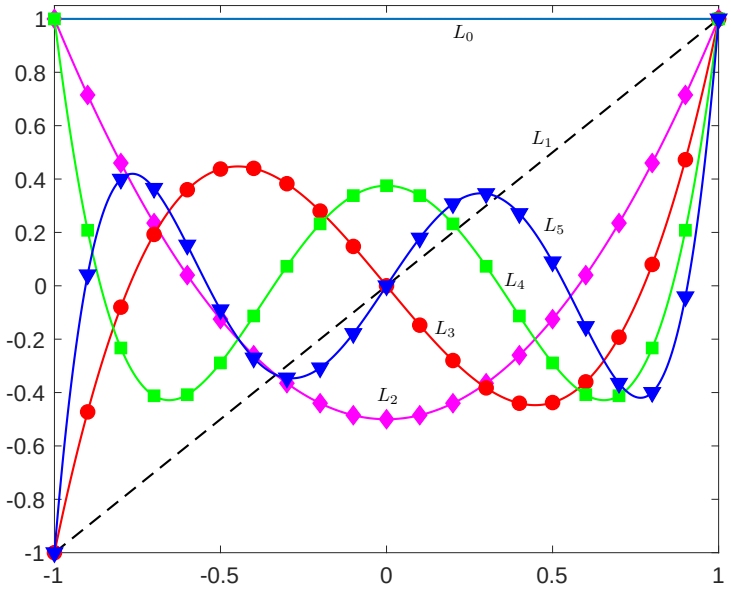
Some even though the Legendre-Gauss rule used one less quadrature point it was a more accurate integral approximation than the Newton-Cotes style Simpson's rule. ⧫

Remark 8.10. This example of a Legendre-Gauss quadrature is typical for the higher order variants with $n+1$ quadrature nodes and weights. That is, the quadrature nodes are placed symmetrically about the midpoint of the interval and the weights will also be symmetric. Further, this simple two-point quadrature highlights that the nodes (and potentially the weights) are *irrational numbers*. This feature makes Legendre-Gauss quadrature inconvenient for pen-and-paper computations, compared to the Newton-Cotes quadrature rules. However, when using a computer the nodes and weights can be easily tabulated in advance and applied to a problem when needed. ⋈

What exactly is going on here? How can we use less function information, but obtain a better integral approximation? To explain why these Gauss-type quadratures are more accurate (even with fewer quadrature points) we return to the discussion of orthogonal families of polynomials. In particular, we use the *Legendre polynomials*. Recall that the Legendre polynomials satisfy a three-term recurrence relation

$$L_0(x) = 1, \qquad L_1(x) = x, \qquad L_{k+1} = \frac{2k+1}{k+1} x L_k(x) - \frac{k}{k+1} L_{k-1}(x),$$

for $k = 1, \ldots, n$. The first few members of this family of polynomials are shown below.



They may look indistinguishable from the Chebyshev polynomials, with their symmetrical oscillation behavior and what not. However, we see the the maximums of the polynomials no longer occur at either -1 or 1 (this is one reason these Legendre points were unsuitable for the optimal interpolation nodes). But there is an important difference for the Legendre polynomials in the context of numerical integration that has to do with the sense in which they are orthogonal to one another:

$$\int_{-1}^{1} L_i(x) L_j(x)\,\mathrm{d}x = \begin{cases} \frac{2}{2j+1} & \text{if } i = j \\ 0 & \text{if } i \neq j \end{cases}$$

Sure they are orthogonal, but we see that the weight function of their inner product is unity, i.e., $w(x) = 1$. This makes this family of polynomials an ideal candidate to build a "better" set of quadrature nodes. Once we know the set of quadrature nodes it is possible to define the quadrature weights using the same form in the Newton-Cotes formulation, see Remark 8.5.

The roots of the Legendre polynomials are distinct, lie in the interval $(-1, 1)$, have a symmetry with respect to the origin, and, most importantly, are the correct choice for determining the parameters that give

us the quadrature nodes and coefficients for the Legendre-Gauss quadrature rule. In other words, the set of nodes $\{x_i\}_{i=0}^n$ that are taken to be the roots of the $(n+1)^{\text{st}}$ degree Legendre polynomial are enough to produce an integral approximation formula that gives exact results for any polynomial of degree less than or equal to $2n+1$, which is precisely our goal.

Remark 8.11. The details of why choosing the roots of the Legendre polynomial works are too technical to get into for this note set. But the basic idea is similar to how we solved rectangular matrix systems using the **QR** factorization. There is a certain amount of information that the problem "reach" and approximate correctly and the rest of the information (i.e. the error) is projected out of the approximate solution by making the error orthogonal to the solution space. This is the basic idea for Gauss-type quadratures only now we work in function "spaces" and we are forcing polynomial functions (and their errors) to be orthogonal to one another, rather than working with vectors in different vector spaces as with **QR**. ⋈

Definition 8.9 ($(n+1)$-point Legendre-Gauss quadrature). The $(n+1)$-point Legendre-Gauss quadrature rule takes the form

$$I_n^{\text{LG}}(f) = \sum_{i=0}^{n} \omega_i f(x_i)$$

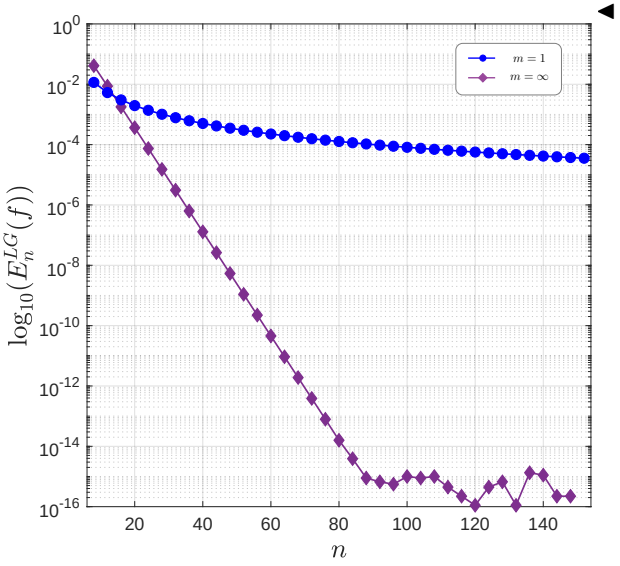
where the quadrature nodes $\{x_i\}_{i=0}^n$ are the roots of the $(n+1)^{\text{st}}$ Legendre polynomial and the quadrature weights are

$$\omega_i = \int_{-1}^{1} \ell_i(x)\,\mathrm{d}x = \int_{-1}^{1} \prod_{\substack{j=0 \\ j\neq i}}^{n} \frac{x - x_j}{x_i - x_j}, \quad \text{where} \quad i = 0, \ldots, n,$$

for the given set of quadrature nodes. This Legendre-Gauss quadrature rule has a degree of exactness $m = 2n+1$, which is optimal. The error for the Legendre-Gauss quadrature is given by

$$E_n^{\text{LG}} = I(f) - I_n^{\text{LG}}(f) = \left(\frac{2^{2n+3}[(n+1)!]^4}{(2n+3)[(2n+2)!]^3} \right) f^{(2n+2)}(\xi), \quad \text{where} \quad \xi \in [-1, 1].$$

We see that this error estimate is looks very odd and cumbersome to work with. But, just like with global interpolation on the Chebyshev nodes. The error will decay incredibly fast as long as the function you want to integrate is smooth and bounded up to its $2n+2$ derivative. So, if a function is ***very*** smooth (i.e. infinitely differentiable and bounded) then the Legendre-Gauss quadrature is almost unbeatable in terms of accuracy and convergence to the solution value. We illustrate this on the left for the absolute value function, $f(x) = |x|$, which is not smooth and $f(x) = (1+25x^2)^{-1}$, the Runge function, which is infinitely differentiable. We see the error approaches machine precision for the very smooth function, but the convergence stagnates for the non-smooth function.



Notice, we have done something else interesting here for this high-order accurate quadrature problem. Numerical methods in this course have been outlined with the strategy that we take a problem we cannot solve and turn it into one we can solve. Before, this usually meant transforming the problem into something involving matrices. However, for quadrature we have done something quite different. We had a goal, to have as accurate of a quadrature rule as possible. This was impossible to achieve at the outset, especially for

an arbitrary integrand $f(x)$. But through some clever mathematical tools we transformed the quadrature problem (something from Calculus) into a polynomial root finding problem (something from algebra)!

Once we know the quadrature nodes we have the complete $(n+1)$-point Legendre-Gauss quadrature rule (modulo computing the weights). But this simply boils down to computing the roots of a polynomial, specifically the $(n+1)^{\text{st}}$ degree Legendre polynomial. This is straightforward because we can create the polynomial from the recurrence relation and then find the roots using the Newton-Raphson method.

Remark 8.12. This illustrates again that there is no "free lunch" in the world of computation mathematics. We had a task that we wanted to compute, in this case approximating an integral, and we always must spend computational effort to get there. We can just be "smart" in where we put the work to create the most accurate approximation we can from the information provided. ⋈

Remark 8.13. Alternatively, we could formulate the problem of finding the nodes and weights for an $(n+1)^{\text{st}}$ order Legendre-Gauss quadrature rule as an eigenvalue problem. This is not necessarily simpler, but it depends on your perspective. Some people prefer solving eigenvalue problems instead of root finding problems because there is less "guess" work (keep in mind for Newton to find these $n+1$ roots you need that many initial guesses as well). This highlights, again, that the work has to go somewhere! If you do not want to deal with root finding the problem becomes one of eigenvalues. Conservation of computational work! ⋈

Remark 8.14. The Legendre-Gauss quadrature rule is only defined on the interval $[-1,1]$; however, just as with the Chebyshev interpolation nodes, we can map the quadrature rule into an arbitrary interval $[a,b]$ with an affine transformation. That is, for the set of Legendre-Gauss quadrature nodes $\{x_i\}_{i=0}^n$ in $[-1,1]$ we have the mapped quadrature nodes

$$t_i = \frac{b-a}{2}x_i + \frac{b+a}{2}, \quad \text{where} \quad t_i \in [a,b] \quad \text{and} \quad i = 0,\dots,n.$$

Fortunately, the quadrature weights $\{\omega_i\}_{i=0}^n$ stay the same under this affine transformation. This is basically using a substitution rule in the integral to obtain the general formula

$$\int_a^b f(t)\,\mathrm{d}t = \frac{b-a}{2}\int_{-1}^{1} f\left(\frac{b-a}{2}x + \frac{b+a}{2}\right)\,\mathrm{d}x \approx \frac{b-a}{2}\sum_{i=0}^{n}\omega_i f\left(\frac{b-a}{2}x_i + \frac{b+a}{2}\right).$$

Take care and remember to include the constant that appears in front of the mapped integral. ⋈

Example 8.5. Approximate the following integral

$$I(e^{-x^2}) = \int_0^1 e^{-x^2}\,\mathrm{d}x = 0.746824132812\dots$$

using the two-point Legendre-Gauss quadrature rule. For this interval of integration, the affine mapping for the quadrature nodes implies that

$$t_i = \frac{1}{2}(x_i+1) \quad \Rightarrow \quad t_0 = \frac{1}{2}\left(-\frac{1}{\sqrt{3}}+1\right), \;\; t_1 = \frac{1}{2}\left(\frac{1}{\sqrt{3}}+1\right),$$

and the quadrature weights are still $\omega_0 = \omega_1 = 1$. Then the integral approximation is given by

$$I_1^{\text{LG}} = \frac{1}{2}\left[\exp\left(-\left(\frac{1}{2}\left(1-\frac{1}{\sqrt{3}}\right)\right)^2\right) + \exp\left(-\left(\frac{1}{2}\left(1+\frac{1}{\sqrt{3}}\right)\right)^2\right)\right] = 0.746594688\dots$$

which compares well to the exact value. It actually has three correct decimals because

$$|I(e^{-x^2}) - I_1^{\text{LG}}| = |0.746824132812\dots - 0.746594688\dots| \approx 0.000229 = 0.229\times 10^{-3} \le 0.5\times 10^{-3}.$$

♦

This further illustrates that is incredibly unwieldy to evaluate Legendre-Gauss quadrature rules by hand. However, that is what computers are for! They can easily handle all these complicated irrational numbers and perform the computations. We just have to keep rounding, approximation, and truncation errors in mind as we do so.

Remark 8.15. Although these Legendre-Gauss quadrature rules might seem very cumbersome. We have actually used one of them several times without realizing it. The midpoint rule from the Newton-Cotes quadrature is actually the Legendre-Gauss quadrature rule for a single quadrature node. This offers another explanation of why the midpoint quadrature rule had higher degree of exactness than we first expected. ⋈

What about "closed" variants of these Gauss-type quadratures? Can we include the endpoints of the interval and still obtain a high accuracy quadrature rule. Of course we can! Although the degree of exactness is slightly less than the "open" Legendre-Gauss variants. This is because we *lose* two of the degrees of freedom when we determine the quadrature nodes and weights because we pin down the endpoints. This family of quadrature rules are known as *Legendre-Gauss-Lobatto quadrature* rules.

Remark 8.16. The degree of exactness for a Legendre-Gauss-Lobatto quadrature rule with $n+1$ quadrature nodes and weights is $m = 2n - 1$. Notice that guaranteeing that the boundary values are included removed two degrees of freedom and, thus, we could only guarantee this slightly reduced degree of exactness. ⋈

Remark 8.17. We have actually done a version of this Legendre-Gauss-Lobatto throughout this note set as well and we didn't realize it. For three quadrature nodes (when $n = 2$) the Legendre-Gauss-Lobatto rule is just Simpson's rule from our Newton-Cotes discussion. If we take $n = 2$ then the quadrature nodes for the Legendre-Gauss-Lobatto quadrature rule on the interval $[-1, 1]$ are

$$x_0 = -1, \qquad x_1 = 0, \qquad x_2 = 1,$$

with the corresponding weights

$$\omega_0 = \omega_2 = \frac{1}{3} \quad \text{and} \quad \omega_1 = \frac{4}{3}.$$

The degree of exactness of this Legendre-Gauss-Lobatto quadrature rule (also Simpson's rule) is

$$m = 2(2) - 1 = 3.$$

So, cubic polynomials are exact under this quadrature rule, which is better than we expect from using a quadratic polynomial to approximate the integrand. ⋈

The quadrature nodes of the $(n + 1)$-point Legendre-Gauss-Lobatto rule are found by locating the roots of the first derivative of the $(n + 1)^{\text{st}}$ Legendre polynomial. Again, this is a root finding problem that, sometimes, results in an explicit form of the quadrature nodes and weights. Usually, however, the Legendre-Gauss-Lobatto nodes and weights are simply tabulated (just like before for Legendre-Gauss quadrature).

Definition 8.10 (Four-point Legendre-Gauss-Lobatto quadrature rule). We define a Legendre-Gauss-Lobatto quadrature rule with four interpolation node ($n = 3$)

$$x_0 = -1, \qquad x_1 = -\frac{1}{\sqrt{5}}, \qquad x_2 = \frac{1}{\sqrt{5}}, \qquad x_3 = 1,$$

and their corresponding weights

$$\omega_0 = \frac{1}{6}, \qquad \omega_1 = \frac{5}{6}, \qquad \omega_2 = \frac{5}{6}, \qquad \omega_3 = \frac{1}{6}.$$

The error in this approximation, again, depends on the smoothness of the integrand function $f(x)$ in that it requires smoothness up to the sixth derivative to guarantee convergence of the approximation of the integral. ◀

EXAMPLE 8.6. We apply the four-point Legendre-Gauss-Lobatto quadrature rule to approximate the integral of the exponential function

$$I(e^x) = \int_{-1}^{1} e^x \, \mathrm{d}x = e - e^{-1} = 2.3504023872\ldots$$

Then evaluating the integrand function at the four Legendre-Gauss-Lobatto nodes we have

$$f(-1) = e^{-1}, \qquad f\left(-\frac{1}{\sqrt{5}}\right) = e^{-\frac{1}{\sqrt{5}}}, \qquad f\left(\frac{1}{\sqrt{5}}\right) = e^{\frac{1}{\sqrt{5}}}, \qquad f(1) = e$$

Then the approximation to the integral with the four-point Legendre-Gauss-Lobatto quadrature rule is

$$\begin{aligned} I_3^{\mathrm{LGL}}(e^x) &= \frac{1}{6} f(-1) + \frac{5}{6} f\left(-\frac{1}{\sqrt{5}}\right) + \frac{5}{6} f\left(\frac{1}{\sqrt{5}}\right) + \frac{1}{6} f(1) \\ &= \frac{e^{-1}}{6} + \frac{5e^{-\frac{1}{\sqrt{5}}}}{6} + \frac{5e^{\frac{1}{\sqrt{5}}}}{6} + \frac{e}{6} \\ &= 2.35048990751\ldots \end{aligned}$$

We compare this approximation to Simpson's rule (which is the Legendre-Gauss-Lobatto method with $n = 2$) to see

$$\begin{aligned} |I(e^x) - I_2^{\mathrm{LGL}}(e^x)| &= 0.011651369 \\ |I(e^x) - I_3^{\mathrm{LGL}}(e^x)| &= 8.75202318 \times 10^{-5} \end{aligned}$$

which illustrates that adding a single quadrature node in a Gauss-type quadrature rule can have a profound impact on the accuracy of the approximation if the underlying integrand function is very smooth. In this case, the number of correct decimals in the integral approximations doubles! ♦

Note Set 9: Ordinary differential equations: Basics and Euler

Modeling quantities that change continuously in time and space have a many applications across science and engineering. Quantifying these rates of change are often done by constructing differential equations. When any derivatives in the model fall on just one of the independent variables these models are called *ordinary differential equations* (ODEs). Differential equations must be supplanted with additional information that define the modeling situation and, from mathematical theory, produce a unique solution to the problem. These conditions can come from prescribed information at the boundary of the domain or, more simply, as an initial value that indicates where the model begins and then is allowed to evolve.

In this note set, we take the independent variable of the ODE to represent time t and examine ***initial value problems***. From an ODE course you may recall that there are a multitude of strategies to solve initial value problems like integrating factors, separation of variables, series solutions, etc. However, these tactics only work for a small class of ODEs. Just like with integration, we will utilize the power of a computer to solve ODEs through more general strategies.

9.1 Background and forward Euler

The solution of ODEs numerically acts as a nice culmination of the different topics covered in this course. This is because it combines ideas from solving nonlinear equations, polynomial interpolation, quadrature, etc. to produce a multitude of techniques to solve initial value problems with a computer. We will see that usually these methods will take the known initial value(s) and then "march" a discrete version of the ODE forward in time. Although there are subtleties between the tactics to do this mostly in terms of accuracy and reliability.

Before we discuss the numerics of this problem we need to set some *groundwork* from our ODE knowledge. Specifically, we need to talk about existence, uniqueness, and stability of the continuous problem before we know how to create discrete analogues of these properties. In this sense, the study and development of solving ODEs numerically is where the line between mathematics and the other sciences blurs as we will see later, through example.

To begin, we consider a very famous ODE that comes from Newton's second law of motion, $F = ma$, or that force equals mass times acceleration. This simple equation relates the state of an object, in this case its position in space, to the second derivative of that state, in this case its acceleration. In the language of mathematics we express this physical law as

$$F(t, y(t)) = m\frac{\mathrm{d}^2 y(t)}{\mathrm{d}t^2}, \qquad \text{or} \qquad F(t, y(t)) = my''(t),$$

where we will use the compact "prime" notation to denote time derivatives. We use this example to illustrate that we will only consider first-order ODEs numerically (and in this note set). This is because we can ***always*** convert a single ODE equation with higher order derivatives into a *system* of first order equations by introducing auxiliary variables. For example, in Newton's second law we can introduce $y_1' = y_2$ (so physically y_2 is the velocity) and the ODE becomes (note we have suppressed the dependency on the variable t)

$$y'' = \frac{F}{m} \qquad \Rightarrow \qquad \begin{pmatrix} y_1' \\ y_2' \end{pmatrix} = \begin{pmatrix} y_2 \\ \frac{F}{m} \end{pmatrix}$$

This produces a *system of coupled ODEs* that describes how the function $y(t)$ evolves in time and its possible interaction with other forces.

This provides us with another important distinction in this world of ODEs. The equation for Newton's second law (whether in scalar or system form) is a ***linear*** ODE because the right-hand-side only depends

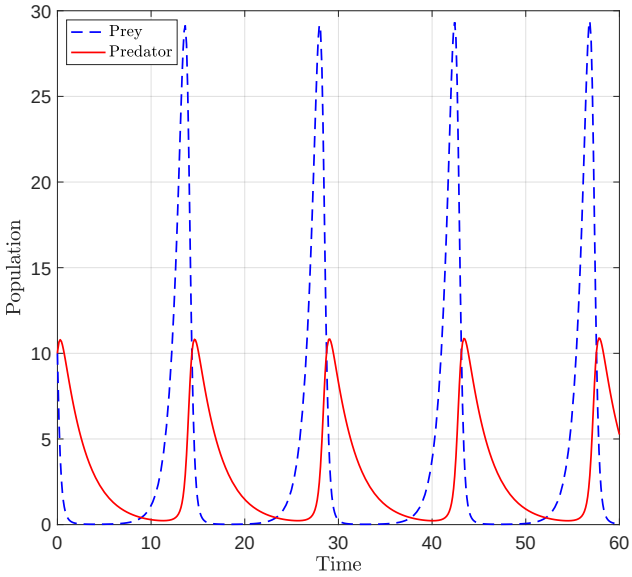
on the independent variable t and the function itself $y(t)$. More complicated phenomena from physics or chemistry or biology often produce mathematical models that are ***nonlinear*** ODE(s) where the right-hand-side has some nonlinear dependence on the solution $y(t)$. This is best seen through an example.

EXAMPLE 9.1. The Lotka-Volterra equations

$$\begin{pmatrix} y_1' \\ y_2' \end{pmatrix} = \begin{pmatrix} \alpha y_1 - \beta y_1 y_2 \\ \delta y_1 y_2 - \gamma y_2 \end{pmatrix},$$

are a nonlinear ODE that describes the interaction of a prey species, denoted y_1, and a predator species, denoted y_2, of animal. The constant α is the population growth of the prey and β is the rate at which the prey get eaten by the predator. Whereas the constant δ is the rate at which the predator encounters the prey and γ is rate at which the predator dies either from old age or lack of food. Note that the coupling of the predator/prey interaction is nonlinear due to the product $y_1 y_2$. On the right we provide a (numerical) solution to the Lotka-Volterra equations where

$$\alpha = 1.1, \quad \beta = 0.4 \quad \delta = 0.1 \quad \gamma = 0.4$$



modeling the population relationship between foxes and rabbits. The initial population of both animals was taken to be $(y_1, y_2)^T = (10, 10)^T$. Even for this relatively simple nonlinear ODE system *there is no explicit form* for the solution $(y_1(t), y_2(t))^T$. This is not a fault of the model, which comes from the biology and population dynamics communities, but highlights why we often need to turn to computers to solve problem we otherwise cannot through traditional mathematical means. ♦

So the stage is set with some notation out of the way such that we can dig into the topic of numerically solving ODEs. For this discussion we will restrict ourselves to a general *scalar* first order initial value problem defined as follows:

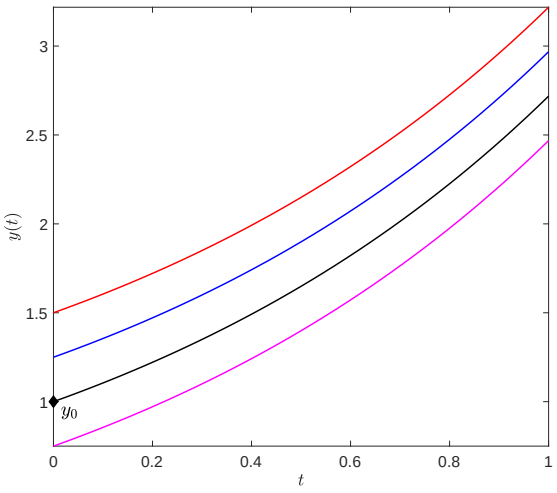
DEFINITION 9.1 (INITIAL VALUE PROBLEM). The general first order initial value problem and its associated initial condition are

$$y'(t) = f(t, y(t)), \quad t_0 \le t \le T, \qquad \text{with} \qquad y(t_0) = y_0,$$

where t_0 is the initial time, T is the final time and y_0 is the initial value of the function $y(t)$ provided. A solution to the initial value problem is a function $y(t)$ that satisfies both the differential equation **and** the initial condition. ◄

It is very important to keep in mind that an ODE $y'(t) = f(t, y)$ by itself does not determine a ***unique*** solution because only the slopes of the solution $y'(t)$ are actually prescribed by the ODE for the independent variable t, not the solution itself $y(t)$. This results in a *family* of solutions that satisfy the ODE. This is where the initial condition y_0 comes in to single out a particular solution. Under some reasonable assumptions about the right-hand-side function $f(t, y(t))$ this initial condition determines a unique solution to the given ODE.

EXAMPLE 9.2. Consider the ODE $y' = \lambda y$. There is an infinite family of solutions to the problem given by $y(t) = ce^{\lambda t}$, where c is a real constant. If we impose the initial condition $y(t_0) = y_0$, then we single out the unique solution that satisfies the initial condition. If $t_0 = 0$ then we have $y(t_0) = y(0) = ce^0 = c$, such that $c = y_0$ and the unique solution satisfying both the ODE and the initial condition is $y(t) = y_0 e^{\lambda t}$. We provide a sketch on the next page of this family of solutions, including the particular unique solution for the initial condition $y(t_0) = y_0$. ♦



So we see that an initial condition is necessary for the solution of an ODE to be unique. But is the condition *sufficient*? Moreover, does a solution always exist? Without some restriction on the problem, the answer to both of these questions is a resounding ***no***. However, under the following reasonable assumptions on the problem, the answer to both questions is ***yes***!

DEFINITION 9.2 (EXISTENCE AND UNIQUENESS OF A SOLUTION). On a domain where the variable $t \in [t_0, T]$ if the Jacobian of the right-hand-side exists and is bounded

$$\frac{\mathrm{d}f}{\mathrm{d}y} \quad \text{exists} \qquad \text{and} \qquad \left|\frac{\mathrm{d}f}{\mathrm{d}y}\right| < L$$

for some constant L, i.e., f is Lipschitz continuous, then the initial value problem with an accompanying initial condition has a unique solution for all $t \in [t_0, T]$. ◀

Under these conditions, solving the initial value problem is said to be ***well-posed*** provided:

1. A solution exists.
2. The solution is unique.
3. The behavior of the solution changes continuously with the initial condition.

EXAMPLE 9.3. Returning to the example with $y' = \lambda y$, the Jacobian of the right-hand-side is bounded under the assumption that λ is a constant

$$\left|\frac{\mathrm{d}f}{\mathrm{d}y}\right| = |\lambda| = L.$$

and the solution, $y(t)$, moves continuously through the infinitely family of solutions as the initial condition changes. ⧫

Now that we have some context for the solution of an initial value problem we are ready to discuss how to discretely solve the problem with a computer. To do this, we need to first discretize the interval of time we are interested in where $t \in [t_0, T]$. A simple approach we have used before is to divide this interval into $n + 1$ uniformly spaced points

$$t_i = t_0 + ih, \qquad h = \frac{T - t_0}{n}, \quad \text{where} \quad i = 0, \ldots, n.$$

just like we have done for interpolation, quadrature, etc.

Remark 9.1. We use h to define the spacing between points in time within the interval $[t_0, T]$ and will refer to h as the "time step" or "step size" throughout this discussion. Due to this naming convention other resources (like those online) may refer to this quantity h as Δt. ⋈

From this discrete version of the time domain we know the starting point as well as the solution because of the initial condition $y(t_0) = y_0$. So, the numerical solution of the initial value problem is obtained by starting at this known information and then attempting to step through the discrete time points and track the solution trajectory dictated by the ODE. That is the general idea, but there are many ways to do this.

The simplest approach to achieve this goal is through the *forward Euler method*, which basically follows the logic that we will evaluate the ODE at the current time, say t_k, because it is information that we know and then we approximate the solution at the next time, t_{k+1}, by moving the current solution one time step forward.

Definition 9.3 (Forward Euler). For the discrete set of points in time $t_i = t_0 + ih$, $h = \frac{T-t_0}{n}$ if we have the solution at the current time t_k the approximate solution at the next time t_{k+1} is given by

$$y_{k+1} = y(t_{k+1}) = y(t_k) + hf(t_k, y(t_k)) = y_k + hf_k$$

where we introduce a compact index notation for evaluating the function and the right-hand-side at the discrete time points. ◀

Remark 9.2. The forward Euler method is an ***explicit*** approximation to the original initial value problem because we only need information from the solution at the current time to advance one time step to the approximate solution at the next time. ⋈

The forward Euler method is seldom used in practice but, due to its simplicity, it is important and helpful to understand the concepts and principles of designing a numerical approximation to solve initial value problems numerically. Additionally, the analysis of the method is straightforward as shown later. There are many ways to arrive at the forward Euler method beginning from the initial value problem

$$y' = f(t, y), \quad y(t_0) = y_0.$$

Two such strategies are of particular interest because they are related to topics covered previously in this course. The first is to replace the time derivative in the initial value problem with a first order finite difference approximation and evaluate the right-hand-side at the current time value

$$\frac{y_{k+1} - y_k}{h} = f(t_k, y_k), \quad y(t_0) = y_0, \quad \text{where} \quad k = 0, \ldots, n-1.$$

This transforms the initial value problem into an algebraic equation that can solved for y_{k+1}. The second strategy is to integrate the initial value problem in time on the time interval $[t_k, t_{k+1}]$. In this case the time derivative can be immediately removed due to the Fundamental Theorem of Calculus:

$$y_{k+1} - y_k = \int_{t_k}^{t_{k+1}} f(s, y(s)) \, \mathrm{d}s$$

We then approximate this integral in time with a simple left rectangle quadrature rule where

$$\int_{t_k}^{t_{k+1}} f(s, y(s)) \, \mathrm{d}s \approx (t_{k+1} - t_k) f(t_k, y_k) = hf_k.$$

Both of these strategies lead directly to the forward Euler method albeit motivated in different ways.

Let's see the forward Euler method in action and observe how the approximation evolves.

Example 9.4. Consider the initial value problem

$$y' = y, \quad y(0) = 1.$$

We know the exact solution to this problem is $y_{ex}(t) = e^t$, but let us apply the forward Euler method to solve it numerically, at least for a few steps. We take the final time to be $T = 1$ and use a time step of $h = 0.5$ so we will have three points in time

$$t_0 = 0, \quad t_1 = 0.5, \quad t_2 = 1.0$$

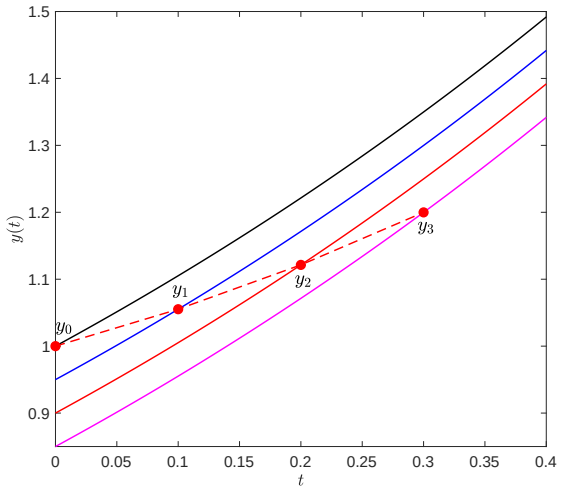
We know the problem data at time $t_0 = 0$ due to the initial condition. We advance to the next solution to time $t_1 = t_0 + h = 0.5$ to find

$$y_1 = y_0 + hf(t_0, y_0) = y_0 + hy_0 = 1 + h = 1.5$$

Note, as we might expect, this approximate value of the solution is *not exact* which is easily seen when we compare this approximate solution to the true solution at time $t_1 = 0.5$

$$y_1 = 1.5, \qquad y_{ex}(0.5) = e^{0.5} \approx 1.649$$

We highlight this to illustrate that the approximate solution given by forward Euler actually lies on a *different solution* of the ODE than that given by the initial condition.



To continue the numerical solution process we take another step from t_1 to $t_2 = t_1 + h = 1.0$ obtaining

$$y_2 = y_1 + hy_1 = 1.5 + (0.5)(1.5) = 2.25.$$

Note that y_2 not only differs from the true solution of the original problem

$$y_2 = 2.25, \qquad y_{ex}(1) = e^1 \approx 2.718$$

but also from another true solution in the family that passes through the point $(t_1, y_1) = (0.5, 1.5)$. This is an important, and subtle, mechanism where error is introduced into the approximate solution within *each time step* of the discrete solution. These local errors accumulate and affect the global error of the scheme. We will return to this discussion on the local versus the global error later. ♦

As shown above, the discrete solution jumps from one member of the family of solutions to the ODE onto another member of the family of solutions as it is marched forward through the time points. That is, each discrete time step is introducing a *perturbation* into the solution. This goes back to one of the original concepts from this course concerning the conditioning of the problem. Will these errors amplify with time or will they decay with time? Under similar assumptions on existence and uniqueness of the solution we can discuss the conditioning (or stability) of an initial value problem

DEFINITION 9.4. If the derivative $\frac{\mathrm{d}f}{\mathrm{d}y}$ exists and $|\frac{\mathrm{d}f}{\mathrm{d}y}|$ is bounded by a constant L for all $t \in [t_0, T]$ then the perturbed solution $y(t, y_0 + \delta)$ of the initial value problem $y' = f(t, y)$ with initial condition $y(0) = y_0 + \delta$ satisfies the bound

$$\max_{t_0 \le t \le T} |y(t, y_0 + \delta) - y(t, y_0)| \le |\delta| e^{L(T - t_0)}$$

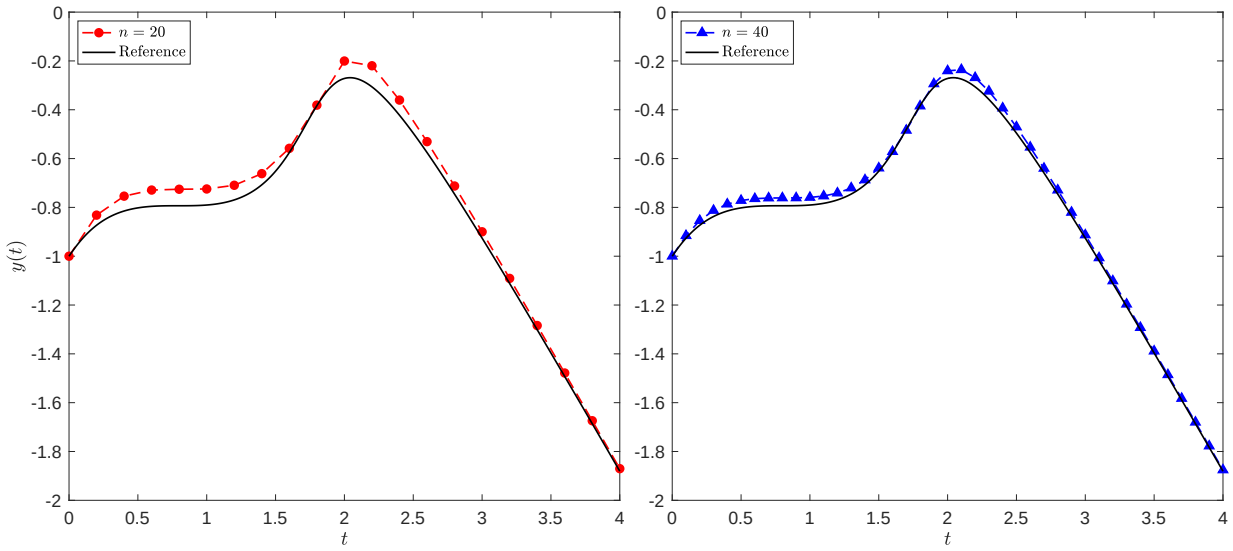
for all sufficiently small values of $|\delta|$. ◄

Remark 9.3. We see that the error in the previous example will amplify within each time step because $\frac{\mathrm{d}f}{\mathrm{d}y} = 1$. This is because the solutions to the ODE $y' = y$ are unstable and the solutions diverge as t increase. However, for an initial value problem with stable solutions the errors do not grow in time. For example, the problem $y' = -y$ has the solution $y(t) = y_0 e^{-t}$ and the errors actually diminish in time as t increases. ⋈

EXAMPLE 9.5. Next, we apply the forward Euler method to a problem to an initial value problem that is a bit more complicated. We consider the initial value problem

$$y' = \sin((t + y)^2), \quad y(0) = -1, \quad 0 \le t \le 4.$$

Below we provide plots of the approximate solution given by the forward Euler method for twenty uniform points in time, i.e., $h = \frac{4-0}{20} = 0.2$ against a reference solution created on a very fine grid of points in time.



We do this because this problem does not have an explicit expression for the true solution against which we can compare. If we double the number of points in the time discretization it changes the approximate solution produced by the forward Euler method considerably and the approximate solution becomes closer to the reference solution. This make sense with how the forward Euler method was derived, say, in the context of a finite difference approximation to the time derivative. We have seen before that decreasing the size of h can make the approximate derivative more accurate and, in turn, it appears to have made the forward Euler method more accurate. ♦

If we increase the number of points in the approximation, can we predict how much better the approximate solution will become using forward Euler? An even more fundamental question is the reliability of adding more points, can we expect the approximation to become better? We know to be cautious here because we have seen with polynomial interpolation that throwing more points at the problem might not actually improve the accuracy.

To answer these questions requires mathematical analysis of the forward Euler method. This analysis is shaped by examining equivalent statements of important concepts from the continuous analysis of an initial value problem. Namely, the topics of interest are:

1. *Consistency* – How well does the discrete equation, like forward Euler, represent the original differential equation?

2. *Convergence* – Does the approximate solution of the discrete equation converge to the solution of the original differential equation?

3. *Stability* – How sensitive is the solution to the discrete equation to perturbations due to round-off errors, finite arithmetic, or inexact problem data?

These are overarching issues throughout the development of time integration solvers for initial value problems whether they are simple, like the forward Euler equations, to more complicated, like Runge-Kutta methods that we will discuss in the next note set.

Next, it is precisely these topics that we will explore in-depth for simple linear step methods, like forward Euler method.

9.2 Consistency, convergence and stability of Euler methods

It is the key components of consistency, convergence, and stability that frame the accuracy and reliability of an approximate solution to an initial value problem. We can think of them as discrete equivalents to the existence, uniqueness, and stability of the original ODE. Throughout this discussion and analysis we will use forward Euler as an example, but the tools we use can be applied to other ODE solvers.

To begin we examine the consistency of the forward Euler method. We know that the numerical solution of an initial value problem will contain discretization errors from two distinct sources:

1. *Rounding error* due to the finite precision of the computer using floating point arithmetic.
2. *Discretization error* (or truncation error) due to the particular method used to approximate the solution to the problem.

This is similar to the errors incurred when approximating the values of derivatives or integrals. Although they arise from different sources, these two types of errors are not independent of one another. For instance, with a finite difference the truncation error can be reduced by using a smaller value of h in the grid spacing, but doing so may lead to greater rounding error. In general, however, truncation error is the dominate factor in determining the accuracy of a numerical solution to an ODE. Thus, we will consider the rounding errors to be negligible.

Consistency of the numerical approximation to an initial value problem is related to the discretization errors made. As such, demonstrating consistency of a numerical method to solve and ODE involves our old friend, the Taylor expansion. That is, to show that the forward Euler method is consistent we examine the Taylor expansion of the solution $y(t)$ at the time t_{k+1}

$$y(t_{k+1}) = y(t_k + h) = y(t_k) + hy'(t_k) + \frac{h^2}{2}y''(t_k) + \mathcal{O}(h^3) = y(t_k) + hf(t_k, y(t_k)) + \frac{h^2}{2}y''(t_k) + \mathcal{O}(h^3)$$

where we substituted the original ODE back into the Taylor expansion to define the $y'(t_k)$ term and we have assumed that the time step h is a constant value. Manipulating this expression we see that

$$\frac{y(t_{k+1}) - y(t_k)}{h} = f(t_k, y(t_k)) + \frac{h}{2}y''(t_k) + \mathcal{O}(h^2).$$

So as the time step becomes arbitrarily small, i.e. $h \to 0$, we recover the original ODE $y' = f(t, y)$ through the limit definition of the derivative. Therefore, the forward Euler method is a *consistent* approximation to the original problem.

It turns out that this Taylor analysis for the consistency of the ODE approximation is also useful to describe the behavior of the errors in the approximation and, ultimately, its convergence. As discussed with a previous example there are two distinct, but related, flavours of error in the ODE approximation.

Definition 9.5 (Global and local errors). The truncation error at the kth step of a method can be thought of in two senses:

- ***Global error*** is the difference between the approximate solution y_k and the true solution of the ODE evaluated at the time t_k, i.e.,
$$e_k = y_k - y_{ex}(t_k)$$
In most cases, we do not know the true solution of the ODE and, hence, the global error must be approximated.
- ***Local error*** is the error made in a *single step* of the numerical method. We denote this quantity as l_k and it is simply the amount by which the new solution of the ODE fails to satisfy the finite difference equation.

◀

Obviously, the global error is of primary interest but it is often impossible to fully know. However, the local error is readily estimated and controlled because it is related to a finite difference approximation. We just need to understand the relationship between these two types of errors. From the Taylor expansion above, we see that the truncation error of a single step for the forward Euler method takes the form

$$y(t_{k+1}) = y(t_k) + hf(t_k, y(t_k)) + \mathcal{O}(h^2) \quad \Rightarrow \quad l_k = \mathcal{O}(h^2).$$

Therefore, the ***local truncation error*** of forward Euler is on the order of $\mathcal{O}(h^2)$. This is the amount of local error made within each time step of the approximation. However, we know that we will take n time steps to move the approximate solution from the initial time t_0 up to the final time of T. This value n is proportional to reciprocal of the time step size $1/h$. So, neglecting the effect of round-off errors, it is reasonable to estimate the amount of global error incurred by the approximation is proportional to the local truncation error divided by h.

Definition 9.6 (Global error for forward Euler). The global truncation error for the forward Euler method is on the order of h, i.e.,

$$e_k = \frac{l_k}{h} = \mathcal{O}(h).$$

Thus, the forward Euler method is convergent and is a *first order* numerical approximation to the initial value problem provided the derivative of the right-hand-side function exists and is bounded. ◀

Remark 9.4. Note that the condition for convergence is similar to the existence and uniqueness requirement of the original ODE problem. ⋈

Remark 9.5. We see that for the forward Euler method *convergence implies consistency*. This is actually a general result for numerical methods to solve ODEs. ⋈

So we have characterized the numerical method to solve an initial value problem in terms of it *accuracy* both locally and globally. In general, we say that the numerical approximation has a global accuracy of p provided it has a local truncation error (or local accuracy) of

$$l_k = \mathcal{O}(h^{p+1}).$$

The motivation for this definition, with the order of global accuracy p being one integer less than the exponent of the local accuracy, is because of the accumulation of local errors over each time step.

Finally, we need to discuss stability of a numerical method to approximate the solution of an initial value problem. This is more subtle because, although it may look like it on its face, the global error is not simply a sum of all the local errors. Keep in mind that as we march forward in time (discretely) the approximate solution is jumping from one solution curve to another. Each of these solution curves has their own trajectory and the local errors might magnify and diverge as the local time steps are taken. Therefore, stability of the numerical method is analogous to the stability of the original ODE.

Let us examine the stability in the context of the forward Euler method applied to the initial value problem

$$y' = \lambda y, \qquad y(0) = y_0$$

where, for full generality, we allow the constant value λ to potentially be a complex number. The exact solution to this initial value problem is

$$y_{ex}(t) = y_0 e^{\lambda t}.$$

A given step of the forward Euler method with a time step size of h produces

$$y_{k+1} = y_k + h\lambda y_k = (1 + h\lambda) y_k.$$

However, we also know that forward Euler in the previous step took the form

$$y_k = y_{k-1} + h\lambda y_{k-1} = (1 + h\lambda) y_{k-1},$$

such that we have

$$y_{k+1} = (1 + h\lambda)y_k = (1 + h\lambda)^2 y_{k-1}.$$

We can apply the definition of the forward Euler method recursively to arrive at an expression for the approximate solution at time t_{k+1} that depends only on the time step h and the initial condition data y_0:

$$y_{k+1} = (1 + h\lambda)^{k+1} y_0.$$

The quantity $1 + h\lambda$ inside the exponent is known as the *growth factor*. If $\mathrm{Re}(\lambda) < 0$, then the exact solution of the ODE decays to zero at t increases. Analogously, the successive numerical values of the approximate solution will decay as t increases provided

$$|1 + h\lambda| < 1.$$

On the other hand, if $|1 + h\lambda| > 1$ then the numerical approximation to the solution ***grows in time*** without bound regardless of the value of the constant λ. This means that the forward Euler method can be *unstable* even when the exact solution is stable. We have no control over the value of λ, but we do have control over the time step size h. Thus, we can define a stability condition for the forward Euler method.

Definition 9.7 (Stability condition on forward Euler). In order for the forward Euler method to be numerically stable for the initial value problem

$$y' = \lambda y, \qquad y(0) = y_0,$$

the step size h must be selected to satisfy the inequality

$$|1 + h\lambda| < 1.$$

This implies that the value $h\lambda$ must lie inside a circle in the complex plane of radius 1 centered at the point $(-1, 0)$. ◀

Example 9.6. Consider the initial value problem

$$y' = -10y, \qquad y(0) = 1, \quad 0 \le t \le 3$$

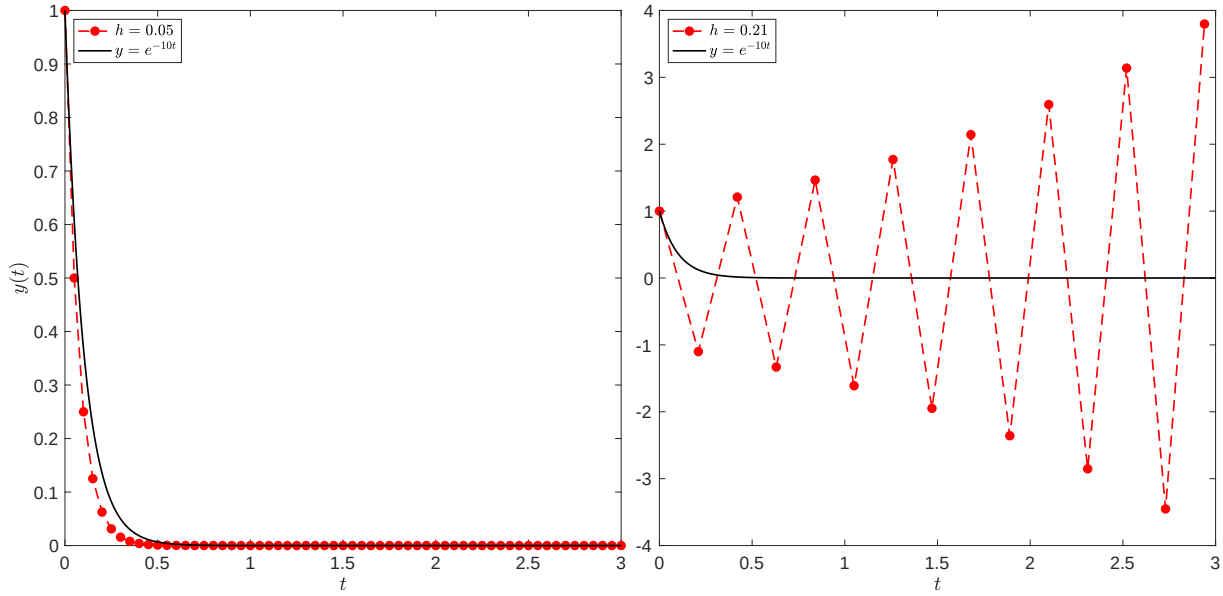
such that the exact solution is given by $y_{ex}(t) = e^{-10t}$. Clearly, the exact solution is stable as it decays for increasing t. If we apply forward Euler to approximate the solution to the initial value problem we see that the stability condition dictates that

$$|1 - 10h| < 1 \qquad \Rightarrow \qquad h < 0.2$$

This restriction on the time step size indicates that we must use at least

$$h = \frac{3 - 0}{n} < 0.2 \qquad \Rightarrow \qquad 15 < n$$

points in the discretization for the numerical method to be stable. Below we illustrate the approximate solution created with forward Euler where we select a stable time step of $h = 0.05$ (on the left) and an unstable time step $h = 0.21$ (on the right).



We see that the stable approximation produces a fairly accurate representation of the exact solution on the interval $[0, 3]$, but the unstable numerical scheme produces oscillations with an amplitude that grows in time without bound, leading to an explosive numerical instability. ♦

This restriction on the time step to guarantee that the numerical approximation is stable is a consequence of using an explicit time integration method. Recall that forward Euler is an explicit method meaning that we only needed information from the current time t_k to advance the approximate solution to the next time level t_{k+1}. This makes the method very easy to implement in practice, but it came with the cost of a limited stability region. What would happen if we relaxed the assumption that we only wanted to use information at the current time?

In particular, what if we used information from the future time at t_{k+1}? This makes the time integration method ***implicit*** but drastically improves the numerical stability. If we return to the initial value problem formulation and integrate in time from t_k to t_{k+1} we had

$$y_{k+1} = y_k + \int_{t_k}^{t_{k+1}} f(s, y(s))\, \mathrm{d}s.$$

If we apply the right rectangle rule as a numerical quadrature then the right-hand-side function has

$$\int_{t_k}^{t_{k+1}} f(s, y(s))\, \mathrm{d}s \approx (t_{k+1} - t_k) f(t_{k+1}, y(t_{k+1}) = h f(t_{k+1}, y_{k+1}).$$

This is the simplest example of an implicit time integration method for initial value problems

Definition 9.8 (Backward Euler method). The backward Euler method is an implicit time integration technique to solve an initial value problem that takes the form

$$y_{k+1} = y_k + h f(t_{k+1}, y_{k+1}).$$

It have a local truncation error of $l_k = \mathcal{O}(h^2)$, an accuracy of $p = 1$, and is *unconditionally stable*. ◀

The truncation error and convergence analysis of the backward Euler method is very similar to the forward Euler analysis. What is most interesting is the stability analysis. What do we mean by *unconditional*? To define this we return to the initial value problem

$$y' = \lambda y, \qquad y(0) = y_0.$$

Applying the backward Euler method we find that

$$y_{k+1} = y_k + h\lambda y_{k+1}, \qquad \Rightarrow \qquad (1 - h\lambda)y_{k+1} = y_k$$

So, applying the same recursive argument from the forward Euler analysis, now for the backward Euler method we find that

$$y_{k+1} = \left(\frac{1}{1 - h\lambda}\right)^{k+1} y_0.$$

That is, the growth factor for the approximate solution has changed for the backward Euler method. To guarantee stability it must be true that

$$\left|\frac{1}{1 - h\lambda}\right| < 1.$$

However, this is true **for any value** of the time step $h > 0$ provided $\mathrm{Re}(\lambda) < 0$. That is, if the original ODE is stable then so is the backward Euler approximation. This is what we mean by unconditional stability. The stability region for the backward Euler method include the entire left half of the complex plane.

Remark 9.6. Implicit methods are a little bit strange conceptually. Basically, the backward Euler method states that we must evaluate the right-hand-side f with the argument y_{k+1} before we know its value. Thus, we are implicit. This statement means that a value for y_{k+1} that satisfies the preceding equation must be determined, and if f is a nonlinear function of y then an iterative solution method, like Newton-Raphson, must be used. ⋈

Example 9.7. Use a step size $h = 0.5$ to setup the backward Euler method for the nonlinear initial value problem

$$y' = -y^3, \quad y(0) = 1.$$

From the backward Euler formula we have for the first step of the implicit method that

$$y_1 = y_0 + hf(t_1, y_1) = 1 - 0.5y_1^3$$

to get the solution at the next step. This nonlinear equation for y_1 is already in a form that can be solved with a fixed point iteration (alternatively we could use Newton-Raphson). In either case, we require an initial guess to kickstart the iteration to find y_1. We could use the previous solution $y_0 = 1$ as this initial guess. Or, a better initial guess, comes from applying the forward Euler method to have $y_1^{ini} = y_0 - 0.5y_0^3 = 0.5$. The iterative method eventually converges to a value $y_1 \approx 0.7709$. Then we have to form a new nonlinear problem and starting guess to advance again to y_2. ⧫

Remark 9.7. We see that we do not get this enhanced stability for free. The computational work has to go somewhere. For the explicit Euler method we have to do more computational work because of a necessarily small time step h for a given problem. Whereas, the implicit Euler method places no restriction on the time step size, ***but*** for each of these time steps we have to solve a nonlinear equation iteratively. ⋈

These forward Euler and backward Euler methods are both first order accurate. This limits their usefulness in practice. However, we can combine them in a certain fashion to create a more accurate method. Basically if we apply the trapezoid quadrature rule to the integral in time we have

$$\int_{t_k}^{t_{k+1}} f(s, y(s))\,\mathrm{d}s \approx \frac{h}{2}(f(t_k, y_k) + f(t_{k+1}, y_{k+1})).$$

To produce the trapezoid time integration rule.

Definition 9.9 (Trapezoid method). For an initial value problem $y' = f(t, y)$ with accompanying initial condition $y(t_0) = y_0$ and uniform time step $h = \frac{T-t_0}{n}$ the *trapezoid method* is given by

$$y_{k+1} = y_k + \frac{h}{2}(f(t_k, y_k) + f(t_{k+1}, y_{k+1})).$$

The local truncation error is $l_k = \mathcal{O}(h^3)$ such that the method is second order accurate ($p = 2$) in its convergence rate. Further, the trapezoid method is implicit and remains unconditionally stable. ◀

Remark 9.8. To see the unconditional stbility of the trapezoid method we again apply the integration technique to the prototype initial value problem $y' = \lambda y$ with $y(0) = y_0$ which yields

$$y_{k+1} = y_k + \frac{h\lambda}{2}(y_k + y_{k+1}),$$

which implies, through a similar recursion argument we have made twice before, that

$$y_{k+1} = \left(\frac{1 + \frac{h\lambda}{2}}{1 - \frac{h\lambda}{2}}\right)^{k+1} y_0.$$

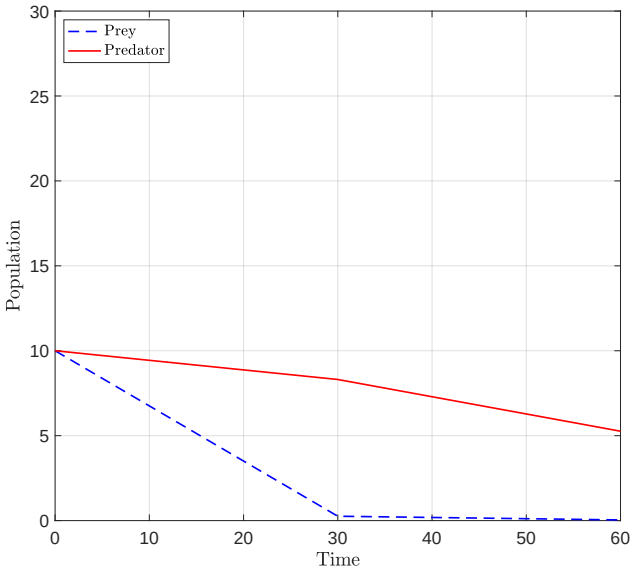
Thus, the method is stable provided

$$\left|\frac{1 + \frac{h\lambda}{2}}{1 - \frac{h\lambda}{2}}\right| < 1,$$

which holds for any $h > 0$. ⋈

Implicit time integration methods, like backward Euler, appear to not place no restriction on the time step. However, this is not technically true. There is still a *physical* time scale that the approximate solution should capture other the numerical solution will be radically different from the physical process we are trying to model.

Example 9.8. Returning to the predator/prey model governed by the Lotka-Volterra equations from the beginning of the lecture where the solution was in the time interval $t \in [0, 60]$. The backward Euler method fully allows us to select a time step size $h = 30$ such that the ODE is solved in two time intervals. This is stable, but the numerical solution produces absolute nonsense when it comes to the physical model. We show this computed solution on the left We see that the excessively large time step has "walked over" all the important physical interactions between the predator/prey in the mathematical model. This highlights, again, the blurring of the line between mathematics and the other sciences. The numerical scheme gave us a solution and it was stable, but we need to interpret the results and ask the question, does this make sense? The numerical solver and computer did exactly what we told it too but the solution it produced is actually garbage. ♦



Note Set 10: Ordinary differential equations: Runge-Kutta

We have seen that implicit time integration methods can drastically improve the numerical stability of an approximation to an initial value problem. However, this came with the cost of increased numerical complexity of the numerical solver as well as increased complexity actually implementing the method. In contrast, an explicit time integration method like forward Euler was very straightforward to implement, but suffered from a (possibly severe) restriction on h, the time step size. In either case, we noted that the accuracy of these simple Euler-type methods was often insufficient for practical problems.

In this note set we will discuss a strategy to improve explicit time integration methods such that we keep their relative simplicity, but improve their accuracy and stability. Just as we saw with finite difference approximations or composite quadrature rules to build a better approximation we will need to take *more information* into account as we advance the solution from one point in time to the next time.

10.1 Runge-Kutta methods

The topic at hand are the Runge-Kutta class of time integration methods. These methods are very popular in practice and offer a nice balance between accuracy and computational work. They remain similar in structure to the forward Euler method, but they *boost* accuracy past first order by evaluating the right-hand-side function $f(t, y)$ more than once for each time step.

To begin we start from the general form of an initial value problem

$$y'(t) = f(t, y(t)), \qquad y(t_0) = y_0,$$

on a time interval $t \in [t_0, T]$. Also, assume that we have divided the time interval into a discrete set of $n+1$ uniform points with a fixed time step size such that

$$t_k = t_0 + kh, \qquad k = 0, \ldots, n \qquad h = \frac{T - t_0}{n}.$$

Next, we integrate the initial value problem on a single time interval of size h, that is from t_k up to t_{k+1}. The Fundamental Theorem of Calculus immediately eliminates the time derivative term and we are left with a time integral term on the right-hand-side

$$y_{k+1} = y_k + \int_{t_k}^{t_{k+1}} f(s, y(s))\,\mathrm{d}t.$$

From here we previously applied simple quadrature rules to this time integral such as:

- The left rectangle rule to create *forward Euler*, which was an explicit method.
- The right rectangle rule to create *backward Euler*, which is implicit.
- The *trapezoid rule* to create an implicit time integration technique of the same name.

But there are many other quadratures available. For example, we have seen that Simpson's rule is often a good choice for an accurate approximation of an integral. However, the goal here is to develop an ***explicit*** time integration technique. We see that using more sophisticated quadrature rules will inherently be implicit because it will require the evaluation of the right-hand-side at points in the "future," which is information that we do not have. To circumvent the need for future information in the approximation of the time integral requires some *bootstrapping*.

To develop a discrete time integration method that relies only on the information we have at the current time t_k we return to the Taylor expansion

$$\begin{aligned} y_{k+1} = y(t_k + h) &= y_k + hy'(t_k) + \frac{h^2}{2}y''(t_k) + \mathcal{O}(h^3) \\ &= y_k + hf(t_k, y_k) + \frac{h^2}{2}y''(t_k) + \mathcal{O}(h^3). \end{aligned}$$

Above, as before, we substituted the known relationship between the first derivative of $y(t)$ and the right-hand-side function $f(t, y)$. If we truncate the approximation at this term we end up with forward Euler. But if we take higher order terms from the Taylor expansion into account we can develop a more accurate approximation, as we have see before. Now, we may not know anything about $y''(t)$ ***but*** we can still eliminate one of the time derivatives by replacing the right-hand-side function $f(t, y)$ to have

$$y''(t) = (f(t, y(t)))',$$

where we explicitly write out the variable dependencies. The right-hand-side f is a function of *two variables* t and y, and y is also a function of t. Therefore, to compute the first time derivative of the right hand side we must employ the (multi-dimensional) chain rule to find

$$y''(t) = (f(t, y(t)))' = f_t(t, y) + f_y(t, y)y'(t) = f_t + ff_y,$$

where the subscript denotes taking a partial derivative. This doesn't appear to help a great deal because we also know very little about these partial derivatives of the right-hand-side function. However, we **can** approximate them by again performing another Taylor expansion only now with respect to the two variables t and y

$$f(t_k + h, y_k + hf_k) = f(t_k, y_k) + hf_t(t_k, y_k) + hf(t_k, y_k)f_y(t_k, y_k) + \mathcal{O}(h^2),$$

let's go ahead and stop there because this multidimensional Taylor expansion gets quite ugly. We see that it is possible to approximate the unknown partial derivative terms above

$$y''(t_k) = f_t + ff_y = \frac{f(t_k + h, y_k + hf_k) - f(t_k, y_k)}{h} + \mathcal{O}(h) \approx \frac{f(t_k + h, y_k + hf_k) - f(t_k, y_k)}{h}.$$

Great! Now we have an approximation that can be used in the original Taylor expansion for y_{k+1}. Substitution gives us

$$y_{k+1} \approx y_k + hf(t_k, y_k) + \frac{h^2}{2}\left(\frac{f(t_k + h, y_k + hf_k) - f(t_k, y_k)}{h}\right) = y_k + \frac{h}{2}\left[f(t_k, y_k) + f(t_k + h, y_k + hf_k)\right].$$

Definition 10.1 (Heun's method). A second order, consistent approximation to the initial value problem is given by *Heun's method*. The right-hand-side is evaluated at two stages

$$k_1 = f(t_k, y_k), \qquad k_2 = f(t_k + h, y_k + hf_k),$$

and then the update to the solution takes the form

$$y_{k+1} = y_k + \frac{h}{2}\left[k_1 + k_2\right].$$

This method is fully explicit, has a local truncation error of $\mathcal{O}(h^3)$, and a global error of $\mathcal{O}(h^2)$. ◀

Remark 10.1. Heun's method is also referred to as two-stage, second order *Runge-Kutta*. ⋈

Remark 10.2. Another point-of-view for Heun's method is that we took the implicit trapezoid rule (also referred to as the Crank-Nicolson method)

$$y_{k+1} = y_k + \frac{h}{2}\left[f(t_k, y_k) + f(t_{k+1}, y_{k+1})\right],$$

and approximated the unknown implicit piece with forward Euler

$$k_2 = y_k + hf(t_k, y_k) \approx y_{k+1}.$$

This makes the scheme a fully explicit method. ⋈

Remark 10.3. This tactic of using a lower order approach (like forward Euler) to approximate unknown information in the future and combine it into the scheme is called the *predictor-corrector* approach. ⋈

EXAMPLE 10.1. To illustrate Heun's method lets compute two steps of the scheme for the nonlinear initial value problem

$$y' = -2ty^2, \qquad y(0) = 1,$$

with the time step $h = 0.25$. Taking the first step from $t_0 = 0$ to $t_1 = 0.25$ we first formulate the stages

$$k_1 = f(t_0, y_0) = -2(0)(1)^2 = 0, \quad k_2 = f(t_0 + h, y_0 + hk_1) = -2(0.25)(1)^2 = -0.5,$$

so that the solution update for Heun's method gives

$$y_1 = y_0 + \frac{h}{2}[k_1 + k_2] = 1 + \frac{0.25}{2}[0 - 0.5] = 0.9375.$$

Continuing to the next step from $t_1 = 0.25$ to $t_2 = 0.5$, we first compute the stages

$$\begin{aligned} k_1 &= f(t_1, y_1) = -2(0.25)(0.9375)^2 = -0.439453125, \\ k_2 &= f(t_1 + h, y_1 + hk_1) = -2(0.5)(0.9375 + (0.25)k_1)^2 = -0.684982538223267, \end{aligned}$$

so that

$$y_2 = y_1 + \frac{h}{2}[k_1 + k_2] = 0.9375 + (0.125)(k_1 + k_2) = 0.796945542097092.$$

For comparison the exact solution for this problem is $y(t) = (1 + t^2)^{-1}$ so the true solution at the final time is $y(0.5) = 0.8$. ♦

The derivation of Heun's method was motivated by the Taylor expansion (a couple of times actually). Thus, it is clear that its local truncation error is designed to be $\mathcal{O}(h^3)$ and, by a similar argument of making this local error once in each time step, we see that the convergence order will be on the order of $\mathcal{O}(h^2)$. So, the method converges, which means that we have consistency. For completeness, we check the consistency by manipulating Heun's method and take the limit as $h \to 0$:

$$\frac{y_{k+1} - y_k}{h} = \frac{1}{2}\left[f(t_k, y_k) + f(t_k + h, y_k + hf_k))\right] \qquad \stackrel{h\to 0}{\Longrightarrow} \qquad y'(t) = \frac{1}{2}[f(t, y) + f(t + 0, y + 0)] = f(t, y).$$

What about the numerical stability of Heun's method? To investigate the stability we must determine the growth factor for the characteristic initial value problem

$$y' = \lambda y, \qquad y(t_0) = y_0,$$

for some (possibly complex) constant λ. To develop the growth factor we examine Heun's method to move the solution a single time step. The stages evaluate to be

$$k_1 = \lambda y_k, \quad \text{and} \quad k_2 = f(t_k + h, y_k + hf_k) = \lambda(y_k + h\lambda y_k) = \lambda y_k + h\lambda^2 y_k.$$

Then the update to the solution is

$$y_{k+1} = y_k + \frac{h}{2}[k_1 + k_2] = y_k + \frac{h}{2}[\lambda y_k + \lambda y_k + h\lambda^2 y_k] = \left(1 + h\lambda + \frac{h^2\lambda^2}{2}\right) y_k.$$

Applying this argument recursively we can bound the approximate solution at the $(k+1)^{\text{st}}$ time step in terms of the initial condition

$$y_{k+1} = \left(1 + h\lambda + \frac{h^2\lambda^2}{2}\right)^{k+1} y_0.$$

Therefore, the growth factor for Heun's method is

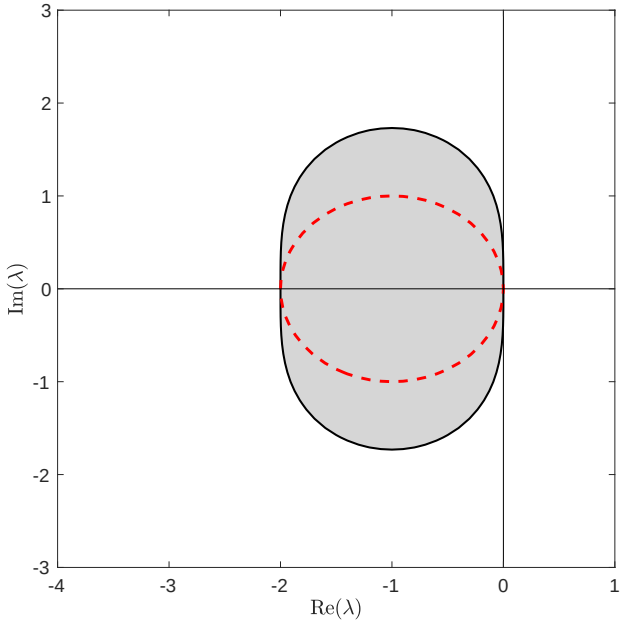
$$\text{growth factor} = 1 + h\lambda + \frac{h^2\lambda^2}{2},$$

and the stability condition for this explicit time integration scheme is

$$\left|1 + h\lambda + \frac{h^2\lambda^2}{2}\right| < 1,$$

under the assumption that $\text{Re}(\lambda) < 0$.

This stability region is a shown on the left and it is a pill shaped region in the complex plane.



Remark 10.4. The stability region for Heun's method includes the same portion of the real axis as the forward Euler method. In other words, this extra stage in the approximation improved the accuracy of the time integrator, but stability is essentially the same as the forward Euler method. ⋈

We can generalize this idea of taking more evaluations of the right-hand-side function within the interval of a single time step from time t_k up to t_{k+1}. The strategy is to improve the local truncation error that, in turn, reduces the global error of the method. The idea is to match terms of higher and higher order within the multidimensional Taylor expansion to develop an **explicit form** for these different right-hand-side evaluations. The amount of algebra to do this grows quite rapidly in terms of size and complexity but the basic idea is rooted in the process of developing the forward Euler method and Heun's method.

The idea of taking these intermediate *stages* where we evaluate the right-hand-side with known information and then combining them as a type of "average" to update the solution from one time step to the next is the basic framework of an ***explicit Runge-Kutta method***. We see that the additional information comes in the form of right-hand-side evaluations at different *stages* to which coefficients and free parameters are matched by the different terms in the truncated (multidimensional) Taylor series.

Foregoing this complicated algebraic manipulation we will present the most famous instance of an explicit Runge-Kutta method. It requires four stage evaluations to advance the solution from one time level to the next and is fourth order accurate globally.

DEFINITION 10.2 (4-STAGE, 4TH ORDER RUNGE-KUTTA). A fourth order, consistent approximation to the initial value problem is given by *classic Runge-Kutta 44 method*. The right-hand-side is evaluated at four stages from the current time t_k:

$$\begin{aligned} k_1 &= f(t_k, y_k), \\ k_2 &= f\left(t_k + \frac{h}{2}, y_k + \frac{h}{2}k_1\right), \\ k_3 &= f\left(t_k + \frac{h}{2}, y_k + \frac{h}{2}k_2\right), \\ k_4 &= f\left(t_k + h, y_k + hk_3\right), \end{aligned}$$

and then the update to the solution takes the form

$$y_{k+1} = y_k + \frac{h}{6}\left(k_1 + 2k_2 + 2k_3 + k_4\right).$$

This method is fully explicit, has a local truncation error of $\mathcal{O}(h^5)$, and a global error of $\mathcal{O}(h^4)$. ◀

Remark 10.5. The Runge-Kutta 44 method is analogous to applying Simpson's rule to the integral in time in the integral form of the initial value problem

$$y_{k+1} = y_k + \int_{t_k}^{t_{k+1}} f(s, y(s))\,\mathrm{d}t,$$

from the beginning of this note set. ⋈

Remark 10.6. Runge-Kutta methods have a number of virtues. Most notably is their ability to obtain high-order accuracy and advance the solution to the next time t_{k+1}while only requiring knowledge of the solution at the current time t_k. This makes explicit Runge-Kutta methods particularly easy to program because, just like forward Euler, the time integration technique does not need to "remember" any part of its solution history to advanced to the next time step. It also means we could *change* the time step size h between steps if desired. ⋈

The Runge-Kutta 44 is an explicit time integration method, so it is still subject to a time step restriction that ensures that its numerical stable. So, we must examine its growth factor and determine the region in which the Runge-Kutta 44 method is stable.

Applying the scheme to the characteristic initial value problem and after many manipulations we obtain the discrete solution at the time t_{k+1} in terms of its initial condition

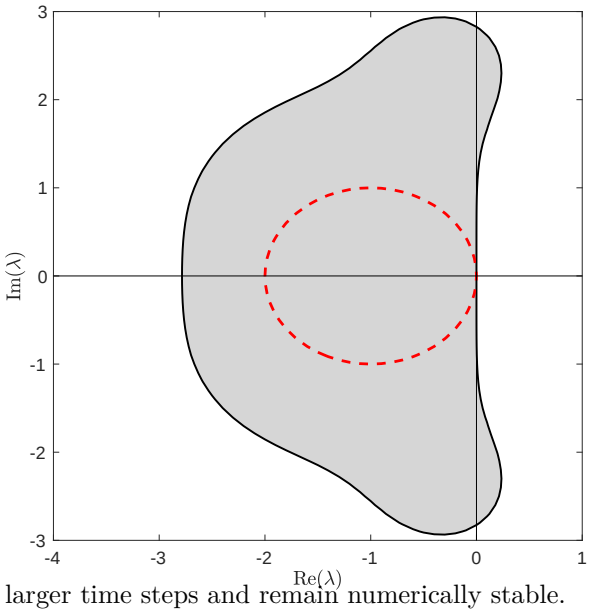
$$y_{k+1} = \left(1 + h\lambda + \frac{(h\lambda)^2}{2} + \frac{(h\lambda)^3}{6} + \frac{(h\lambda)^4}{24}\right)^{k+1} y_0.$$

Therefore, the growth factor for the Runge-Kutta 44 method is given by

$$\text{growth factor} = 1 + h\lambda + \frac{(h\lambda)^2}{2} + \frac{(h\lambda)^3}{6} + \frac{(h\lambda)^4}{24}.$$

The corresponding stability condition is

$$\left|1 + h\lambda + \frac{(h\lambda)^2}{2} + \frac{(h\lambda)^3}{6} + \frac{(h\lambda)^4}{24}\right| < 1.$$



This stability region is shaded gray on the right and is plotted against the stability region of forward Euler (dotted line). We see that Runge-Kutta 44 is not only significantly more accurate, but it can take much larger time steps and remain numerically stable.

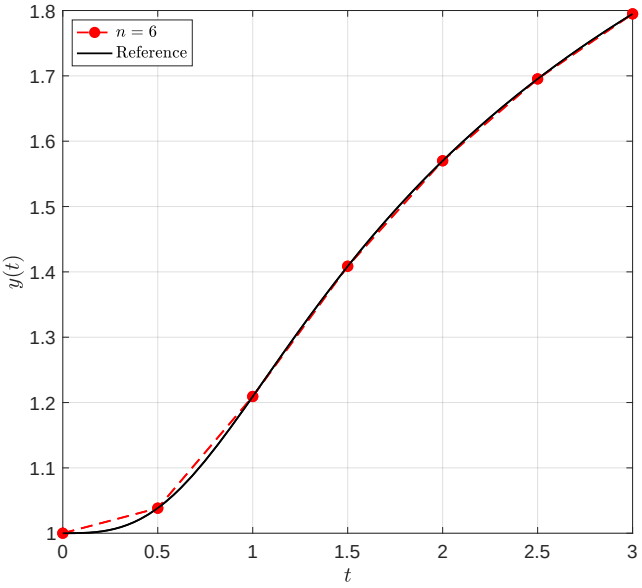
EXAMPLE 10.2. As an example of the high-order accuracy and stability of the Runge-Kutta 44 method we consider the nonlinear initial value problem

$$y' = \frac{t^2}{(1+t^3)y}, \quad y(0) = 1, \qquad 0 \le t \le 3.$$

An exact solution for this initial value problem is

$$y_{ex}(t) = \sqrt{1 + \frac{2}{3}\ln(1+t^3)}.$$

We verify the fourth order accuracy of the Runge-Kutta 44 method in the Table below by examining the error between the approximation and the exact solution at the final time. We see that the ratio of the errors is around a factor of 16 as expected when we double the number of points in time. We also compute the experimental order of convergence (EOC) and see that it is approximately 4, which matches the theoretical fourth order nature of Runge-Kutta 44. We also see the stability advantage of the Runge-Kutta 44 method



$$\texttt{EOC} = \frac{1}{\ln(2)} \ln\left(\frac{\text{error for } n \text{ points}}{\text{error for } 2n \text{ points}}\right)$$

n	$\lvert y_{ex}(3) - y_T \rvert$	Ratio	EOC
6	5.7179e-05	—	—
12	3.0253e-06	18.90	4.24
24	1.6501e-07	18.33	4.20
48	9.3525e-09	17.64	4.14
96	5.5153e-10	16.95	4.08
192	3.3396e-11	16.52	4.06

because we obtain a quite accurate (about four accurate digits) approximate solution for the method even with a very coarse discretization in time with $n = 6$ points. The forward Euler (and Heun's method) are both unstable for this very coarse resolution. We also plot the Runge-Kutta 44 solution on this very coarse resolution against the exact solution and see that it accurately tracks the solution in time even for the rather large step size. ♦

10.2 Outlook to numerical partial differential equations

So where do we go from here? We have covered the fundamental tools of numerical mathematics for solving linear systems, approximating derivatives and integrals, as well as discretely modeling a solution as it evolve over time. These are all necessary components to solve a broad range of problems using a computer. Numerical analysis is powerful tool that can model the solution for physical processes when experiments are too expensive, too dangerous, or impossible to perform.

It may seem like the tools from this course are just the basics and using a computer to solve these more complicated physical problems is in the far-off future. But you *know more than you think you know!* We (and many others) can immediately solve these physical problems with the tools to solve linear systems, approximate derivatives or integrate ODEs in time.

It turns out that many physical phenomena of interest can be modeled mathematically with partial differential equations (PDEs). A PDE is an equation that involves the rate of change of unknown multivariable function(s) with respect to several continuous variables. To pose the PDE properly also requires information about the initial configuration of the physical system (initial conditions) as well as the possible prescription of how the physics should behave at any boundary (boundary conditions), e.g., the surface of an airfoil.

Broadly, PDEs fall are categorized into three classes:

- *Elliptic* PDEs have no time-like variable. They can be used to model potential functions and static processes. A physical example is gravity. They also come up in optimization problems, for example, maximizing the surface area of an object.
- *Parabolic* PDEs have a time like variable and are characterizaed by diffusion processes. For example, how heat diffuses and spreads throughout a spatial domain.
- *Hyperbolic* PDEs have a time like variable and are characterized by wave-type solutions that propagate information from their initial conditions throughout space and time.

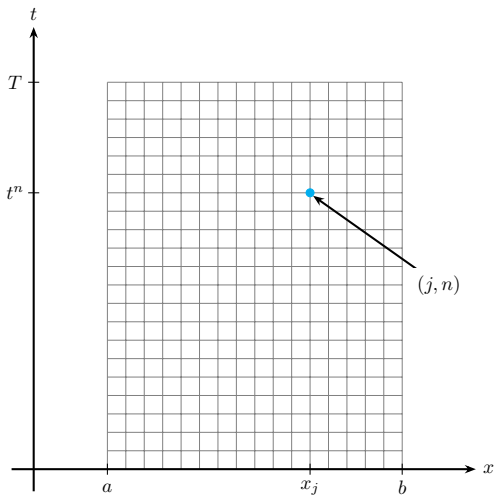
It turns out we have the required knowledge from this course to solve problems involving PDEs as it basically involves approximating derivatives in space and time using our numerical techniques. To illustrate this and demonstrate the blending of numerical mathematics and physics we will consider a simple example of a hyperbolic PDE in one spatial dimension. For this we examine the linear advection equation

$$u_t + au_x = 0, \quad u(x, t = 0) = u_0(x),$$

where the domain is periodic and we have some initial condition. Without loss of generality we assume the constant $a > 0$. This problem has a unique solution of the form

$$u(x, t) = u_0(x - at).$$

This general solution tells us two important things. First, the solution at any time t^* is the original initial condition shifted to the right, as $a > 0$, by the amount at^*. In other words, the solution at a point (x, t) depends only on the value of $\xi = x - at$. The lines in the (x, t) plane on which ξ is a constant are referred to as *characteristics*. The parameter a is the speed of propagation of the solution along each characteristic line. Thus, the solution of the advection equation can be regarded as a wave that propagates with speed a without changing its shape. From the physics of the model, this means that information for a hyperbolic problem has a *preferred direction* of travel. A second important observation is that it appears the solution of the advection equation only makes sense if the function u is differentiable in space and time. However, the general solution $u_0(x - at)$ prescribes **no differentiability constraint** on the initial condition u_0. In general, this means that *discontinuous solutions are allowed* for hyperbolic problems.



Foregoing further analysis of the PDE let's try to discretize this problem and model wave propagation with a computer. We see that the advection equation contains a spatial derivative and a time derivative. We have learned that we can approximate spatial derivatives with finite difference approximations on a discrete set of points where we denote this spatial grid with a subscript

$$u(x, t) \approx u(x_j, t) = u_j(t).$$

We denote the spatial grid spacing by Δx (analogous to the h variable we used previously). We learned that a central finite difference is second order accurate, so it seems like a good idea to approximate the spatial derivative in the linear advection equation with this accurate discrete derivative. Now the PDE becomes a set of ODEs

$$(u_j)_t + \frac{u_{j+1} - u_{j-1}}{\Delta x} = 0,$$

which is a semi-discrete form of the equation. To complete the discretization we need to integrate the ODEs in time. We discretize with a uniform set of points in time as well, with time step spacing Δt, and we denote this temporal grid with superscripts. Furthermore, capital letters denote a fully discrete quantity, e.g., $u_j(t) \approx u_j(t^n) = U_j^n$. This yields the space-time grid of points as shown above.

We apply forward Euler to create a fully discrete set of equations to determine how each of the spatial points evolves in time. Putting everything together, we have a discrete version of the advection equation

$$\frac{U_j^{n+1} - U_j^n}{\Delta t} + a\frac{U_{j+1}^n - U_{j-1}^n}{2\,\Delta x} = 0,$$

for $i = 0, ..., N$ points in space and $n = 1, ..., M$ points in time with the initial values

$$U_j^0 = U_0(x_j), \quad i = 0, ..., N.$$

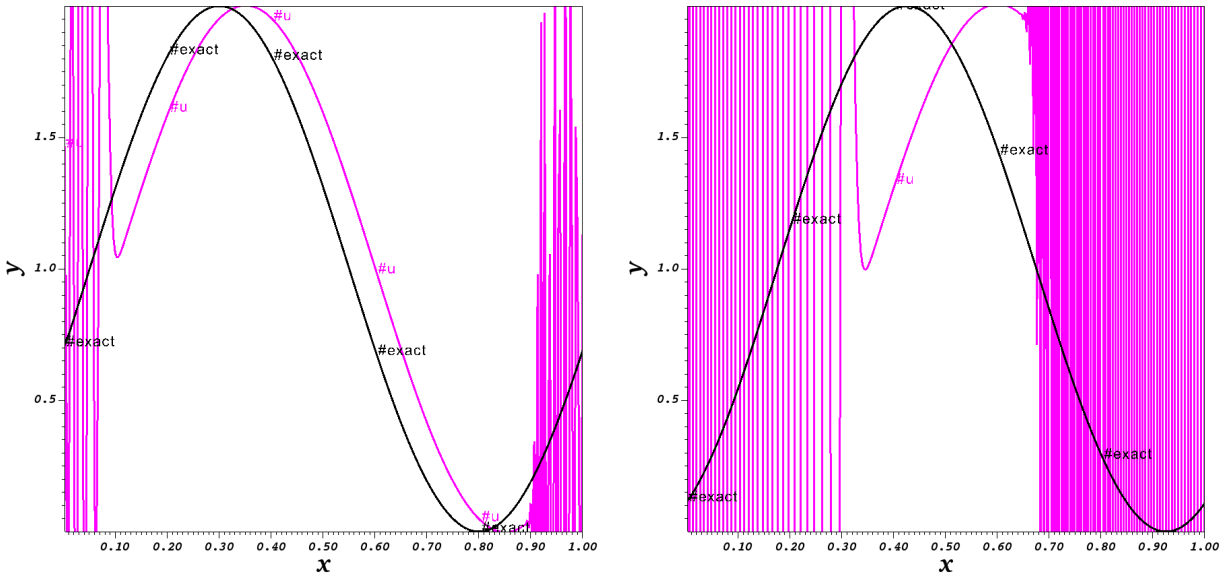
and the periodic boundary conditions that $U_0^n = U_N^n$. Rearranging this discrete equation we have

$$U_j^{n+1} = U_j^n - \frac{a\Delta t}{2\Delta x}\,(U_{j+1}^n - U_{j-1}^n).$$

This is now an explicit time integration problem so lets try it out for the initial condition

$$u_0(x) = \sin(2\pi x) + 1, \quad x \in [0, 1].$$

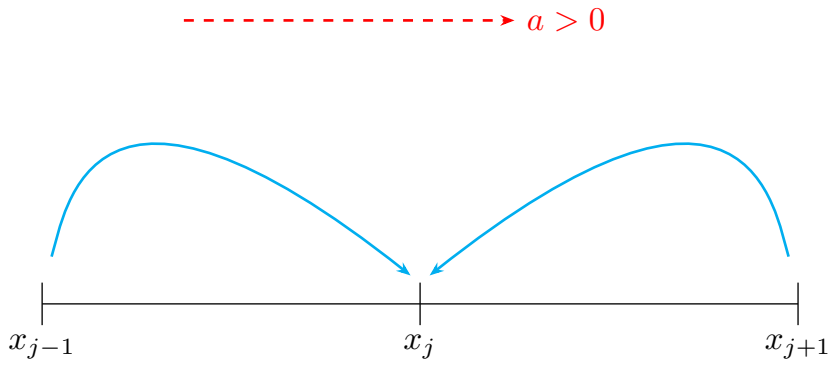
Let's solve this numerical approximation with the wave speed $a = 1$ and $\Delta t = \Delta x = 0.05$. Below we show the evolution of the computed solution at the times $T = 0.1$ and $T = 0.5$ in the plots.



Clearly something went wrong! The numerical solution is unstable and is not a wave.

Remark 10.7. It turns out that using forward Euler in time and a central spatial approximation ***will never work!*** ⋈

In fact, the problem lies in the selection of the approximation we selected for the spatial derivative. The central difference pulls information from the left and the right of each grid node as shown in the figure below.



However, this discretization technique ***goes against the physics of the problem!*** We know that the advection equation transports information with a preferred direction, but the central difference pulls information (and physics) from the wrong direction. It is possible to analyze the finite difference scheme and demonstrate that the amount of "energy" in the discrete solution at the time t^n.

$$E^n = \sum_{i=0}^{N} (U_j^n)^2 \Delta x,$$

evolves according to the formula

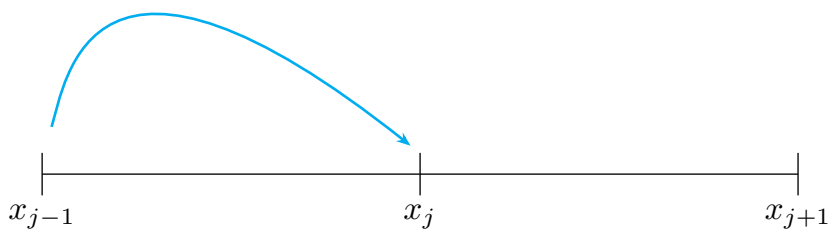
$$E^{n+1} = E^n + \frac{\Delta x}{2} \sum_{i=0}^{N} (U_j^{n+1} - U_j^n)^2.$$

Thus, the discrete energy E^n grows in every time step, independent of the choice of Δx, Δt. This leads to the blowup type behavior we have observed.

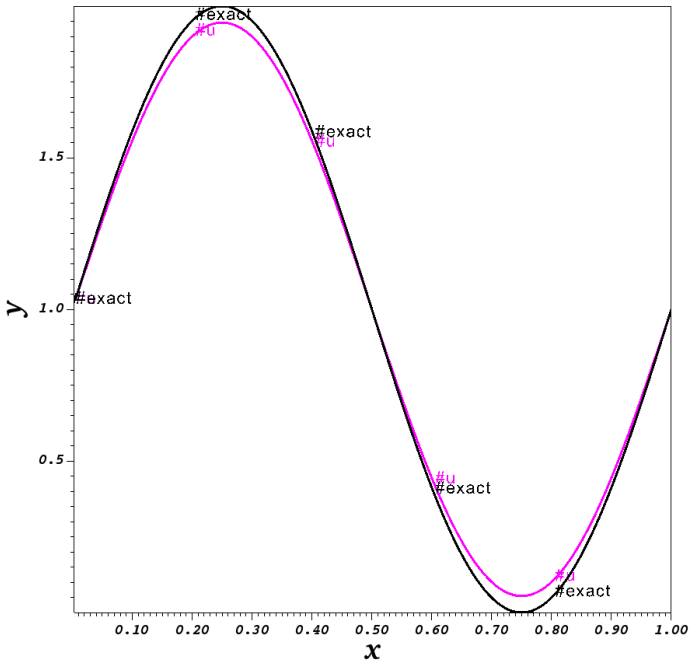
To fix this issue we change the spatial discretization to use an upwind finite difference method that is biased toward where the wave information is coming from, i.e.,

$$\frac{U_j^{n+1} - U_j^n}{\Delta t} + a \frac{U_j^n - U_{j-1}^n}{\Delta x} = 0.$$

as shown in this schematic



If we again consider the initial condition $u_0(x) = \sin(2\pi x) + 1$ we show the approximate solution against the exact solution in the plots below at the final time $T = 1$ such that the wave has traveled one period through the domain.



Now the numerical solution looks like we expect. So, what happened? Why did this work? Heuristically, the math now matches the physics and the approximation appears stable. But how does this manifest in the mathematical formula(s)? We can reformulate the upwind scheme to have the form

$$\frac{U_j^{n+1}-U_j^n}{\Delta t}+a\frac{U_j^n-U_{j-1}^n}{\Delta x}=0 \quad \Rightarrow \quad \frac{U_j^{n+1}-U_j^n}{\Delta t}+a\frac{U_{j+1}^n-U_{j-1}^n}{2\,\Delta x}=\frac{a\Delta x}{2}\frac{U_{j+1}^n-2\,U_i^n+U_{j-1}^n}{\Delta x^2}.$$

The additional term on the right-hand-side is an approximation to the second derivative

$$\frac{a\Delta x}{2}\frac{U_{j+1}^n-2\,U_j^n+U_{j-1}^n}{\Delta x^2}\approx\frac{a\Delta x}{2}u_{xx}$$

which is actually a diffusion term! Thus, the upwinding of the finite difference approximation introduces ***artificial diffusion*** that has a stabilizing effect on the approximate solution. We also see the artificial dissipation in the plot of the solution above after one period of propagation. There has been some information lost from the peak of wave. This numerical diffusion (sometimes referred to as dissipation) plays a crucial role for the stability of numerical methods applied to hyperbolic PDEs.

What about the solution energy? For the upwind finite difference approximation with forward Euler time integration the discrete solution energy now evolves according to

$$E^{n+1}=E^n+\left(\frac{a^2\Delta t^2}{\Delta x}-a\Delta t\right)\sum_{i=0}^{N}(U_j^n-U_{j-1}^n)^2.$$

Now we have ***control*** over the discrete solution energy through the spatial and temporal resolution. That is, the discrete energy will decay provided

$$\frac{a^2\Delta t^2}{\Delta x}-a\Delta t\leq 0 \qquad \Rightarrow \qquad \frac{a\Delta t}{\Delta x}\leq 1.$$

Remark 10.8. This is known as the Courant-Friedrichs-Lewy (CFL) condition. ⋈

This condition tells us how we have to discretize space and time, i.e., selecting the values for Δt and Δx in order for the upwind approximation to be stable.

For numerical methods applied to hyperbolic PDES, numerical dissipation is **good**, as it enables stability but numerical dissipation is **bad**, as it introduced numerical errors (damping) of the solution. An optimal amount of numerical dissipation follows the adage "as small as possible, but sufficient to give stability!" In practice, this is always easier said than done.

This illustrates an important conclusion about applying numerical mathematics and algorithms for PDEs, we need to understand the physics of the problem and critically analyze the resulting the computer and algorithms produce for us. Do they make sense in context? Also, interesting things can happen when we discretize a physical model. As we saw above for the advection equation, the physical model contained no dissipation, but the upwind numerical method ***does*** contain (numerical) dissipation that is necessary for the scheme to run and produce a solution. Understanding this interplay between computational mathematics and the other sciences is a crucial and very interesting task.

Appendix A: Note sets for removed topics

This appendix contains course notes that concern topics which are no longer covered in the curriculum. However, the notes may be useful in your future studies, so I include them here for completeness.

A.1 Solving linear systems: Descent methods

We expand our knowledge of iterative methods to more sophisticated techniques that will converge to the solution faster. Our focus remains on tactics to find the solution of an $n \times n$ linear system $\mathbf{Ax} = \mathbf{b}$, but we will reinterpret the problem into another context. Throughout this discussion we will assume that the matrix $\mathbf{A}$ is *symmetric positive definite*. As a consequence we immediately know that the matrix is nonsingular. At first glance this appears to be a very restrictive assumption, however we can recast any nondegenerate linear system into such a problem because

$$\mathbf{Ax} = \mathbf{b} \qquad \Leftrightarrow \qquad \mathbf{A}^T\mathbf{Ax} = \mathbf{A}^T\mathbf{b}.$$

Here, the matrix $\mathbf{A}^T\mathbf{A}$ is symmetric positive definite by construction.

Remark A.1. The formulation on the right hand side is referred to as the **normal equations**. This manipulation is theoretically valid, but it is less clear whether or not such it will be practical. We are guaranteed that the matrix $\mathbf{A}^T\mathbf{A}$ is symmetric positive definite, but matrix multiplication is computationally expensive and could destroy any structure the matrix might have (like sparsity). Also, the conditioning of the normal equations is

$$\kappa_2(\mathbf{A}^T\mathbf{A}) = \kappa_2(\mathbf{A})^2.$$

Thus, if the original system is ill-conditioned solving the normal equations is even more so. ⋈

Once we assume that the matrix $\mathbf{A}$ is symmetric positive definite we can recast the solution of the linear system into a *minimization* problem. For this we actually convert the linear problem into a quadratic problem. That is, $\mathbf{Ax} = \mathbf{b}$ is a condition on the vector $\mathbf{x}$ that will minimize the value of the quadratic form

$$f(\mathbf{x}) = \frac{1}{2}\mathbf{x}^T\mathbf{Ax} - \mathbf{x}^T\mathbf{b}.$$

We see that when we take the gradient of this quadratic form that

$$\nabla f(\mathbf{x}) = \frac{1}{2}(\mathbf{A} + \mathbf{A}^T)\mathbf{x} - \mathbf{b} = \mathbf{Ax} - \mathbf{b},$$

because $\mathbf{A}$ is symmetric. Further, due to the positive definiteness of $\mathbf{A}$ the function $f(\mathbf{x})$ is strictly convex and attains a *unique minimizer* at $\mathbf{x} = \mathbf{A}^{-1}\mathbf{b}$ when the gradient vanishes $\nabla f(\mathbf{x}) = 0$.

Next, the strategy for the iterative technique is to create a fixed point iteration where $\mathbf{x} = \mathbf{A}^{-1}\mathbf{b}$ is the fixed point where *within each step*:

- Select a direction $\mathbf{p}^{(k)}$ and choose a scale $\alpha^{(k)} > 0$ so the next value is given by

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \alpha^{(k)}\mathbf{p}^{(k)}.$$

- The scalar α is chosen to minimize $f(\mathbf{x}^{(k)} + \alpha\mathbf{p}^{(k)})$ from a *simple* analytical expression.

Remark A.2. The construction of a function $f : \mathbb{R}^n \to \mathbb{R}^+$ that possesses a unique minimum when $\mathbf{x} = \mathbf{A}^{-1}\mathbf{b}$ as well as the creation of iterative techniques of the form above is a common task in *optimization*. Further discussion on this will be given in TAOP07. ⋈

Great! Now, we have an ansatz for the iteration technique, but there is still the question of how to select the direction $\mathbf{p}^{(k)}$ and how to compute the scaling $\alpha^{(k)}$.

First, we will determine the scaling $\alpha^{(k)}$. Note that it is allowed to *depend* upon the direction $\mathbf{p}^{(k)}$ which is yet to be determined. In general, finding the magnitude of this "step size" α is called a *line search* that may or may not have a closed form. In our case, for the quadratic form $f(\mathbf{x})$ above, it does! We examine

$$\begin{aligned} f(\mathbf{x}^{(k)} + \alpha\mathbf{p}^{(k)}) &= \frac{1}{2}(\mathbf{x}^{(k)} + \alpha\mathbf{p}^{(k)})^T\mathbf{A}(\mathbf{x}^{(k)} + \alpha\mathbf{p}^{(k)}) - (\mathbf{x}^{(k)} + \alpha\mathbf{p}^{(k)})^T\mathbf{b} \\ &= \frac{1}{2}\alpha^2(\mathbf{p}^{(k)})^T\mathbf{A}\mathbf{p}^{(k)} - \alpha(\mathbf{p}^{(k)})^T\mathbf{b} + \frac{1}{2}\alpha(\mathbf{p}^{(k)})^T\mathbf{A}\mathbf{x}^{(k)} + \frac{1}{2}\alpha(\mathbf{x}^{(k)})^T\mathbf{A}\mathbf{p}^{(k)} + \frac{1}{2}(\mathbf{x}^{(k)})^T\mathbf{A}\mathbf{x}^{(k)} - (\mathbf{x}^{(k)})^T\mathbf{b} \\ &= \frac{1}{2}\alpha^2(\mathbf{p}^{(k)})^T\mathbf{A}\mathbf{p}^{(k)} - \alpha(\mathbf{p}^{(k)})^T\left[\mathbf{b} - \mathbf{A}\mathbf{x}^{(k)}\right] + \frac{1}{2}(\mathbf{x}^{(k)})^T\mathbf{A}\mathbf{x}^{(k)} - (\mathbf{x}^{(k)})^T\mathbf{b} \\ &= \frac{1}{2}\alpha^2(\mathbf{p}^{(k)})^T\mathbf{A}\mathbf{p}^{(k)} - \alpha(\mathbf{p}^{(k)})^T\mathbf{r}^{(k)} + \frac{1}{2}(\mathbf{x}^{(k)})^T\mathbf{A}\mathbf{x}^{(k)} - (\mathbf{x}^{(k)})^T\mathbf{b}, \end{aligned}$$

where we have used the facts that $\mathbf{A}$ is symmetric and all the vectors/matrices only contain real numbers. Also, we substituted the formula for the *residual* of the current solution $\mathbf{r}^{(k)} = \mathbf{b} - \mathbf{A}\mathbf{x}^{(k)}$. This gives us a quadratic equation in terms of the variable α. If we differentiate with respect to α and set the expression equal to zero we find

$$\alpha(\mathbf{p}^{(k)})^T\mathbf{A}\mathbf{p}^{(k)} - (\mathbf{p}^{(k)})^T\mathbf{r}^{(k)} = 0 \qquad \Rightarrow \qquad \alpha^{(k)} = \frac{(\mathbf{p}^{(k)})^T\mathbf{r}^{(k)}}{(\mathbf{p}^{(k)})^T\mathbf{A}\mathbf{p}^{(k)}}.$$

Definition A.1 (Step size selection). If we have the current approximate solution $\mathbf{x}^{(k)} \in \mathbb{R}^n$ and a direction $\mathbf{p}^{(k)} \in \mathbb{R}^n$ in which the next step of the iteration is to be taken, then the scalar $\alpha^{(k)}$ that minimizes $f(\mathbf{x}^{(k+1)})$ is

$$\alpha^{(k)} = \frac{(\mathbf{p}^{(k)})^T\mathbf{r}^{(k)}}{(\mathbf{p}^{(k)})^T\mathbf{A}\mathbf{p}^{(k)}},$$

such that the value of the next iterative solution is $\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \alpha^{(k)}\mathbf{p}^{(k)}$. ◀

We see that the step size depends upon that current solution residual and the search direction. Therefore, once we select an approach direction for the iteration to go, the iterative method is entirely determined.

One of the simplest strategies to select the direction is to use our knowledge from multivariable calculus. Say we have a current estimate for the solution $\mathbf{x}^{(k)}$. We then know that the negative gradient, $-\nabla f(\mathbf{x})$, gives the direction in which the function $f(\mathbf{x})$ is decreasing most rapidly. In turn, we would expect that taking a step in this direction should bring us closer to the minimum value we seek. We use the gradient equation above to find the direction in the iterative method is

$$\mathbf{p}^{(k)} = -\nabla f(\mathbf{x}^{(k)}) = \mathbf{b} - \mathbf{A}\mathbf{x}^{(k)} = \mathbf{r}^{(k)}.$$

So, the direction is simply the residual of the current solution $\mathbf{x}^{(k)}$.

Definition A.2 (Steepest descent method). The method of steepest descent takes the search direction to be the current solution residual $\mathbf{p}^{(k)} = \mathbf{r}^{(k)}$ such that the iteration is given by

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \alpha^{(k)}\mathbf{r}^{(k)} \quad \text{where} \quad \alpha^{(k)} = \frac{(\mathbf{r}^{(k)})^T\mathbf{r}^{(k)}}{(\mathbf{r}^{(k)})^T\mathbf{A}\mathbf{r}^{(k)}}.$$

Here, the matrix $\mathbf{A}$ is symmetric positive definite and the initial guess $\mathbf{x}^{(0)}$ is arbitrary. ◀

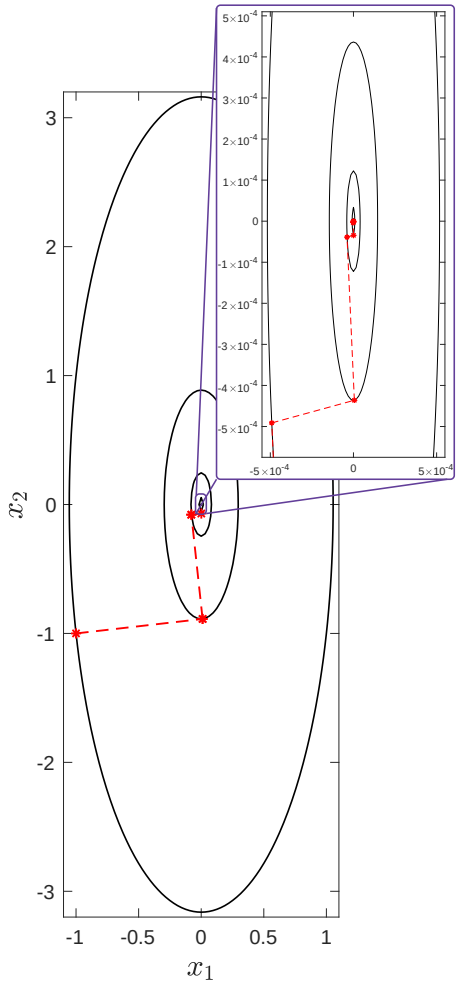
We see that the computational cost of the steepest descent algorithm is $\mathcal{O}(n^2)$ being dominated by two matrix-vector products: one needed to compute the residual of step k and another to compute the step size $\alpha^{(k)}$. We can save computational cost by noting that the residual of the next iteration is

$$\mathbf{r}^{(k+1)} = \mathbf{b} - \mathbf{A}\mathbf{x}^{(k+1)} = \mathbf{b} - \mathbf{A}(\mathbf{x}^{(k)} + \alpha^{(k)}\mathbf{r}^{(k)}) = \mathbf{r}^{(k)} - \alpha^{(k)}\mathbf{A}\mathbf{r}^{(k)}.$$

Thus, if we introduce an auxiliary vector $\mathbf{v}^{(k)} = \mathbf{A}\mathbf{r}^{(k)}$ each step of the steepest descent method only requires one matrix-vector product. We summarize the steepest descent algorithm in the pseudocode below where we use an identical kill condition to stop the iteration as we used for the Jacobi and Gauss-Seidel methods.

Algorithm A.1: *Steepest descent*: Solve a linear system with steepest descent iteration

Procedure Steepest Descent
Input: $\mathbf{A}$, $\mathbf{b}$, $\mathbf{x}_0$, `tol` `// s.p.d. matrix, right hand side, initial guess, and tolerance`
$max_{its} \leftarrow 250$ `// maximum number of iterations allowed`
$\mathbf{r}_0 \leftarrow \mathbf{b} - \mathbf{A}\mathbf{x}_0$
$\mathbf{v}_0 \leftarrow \mathbf{A}\mathbf{r}_0$
for $k = 0$ **to** max_{its} **do**
  $\alpha_k \leftarrow \mathbf{r}_k^T\mathbf{r}_k / (\mathbf{r}_k^T\mathbf{v}_k)$
  $\mathbf{x}_{k+1} \leftarrow \mathbf{x}_k + \alpha_k\mathbf{r}_k$
  $\mathbf{r}_{k+1} \leftarrow \mathbf{r}_k - \alpha_k\mathbf{v}_k$
  $\mathbf{v}_{k+1} \leftarrow \mathbf{A}\mathbf{r}_{k+1}$
  if $\|\mathbf{r}_{k+1}\|_2 < \texttt{tol} \cdot \|\mathbf{b}\|_2$ **then**
    break
Output: $\mathbf{x}_{k+1}$ `// solution vector`
End Procedure Steepest Descent



Next, we consider a simple two-dimensional example to illustrate how the steepest descent method performs.

EXAMPLE A.1. We will apply the steepest descent method to the very simple system

$$\begin{pmatrix} 1 & 0 \\ 0 & 9 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix},$$

where we know the solution is $\mathbf{x} = (0\,,\,0)^T$. We choose an initial guess of $\mathbf{x}^{(0)} = (-1\,,\,-1)^T$. We select a two dimensional problem such that we can visualize how the steepest descent iteration performs. In the figure at the left, we see that the method always moves orthogonal to the current level curve of the function $f(\mathbf{x})$ as expected. However, we see that the method zig-zags along the level curves and slowly "falls" onto the minimum of the function $f(\mathbf{x})$, which is also the desired solution. It takes 15 iterations of steepest descent to obtain a solution with $\texttt{tol} = 10^{-7}$.

We can geometrically interpret this zig-zag type convergence shown in the figure. Locally the method moves in a direction until it is tangent to a level curve, then it selects the new *locally* fastest direction based on the gradient. However, we see that the level curves of the function $f(\mathbf{x})$ are eccentric ellipses. This causes the "search" for a descent direction to ping-pong across level curves rather than following a more global "trend" down to the solution. ♦

We see that steepest descent converges, but is does so very slowly. To quantify the convergence rate we first define a convenient metric to measure the error.

DEFINITION A.3 (ENERGY NORM). If $\mathbf{A} \in \mathbb{R}^{n\times n}$ is a symmetric positive definite matrix then it defines an *inner product* on $\mathbb{R}^n$. Further, this induces a particular vector norm where

$$\|\mathbf{v}\|_{\mathbf{A}}^2 = \mathbf{v}^T\mathbf{A}\mathbf{v},$$

for some vector $\mathbf{v} \in \mathbb{R}^n$. This is often called the *energy norm* or the $\mathbf{A}$-*norm*. ◀

We will return to this energy norm and its usefulness later. For now we use it to state how the error evolves for the steepest descent method:

DEFINITION A.4 (ERROR IN STEEPEST DESCENT). The error in the steepest descent method is damped in each iteration according to

$$\|\boldsymbol{\epsilon}^{(k+1)}\|_{\mathbf{A}} \leq \left[\frac{\kappa_2(\mathbf{A})-1}{\kappa_2(\mathbf{A})+1}\right] \|\boldsymbol{\epsilon}^{(k)}\|_{\mathbf{A}},$$

where $\kappa_2(\mathbf{A})$ is the condition number of $\mathbf{A}$ in the matrix 2-norm. Because $\mathbf{A}$ is a symmetric positive definite matrix this condition number is given by

$$\kappa_2(\mathbf{A}) = \frac{\lambda_{\max}}{\lambda_{\min}},$$

which is the ratio of the largest eigenvalue over the smallest eigenvalue of $\mathbf{A}$. ◀

Remark A.3. We see that when $\kappa_2(\mathbf{A}) \approx 1$ then steepest descent converges quickly. In this case, the largest and smallest eigenvalues are very close in value for the symmetric positive definite matrix $\mathbf{A}$. ⋈

EXAMPLE A.2. From the previous example with a diagonal 2×2 matrix we immediately know that $\lambda_{\max} = 9$ and $\lambda_{\min} = 1$. Therefore, the error between steps of the iteration in the steepest descent method is not very well damped because

$$\frac{\kappa_2(\mathbf{A})-1}{\kappa_2(\mathbf{A})+1} = \frac{9-1}{9+1} = \frac{4}{5}.$$

This also helps to quantify the geometric discussion earlier because as the condition number $\kappa_2(\mathbf{A})$ increases it means that the level curves become more eccentric and convergence slows. ⧫

This leads us to the topic of improving the convergence rate of these gradient-type iterative methods. We see that the eigenvalues of the matrix $\mathbf{A}$, particularly their spread, play a crucial role in how fast the method converges. So, if we were able to change the eigenvalues of the iteration and bring their maximum and minimum values closer together convergence would become quicker. This is the idea behind ***preconditioning***. This is named as such because it involves applying a transformation to the method that *shrinks* the condition number of the problem.

Remark A.4. Recall the matrix $\mathbf{M}$ from the splitting techniques of the previous lecture. This matrix was selected such that $\mathbf{M}^{-1} \approx \mathbf{A}^{-1}$, but the matrix $\mathbf{M}$ was much cheaper to invert. This was why we denoted this as the *predconditioner* matrix. ⋈

We mention preconditioning for completeness of this presentation, but details of how to analyze and implement preconditioners into these iterative methods is outside the scope of this course.

There is another way to improve convergence of a descent-type iterative method that does not require knowledge of the eigenvalues. It has to do with selecting the *direction* $\mathbf{p}^{(k)}$ in a given step. For steepest descent we picked the negative gradient (i.e. the residual vector) as the descent direction that is ***locally*** fastest. However, we saw that this could lead to a zig-zag effect as the method searched for the solution. In the steepest descent method the direction of the current step is orthogonal to the direction of the next step:

$$\mathbf{p}^{(k)} \perp \mathbf{p}^{(k+1)} \qquad \text{or} \qquad (\mathbf{p}^{(k)})^T\mathbf{p}^{(k+1)} = 0.$$

However, there is no such orthogonality enforced with previous descent directions. For example,

$$\mathbf{p}^{(k-1)} \not\perp \mathbf{p}^{(k+1)} \qquad \text{or} \qquad (\mathbf{p}^{(k-1)})^T\mathbf{p}^{(k+1)} \neq 0.$$

This illustrates what we mean when we say that steepest descent only selects "local" direction information.

Instead, there is often a better direction to consider that is based on more "global" information. To determine this improved direction we will use the **A**-norm as well as information encoded into each of the search directions. Keep in mind that the gradient method consists of two phases: selecting the descent direction and the step size. However, we have already computed, in DEFINITION A.1, an exact value of the step size $\alpha^{(k)}$ for a given direction $\mathbf{p}^{(k)}$. So, the burning question remains of how to determine $\mathbf{p}^{(k)}$. First, we state a property that the different descent direction should possess.

DEFINITION A.5 (**A**-ORTHOGONAL). The set of nonzero direction vectors $\{\mathbf{p}^{(0)}, \ldots, \mathbf{p}^{(k)}\}$ should be mutually orthogonal in the inner product induced by the symmetric positive definite matrix **A**. That is,

$$(\mathbf{p}^{(i)})^T \mathbf{A}\mathbf{p}^{(j)} = 0, \qquad \text{if} \qquad i \neq j.$$

Such vectors are also said to be **A**-*conjugate*. ◀

With this property in mind we need a way to automatically generate such a sequence of *conjugate directions*. We start by selecting the initial direction to be identical to that of the steepest descent method (after all it is locally fastest at the beginning) with $\mathbf{p}^{(0)} = \mathbf{r}^{(0)}$. The other directions are constructed from

$$\mathbf{p}^{(k+1)} = \mathbf{r}^{(k+1)} - \beta^{(k)}\mathbf{p}^{(k)}, \qquad k = 0, 1, 2, \ldots$$

where the real number $\beta^{(k)}$ is determined to ensure the **A**-orthogonality of the next direction $k+1$ with all previous direction

$$(\mathbf{p}^{(j)})^T \mathbf{A}\mathbf{p}^{(k+1)} = 0, \qquad j = 0, 1, \ldots, k.$$

Bypassing the gory details of this derivation we have that the value to select the next search direction is

$$\beta^{(k)} = \frac{(\mathbf{A}\mathbf{p}^{(k)})^T \mathbf{r}^{(k+1)}}{(\mathbf{A}\mathbf{p}^{(k)})^T \mathbf{p}^{(k)}}, \qquad k = 0, 1, 2, \ldots$$

This form of $\beta^{(k)}$ is conducive to an efficient implementation.

DEFINITION A.6 (CONJUGATE GRADIENT). If we select the descent directions $\mathbf{p}^{(k)}$ to be **A**-orthogonal as described above and take the step size $\alpha^{(k)}$ to be that given in DEFINITION 5.1, then we obtain the so-called *conjugate gradient method*. ◀

It is difficult to summarize the conjugate gradient method in a few statements. Instead, we provide pseudocode below with comments that outlines this improved iterative technique.

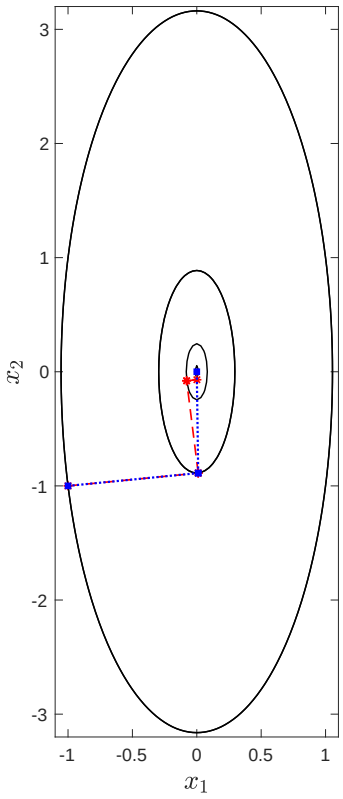
Algorithm A.2: *Conjugate gradient*: Solve a linear system with conjugate gradient iteration

Procedure Conjugate Gradient
Input: $\mathbf{A}$, $\mathbf{b}$, $\mathbf{x}_0$, `tol` `// s.p.d. matrix, right hand side, initial guess, and tolerance`
$\mathbf{r}_0 \leftarrow \mathbf{b} - \mathbf{A}\mathbf{x}_0$ `// initialize the residual`
$\mathbf{p}_0 \leftarrow \mathbf{r}_0$ `// initialize the conjugate directions`
for $k = 0$ **to** $n - 1$ **do**
  $\mathbf{v}_k \leftarrow \mathbf{A}\mathbf{p}_k$ `// compute matrix-vector product once for reuse`
  $\alpha_k \leftarrow \mathbf{p}_k^T \mathbf{r}_k / (\mathbf{p}_k^T \mathbf{v}_k)$ `// compute the step size`
  $\mathbf{x}_{k+1} \leftarrow \mathbf{x}_k + \alpha_k \mathbf{p}_k$ `// update the solution`
  $\mathbf{r}_{k+1} \leftarrow \mathbf{r}_k - \alpha_k \mathbf{v}_k$ `// update the residual`
  $\beta_k \leftarrow \mathbf{v}_k^T \mathbf{r}_{k+1} / (\mathbf{v}_k^T \mathbf{p}_k)$ `// compute the directional scaling`
  $\mathbf{p}_{k+1} \leftarrow \mathbf{r}_{k+1} - \beta_k \mathbf{p}_k$ `// compute the next conjugate direction`
  if $\|\mathbf{r}_{k+1}\|_2 < \texttt{tol} \cdot \|\mathbf{b}\|_2$ **then**
    break
Output: $\mathbf{x}_{k+1}$ `//solution vector`
End Procedure Conjugate Gradient

Just as with the steepest descent method we note the very low computational complexity for each step. The most expensive part is a single matrix-vector product which we precompute and store for reuse in the variable $\mathbf{v}^{(k)}$. The remaining tasks, like computing the step size or the next conjugate direction, are inner product calculations. So, the conjugate gradient method has a cost of $\mathcal{O}(n^2)$ and is a BLAS2 operation.

Remark A.5. A consequence of employing these conjugate directions in the gradient method to solve the linear system $\mathbf{Ax} = \mathbf{b}$ is that the iteration with *terminate* after at most n steps, yielding the exact solution. ⋈

So, we have markedly improved the rate of convergence for the iterative technique compared to steepest descent (as well as Jacobi or Gauss-Seidel from the previous lecture). For all those methods it could be the case that the number of iterations was larger (possibly much larger) that the dimension of the system. However, with conjugate gradient we see that a tolerance for the stopping criterion is typically reached in fewer than n steps.



EXAMPLE A.3. We use three example problems to explore the performance of the conjugate gradient method. Also, along the way we will compare the iteration count of conjugate gradient to steepest descent, Jacobi, and Gauss-Seidel iterative methods. First, we revisit the 2×2 system

$$\begin{pmatrix} 1 & 0 \\ 0 & 9 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}.$$

We illustrate the convergence of the conjugate gradient method compared to the steepest descent method in the figure on the right. There, conjugate gradient is the dotted blue line and steepest descent is the dashed red line. We see that the smarter selection of orthogonal search directions provides rapid convergence in two steps of the method. The iteration no longer zig-zags like the steepest descent method did. Next, we consider the two symmetric positive definite matrices

$$\mathbf{A}_2 = \begin{pmatrix} 2 & -1 & 0 & 0 & 0 & 0 & 0 \\ -1 & 2 & -1 & 0 & 0 & 0 & 0 \\ 0 & -1 & 2 & -1 & 0 & 0 & 0 \\ 0 & 0 & -1 & 2 & -1 & 0 & 0 \\ 0 & 0 & 0 & -1 & 2 & -1 & 0 \\ 0 & 0 & 0 & 0 & -1 & 2 & -1 \\ 0 & 0 & 0 & 0 & 0 & -1 & 2 \end{pmatrix}, \quad \mathbf{A}_3 = \begin{pmatrix} 4 & -1 & 0 & 0 & 0 & 0 & 0 \\ -1 & 4 & -1 & 0 & 0 & 0 & 0 \\ 0 & -1 & 4 & -1 & 0 & 0 & 0 \\ 0 & 0 & -1 & 4 & -1 & 0 & 0 \\ 0 & 0 & 0 & -1 & 4 & -1 & 0 \\ 0 & 0 & 0 & 0 & -1 & 4 & -1 \\ 0 & 0 & 0 & 0 & 0 & -1 & 4 \end{pmatrix}$$

Just as before we manufacture the right hand side vector $\mathbf{b}$ such that the exact solution will be $\mathbf{x}_{ex} = \mathbf{1}$. We set the tolerance in the kill condition to be $\texttt{tol} = 10^{-7}$ and use the initial guess $\mathbf{x}^{(0)} = \mathbf{0}$. For the matrix $\mathbf{A}_2$: conjugate gradient takes *four iterations* whereas steepest descent requires 195 iterations, Jacobi needs 163 iterations, and Gauss-Seidel needs 81 iterations. For the other matrix $\mathbf{A}_3$: conjugate gradient takes *four iterations* whereas steepest descent requires 20 iterations, Jacobi needs 18 iterations, and Gauss-Seidel needs 11 iterations. Thus, we see that conjugate gradient outperforms the other methods we have discussed while maintaining a similar computational cost. ⧫

Remark A.6. Just as before, the eigenvalues of $\mathbf{A}$ play an important role to describe the convergence of conjugate gradient. The error in the kth step is bounded by the initial error and the condition number

$$\|\boldsymbol{\epsilon}^{(k+1)}\|_{\mathbf{A}} \leq 2 \left[\frac{\sqrt{\kappa_2(\mathbf{A})} - 1}{\sqrt{\kappa_2(\mathbf{A})} + 1} \right]^k \|\boldsymbol{\epsilon}^{(0)}\|_{\mathbf{A}}.$$

⋈

Remark A.7. As a final takeaway concerning techniques for solving square linear systems. If the matrix of interest $\mathbf{A}$ is symmetric positive definite, then ***conjugate gradient*** is very straightforward and computationally efficient strategy to compute the solution. ⋈